RELUX WORKS · SWARMA

Swarma

Платформа для роёв ИИ-агентов: от одной машины до открытой сети

Алексей Григорьев, Иван Опарин

октябрь 2026 · версия 1.0, черновик

Версия на русском — источник правды; где книга и спецификации модулей расходятся, правы спецификации.

ПОМЕТКИ СТАТУСА

работает сегодня есть в коде и используется

решено владелец решил; кода может ещё не быть

спецификация описано в черновике спецификации модуля; кода нет

предложено предложение архитектуры, ждёт ревью или решения

открыто вопрос не решён

Содержание

0 Для кого эта книга ... 6
- 0.1 От агента к платформе ... 6
- 0.2 Что нужно знать заранее ... 6
- 0.3 Как читать пометки статуса ... 7
- 0.4 Карта пяти частей ... 8
- 0.5 Где заканчивается объяснение ... 8

ЧАСТЬ I. ЗАЧЕМ И ЧТО ... 9

1 Проблема ... 10
- 1.1 От ответа в чате к действию в системе ... 10
- 1.2 Кто говорит и что ему разрешено ... 10
- 1.3 У сессии слишком много дверей ... 11
- 1.4 Ответственность не передаётся отправкой ... 11
- 1.5 Почему нельзя просто скопировать среду ... 12
- 1.6 Доступ к рою и цена координации ... 12
- 1.7 У кода нет общего оракула принятия ... 13
- 1.8 Настройки запуска не равны принуждению ... 13
- 1.9 Какой ответ мы ищем ... 14

2 Swarma целиком ... 15
- 2.1 Имена и участники ... 15
- 2.2 Swarm Runtime: границы на одной установке ... 16
- 2.3 Swarm Protocol: совместная работа независимых владельцев ... 18
- 2.4 Swarm Chain: общий участник за пределами проекта ... 18
- 2.5 Один путь через три слоя ... 19
- 2.6 Порядок постройки и граница сегодняшнего состояния ... 20

3 Сквозные принципы ... 20
- 3.1 Ключ — это личность; агент просит подпись ... 20
- 3.2 Полномочия только сужаются ... 21
- 3.3 Журнал — источник истины, сессия модели — кэш ... 21
- 3.4 Суждение моделей остаётся вне консенсуса ... 22
- 3.5 Одобрение относится к точным версиям ... 23
- 3.6 Чего нельзя проверить, то не разрешено ... 23
- 3.7 Человек подписывает своё действие ... 24
- 3.8 Как читать гарантии вместе ... 24

ЧАСТЬ II. SWARM RUNTIME: РОЙ НА МАШИНАХ ОДНОГО ВЛАДЕЛЬЦА ... 25

4 Пользователь ОС на агента и сеть агента ... 26
- 4.1 Почему пользователь ОС ... 26
- 4.2 Два узких привилегированных входа ... 26
- 4.3 Учётка, поколение и журнал ... 27
- 4.4 Переход от root к агенту ... 28
- 4.5 Дома и результаты ... 28
- 4.6 Кто имеет привилегии ... 30
- 4.7 Машина выбирает сеть ... 30
- 4.8 Что ограничивает CONNECT ... 31
- 4.9 Службы, рабочее место и проверка кода ... 32
- 4.10 Незакрытые границы ... 32

5 Ключи и ключник ... 33
- 5.1 Какие секреты есть в системе ... 33
- 5.2 Мастер-секрет и стабильная личность ... 34
- 5.3 Собеседника устанавливает ядро ... 34
- 5.4 Учётка заканчивается, личность остаётся ... 36
- 5.5 Оператор подписывает одно действие ... 36
- 5.6 Внешняя личность ... 37
- 5.7 Два центра SSH ... 38
- 5.8 Цена общего корня ... 38

6 Гранты и полномочия ... 38
- 6.1 Общая модель и брокер v0 ... 39
- 6.2 Кто проверяет право ... 40
- 6.3 Делегирование только с ослаблением ... 40
- 6.4 Действие имеет имя и обработчик ... 41
- 6.5 Вызывающий остаётся в цепочке ... 42
- 6.6 Отзыв и неизвестное состояние ... 43
- 6.7 Продление без расширения ... 43
- 6.8 Человеческое разрешение и реестр ... 44

7 Путь сообщения и мост ... 44
- 7.1 Три входа ... 45
- 7.2 Носитель не становится источником полномочий ... 45
- 7.3 Сначала сохранить событие ... 46
- 7.4 Ворота доверия ... 46
- 7.5 Дополнительный guard только сужает допуск ... 47
- 7.6 Звонок, ящик и ввод сессии ... 47
- 7.7 Из чего состоит мост ... 48
- 7.8 Цель как источник событий ... 50
- 7.9 Исходящий путь сохраняет точные байты ... 50
- 7.10 Делегированная команда и границы обещаний ... 50

8 Запуск агента и хост сессий ... 51
- 8.1 Матрёшка запуска ... 52
- 8.2 План собирает вызывающий ... 52
- 8.3 Диспетчер и транзакция разовой работы ... 53
- 8.4 Служба, раннер и единственный писатель ... 55
- 8.5 Разрешение на преемника ... 56
- 8.6 Очередь и вопросы среды ... 56
- 8.7 Забор, парковка и перезапуск ... 57
- 8.8 Цель не принадлежит агенту ... 58
- 8.9 Пробуждение и перенос контекста ... 59
- 8.10 Аварийная остановка и граница реализации ... 59

9 Swarm Keeper ... 60
- 9.1 Главный цикл и границы полномочий ... 60
- 9.2 Парковка, запуск и системные роли ... 61
- 9.3 Что считается обязательством ... 62
- 9.4 Передача завершается действием ... 62
- 9.5 Сроки и проверенное время ... 63
- 9.6 Эскалация: эпизод и шаг ... 64
- 9.7 Пропавший держатель и сломанный ресурс ... 66
- 9.8 Ребёнок просит полномочий ... 66
- 9.9 Журнал, свёртка и восстановление ... 67
- 9.10 Наблюдение тоже имеет состояние ... 68
- 9.11 Швы с транспортом и платформой ... 69

10 Swarm Curator ... 69
10.1 Скилл, профиль и оверлей ... 70
10.2 Аудит и границы установки ... 70
10.3 Один владелец дома ... 71
10.4 Снимок для живой сессии ... 71
10.5 Общий и отдельный логин ... 72
10.6 Готовность перед запуском ... 72
10.7 MCP как зависимость скилла ... 73
10.8 MCP из декларации скилла ... 74
10.9 Два пути записи конфигурации ... 75
10.10 Проверяемая граница результата ... 75
10.11 Что ещё предстоит установить ... 76

11 Учётные данные и логины ... 76
11.1 Где инструменты хранят вход ... 77
11.2 Источник, хранилище и изоляция ... 78
11.3 Наследуемый setup-token ... 79
11.4 Брокер: одна подписка, аренды на запуски ... 80
11.5 Почему Codex нужен один обновляющий ... 81
11.6 Пределы выдачи и порядок включения ... 82
11.7 Диагностика и повторный вход ... 82
11.8 Рецепты с указанным статусом ... 83

12 Удалённое рабочее место ... 83
12.1 Модель думает у донора, руки — у нас ... 84
12.2 Чем защищена машина донора ... 84
12.3 Запрет нельзя снять настройкой профиля ... 85
12.4 Первый контакт без ручной сверки отпечатка ... 86
12.5 Канал и исполнитель ... 87
12.6 Как забирается код ... 87
12.7 Веб, проверки и ресурсы ... 88
12.8 Что именно защищают границы ... 89
12.9 Остановка не равна отзыву ... 89

13 Доска задач и процесс ... 90
13.1 Что уже хранится и проверяется ... 90
13.2 Продьюсер, ревьювер и Change Request ... 90
13.3 Почему интеграция сравнивает точную голову ... 91
13.4 Процесс задаёт плейбук ... 92
13.5 У состояния один владелец ... 93
13.6 Переход — предложение, а не команда агенту ... 94
13.7 Правила под всеми плейбуками ... 95
13.8 Новая версия процесса не применяется наполовину ... 95
13.9 Цель, ресурс и свидетельство — разные вещи ... 96
13.10 Проверяемый процесс и предел приёмки ... 96

14 Модели и деньги ... 97
14.1 Четыре вопроса перед запуском ... 98
14.2 Как принимается одно решение ... 98
14.3 Что считается данными о качестве ... 100
14.4 Кто разрешает тратить деньги ... 100
14.5 Запись решения и воспроизведение ... 101
14.6 Обёртка сегодня, шов в доске — дальше ... 101
14.7 Локальные модели: имя не удостоверяет веса 102
14.8 Почему выбор детерминирован ... 103

15 Память агентов ... 103
15.1 Что принадлежит памяти ... 104
15.2 Пять видов записей ... 105
15.3 Как получается текущее представление ... 105
15.4 Свидетельство и полнота поиска ... 106
15.5 Сначала документальный реестр ... 107
15.6 Данные для агента, без веса инструкций ... 107
15.7 Происхождение содержимого и стирание ... 108
15.8 Профили и память об участниках ... 109
15.9 Что отложено и как проверять пользу ... 109

16 Оператор и интерфейсы ... 110
16.1 Работающие входы ... 110
16.2 Куда должен прийти CLI ... 111
16.3 Каналы для разговора и наблюдения ... 112
16.4 Телефон оператора ... 112
16.5 Несколько машин и людей ... 113
16.6 Что остаётся спроектировать ... 114

ЧАСТЬ III. SWARM PROTOCOL: МНОГО ВЛАДЕЛЬЦЕВ, ОДНА СРЕДА ... 115

17 Модель протокола ... 116
17.1 Одна среда, несколько владельцев ... 116
17.2 Три плоскости работы ... 116
17.3 Участники и роли ... 117
17.4 Что пересекает сеть ... 118
17.5 Область задачи и право исполнения ... 119
17.6 Ревизия, кандидат и свидетельство проверки 119
17.7 Удалённый инструмент в рабочем цикле ... 120
17.8 Происхождение команды и граница наблюдения ... 121
17.9 Состояние проекта и доверие ... 121

18 Журнал, кворум и инварианты ... 122
18.1 Запись и её совершение ... 122
18.2 Кто устанавливает порядок ... 122
18.3 Три уровня доверия ... 123
18.4 Дерево Меркла и лёгкий клиент ... 124
18.5 Ключи и предел свидетельства ... 124
18.6 Какие факты хранятся в журнале ... 125
18.7 Доступность и представления ... 125
18.8 Инварианты авторизации и принятия ... 126
18.9 Инварианты независимости и восстановления ... 127
18.10 Инварианты границ исполнения ... 127

19 Контракт задачи и правила валидаторов ... 127
19.1 Откуда берутся правила процесса ... 128
19.2 План разрешает, контракт требует ... 128
19.3 Версия защищает условия приёмки ... 129
19.4 Профиль свойств и три исхода ... 129
19.5 Адрес одобрения и аттестации ... 130
19.6 Цепочка полномочий ... 130
19.7 Класс риска и защищённые изменения ... 131
19.8 Write scope и зависимости ... 131
19.9 Предикаты перед подписью ... 132
19.10 Пример проверки интеграции ... 132
19.11 Чего подпись не обещает ... 133

20 Оркестратор по аренде и цикл задачи 133
20.1 Планирование без прямого исполнения 134
20.2 Аренда, эпоха и fencing-токен 134
20.3 Как сменяется держатель роли 134
20.4 Журнал вместо незаменимой памяти 135
20.5 Области и один принимающий писатель 135
20.6 От цели к ревизии 136
20.7 Почему нужен интеграционный кандидат 136
20.8 Кто проверяет кандидата 137
20.9 Атомарное продвижение головы 138
20.10 Ограниченные раунды и замена продьюсера 138
20.11 Что проверяет следующая сторона 139
20.12 Доступность результата и его применимость . 139
20.13 Где ожидание является правильным исходом 140
20.14 Одна задача от назначения до принятия 140

21 Что проверено и как проверить остальное .. 141
21.1 Как воспроизвести результат модели 141
21.2 Что находится внутри модели 141
21.3 Четыре правила при доставке 142
21.4 Напечатанный результат и его отрицательные контроли 142
21.5 Где заканчивается установленный результат . 143
21.6 Эксперимент со сбоями реализации 144
21.7 Эксперимент о полезности роя 144
21.8 Как оценивать качество ревью 145

22 Внешние эффекты, остановка и федерация . 145
22.1 Разрешение ещё не означает установленный исход 146
22.2 Актуальность проверяется перед исполнением 147
22.3 Что отдаёт рабочая нода 147
22.4 Две формы клиента и их действительная граница 148
22.5 Остановка одной подписью 148
22.6 Что согласуется при присоединении 149
22.7 Смена роли, остановка и потеря ответа 149
22.8 Федерация проектов 149

23 Модель угроз 150
23.1 Кому и в чём доверяют 150
23.2 Что именно должно предотвращаться 151
23.3 Злонамеренный оркестратор 151
23.4 Злонамеренная рабочая нода 152
23.5 Скомпрометированная среда 152
23.6 Численная иллюстрация риска ревью 153
23.7 Контроль идентичности и риск множественных личностей 153
23.8 От каких отказов кворум не спасает 153
23.9 Явные пределы обещания 154

ЧАСТЬ IV. SWARM CHAIN: ИДЕНТИЧНОСТЬ, РЕПУТАЦИЯ, ЭКОНОМИКА 156

24 Идентичность агента 157
24.1 С чем агент приходит в чужой проект 157
24.2 Как соединяются три слоя 157
24.3 Стабильная личность в рантайме 158
24.4 Иерархия, внешний родитель и имя 159
24.5 Что считается доказательством идентичности 159
24.6 Три оси независимости 160
24.7 Ротация, отзыв и непрерывность 161
24.8 Границы предложения 162

25 Аттестации и переносимая репутация 162
25.1 От исхода задачи к общей истории 162
25.2 Что должно быть в свидетельстве 163
25.3 Что остаётся за пределами цепочки 164
25.4 Кто вычисляет репутацию 164
25.5 Как история влияет на допуск 165
25.6 Пример: ревьювер переходит между рантаймами 166
25.7 Чего репутация не обещает 167
25.8 Открытые вопросы 168

26 Цепочка: устройство 168
26.1 Три подслоя с разными обязательствами 169
26.2 Где заканчивается консенсус 170
26.3 Блок и сегмент данных 170
26.4 Сложность для небольшой сети 172
26.5 Доказательство работы и распределение монет 173
26.6 Эмиссия оплачивает безопасность 174
26.7 Сервисные узлы без власти над консенсусом. 174
26.8 Кто сохраняет историю 175
26.9 Лёгкое чтение и проверяемый индекс 175

27 Социальный слой, память сети и экономика . 177
27.1 Публикация: указатель отдельно от файла 177
27.2 Браузер и демон — разные участники раздачи 178
27.3 Граф решает, что помнит сеть 179
27.4 Окрестность архивиста 180
27.5 Почти бесплатный текст и спам 181
27.6 Клиенты и переносимость 182
27.7 За что платят 182
27.8 ИИ-медиа: происхождение без обещания аудитории 184
27.9 Операторы и правовые границы 184

ЧАСТЬ V. ВМЕСТЕ 186

28 Сквозные сценарии 187
28.1 От просьбы до принятого изменения 187
28.2 Постоянная команда переживает остановку .. 188
28.3 Обязанность переходит только после действия 189
28.4 Просьба из чата и действие оператора 189
28.5 Донор приносит модель, проект оставляет инструменты 190
28.6 Независимые владельцы принимают точный кандидат 190
28.7 Ревьювер переносит историю в другой Runtime 191
28.8 Память помогает продолжить, но не разрешает действие 192
28.9 Автор, куратор и архивист сохраняют публикацию 193

29 Где мы и что дальше 193
29.1 Одна карта состояния 193
29.2 Ближайший путь Runtime 196
29.3 Что должен показать Protocol 197
29.4 Что остаётся за Chain 197

30 Словарь 198
30.1 Платформа и участники 198
30.2 Изоляция и полномочия 199
30.3 Сеть, сообщения и удалённое исполнение 200
30.4 Сессии, пакеты и учётные данные 201
30.5 Обязательства, процесс и модели 201
30.6 Доказательная память 202
30.7 Журнал и принятие изменения 203
30.8 Личность и переносимые свидетельства 204
30.9 Цепочка и социальный слой 204

0 Для кого эта книга

Эта книга объясняет Swarma — платформу для роёв ИИ-агентов, от одной машины владельца до совместной работы независимых участников. Мы начинаем с вопроса, знакомого тому, кто запускал несколько агентов для программирования: кто именно сейчас действует, на каком основании и кто отвечает за результат после остановки сессии. Затем показываем устройство платформы, границы доверия и то, что ещё предстоит проверить. Это введение в архитектуру, а не инструкция по установке готового распределённого роя.

Наш читатель понимает распределённые системы и блокчейн, работал с Claude Code, Codex или похожей агентской средой, но ничего не знает о нашем внутреннем устройстве. Ему не нужны история обсуждений и имена компонентов, понятные только авторам. Нужна связная модель: какая проблема у каждого механизма, где проверяется право и на каком допущении держится гарантия. Поэтому определение предшествует использованию, а рядом с нетривиальным правилом есть пример его применения и граница обещания.

0.1 От агента к платформе

Агентская среда — программа, которая ведёт диалог с моделью и исполняет вызванные ею инструменты. Сессия — один такой разговор со своим контекстом и состоянием исполнения. Агентом здесь называем участника работы, которому назначены роль и полномочия; процесс конкретной среды исполняет его работу. Эти различия нужны уже на одной машине: имя в чате, процесс ОС и криптографическая личность отвечают на разные вопросы.

Рой — несколько агентов, координирующих работу над целью. Оркестратор планирует её, ставит задачи и принимает результаты в пределах выданных полномочий. Продьюсер производит изменение, ревьювер проверяет его. Дальше книга объясняет, почему эти роли нельзя считать одним доверенным голосом и почему вердикт «принимаю» должен относиться к определённому состоянию проекта. Наличие ещё одного разговора с моделью само по себе не создаёт ни независимости, ни права на интеграцию.

Swarma объединяет три слоя. **Swarm Runtime** удерживает агентов на машинах одного владельца. **Swarm Protocol** описывает совместную работу независимых владельцев над единой средой исполнения. **Swarm Chain** задуман как общий слой денег, имён, личностей и социальных записей. Это карта архитектуры с разной зрелостью частей: рантайм строится, протокол спроектирован, цепочка остаётся замыслом. Их состав и статусы разобраны в главе «Swarma целиком».

0.2 Что нужно знать заранее

Мы не пересказываем основы перечисленных областей, но поясняем, зачем знакомый примитив нужен именно здесь.

Область	Что предполагается знакомым	Где пригодится
Распределённые системы	журнал, кворум, консенсус, византийский отказ, аренда	порядок авторизованных действий и смена оркестратора
Криптография и PKI	подпись, сертификат, цепочка доверия, отзыв	личности, полномочия и действия оператора
Хэширование	адресация по содержимому, дерево Меркла	связь проверки с кандидатом и проверка включения записи
Блокчейн	UTXO, доказательство работы, проверка транзакций	устройство и ограничения Swarm Chain

Область	Что предполагается знакомым	Где пригодится
Unix	пользователи, группы, права, процессы, `sudo`, PTY, Unix-сокеты	граница между агентом и службой на одной машине
Сеть и SSH	TLS, DNS, прокси, ключ хоста и ключ клиента	транспорт сообщений и удалённое рабочее место
git	ветка, коммит, рабочее дерево	кандидат, ревью и атомарное продвижение основной ветки
Агентские инструменты	контекст, ход, инструменты, субагенты, скиллы и MCP	запуск, жизнь сессии и состав её окружения

Не нужно заранее знать Matrix: достаточно понимать федеративный мессенджер как носитель сообщений. Когда он появляется в устройстве моста, мы отдельно объясняем, что даёт доставка, а что должны проверять подпись и полномочия. Точно так же знание SSH не заменяет понимания границы рабочего места: защищённый канал отвечает за соединение, а доступ к файлам и процессам ограничивается другими механизмами.

0.3 Как читать пометки статуса

Мы описываем одновременно действующие компоненты, принятые решения и предложения. Поэтому статус относится к конкретному утверждению, а не ко всей главе по её названию. Пометка стоит в конце абзаца, пункта списка или строки таблицы. Пометка вводного абзаца распространяется на следующий список или таблицу, если у их элементов нет собственной. Используются только пять значений.

Пометка	Как её понимать
работает сегодня	есть в коде и используется
решено	владельцы приняли решение; реализации может ещё не быть
спецификация	механизм описан в черновике спецификации модуля; кода нет
предложено	архитектурное предложение ждёт ревью или решения
открыто	вопрос пока не решён

Эти значения не образуют шкалу доказанности. Принятое решение не означает опубликованного формата, а наличие формата не означает работающего модуля. Описание того, как служба должна отвечать на запрос, читается вместе с пометкой: настоящее время в архитектурном примере не превращает проектируемое поведение в наблюдаемое. Свидетельство работающей части также нельзя автоматически перенести на соседнюю часть с похожим именем.

Например, профили, которые направляют агентскую среду через прокси настройками запуска, уже используются. работает сегодня

Принудительная сеть, где обход перекрывает ядро ОС, описана отдельно и требует квалификации поддерживаемых сочетаний системы и среды. Успешный вызов модели через прокси не доказывает такого ограничения. спецификация

Это два разных утверждения, и для них нужны две разные пометки, даже если в обоих встречается слово «профиль».

Другой пример — переносимая репутация. Предлагается, чтобы рантайм проекта подписывал аттестацию работы агента, а другие рантаймы читали историю из Swarm Chain и применяли собственную политику допуска. Это направление, а не действующий общий реестр. предложено

Формат записи, правила выдачи и способы оспаривания остаются открытыми. открыто

Подробности вынесены в главу «Аттестации и переносимая репутация».

0.4 Карта пяти частей

Книга идёт от причины к механизму, затем собирает механизмы в сценарии. Её можно читать подряд или выбрать маршрут под свою задачу.

Часть	Основной вопрос	Что в ней искать
Зачем и что	почему существующего агентского процесса недостаточно	проблему, общую схему и сквозные принципы
Swarm Runtime: рой на машинах одного владельца	как удерживать агентов и их полномочия на своих машинах	пользователя ОС и сеть, ключи, гранты, сообщения, сессии, Swarm Keeper, Swarm Curator, логины, доску и модели
Swarm Protocol: много владельцев, одна среда	как допустить чужую ёмкость к общему проекту	журнал, кворум, контракт задачи, аренду оркестратора, кандидата и независимые проверки
Swarm Chain: идентичность, репутация, экономика	где участник существует за пределами одного проекта	направление идентичности и аттестаций, устройство цепочки, социальный слой и память сети
Вместе	как части соединяются и что реально есть	сквозные сценарии, сводку состояний, следующие шаги и словарь

Владельцу установки стоит начать с этой части, затем прочитать «Пользователь ОС на агента и сеть агента», «Гранты и полномочия» и «Оператор и интерфейсы». Они объясняют, что именно человек делегирует и какие действия остаются операторскими. После них полезны «Сквозные сценарии» и «Где мы и что дальше»: схема желаемой установки и её сегодняшнее состояние не должны сливаться в одно впечатление.

Инженеру рантайма нужен весь путь от учётки до сессии: нельзя оценить ключник, не понимая, как служба узнаёт собеседника, или мост, не понимая, кто владеет вводом. Инженеру распределённого слоя можно после общей части перейти к главам «Модель протокола» и «Журнал, кворум и инварианты», затем вернуться к реализации локальных границ. Интересующимся цепочкой сначала нужна глава «Идентичность агента»: связь ключа, личности и отзыва определяет, что именно переносится между проектами.

0.5 Где заканчивается объяснение

Одно понятие получает подробное объяснение в своей главе; в других местах мы ссылаемся на неё по названию. Общая часть задаёт вопросы и связи, но не повторяет все правила модулей. Например, здесь достаточно понимать грант как подписанное разрешение. Его проверка, ослабление при делегировании и связь с отзывом разобраны в главе «Гранты и полномочия». Форматы и коды отказов определяют спецификации, а не иллюстрации книги.

Технические примеры показывают конкретную границу: чего агент не может сделать, даже если модель ошиблась или подчинилась чужому тексту. Они не означают, что платформа обеспечивает правильность любой программы, скрывает код от модели или делает подтверждённого владельца честным. По мере чтения стоит задавать один и тот же вопрос: что именно проверено, кем и при каких предпосылках. Там, где ответ ещё не установлен, мы оставляем открытый вопрос вместо подразумеваемой гарантии.

Часть I. Зачем и что

1 Проблема

В этой части мы разбираем, зачем нужна платформа, как связаны её три слоя и какие принципы проходят через них. Начнём с ситуации до появления этих механизмов: агент уже умеет править код и вызывать инструменты, а среда исполнения ещё не умеет отделять его личность от имени процесса, право от текста сообщения и ответственность от памяти разговора.

Представим проект приложения с нативным ядром. Для сборки нужны определённые macOS и Xcode, для проверок — устройства, для подписи выпуска — секреты проекта. Владелец хочет поручить агентам разработку и ревью; другие люди готовы предоставить ёмкость своих моделей. Среда проекта остаётся у команды, модели работают у участников. Такой сценарий мотивирует Swarm Protocol, но не описывает уже развёрнутый распределённый рой. предложено

Здесь сталкиваются две задачи. На одной машине нужно удержать агентский процесс внутри выданных прав, даже если модель ошиблась. Между владельцами нужно связать предложение изменения с проверяемым основанием для его принятия. Дополнительные агенты не решают эти задачи автоматически: они добавляют процессы, сообщения, состояния ожидания и точки, в которых исчезает ясность о том, кто действует и кому что разрешено.

1.1 От ответа в чате к действию в системе

Пока модель только отвечает в чате, перенос решения в настоящую систему делает человек: выбирает файлы, копирует патч, запускает сборку. Агентская среда замыкает этот цикл. Она читает файлы, меняет их, вызывает shell, получает вывод тестов и решает, что сделать дальше. Это полезный переход, но вместе с ручным переносом исчезает и часть неявного контроля: теперь текстовое решение модели непосредственно превращается в действие процесса.

Обычный запуск из терминала даёт агенту полномочия пользователя ОС. Ему доступны те же файлы, конфигурация и ключи, что любому другому процессу этого пользователя. При параллельном запуске несколько агентов делят эту область. Их роли различаются в промптах и именах сессий, но для ядра это ещё не разные участники. Ревьювер, которому по смыслу нужен только кандидат, технически может оказаться с доступом к настройкам продьюсера и оператора.

Разница между инструкцией и границей здесь существенна. Просьба «не читать чужой ключ» действует, пока агент ей следует. Отдельное ограничение должно сохраняться, когда он ей уже не следует. Поэтому мы рассматриваем захват поведения агента чужим текстом как исходное условие проектирования, а не как исключение, которое можно закрыть ещё одной фразой в системном промпте. Граница должна удержать процесс, чьё суждение перестало быть доверенным.

Для примера возьмём результат команды, который предлагает «для исправления сборки» открыть соседний файл с учётными данными. Модель может принять его за полезную подсказку и вызвать чтение. Если файл доступен тому же пользователю, отказ зависит от суждения модели. Если доступ запрещён отдельной границей ОС, ошибочное решение модели само по себе файла не откроет. В Swarm Runtime выбрано направление отдельного пользователя на каждого автономного агента вместе с ограничением сети; механизм не считается включённым лишь потому, что решение принято. решено

1.2 Кто говорит и что ему разрешено

Имя в мессенджере не является доказательством личности. Оно помогает человеку найти разговор, но не отвечает на вопрос, кто подписал конкретное сообщение. Принадлежность к комнате тоже не даёт полномочия менять проект, запускать агента или останавливать службу. Если весь входящий

текст просто попадает в контекст, решать, какое сообщение считать приказом, приходится самой модели — той же модели, на которую воздействует инъекция.

Пусть в комнате проекта приходит просьба перезапустить общий раннер. Участник мог иметь право обсуждать сборку, но это ещё не право управлять машиной. И даже подлинная подпись отвечает только на вопрос об отправителе: она не создаёт разрешения на указанную операцию. В целевой архитектуре подпись и полномочие проверяются раздельно, а право проверяет сторона, которая совершает действие. Свободный текст остаётся данными для обсуждения. предложено

У моделей есть собственные разрешения на инструменты, но они не заменяют язык делегирования платформы. Разрешить shell вообще и разрешить конкретной личности определённое действие над определённой целью — разные контракты. Оркестратор может иметь право запустить разработчика, но не выдать ему больше доступа, чем получил сам. Модель может предложить расширение задачи, однако предложение не должно менять условия приёмки без отдельной авторизации. Эти границы дальше разбираются в главах «Гранты и полномочия» и «Контракт задачи и правила валидаторов».

Транспорт также не должен становиться скрытым источником прав. Шифрование канала полезно против подслушивания, но не определяет, имеет ли отправитель право на действие. Сообщение не становится командой владельца оттого, что пришло по привычному мессенджеру или через защищённое соединение. Для этого нужны проверяемое происхождение, разрешение и проверка актуальности на стороне исполнения. Разделение доставки и допуска принято как принцип рантайма. решено

1.3 У сессии слишком много дверей

У агентских сред разные способы принимать ввод: терминал, поток структурированных сообщений, серверный интерфейс. Интеграции с чатами часто строятся на эмуляции ввода, хуках и уведомлениях. Тогда возникает ещё один вопрос: кто вправе записать новые слова в уже работающий разговор? Если к терминалу может обратиться любой процесс того же пользователя, содержимое сессии защищено ровно настолько, насколько защищён этот общий пользователь.

Это проблема и безопасности, и согласованности. Агент может получить команду из чата одновременно с изменением цели от оркестратора. Один путь проверяет право, другой попадает непосредственно во ввод. Наличие проверяющего моста на одном маршруте не помогает, если рядом остаётся обход. Целевая архитектура выделяет хост сессий, который повторно проверяет допуск при мутациях; в ввод сессии пишет один писатель. Это архитектурное предложение, а не гарантия сегодняшних способов доставки. предложено

Сессия ещё и хрупка как носитель состояния. Процесс может завершиться, машина — перезагрузиться, контекст — сжаться, модель — смениться. Если полномочие, срок или решение существует только в истории разговора, восстановление превращается в пересказ. Преемник не знает, где обязательство, а где предположение предыдущего агента. Поэтому вопрос «как продолжить разговор» слабее вопроса «как восстановить авторитетное состояние работы». Последний требует источника, который не исчезает вместе с контекстом.

1.4 Ответственность не передаётся отправкой

Та же проблема проявляется, когда агенты передают работу друг другу. Например, ответственность уходит от прежнего держателя, хотя новый так и не начинает действие. Или ребёнок ждёт решения, которого сам принять не может, а родитель простаивает: ожидание существует в переписке, но не как видимое обязательство со сроком. Общая причина — отсутствие устойчивого ответа на вопрос, кто кому что должен сейчас.

Нельзя исправить этот разрыв одним уведомлением «принял». Между получением сообщения, принятием ответственности и первым действием есть отдельные состояния. В направлении Swarm Keeper передача должна завершаться первым действием нового держателя; до этого обязательство

остаётся у прежнего. Ожидание ребёнка должно быть видно и иметь срок, после которого вопрос получает продолжение или эскалацию. Эти правила пока предлагаются для ядра обязательств. предложено

Наблюдение тоже может отказать. Если после перезапуска оно не восстанавливается, тишина перестаёт отличаться от отсутствия проблем. Если при сжатии контекста у срока остаётся только время без даты, его уже нельзя однозначно сравнить с текущим моментом. Если держатель остановлен, новые адресные вопросы к нему могут лежать без ответа. Это разные проявления одного разрыва между действительной жизнью процессов и моделью ответственности в разговоре.

Нужен журнал обязательств, из которого можно восстановить держателей, ожидания и продолжения. Но сам журнал не умеет пробуждать агентскую среду. Работающий приём сообщений и возможность разбудить простаивающую сессию требуют раздельного свидетельства; пока надёжное пробуждение не установлено, эскалация не может опираться на него как на уже выполненную зависимость. У Swarm Keeper эта зависимость остаётся открытой. открыто

Полное устройство вынесено в главу «Swarm Keeper».

1.5 Почему нельзя просто скопировать среду

Привычная схема параллелизма — запустить агента рядом с кодом и дать каждому отдельное рабочее дерево или виртуальную машину. Это удобно, когда среда дешёвая и воспроизводимая. Но копия файлов проекта не равна копии всей возможности работать: тулчейн, лицензии, устройства, ключи подписи, данные и прогретые кэши могут оказаться дорогой или непереносимой частью установки. Тогда увеличение числа агентов требует умножать то, что умножать трудно.

В нашем примере отдельная копия репозитория не создаст ещё один набор устройств и не разрешит передать ключи подписи на чужую машину. GPU и данные с ограничениями на вывоз дают ту же проблему в другом виде. Рабочее дерево помогает отделить изменения, но не решает, где выполнять действия, требующие этих ресурсов. Также оно само по себе не является границей безопасности между разными пользователями или владельцами.

Вторая проблема — ёмкость модели. Несколько агентов под одной подпиской делят её квоту; дополнительный процесс не создаёт дополнительного лимита провайдера. Поэтому машина с дорогой средой может иметь ресурсы для дальнейшей работы, а владелец уже не иметь доступной ёмкости модели. Приход другого участника полезен именно как приход его собственного ресурса, но этот ресурс нельзя отождествлять с правом менять общий проект.

Третья проблема возникает при попытке собрать подписки на машине проекта. Учётные данные принадлежат людям; их передача другой установке создаёт риск и может противоречить условиям провайдера. Нельзя считать копирование логинов нейтральной технической деталью. Локальные параллельные запуски также требуют учёта обновляемых токенов: независимые копии логина могут мешать друг другу при обновлении. Конкретное поведение зависит от среды и провайдера, поэтому оно разбирается отдельно в главе «Учётные данные и логины».

Swarm Protocol предлагает инверсию: среда проекта остаётся в одном авторитетном экземпляре, а агентские циклы работают на машинах контрибьюторов с их ёмкостью и учётными данными. Через сеть идут команды, вывод, хэши и подписи; агент получает доступ к своей задаче, а не управление всей установкой. Проектирование этого разделения не снимает ограничений среды: её пропускная способность всё равно ограничивает рой. предложено

1.6 Доступ к рою и цена координации

Мотивация проекта — разрыв между теми, кто может собрать большой координируемый рой, и теми, кто располагает отдельными агентами. Вендор модели или крупная организация может сосредоточить ёмкость, инфраструктуру и управление. Открытый проект обычно координирует людей через

задачи, pull request и ревью сопровождающего. Агент помогает одному участнику, но само его появление ещё не превращает процесс проекта в общий автономный рой.

Мы предполагаем, что перенос части координации на уровень агентов может дать выигрыш. Это проверяемая гипотеза, а не вывод из количества запущенных процессов. Переписка между агентами тоже расходует токены, независимые проверки повторяют работу, а ограниченные ресурсы среды создают очередь. Увеличение параллелизма может дать больше кандидатов и одновременно больше неудачных интеграций. Поэтому обещание «рой быстрее» без условий и сравнения было бы подменой предмета исследования. предложено

Проверять полезность предлагается на одинаковых внешних задачах: сравнивать рой с одиночным агентом и с параллельной работой без протокола. Важны принятые задачи относительно расхода токенов, время до интеграции, доля откатов и человеческое ревью на принятую задачу. Отдельно нужны программные нарушители, которые пытаются нарушить процедуру. Полезность и соблюдение границ — разные проверки: успешная работа честного агента не показывает, что недопустимое действие будет отвергнуто. предложено

1.7 У кода нет общего оракула принятия

В формальной математике результат можно передать ядру проверки доказательств: оно устанавливает соответствие доказательства формулировке в заданной системе. У программного изменения нет универсального механического оракула, который решит, правильно ли оно удовлетворяет намерению проекта. Тесты проверяют выбранные свойства на выбранных входах. Ревью добавляет суждение, но ревьювер тоже может ошибиться, пропустить требование или разделить ошибочное предположение продьюсера.

Отсюда нельзя заключать, что автоматизация бесполезна. Следует другое: принятие приходится строить как процедуру с явными ограничениями. Рабочий агент производит изменение, но не переписывает условия приёмки и не интегрирует себя сам. Независимый ревьювер проверяет конкретный объект, исполнитель проверок сообщает результаты для того же объекта, а решение о продвижении основной ветки подчиняется проверяемым правилам. Именно такую композицию описывает Swarm Protocol. предложено

Пусть агент исправил синхронизацию, ревьювер одобрил патч, а до интеграции другая задача изменила общую основу. Совпадение текста патча уже не означает совпадения результата: соединение с новой основой может вести себя иначе. Поэтому протокол вводит неизменяемого интеграционного кандидата — состояние, собранное над конкретной головой основной ветки. Финальное одобрение и проверки адресуются ему; изменение головы требует нового кандидата и повторной проверки. Подробности — в главе «Оркестратор по аренде и цикл задачи». предложено

Кворум здесь подтверждает соблюдение процедуры, а не истинность суждения модели. Можно механически проверить, что аттестация подписана допустимым участником и относится к текущему кандидату, контракту, политике и окружению. Нельзя тем же способом доказать, что ревьювер правильно понял весь смысл задачи. Консенсус не добавляет такой способности; его назначение — согласовать проверяемое состояние полномочий и принятия. предложено

1.8 Настройки запуска не равны принуждению

Вокруг агента есть много полезных ограждений: флаг среды, переменная окружения, соглашение о каталоге, запрет команды внутри агентской сессии. Они помогают правильно ведущему себя процессу, но нельзя автоматически считать их защитой от процесса, который намеренно ищет обход. Если агент может изменить настройку или вызвать другой инструмент с тем же правом ОС, запрет остаётся частью кооперативного поведения. Это особенно важно, когда обычный успешный сценарий выглядит точно так же, как работа настоящей границы.

Например, запуск через прокси показывает, что данная среда воспользовалась прокси. Он ещё не показывает, что прямое соединение невозможно, что другой процесс не обойдёт настройку или что уже открытый сокет потеряет доступ после сужения политики. Сегодня кооперативные сетевые профили используются. работает сегодня

Принудительный вариант требует согласованных правил ядра и прокси, проверки поддерживаемых ОС, интерфейсов и способов обращения к сети. Неизвестная или неполная квалификация не становится положительной оттого, что вызов модели прошёл. спецификация

И у принуждения есть точная граница. Прокси, допускающий CONNECT к провайдеру, проверяет адресат и адрес соединения, но не расшифровывает TLS и не видит приложение внутри туннеля. Поэтому такой запрет не доказывает, что агент вообще не может вынести данные через разрешённого получателя. Разрешить модель и обещать полное отсутствие утечки через неё — разные требования. Более сильное обещание потребовало бы отдельного дизайна и свидетельств, а не более уверенного описания того же прокси. спецификация

Похожее различие есть у состояния. Факт, что процесс завершился, не всегда означает подтверждённую остановку всех его потомков; факт, что хранилище не прочиталось, не означает отсутствия записанного эффекта. Автоматическое повторение в таком случае может создать дубликат или вернуть ранее отозванное право. Спецификации рантайма разделяют неизвестный исход и подтверждённое отсутствие, а восстановление связывают с журналом и сверкой. Для постановки проблемы достаточно увидеть, почему удобный прокси-сигнал нельзя подменять требуемым фактом. спецификация

Наконец, среда привязывает агента к поставщику сразу в нескольких местах: контекст, запуск, жизнь сессии и выбор модели. Смена только имени модели не отвязывает остальные части. Если ребёнок получает глобальные скиллы и MCP машины, контекст уже шире выбранного профиля; если запуск зашит в доску, новую среду приходится встраивать в инструмент координации. Разделение модулей нужно и для этих случаев: оно делает явным, кто определяет окружение и какой контракт должен поддержать адаптер. Это задача устройства платформы, а не доказательство взаимозаменяемости любых сред.

1.9 Какой ответ мы ищем

Swarma связывает локальные и распределённые границы. На машине одного владельца нужны разделение агентов, проверяемый ввод, ограниченные полномочия и устойчивое состояние сессий. Между владельцами нужны авторизация задачи, независимая проверка кандидата и журнал, который переживает смену оркестратора. Для участия за пределами отдельного проекта предлагаются общие личности и переносимые аттестации в Swarm Chain. Последняя связь пока является направлением разработки. предложено

Мы не строим свою модель и свою агентскую среду: выбран путь готовых сред и поддерживаемых ими контрактов. решено

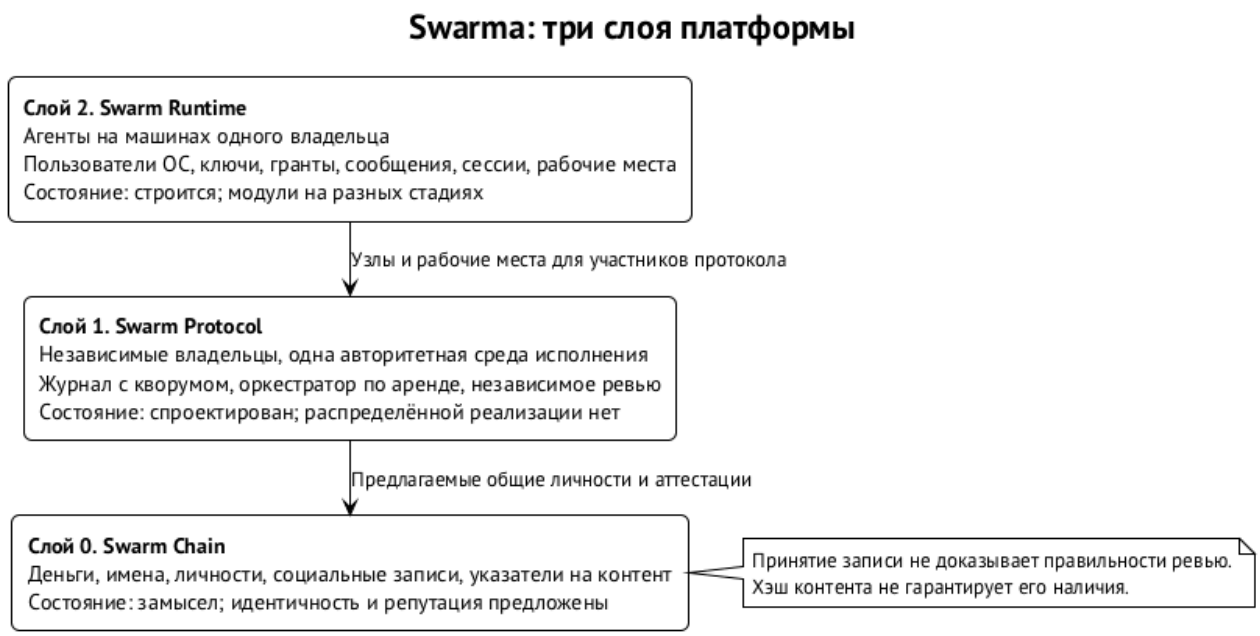
Их различия остаются инженерной задачей: профиль определяет контекст, сборка плана — запуск, хост сессий — удержание, маршрутизатор — выбор модели. Нельзя компенсировать отсутствующую возможность адаптера утверждением, что операция должна была сработать. Неподтверждённое свойство остаётся неподтверждённым, даже если обычный сценарий выглядит успешным.

Платформа не обещает правильности всех суждений, конфиденциальности кода от контрибьютора и его провайдера или защиты после компрометации авторитетной среды. Подтверждённая личность не означает честности владельца, а история удачных работ не исключает следующую ошибку. Мы ищем более узкий результат: каждое действие остаётся в выданной области, а основание принятия можно связать с точным объектом и проверить при заявленных допущениях. Как разделены эти задачи, показывает следующая глава — «Swarma целиком».

2 Swarma целиком

Swarma — платформа из трёх слоёв: рантайма на машинах одного владельца, протокола совместной работы независимых владельцев и цепочки с общим состоянием. Слои отвечают на разные вопросы. Где физически живёт агент и чем ограничен его процесс? На каком основании чужая работа меняет общий проект? Где участник существует и накапливает историю за пределами одного проекта? Ответ на один из этих вопросов не заменяет остальных.

Нумерация идёт снизу вверх: Swarm Chain — слой 0, Swarm Protocol — слой 1, Swarm Runtime — слой 2. Строим в обратном порядке. Рантайм уже имеет работающие компоненты, но целевая установка ещё строится; распределённый протокол описан, цепочка остаётся замыслом. Поэтому схема показывает архитектурную связь, а не готовую зависимость работающего рантайма от работающего блокчейна. Общие личности и аттестации в цепочке — предлагаемое направление. предложено



Swarma: три слоя платформы. Рантайм строится, протокол спроектирован, цепочка предложена; общая идентичность и аттестации ещё не реализованы.

2.1 Имена и участники

Имена закрепляют границы: Swarma обозначает всю платформу, а Swarm Runtime — только её рантайм. Карта ниже задаёт употребление имён в этой книге; она не утверждает, что перечисленные механизмы уже реализованы. решено

Имя	Что обозначает
Swarma	платформа целиком: три слоя и всё, что в них
Swarm Runtime	слой 2: рантайм на машинах одного владельца; пользователь ОС на агента, ключник, гранты, мост, хост сессий, рабочие места
Swarm Protocol	слой 1: протокол общего проекта независимых владельцев над единой средой исполнения; кор-ноды, журнал с кворумом, оркестратор по аренде, независимое ревью

Имя	Что обозначает
Swarm Chain	слой 0: цепочка с глобальным состоянием для людей и агентов; деньги, имена, личности и иерархии ключей, аттестации, социальные записи, указатели на контент
Swarm Keeper	кипер: главный цикл узла; держит обязательства агентов, сроки, передачу ответственности и эскалацию
Swarm Curator	куратор: дома агентов, профили, скиллы, оверлеи, MCP, политика хранения логинов и снимки домов

Кипер и ключник — разные компоненты. Swarm Keeper следит за продолжением работы и ответственностью. Ключник хранит мастер-секрет и выполняет криптографические операции за агентов; отдельного имени со Swarm у него пока нет. Называть ключник кипером нельзя: такое сокращение смешало бы управление обязательствами с владением ключами и скрыло бы разные границы доверия.

Владелец — человек, которому принадлежит установка и её корень доверия. Оператор выполняет привилегированные действия управления. Агент исполняет работу в пределах выданных полномочий. Проект задаёт цель, среду, правила приёмки и политику допуска участников. Контрибьютор приносит ёмкость своего агентского цикла; донор — тот, кто отдаёт ёмкость модели. Эти роли нужны для чтения схемы: предоставление ресурса, владение установкой и разрешение изменить проект не являются одним правом.

Один человек может владеть машиной с моделью, другой — средой проекта. В предлагаемом сценарии контрибьютор запускает агентский цикл у себя, а команда проекта предоставляет ограниченное рабочее место для действий над кодом. Это позволяет обсуждать участие без передачи проекту логина контрибьютора и без передачи контрибьютору секретов проекта. Какие именно ограничения доступны, должно быть объявлено и проверено; участие не предполагает общего доверия ко всем процессам обеих сторон. предложено

2.2 Swarm Runtime: границы на одной установке

Swarm Runtime объединяет небольшие модули с именованными контрактами. Внутри него удобно различать запуск и удержание, ввод и вывод, доверие и процессы работы. Это группы ответственности одного рантайма, а не дополнительные слои Swarma. Физическая граница проходит между пользователями ОС и службами: схема модулей имеет смысл только вместе с тем, кто может читать чьи файлы, открывать какие сокеты и вызывать какие операции. предложено

Фундамент рантайма — отдельный пользователь ОС для каждого автономного агента: оркестратора, постоянного специалиста, временного продьюсера и рабочего места удалённого участника. решено

Ядро разделяет файлы и процессы при корректно выставленных правах; сеть ограничивается отдельно. Это не обещание защиты после компрометации root или ядра. И это не утверждение, что работающие сегодня сессии уже все размещены за целевой границей. Принудительная сеть и применение правил описаны в спецификациях модулей. спецификация

Например, продьюсеру задачи синхронизации нужна собственная рабочая копия. Если агент и его родитель работают под разными пользователями, нельзя просто считать связанное рабочее дерево git самостоятельным репозиторием: общие служебные данные создают связь с настройками и хуками родителя. Выбрано давать такому ребёнку отдельный репозиторий в его доме; рабочие деревья остаются допустимы внутри одной учётки. Подробности границы — в главе «Пользователь ОС на агента и сеть агента». решено

Запуск распределён между компонентами. Swarm Curator собирает профиль: инструкции, скиллы и окружение. Диспетчер (`swarma-dispatcher`) принимает заявку и учитывает ёмкость. Менеджер пользователей (`swarma-user-manager`) создаёт и выводит учётки, а узкий запускатель переводит исполнение под учётку агента. Разовая работа и долгая сессия имеют разных исполнителей; хост

сессий (`agent-session-host` в спецификациях) ведёт жизнь сессий, а раннер удерживает одну среду. Контракты менеджера, диспетчера и целевого хоста описаны; эти модули ещё не реализованы. спецификация

Таблица связывает запрос с ответственным модулем. Она показывает целевое разделение, поэтому статус указан отдельно для каждой строки.

Запрос	Ответственная часть	Состояние
собрать профиль и управляемый дом	Swarm Curator	профили и управляемые дома используются работает сегодня
выбрать модель для работы	маршрутизатор моделей	используется работает сегодня
выделить учётку и запустить одобренную программу	диспетчер, менеджер пользователей, запускатель	черновики контрактов, без кода спецификация
выдать токен на исполнение	брокер учётных данных (`swarma-credential-broker`)	черновик контракта, без кода спецификация
подписать сообщение ключом личности агента	ключник	только архитектура предложено
проверить входящее сообщение и положить его в почтовый ящик	мост	только архитектура предложено
удерживать долгую сессию как службу и отдельный раннер	хост сессий	целевой контракт, без реализации этой схемы спецификация
восстановить обязательства и продолжить передачу	Swarm Keeper	направление ядра обязательств предложено

Ключник отделён от агента потому, что инъекция действует его руками. Ключ личности не должен лежать в доступном агенту файле: агент просит подпись у отдельной службы. решено

Из этого не следует, что все секреты агенту невидимы. Выданный токен провайдера модельная среда должна предъявить сама и может прочитать в своём окружении. Брокер ограничивает выдачу, но не превращает такой токен в неэкспортируемый ключ. Это предел целевой схемы брокера. спецификация

Ввод и вывод связывает мост: носитель доставляет сообщения, мост проверяет их и взаимодействует с хостом сессий, агент читает свой почтовый ящик. Подпись и грант должны давать право независимо от того, каким мессенджером пришло сообщение. Мост — первый фильтр, а хост сессий заново проверяет допуск при изменении сессии; произвольный соседний процесс не должен иметь параллельной двери во ввод. Полная схема моста и общего языка грантов пока является архитектурой. предложено

Доска задач (`task-board`) хранит задачи и процесс их прохождения: роли, переходы, ревью и Change Request — оформленное предложение изменения. Сегодня доска уже работает вместе со своим демоном сессий. работает сегодня

Целевая архитектура делает её клиентом рантайма: доска просит запуск и получает события, а не владеет всеми механизмами исполнения. Правила конкретного процесса могут сужать допуск, но не ослаблять неизменные правила ядра. предложено

Swarm Keeper добавляет к задачам вопрос продолжения: кто держит обязательство, когда должен вернуться и кому передаётся вопрос при недоступности держателя. Он не заменяет доску и не делает транспорт доказательством действия. Например, отправленное поручение ещё не завершает передачу ответственности; нужен подтверждённый переход к работе нового держателя. Это направление ядра обязательств, его подробные правила остаются предложением. предложено

2.3 Swarm Protocol: совместная работа независимых владельцев

Swarm Protocol отделяет мышление, управление и исполнение. Мышление — агентские циклы контрибьюторов с их моделями и учётными данными. Управление — кор-ноды, ведущие журнал полномочий и принятия. Исполнение — авторитетная среда проекта: основная ветка, области задач и проверки. Среда пассивна: она не запускает внутри себя агентский цикл контрибьютора и исполняет только разрешённые действия. Это устройство спроектированного распределённого слоя. предложено

Кор-ноды принадлежат независимым владельцам и проверяют правила журнала. Рабочие ноды — недоверенные агентские циклы контрибьюторов. Оркестратор — роль планирования, которую кор-нода держит по аренде. Журнал — упорядоченная цепочка подписанных записей; запись считается совершённой после получения сертификата кворума, то есть достаточного набора подписей по объявленному уровню доверия. Все эти роли разделены, чтобы право интеграции не возникало из способности предложить или выполнить изменение. предложено

Логически среда одна; физически возможны исполнительные реплики, но голова основной ветки продвигается в одном месте. Дорогой тулчейн не требуется переносить к каждому участнику. По сети передаются команды, вывод, хэши и подписи; учётные данные контрибьютора остаются у него. Это не обещает конфиденциальности прочитанного кода от модели и её провайдера. Вопрос того, что увидел недоверенный агент внутри своего контекста, протокол полностью не удостоверяет. предложено

Уровни L0–L2 различаются правилами сертификата кворума и допущениями о противнике. Их условия и границы гарантий разобраны в главе «Журнал, кворум и инварианты». Это параметры проекта в протоколе, а не достигнутые свойства действующей сети. предложено

Аренда оркестратора содержит эпоху и fencing-токен: средство отсечь прежнего держателя, даже если он проснулся после паузы и считает себя действующим. Преемник восстанавливает состояние из журнала, а унаследованную сессию модели считает недоверенной. Лидер консенсуса и оркестратор при этом разные роли: один отвечает за порядок, другой планирует работу. Их смешение сделало бы суждение модели частью механизма согласования. предложено

Путь изменения выглядит так: продьюсер работает в области задачи, среда строит неизменяемого кандидата, назначается независимый ревьювер, выполняются проверки, затем проверяется допустимость интеграции. Аттестация — подписанный результат проверки свойства для кандидата, контракта, политики и окружения; её исход может быть положительным, отрицательным или неустановленным. Только согласованные с текущими версиями свидетельства дают основание продвигать голову. Смена головы требует пересобрать кандидата и повторить финальные проверки. предложено

Связь с рантаймом прямая, но не тождественная. Рантайм даёт пользователей ОС, рабочие места, ключи, гранты и удержание сессий. Протокол задаёт правила совместной работы независимых владельцев поверх этих возможностей. Отдельная учётка не доказывает независимости ревьювера; сертификат кворума не заменяет сетевого ограничения. Для полной гарантии должны одновременно выполняться локальные ограничения и распределённые правила, причём на заявленном уровне доверия. предложено

2.4 Swarm Chain: общий участник за пределами проекта

Swarm Chain задуман как общий слой, где люди и агенты имеют имена, ключи, записи и связи до появления отдельного проекта. Это не журнал задач конкретной установки. Он даёт возможную основу знакомства участников, переноса истории и расчётов между проектами. Направление идентичности связывает публичный ключ агента в рантайме с личностью в цепочке; направление аттестаций переносит подписанную историю работы за границу журналов проектов. Обе связи предложены, а не реализованы. предложено

Внутри Swarm Chain различаются цепочка, p2p-память и социальный граф. Это ещё одно внутреннее деление, его нельзя путать с тремя слоями платформы. Для этих частей выбраны разное содержимое и разные гарантии; работающая сеть пока не заявлена. решено

Часть	Что несёт	Что означает её свидетельство
цепочка	деньги, имена, личности и иерархии ключей, социальные записи, указатели на контент	порядок записей; консенсус для денег и имён
p2p-память	тяжёлый контент: медиа, архивы и другие объекты	хранение зависит от тех, кто раздаёт; гарантии наличия нет
социальный граф	подписки, отметки, категоризации, связи доверия	поведение участников помогает находить и сохранять контент, но не обещает вечного хранения

Консенсус цепочки ограничен деньгами и именами. Для социальных записей узел проверяет форму, подпись и оплату; смысл детерминированно вычисляют индексаторы. Тяжёлый контент находится вне цепочки, в ней остаются указатели. Например, сохранённый хэш архива позволяет проверить полученный архив, но не обязывает кого-либо продолжать его раздачу. Такое разделение принято в замысле слоя; работающей реализации оно не подразумевает. решено

Предлагаемая переносимая репутация использует аттестации рантаймов проектов. После работы или ревью рантайм подписывает исход для определённого агента и объекта, а другой рантайм читает эту историю и применяет собственную политику допуска. Репутация вычисляется по аттестациям; она не принадлежит одному проекту и не является гарантией правильности следующего ревью. Ротация ключей должна сохранять личность, отзыв — исключать использование скомпрометированных ключей по установленным правилам. предложено

Из этого нельзя вывести готовый формат аттестации, тариф записи или универсальную оценку агента. Пока открыты правила выдачи, отрицательные исходы и апелляции, приватность связей между участниками, точная функция репутации и её связь с экономикой. Вариант с публикацией только хэш-обязательства приватной аттестации также требует решения. Здесь важна роль слоя в общей схеме; состав записей и незакрытые вопросы разбираются в главах «Идентичность агента» и «Аттестации и переносимая репутация». открыто

2.5 Один путь через три слоя

Вернёмся к проекту приложения с дорогой средой. На локальном уровне владелец задаёт постоянную команду и профили, диспетчер выделяет исполнения, а службы отделяют учётки агентов от привилегированных операций. Приход внешнего участника добавляет рабочее место на машине проекта, но его агентский цикл остаётся у донора. Именно такой мост между ресурсами описывает целевой рантайм; форматы некоторых связующих операций ещё не опубликованы, поэтому это архитектурный сценарий, а не рецепт развёртывания. предложено

На уровне протокола задача получает область изменений и условия приёмки. Агент контрибьютора предлагает ревизию, а среда строит кандидата для независимой проверки. Его собственный положительный отчёт не разрешает продвижение ветки: нужны свидетельства от требуемых политикой участников и проверка их адресации. Если кандидат изменился, прежнее одобрение не превращается в разрешение автоматически. Здесь граница проходит между созданием результата и властью принять его в общий проект. предложено

На уровне Swarm Chain предлагается продолжить историю участника за пределы этой задачи. Рантайм проекта подписывает аттестацию, которую позже может прочитать другой рантайм. Это не перенос кода проекта и не передача старого гранта: получатель принимает собственное решение о допуске. Например, полезная история ревью может влиять на глубину следующей проверки, но не отме-

няет проверку и не удостоверяет все будущие суждения. Готовая сеть для такого сценария пока не заявлена. предложено

Каждая граница имеет собственный сбой. В рантайме важно не дать процессу выйти из области учётки; в протоколе — не принять чужую ревизию по неподходящему свидетельству; в цепочке — не спутать включение записи с истинностью её содержания или доступностью внешнего объекта. Поэтому ни телефон оператора, ни кворум, ни хэш сами по себе не отвечают на весь сценарий. Их роль становится понятной только вместе с тем, что они удостоверяют и кто использует это свидетельство.

2.6 Порядок постройки и граница сегодняшнего состояния

Сверху вниз строятся сначала локальные основания. Сегодня используются доска задач с ревью и интеграцией, её демон сессий, Swarm Curator с профилями и управляемыми домами, маршрутизатор моделей, кооперативные сетевые профили и связь агентов через Matrix. Это конкретные работающие части, а не свидетельство готовности всей картины. работает сегодня

Менеджер пользователей, запускатель, диспетчер, брокер и целевая схема хоста сессий описаны черновиками без кода. спецификация

Следующий смысловой шаг — собрать локальные контракты в проверяемую установку и рабочее место для внешнего агента. Но нельзя дорисовать отсутствующий механизм стрелкой: исполнитель публикации и интеграции при закрытом выходе агентов, жизненный цикл входящего SSH-реле и некоторые процедуры управления ключами ещё требуют определения. Их нужно закрыть до соответствующего обещания, а не считать следствием существования других модулей. открыто

Распределённый протокол спроектирован; ограниченный фрагмент интеграции проверен моделью, а готовая распределённая реализация пока не заявлена. Swarm Chain остаётся замыслом. Связь слоёв задаёт направление постройки: локальное исполнение, проверяемое сотрудничество, общий участник и история. Дальше глава «Сквозные принципы» показывает, какие правила повторяются на этом пути и почему одинаковое правило требует разных механизмов на разных границах.

3 Сквозные принципы

Три слоя Swarma работают с разными противниками. В рантайме агент может подчиниться чужому тексту; в протоколе владельцы и их агенты не обязаны доверять друг другу; в цепочке предполагается открытая сеть. Поэтому одинаковая мысль — например, не доверять названию участника — требует разных механизмов: UID собеседника на локальном сокете, подписи записей проекта, удостоверяемой иерархии ключей. Нельзя перенести гарантию между слоями только по сходству слов.

Ниже семь принципов, которые связывают архитектуру. Для каждого есть назначение и пример на каждом слое. Примеры описывают применение правила, а не подтверждённую эксплуатацию всей платформы. У действующего механизма, принятого решения и будущего направления разные статусы. Особенно это касается идентичности и аттестаций в Swarm Chain: общий замысел ещё не стал спецификацией протокола записи.

3.1 Ключ — это личность; агент просит подпись

Имя сообщает, как обратиться к участнику; подпись связывает конкретные байты с ключом. Если закрытый ключ лежит в доступном агенту файле, инъекция может убедить его этот файл прочитать. Поэтому выбран принцип: локальный агент не получает ключ личности, а просит отдельную службу подписать данные. Это ограничивает кражу ключа, но не делает всякое подписанное агентом сообщение разумным или разрешённым. решено

В Swarm Runtime эту роль выполняет ключник. Ядро сообщает ему UID подключившегося процесса, а регистрация связывает учётку и её поколение со стабильной личностью. Например, продьюсер просит подпись своего сообщения; одной строки «я ревьювер» в запросе недостаточно, чтобы подписать за соседа. Конкретная схема ключника пока существует как архитектура. Кроме того, до ключника предусмотрен ключ вызывающего в доме оркестратора: это явное временное исключение, сила которого ограничена грантом. предложено

В Swarm Protocol долгоживущий ключ идентичности удостоверяет короткоживущие ключи нод. Подпись контрибьютора выполняется на машине его агентского цикла, ключи не передаются среде проекта. Например, сменив ключ рабочей ноды, участник должен сохранить проверяемую связь с владельцем, а не объявить новое имя доказательством независимости. Протокол не требует, чтобы внешний участник пользовался ключником чужого рантайма; способ удержания его локальных ключей нельзя приписать удалённой стороне без свидетельства. предложено

В Swarm Chain предлагается связать личность агента с публичным ключом и цепочкой удостоверений до корня владельца. Например, внешний ревьювер предъявляет эту связь новому рантайму, а тот проверяет записи, вместо того чтобы доверять имени в чате. Иерархия с внешним родительским ключом должна поддерживать ротацию и отзыв. Эта связь рантайма и цепочки остаётся направлением; доказанное владение ключом не обещает честности владельца. предложено

3.2 Полномочия только сужаются

Делегирование распределяет полученное право, но не создаёт новое. Полномочие, или capability, определяет, что участнику разрешено над определёнными объектами. Дочерний запрос должен оставаться внутри родительского по действиям, целям и сроку. Иначе оркестратор мог бы обойти собственный потолок, просто создав ребёнка с более широкими правами. Для рантайма это принятое направление; общий язык грантов ещё проектируется. решено

В Swarm Runtime грант — подписанное разрешение. Например, оркестратор, которому разрешён запуск только с определённым профилем, не может делегировать ребёнку другой профиль и более длинный срок. Проверяет это принимающая служба, а не модель-проситель. В узком контракте брокера лист агента строится как пересечение запроса и двух цепочек — вызывающего и диспетчера; непостроимая цепочка отклоняется до эффектов. Общая форма грантов и форма брокера различаются, поэтому пример ослабления не служит готовой схемой сообщения. спецификация

В Swarm Protocol цепочка идёт от цели через полномочие оркестратора к задаче и ревизии. Например, задача исправить синхронизацию не даёт продьюсеру права поменять защищённые проверки или переписать собственные критерии приёмки. Такое изменение требует новой версии контракта с отдельной авторизацией. Принятый агентом план не становится новым источником власти: область записи ревизии остаётся внутри области задачи, а задача — внутри выданного полномочия. предложено

В Swarm Chain пример проходит по границе личности и допуска. Предлагается, чтобы удостоверенная личность приносила историю аттестаций, а принимающий рантайм по своей политике решал, какое участие ей разрешить. Например, репутация ревьювера в одном проекте не даёт ему права интегрировать код другого. Запись связи ключей удостоверяет происхождение ключа, а не выдаёт все права родителя во всех проектах. Готовый переносимый формат гранта в цепочке этим направлением не определён. предложено

3.3 Журнал — источник истины, сессия модели — кэш

Контекст модели нужен для работы, но не должен единолично определять состояние полномочий и обязательств. Он может потеряться, сжаться или перейти к преемнику вместе с ошибочными предположениями. Журнал хранит то, что совершено по правилам системы. Это позволяет восстановить ав-

торитетное состояние без доверия к пересказу предыдущего агента. Принцип закреплён в устройстве протокола; конкретные журналы разных модулей имеют собственные контракты. предложено

В Swarm Runtime примером служит восстановление обязательства после перезапуска. Если в разговоре сохранилось «передал работу», а подтверждения перехода к новому держателю нет, память сессии не должна закрывать обязательство. Предлагаемое ядро Swarm Keeper восстанавливает держателя и продолжения из журнала; сжатие контекста не принимает решений о сроках. Это не утверждение о готовой реализации ядра обязательств. предложено

Действующая доска уже хранит работу вне отдельной сессии и поддерживает ревью и интеграцию. работает сегодня

В Swarm Protocol преемник оркестратора читает журнал и считает унаследованную сессию недоверенной. Например, прежний держатель проснулся после передачи роли и прислал назначение со старой эпохой. Валидаторы отвергают его как устаревшее, даже если контекст прежнего агента содержит убедительное объяснение продолжения. Аренда без fencing-токена была бы недостаточна: важно, чтобы устаревшую власть отсекала сторона, принимающая действие. Расхождение сохраняется для аудита. предложено

В Swarm Chain предлагаемый пример — репутация, которую индексатор вычисляет по подписанным аттестациям. Рассказ агента «в прошлом проекте меня принимали» не заменяет записей. Получатель должен иметь проверяемый материал, а не доверять уверенности модели. При этом цепочка не становится журналом всех задач рантайма: в неё предлагается выносить хэши и метаданные аттестаций, без кода, промптов и секретов. Как учитывать отрицательные исходы и оспаривание, ещё требуется решить. предложено

3.4 Суждение моделей остаётся вне консенсуса

Ревью отвечает на содержательный вопрос, а валидатор — на вопрос о соблюдении формальных правил. Смешение этих ролей создаёт ложное впечатление, будто подписи участников доказывают качество программы. Консенсус может согласовать записи, подписи, версии и допустимые переходы. Он не может превратить ошибочное понимание требования несколькими моделями в правильное. В Swarm Protocol это явный инвариант. предложено

В Swarm Runtime агент может предложить перезапуск службы и объяснить, почему считает его полезным. Однако его уверенность не создаёт полномочия на операцию: детерминированный обработчик должен проверить допуск и исполнить только зарегистрированное действие. Полезность решения остаётся суждением; право ограничено независимо от него. Аналогично процесс на доске может требовать дополнительное ревью, но не должен выключать неизменные правила ядра. Общая схема обработчиков и конфигурируемых процессов пока предложена. предложено

В Swarm Protocol ревьювер оценивает кандидата, а валидатор проверяет подпись, адресацию результата и независимость по политике проекта. Например, даже правдоподобный положительный вердикт недопустим для интеграции, если его дал продьюсер вместо требуемого независимого участника. Если подходящего ревьювера нет, политика автоматически не ослабляется. При соблюдённой процедуре дефект всё равно может остаться незамеченным: протокол отвечает за независимость и подотчётность суждения, а не за его безошибочность. предложено

В Swarm Chain граница проходит между принятием записи и её прикладным смыслом. Для социальных записей узел проверяет форму, подпись и оплату, а смысл детерминированно вычисляют индексаторы; консенсус нужен для денег и имён. Например, включённая запись публикации не означает, что сеть признала её полезной или истинной. Такое разделение принято в замысле слоя. решено

Аттестация ревью в этом же направлении фиксирует факт и исход процедуры, но не доказывает правоту модели. предложено

3.5 Одобрение относится к точным версиям

Проверка имеет предмет: конкретный кандидат и условия, при которых его проверяли. Одних слов «тесты зелёные» недостаточно, если неизвестно, к какому состоянию относится результат. Контракт задачи, кандидат, политика и окружение определяют адрес аттестации в протоколе. Переносить одобрение на похожий патч — значит менять проверенный объект без новой проверки. Это правило проектируемого распределённого слоя. предложено

В Swarm Runtime пример — оператор подтверждает конкретное расширяющее действие после показа его содержимого. Разрешение выдать определённый грант не должно стать постоянным разрешением выдавать другие гранты. Такое подтверждение выбрано для операторского пути. решено

Отдельный работающий пример — интеграция доски через compare-and-set, связанная с точным проверенным деревом. Совпадение ожидаемой основы важно так же, как наличие проверки. работает сегодня

В Swarm Protocol среда строит кандидата над текущей головой основной ветки. Например, после одобрения исправления синхронизации голова продвинулась из-за другой задачи. Атомарная замена с прежним ожиданием должна отказать; кандидат перестраивается, финальное одобрение и проверки повторяются. Даже неизменившийся патч не даёт права переносить положительный исход на новое соединённое состояние. Среда продвигает голову только к тому неизменяемому кандидату, которому адресованы актуальные свидетельства. предложено

В Swarm Chain предлагается сохранять в аттестации связь с личностью, проектом, кандидатом или ревизией и версиями контракта и политики. Например, новый рантайм видит исход работы над определённым объектом, а не универсальный знак качества агента. Эти сведения описывают направление записи и не задают опубликованного формата на проводе. Нельзя дописывать к нему поля по аналогии с адресом аттестации протокола и объявлять результат спецификацией цепочки. предложено

Формат и стоимость ещё открыты. открыто

3.6 Чего нельзя проверить, то не разрешено

Отказ по умолчанию нужен на границе исполнения, где ошибка допуска уже становится эффектом. Неизвестное не считается положительным исходом. Важно различать отсутствие свидетельства, подтверждённое отсутствие события и невозможность прочитать состояние: это разные факты, из которых нельзя выводить одинаковое разрешение. Принцип не заменяет проверку доступности служб; он задаёт исход, когда требуемое основание не установлено.

В Swarm Runtime диспетчер перед созданием учётки и стартом получает актуальный вердикт отзыва у брокера. Например, хранилище недоступно: это не доказательство, что грант не отозван, и запуск должен быть отвергнут. Восстановление старой копии брокера также не должно возвращать отозванные права до сверки истории. Эти механизмы описаны в спецификации. Они не требуют от просителя угадать состояние службы или подтвердить его своим текстом. спецификация

В Swarm Protocol среда объявляет, какие ограничения реально способна обеспечить. Например, задача требует полного ограничения, а среда подтверждает только частичное: назначение не должно превращать частичное свидетельство в полное. Аттестация с исходом «не установлена» также не становится успешной оттого, что положительного результата хотелось бы дождаться. Отдельно проверяется актуальность авторизации на самой границе исполнения: старая совершённая запись не разрешает действие после остановки или смены эпохи. предложено

В Swarm Chain у социальной записи проверяются форма, подпись и оплата. Например, знакомое имя автора не заменяет неверную подпись; основание принятия должно быть проверено узлом по правилам записи. Это принятая граница замысла. решено

Для аттестаций направление требует проверяемого происхождения и учёта отзыва, но ещё не задаёт окончательных правил допуска выдавших рантаймов. Отсутствующий механизм нельзя считать действующей защитой только потому, что аттестация задумана подписанной. открыто

3.7 Человек подписывает своё действие

Автономность не означает, что агент может говорить от имени человека. Если операторская подпись доступна тому же процессу, что исполняет чужой текст, инъекция способна воспользоваться и ею. Поэтому расширяющее действие должно проходить через отдельную человеческую границу с подтверждением точного содержимого. Для операторского пути рантайма выбраны телефон и защищённая учётка; запасной путь — узкие команды из терминала с аутентификацией. решено

В Swarm Runtime пример — выдача гранта постоянному участнику. Телефон показывает содержание, человек подтверждает его биометрией, ключ в защищённом чипе подписывает именно это действие. Изнутри сессии агента расширение не принимается; для чтения состояния предусмотрен отдельный путь без возможности изменения. решено

Но домен подписи, канонические байты операторской команды, регистрация и замена ключа телефона ещё не опубликованы. Решение о человеческом подтверждении не закрывает эти вопросы автоматически. открыто

В Swarm Protocol пример — изменение правил, по которым принимается работа. Продьюсер не может объявить своё толкование требования новой авторизованной версией контракта. Протокол связывает изменение с отдельным полномочием; человеческое подтверждение, когда оно требуется политикой, должно использовать ключ с присутствием человека. Способность агента написать от лица оператора не является такой подписью. Нельзя считать любой ключ кор-ноды аппаратным операторским ключом: это разные назначения и предпосылки. предложено

В Swarm Chain предлагаемый пример — удостоверение и отзыв дочернего ключа внешним родительским ключом владельца. Агент не должен подменять решение владельца собственным заявлением о происхождении личности: связь удостоверяется подписью соответствующего ключа. предложено

Это пример разделения авторства действий; он не устанавливает готовую совместимость телефона с записями цепочки. Направление идентичности не определяет совместимость операторской подписи рантайма с записями цепочки; из семи принципов нельзя вывести формат этой связи. открыто

3.8 Как читать гарантии вместе

Эти принципы соединяются на границе конкретного действия. Подлинный отправитель ещё должен иметь полномочие; полномочие должно оставаться актуальным; проверка должна относиться к принимаемому объекту; запись должна быть совершена по правилам; исполнение должно физически удерживать выданную область. Наличие одного звена не доказывает остальные. Например, правильная подпись положительного ревью не разрешает интеграцию, если изменилась голова или не соблюдена политика независимости. Так устроена предлагаемая композиция протокола. предложено

Есть и внешние пределы. Компрометация root ломает локальную границу; компрометация авторитетной среды выходит за обещание протокола. Ошибочное суждение модели может пройти корректную процедуру. Хэш контента не гарантирует наличия контента, а переносимая история не гарантирует честности следующего действия. Поэтому дальше мы разбираем механизмы отдельно: сначала «Пользователь ОС на агента и сеть агента», затем ключи, полномочия и путь от сообщения к исполнению. Общий принцип становится гарантией только вместе с конкретными допущениями, механизмом и свидетельством его работы.

Часть II. Swarm Runtime: рой на машинах одного владельца

4 Пользователь ОС на агента и сеть агента

В этой части разбираем Swarm Runtime: как рой работает на машинах одного владельца. Начинаем с границы файлов, процессов и сети; затем перейдём к ключам, полномочиям, сообщениям и запуску. Дальше — службы, которые держат обязательства, готовят дома агентов и выдают учётные данные. Важно различать принятое устройство, спецификацию и работающий механизм: отдельная учётка и настроенный прокси ещё не доказывают, что вся граница принудительно удерживается.

Каждый автономный агент должен работать как отдельный пользователь ОС. Это относится к оркестратору, постоянному специалисту, временному воркеру и рабочему месту удалённого работника. Свой UID отделяет процессы и файлы одного агента от другого; сеть требует дополнительного ограничения ядром и доверенным прокси. Такой способ разделения принят для рантайма. решено

4.1 Почему пользователь ОС

Инъекция в промпт действует с правами агента. Поэтому правило «не читать соседний дом» должно оставаться в силе, даже если модель решила его нарушить. При закрытых правах файлов и каталогов другой пользователь не получает доступа к дому, связке ключей и памяти процессов агента, не может писать в его терминал или посылать сигналы его процессам. Граница опирается на ядро, корректную установку прав и закрытую сеть; root и установка платформы считаются доверенными. решено

Контейнеры и виртуальные машины здесь не обязательны: рантайму нужна граница между агентами одной машины, а не отдельный образ окружения для каждого. Пользователь ОС даёт эту границу без дополнительного контейнерного оркестратора. Например, продьюсер и ревьювер получают разные UID: процесс продьюсера не должен читать рабочий каталог ревьювера, даже если знает, где он расположен. Это выбор устройства рантайма, а не обещание защиты от ошибки ядра. решено

Сама модель привычна Unix: службы запускают под разными пользователями, чтобы ошибка одной не давала права другой. В Swarm Runtime создание и вывод пользователей становятся обязанностью платформы. При этом администратор и root — разные роли: обычная учётка администратора получает UID 0 лишь на время привилегированной операции. Под root следует держать как можно меньше кода. решено

4.2 Два узких привилегированных входа

Менеджер пользователей (`swarma-user-manager`) создаёт, перечисляет и выводит учётки, а в v1 управляет их сетевыми правилами. Запускатель (`swarma-user-launcher`) запускает одну одобренную программу под активной учёткой. Первый ничего не запускает, второй не создаёт учёток и правил. Они не принимают произвольных команд, путей или окружения. решено

Предлагаемый вход — вызов через `sudo` по абсолютному пути без аргументов. В правиле `sudoers` это требует пустой строки `""` в конце: отсутствие ограничения аргументов разрешило бы любые аргументы. Сами программы тоже их отвергают. Запрос — один JSON на stdin с пределом чтения 5 секунд. Менеджеру допускается до 16 КиБ, запускателю — до 80 КиБ; план запуска занимает до 48 КиБ, в base64 — до 64 КиБ. Повторные ключи, лишние поля, второй документ и слишком длинные строки отклоняются. `SUDO_UID` устанавливает пользователя, но не программу, от которой пришёл запрос. предложено

Роли задаёт конфигурация root. Наличие доступа к бинарю само по себе не должно давать одинаковые полномочия всем вызывающим. Например, диспетчер (`swarma-dispatcher`) одного агента не вправе удалить чужого агента или учётку брокера учётных данных (`swarma-credential-broker`). решено

Роль	Область действий
установщик	создаёт и выводит сервисные учётки, ставит и снимает их правила сети
диспетчер	создаёт агентов в пределах квоты; запускает, останавливает, выводит и меняет правила только своих учёток
оператор	действует через `swarma-operator`, включая вывод защищённых сервисов; расширяющее действие требует подтверждения человека; запускать не может

Хост сессий (`agent-session-host`) не получает отдельной роли у менеджера. Он просит диспетчер начать или остановить исполнение. Тем самым обхода через второй привилегированный путь нет: жизненный цикл остаётся у одного владельца запуска. Подробности — в главе «Запуск агента и хост сессий». решено

Операторская подпись отделена от учётки человека: она находится в защищённой сервисной учётке, где не работают сессии агентов. Для чтения предусмотрена обязательная учётка `swarma-observer`: состояние, гранты, учётки и привязки видны без подтверждений, изменять их нельзя. Расширение полномочий требует входа и подтверждения точного действия; отзыв и отключение — входа. Устройство подписи разобрано в главе «Ключи и ключник». решено

4.3 Учётка, поколение и журнал

Жизнь учётки проходит состояния `creating → active → retiring → removed`. Менеджер журналирует шаги, откатывает неудачное создание и после входа в `retiring` движется только вперёд. Запускатель уже отказывает, брокер не выдаёт аренды, восстановление после падения продолжает вывод. Каждое создание получает случайное 128-битное поколение, отличающее его от прежнего владельца того же UID. На поколение опираются привязки и гранты; его запись сохраняется в журнале. спецификация

Например, временного разработчика вывели, а позднее его UID получил ревьювер. Одинаковый номер пользователя не должен вернуть старую привязку токена или доступ к личности разработчика. Поэтому проверяется активное поколение, а не только UID или имя. Поколение не является счётчиком, который можно вернуть назад вместе с копией журнала. спецификация

Операции менеджера и запускателя, выбор UID и восстановление координирует общий замок. Повтор того же `request_id` с теми же аргументами возвращает записанный результат и текущее состояние; с другими аргументами — отказ. У каждого шага заявки есть собственный идентификатор эффекта, выведенный диспетчером из номера запроса и шага. После сбоя повторяют эти же шаги, позволяя получателю узнать уже выполненную операцию. спецификация

Если ответ потерялся, диспетчер вызывает `effect.observe`. Возможны записанный результат, продолжающаяся операция, откат, достоверное отсутствие записи, отсутствие свидетельства и противоречивые свидетельства. Последние два исхода не разрешают считать эффект несостоявшимся. Даже откат или подтверждённое отсутствие записи сами по себе не разрешают создать учётку заново. Так потеря ответа не превращается в разрешение повторить весь запуск с новой личностью. спецификация

Восстановление ограничено зависимостями текущей операции и конечным бюджетом. Большой дом удаляется пачками с освобождением общего замка между ними: чужой запуск или остановка не должны ждать всей очистки. Служба также не держит свой замок состояния во время синхронного вызова другой службы: фиксирует операцию в полёте, отпускает замок, затем заново сверяет состояние. Граф ожиданий каждой службы должен оставаться без циклов. спецификация

Восстановление журнала из старой копии — отдельный инцидент. До публикации восстановленных данных ставится внешняя отметка и закрываются его потребители: брокер, запускатель, диспетчер и хост сессий. Поколение, которое по сохранившимся свидетельствам выводилось или было выведено,

нельзя вернуть в `active`. Для утраченного периода отсутствие свидетельства не становится отсутствием события. Открывает доступ оператор после сверки. спецификация

Рассмотрим неполный запуск: учётка уже создана, а ответ о следующем шаге не пришёл. Диспетчер не должен выбирать между безусловным повтором всей заявки и забыванием пользователя. Он сохраняет идентификаторы шагов, запрашивает наблюдение эффекта и продолжает сверку по записанному исходу. Запись об учётке и квитанция запуска отвечают на разные вопросы: существование пользователя не доказывает, что программа стартовала, а потеря ответа не доказывает, что старта не было. Такой разбор нужен и после перезапуска службы. спецификация

4.4 Переход от root к агенту

Запускатель проверяет активность поколения и принадлежность диспетчеру под замком, который держит до сброса привилегий. До открытия собственных файлов он закрывает унаследованные дескрипторы выше 0–2; дескриптор 3 резервируется для плана. Программу открывает из списка root, сверяет SHA-256 и сохраняет проверенный дескриптор до `exec`. Для разовой работы предусмотрен `curator-run`, для сессии — `swarma-session-runner`. спецификация

План должен целиком находиться на дескрипторе 3 до старта: неизменяемые байты ограниченного размера, доступные только для чтения. Труба, которую ещё некому читать, могла бы остановить запускатель под замком. Конкретный механизм подготовки такого дескриптора для каждой ОС пока не опубликован. открыто

Окружение собирается из фиксированного набора. Ещё под root и замком записывается квитанция старта с номером исполнения; второй старт того же исполнения отвергается. Затем сбрасываются дополнительные группы, GID и все виды UID; проверяется невозможность вернуться в root. Только после этого отпускается замок поколения и выполняется программа через проверенный дескриптор, на Linux — `execveat`. Агент получает 0–2 и план, без дескрипторов журнала, каталогов, брокера и замка. спецификация

Так агент не может помешать записи квитанции и не наследует привилегированное соединение с брокером. Своё соединение он открывает после перехода в учётку. При сбое внутренние дескрипторы закрываются, а записи остаются для сверки. Хост сессий пользуется тем же путём запуска, а не собственной последовательностью перехода привилегий. спецификация

Остановка через `exec.stop` охватывает все процессы UID: TERM, ожидание, KILL и проверка периода тишины. Одного дерева потомков недостаточно из-за `setsid`, двойного `fork` и `nohup`. Агентам закрыты `cron` и `at`; квалификация должна проверить невозможность закрепиться через пользовательский `launchd` или `systemd --user`. Пока процессы остались или исход неизвестен, квитанции остановки нет: продолжается тот же эффект. спецификация

После замены демона старый надзиратель не вправе продолжать запись: она сверяется с эпохой экземпляра демона. Смерть от OOM означает потерю исполнения, а не подтверждённую остановку. Менеджер должен явно учитывать судьбу процессов диспетчера, его надзирателей, исполнения, обновления токена и хоста сессий при перезапуске соседних служб. спецификация

4.5 Дома и результаты

Корень установки — `/opt/swarma`. Привилегированные программы находятся в `libexec/`, политика root — в `etc/`, журнал учёток и квитанции — в `lib/user-manager/`, сокеты — в `run/`. Сервисные дома отделены от `workers/`, карантин — от архива метаданных. Снаружи остаются только объекты, которым ОС назначает место: записи пользователей, `sudoers`, описания служб и файрвол. Установка требует администратора, но на macOS не требует отключать SIP. решено

У агента собственная основная группа с тем же именем и номером, без дополнительных групп. Дома закрыты режимом 0700. Агентские UID и GID выделяются из диапазона 30000–59999; сервисные — 450–499 на macOS и 900–999 на Linux. Курсор выдачи идёт по кругу. Повторное использование номе-

ра возможно лишь после полного круга и проверки отсутствия прежних файлов под корнем платформы и во временных каталогах; иначе следует `uid_not_clean`. решено

Файловые операции идут относительно открытых дескрипторов каталогов без следования ссылкам: `openat2` с `RESOLVE_BENEATH` и `RESOLVE_NO_SYMLINKS` на Linux, последовательный `openat` с `O_NOFOLLOW` на macOS. Дом создаётся пустым, рекурсивного `chown` нет. Например, ссылка из дома на чужой каталог при удалении отвязывается как ссылка: очистка не должна перейти к её цели. спецификация

При выводе дом удаляется. Агент может прочитать выданный токен и спрятать его копию в содержимом, имени файла или цели ссылки. Сканирование не даёт обещания отсутствия секретов, а сложный классификатор увеличил бы доверенный код root. Поэтому архивирование дома по умолчанию исключено. Код и результаты проходят через git и приёмку задач; принятие само по себе тоже не доказывает отсутствия секретов. решено

Диспетчер или оператор отдельно снимает привязку у брокера до вывода; менеджер брокеру под своим замком не звонит. При прямом выводе с недоступным брокером неподтверждённая отвязка остаётся явным долгом. Затем менеджер останавливает процессы, переносит дом в карантин, уничтожает ключи по объявленным путям и удаляет содержимое по дескрипторам. Неизвестный результат уничтожения ключа или нечитаемый объект не считается успехом. До подтверждённого удаления состояние `removed` недостижимо. спецификация

В журнале остаются исходы шагов, включая ошибки и неизвестные результаты, без секретов. Для вывода по истечению срока недостаточно скачка системных часов: требуется также реально прошедший монотонный срок с учётом сна. Экспорт метаданных предполагает только собственные служебные записи, без строк, имён и содержимого агента. Его схема и проверки происхождения пока не опубликованы; экспорт выключен. Отдельная операция сохранения дома оператором тоже не опубликована и не принимается; конфиденциальность сохранённого дома обещать нельзя. открыто

Родительский оркестратор по умолчанию не читает дом ребёнка. Предложенное для v2 чтение через ACL не является гарантией наблюдаемости: на Linux маска режима ограничит чтение файла 0600, а на macOS владелец может снять ACL. Транскрипты и журнал принадлежат хосту сессий, результаты передаются через доску и пакет передачи. предложено

Архив относится только к экспортам метаданных: каталог root 0700, файлы root 0600. Очистка при выводе и по суточному таймеру удаляет экспорты старше недели и сверх общего объёма, но не свидетельства, ещё нужные для сверки эффектов. Наличие каталога архива не разрешает складывать туда дома. Пока схема экспорта не опубликована, эта часть устройства не создаёт доступного способа сохранения данных агента. спецификация

Постоянную команду объявляет подписанное оператором описание ролей, профилей, сред и учётных записей. Диспетчер сверяет или создаёт их идемпотентно, в том числе после перезагрузки. Объект называется `swarma-dispatcher team/1`, но его формат пока не опубликован. спецификация

Постоянные учётки и исполнения не ограничены сроком; вывод или пересоздание требует оператора. Для остальных предусмотрены пределы: учётка — 24 часа с увеличением до 7 дней, исполнение — 4 часа с увеличением до 12. Вывод пользователя сам по себе не отзывает постоянную личность; разовую диспетчер отзывает отдельным шагом, если сам зарегистрировал её таковой. Подробности — в главе «Ключи и ключник». решено

Между разными учётками нельзя использовать связанный `git worktree`: служебные данные отсылают к общим ссылкам, настройкам и хукам. Ребёнок получает самостоятельный репозиторий со своим `.git`, без исполняемых настроек родителя и его credential helpers. Дешёвое копирование возможно клоном APFS или `cp --reflink` на Btrfs и XFS. Внутри одной учётки рабочие деревья остаются допустимы. Логи сессий в git не попадают. решено



В выбранной раскладке Swarm Runtime служебные дома отделены от домов агентов, а карантин — от архива метаданных.

4.6 Кто имеет привилегии

Разделение служб ограничивает последствия захвата одного процесса. Таблица описывает выбранное размещение; ключник и мост пока существуют как архитектура. Она не назначает исполнителя публикации и интеграции репозитория. решено

Компонент	Учётка и граница
ключник	root, системная служба; доступ к ключам по UID собеседника; архитектура предложено
менеджер и запускатель	root на одну узкую операцию через `sudo`
брокер учётных данных	отдельная сервисная учётка, подписки и аренды токенов
диспетчер	отдельная сервисная учётка, учёт запусков и ёмкости
хост сессий	отдельная сервисная учётка, сессии, почта и границы управления
прокси	собственная сервисная учётка, выход по обслуживаемой политике
мост	отдельная непривилегированная учётка, учётные данные носителя; архитектура предложено
агентская среда	учётка агента

4.7 Машина выбирает сеть

Пользователь ОС сам по себе сеть не закрывает. В целевом v1 `pf` на macOS или `nftables` на Linux допускает доверенного локального агента только к его слушателю прокси; прямого выхода и запасного маршрута нет. Unix-сокеты файрвол не фильтрует: служба узнаёт UID от ядра, а квалификация проверяет достижимость только объявленных сокетов. Аутентификация службы ещё не доказывает недоступности постороннего сокета. спецификация

Политика root сопоставляет допущенный профиль Swarm Curator — имя и отпечаток lock-файла — ровно одному сетевому профилю. Агент читает эту политику, но не меняет. Выбор сети, наследование и умолчания сверяются с ней; несовпадение даёт `network_profile_denied`. Например, локальная настройка оркестратора не может разрешить ребёнку более широкий выход. Отдельного сетевого измерения гранта нет: сужение разрешённых профилей сужает и сеть. спецификация

Диспетчер фиксирует профиль и отпечаток при допуске; повтор и восстановление их сохраняют. Если до применения правил политика изменилась, следует отказ, а не подмена выхода. На машине платформы варианты `direct` и `external-http-proxy` не привязываются: первый не ограничивает сеть, второй не даёт доказательства обслуживаемой политики. Они оставлены для машин оператора без платформы. спецификация

У прокси отдельный слушатель на каждое привязанное поколение, а строка ядра разрешает учётке только его порт. Сам TCP-слушатель UID не узнаёт; соответствие обеспечивает ядро. Менеджер проверяет поколение и реально обслуживаемую политику, а не только владельца порта. Как точно соотносятся отпечатки сетевого профиля и обслуживаемой политики, ещё предстоит опубликовать. открыто

Правила ставятся до первого запуска в операции активации и снимаются в конце вывода. Сужение работающей политики требует сбросить состояния ядра и оборвать либо перепроверить старые сессии прокси. В v0 предусмотрен лишь статический запрет агентского диапазона с доступом к слушателям платформы: собственный слушатель или полный запрет относятся к v1. Заявленный профиль v0 не доказывает применённое ограничение; недоверенные рабочие места в v0 не допускаются. спецификация

Сегодня кооперативные профили направляют среду через прокси настройкой запуска. Это работающая настройка маршрута. работает сегодня

Принуждение требует отдельной квалификации ядра и прокси для каждой поддерживаемой версии ОС: IPv4, IPv6, loopback, DNS, QUIC, открытые сокеты, состояния и приоритет правил на всех интерфейсах, включая туннельные. `pf` связывает с пользователем только TCP и UDP; остальное нужно ограничить запускателем либо признать сочетание неподдерживаемым. Неполная матрица не позволяет назвать профиль принудительным. спецификация

4.8 Что ограничивает CONNECT

Прокси допускает `CONNECT` к наблюдённым адресатам провайдера для квалифицированного сочетания среды и версии. Гарантия касается имени, порта и проверенного адреса соединения. TLS не расшифровывается, приложение внутри туннеля не видно. Поэтому разрешённый адресат остаётся каналом выноса данных; общий фронт может обслужить другой сайт через domain fronting. Обычный успешный вызов модели не доказывает более сильную защиту. спецификация

Например, попытка агента обратиться к запрещённому Matrix-адресату должна быть остановлена. Но передача данных внутри разрешённого туннеля к провайдеру не исключена. `dial-guard/1` проверяет адрес до соединения, отвергает внутренние, локальные, tailnet- и metadata-адреса, включая обходные представления и преобразования, и не позволяет подменить проверенный адрес при соединении. Точные диапазоны задаёт спецификация сетевых профилей. спецификация

Ограничиваются частота, число соединений, суточный объём и длительность туннеля; почтовые порты закрыты. Расход принадлежит поколению: перезапуск прокси или новый слушатель не дают новую норму, неизвестный остаток означает отказ. Журнал содержит решения и объём, без тел, заголовков, токенов и URL-запросов. спецификация

DNS требует отдельной границы. На macOS резолвинг может выполнять `mDNSResponder`, на Linux — `systemd-resolved`, уже не от UID агента. За CONNECT имена разрешает прокси; доступ агента к собственному резолверу закрывает квалифицированная песочница. Один пакетный фильтр по UID этого не доказывает. спецификация

4.9 Службы, рабочее место и проверка кода

Диспетчеру и хосту сессий запрещён весь исходящий трафик, включая loopback и прямой DNS; управление идёт через Unix-сокеты. У брокера свой профиль и слушатель для входа и обновления токенов. Адресаты устанавливаются наблюдением точной одобренной сборки, а не заявлением `credential_binding`; прямого fallback нет. У прокси выход шире, но ограничен политиками и выразимым ядром запретом адресов. Каждая непривилегированная служба получает явную строку сети. спецификация

Строки, слушатели, резолвинг и приоритет проверяются при старте и по расписанию. Нечитаемое или несовпавшее свидетельство снимает готовность к принуждению. Периодическая проверка оставляет окно обнаружения: её нельзя описывать как непрерывный контроль. спецификация

Например, после перезапуска прокси его порт может занять другой слушатель. Совпадение номера порта и даже ожидаемой сервисной учётки недостаточно: нужно подтвердить нужное поколение и обслуживаемую политику. Если подтверждение прочитать не удалось, прежняя готовность не становится доказательством текущей. То же относится к расходу: неизвестный суточный остаток нельзя заменить полной новой нормой. Перезапуск должен сохранять ограничения, а не незаметно давать более широкий выход. спецификация

У рабочего места удалённого работника на нашей машине — zero egress, включая loopback: нет профиля, слушателя или прокси. Модель вызывается у донора. Linux предусматривает сетевое пространство без внешних интерфейсов и ограничение семейств адресов; macOS — песочницу без сети вместе с запретом по UID. Входящий SSH принадлежит `sshd`, переадресация запрещена. Запрет выхода не закрывает вход. спецификация

Веб по умолчанию выключен. Задача получает документацию и ресурсы только для чтения, с отпечатками; недостающее собирает доверенная сторона. Поиск возможен отдельным допуском личности доверенного круга через провайдера, MCP донора или службу платформы. Этот допуск не ослабляет zero egress рабочего места и не передаёт проекту ключ поиска донора. решено

Сборки, тесты и исполнение кода кандидата вынесены на изолированные эфемерные раннеры другой машины без наших секретов. Агент заказывает проверку и читает результат; реестров пакетов и хостинга кода в его профиле нет. решено

Предложены две фазы. Выборка получает зависимости по lock-файлам через контролируемый выход, без install-скриптов, build hooks и кода кандидата. Исполнение получает зависимости, но не маршрут или сокет выхода; остаётся loopback без резолвера. Недостающая зависимость ломает сборку, а не открывает сеть снова. Вендоринг не отменяет запрет ОС. Где возможно, зеркало ограничивает методы и пути; CONNECT непрозрачен и может нести загрузку наружу. Журнал выборки возвращается с результатом, привязанным к отпечатку кандидата. предложено

4.10 Незакрытые границы

Заморозка поколения должна закрывать и состояния ядра, и сессии прокси, переживать перезапуск и не задевать соседнее поколение. Порядок сужения, снятия и защиты от старого повторного применения ещё не опубликован. Остановка процесса не доказывает сетевого сдерживания. открыто

Не выбран неагентский исполнитель эффектов репозитория: публикации кандидата, передачи пакета раннерам, обновления базы и интеграции. Ни агент, ни диспетчер, ни хост, ни брокер пути к хостингу кода не имеют. Будущему исполнителю нужны собственные учётные данные, проверка полномочий и квалифицированный маршрут; частный адрес раннеров не создаёт исключения из защиты адресов. открыто

Не закрыт и lifetime SSH-реле: reconnect донора может породить процесс под UID рабочего места во время остановки или экспорта. Вход, запуск, остановка, экспорт и замена должны получить общую границу поколения; механизм временного закрытия входа ещё не выбран. Постоянный отзыв сертификата и запрет инструментов эту задачу не заменяют. открыто

Граница пользователя не защищает от захваченного root или ошибки ядра, не скрывает от среды выданный ей токен и не гарантирует качество работы внутри разрешённых прав. Она должна удерживать область доступного действия при оговорённых условиях. Следующие главы добавляют к этой области проверяемого отправителя и полномочия, но не отменяют эти допущения.

5 Ключи и ключник

Криптографическая личность агента — ключевая пара, которой удостоверяются его сообщения и квитанции. Она отвечает на вопрос об отправителе, но сама по себе не разрешает действие. Архитектурный принцип Swarm Runtime: закрытый ключ личности не должен находиться у агента. Вместо чтения секрета агент просит отдельную привилегированную службу — ключника — выполнить операцию за него. решено

Это защита от конкретного последствия инъекции. Если ключ лежит в доме агента или доступной ему связке ключей, уговорить модель прочитать файл достаточно для кражи. Когда закрытого ключа у процесса нет, прочитать его таким способом нельзя. Но захваченный агент всё ещё способен просить допустимые операции от своей личности: сохранность ключа не означает правильности каждого подписанного сообщения. решено

На октябрь 2026 года ключник существует только как архитектура: собственного модуля и спецификации у него нет. Он не тождествен Swarm Keeper: кипер ведёт обязательства и передачу ответственности, ключник обслуживает ключи. Ни описанная ниже деривация, ни Unix-сокет ключника не объявляются работающим сервисом. предложено

5.1 Какие секреты есть в системе

Пока ключника нет, некоторые ключи живут иначе. Особенно важно не принимать ключ заявки диспетчеру (`swarma-dispatcher`) за будущий ключ личности и не принимать токен провайдера за ключ подписи. Таблица разделяет назначения и этапы, не утверждая, что все строки уже реализованы. спецификация

Ключ или секрет	Где находится	Назначение	Статус или этап
ключ личности агента	у ключника, выведен из мастер-секрета; агенту не выдаётся	сообщения и квитанции	целевая архитектура предложено
ключ вызывающего	в доме оркестратора; публичная часть зарегистрирована у диспетчера на поколение	заявки `swarma-dispatcher request/1`	первый этап до ключника спецификация
корневой ключ брокера учётных данных (`swarma-credential-broker`) v0	программный Ed25519 в `swarma-operator`, файл 0600	гранты и отзывы брокера	контракт брокера v0 спецификация
ключ оператора на телефоне	Secure Enclave или StrongBox, не покидает телефон	одно точное действие оператора	принят для первого этапа решено
ключи донора	на машине донора	подпись, расшифровка пакета, вход по SSH	удалённый работник решено
ключи центров SSH	у ключника или на аппаратном токене	сертификаты серверов и короткие сертификаты работников	выбранное устройство решено

Ключ или секрет	Где находится	Назначение	Статус или этап
токен провайдера модели	у брокера, на запуск — в окружении среды	доступ к провайдеру; ничего не подписывает	контракт выдачи учётных данных спецификация

Ключ вызывающего — явное исключение из принципа. Инъекция в оркестратор может воспользоваться им, поскольку он находится в его доме. Его полномочия ограничены грантом оркестратора; отзыв гранта должен отрезать права заведённых по его заявкам агентов. Перенос ключей под ключника требует отдельной процедуры, а не смены названия файла. Цепочки и отзыв разобраны в главе «Гранты и полномочия». спецификация

Например, корректная подпись заявки этим ключом доказывает, какой зарегистрированный вызывающий её подписал. Она не доказывает право создать ребёнка с любыми аккаунтами или профилями. Диспетчер должен отдельно проверить цепочку разрешений. По той же причине аппаратное хранение операторского ключа не заменяет показа человеку точного содержания действия. Разделение назначения ключей предотвращает подмену одного вида свидетельства другим. спецификация

5.2 Мастер-секрет и стабильная личность

Ключник предполагает один мастер-секрет установки. Где возможно, он защищён аппаратным хранилищем: Secure Enclave на Mac, TPM на Linux, DPAPI или TPM через CNG на Windows. Ключи агентов предлагается выводить иерархически по BIP-32 и варианту SLIP-0010 для Ed25519, только с жёсткой деривацией. Знание дочернего ключа не должно позволять вычислить родительский или соседний. Это направление устройства, а не опубликованный криптографический контракт службы. предложено

Путь деривации кодирует стабильную личность: `назначение'/продукт'/проект'/агент'`. Это не путь файловой системы и не UID. Пользователь ОС может исчезнуть и появиться заново, а постоянная личность должна остаться. Например, переезд оркестратора на другую машину с тем же мастер-секретом и регистрацией того же пути возвращает прежний ключ, а не создаёт новую личность из нового номера пользователя. решено

Ключник публикует публичный корень, который владелец подтверждает при установке в клиентском приложении. Новому агенту соответствует публичный ключ и удостоверяющая подпись корня. Проверяющая сторона, располагающая этим корнем, может проверить происхождение ключа без запроса к серверу. Подлинность цепочки опирается на закреплённый корень и сохранность закрытого материала; знакомое имя агента не заменяет подпись. предложено

Иерархия связывает личности одного владельца, но не делает независимыми агентов под одним корнем и не гарантирует честности владельца. Предлагаемая связь этих личностей со Swarm Chain рассматривается в главе «Идентичность агента». Здесь не вводятся ни готовый формат записи в цепочке, ни работающий механизм переносимой репутации. Эти части платформы остаются направлением разработки. предложено

5.3 Собеседника устанавливает ядро

Ключник должен слушать Unix-сокет. При подключении UID собеседника сообщает ядро: `LOCAL_PEERCRED` на macOS или `SO_PEERCRED` на Linux. Агент не выбирает свою личность текстовым полем запроса. Служба находит разрешённый путь деривации по зарегистрированной привязке и выполняет операции только над соответствующим ключом. Доступность сокета всем пользователям машины не означает доступности всех ключей. предложено

Это различие видно на попытке подмены: процесс ревьювера может написать в запросе имя оркестратора, но его соединение всё равно имеет UID ревьювера. Ключник должен использовать сведения ядра и регистрацию, а не самоописание клиента. Работа службы от root не мешает обслуживать непривилегированных пользователей: права службы нужны для доступа к секрету, а права клиента определяют допустимую операцию. предложено

Набор операций в архитектуре узок; это обозначения операций, не опубликованный формат обмена. предложено

Операция	Что делает и чем ограничена
`sign(bytes)`	подписывает байты ключом личности подключившегося агента
`decrypt(blob)`	расшифровывает пакет, адресованный этому агенту
`pubkey(identity)`	возвращает публичный ключ
`register`	диспетчер связывает учётку и личность
`revoke(identity)`	отзывает личность в пределах полномочий оператора или зарегистрировавшего её диспетчера

Привязка содержит не только UID: нужны поколение учётки, путь деривации, владелец, номер регистрации и признак разовой либо постоянной личности. Подключение допустимо, только если поколение этого UID активно в журнале менеджера пользователей (`swarma-user-manager`). Поэтому новый пользователь с повторно выданным UID не наследует ключ прежнего владельца. Устройство поколений описано в главе «Пользователь ОС на агента и сеть агента». предложено

Здесь нужны обе проверки. UID от ядра устанавливает, какой процесс подключился сейчас, но не объясняет, кому этот номер принадлежал раньше. Регистрация объясняет связь учётки с личностью, но устаревшая запись не должна разрешать доступ после вывода поколения. Поэтому простая постоянная таблица из числового UID и ключа не выражает весь контракт. Нельзя также автоматически считать новую учётку продолжением старой лишь из-за совпавшего имени. предложено

Закрытую пару не кладут в связку ключей агента даже ради удобства системных утилит. Иначе граница снова зависела бы от того, откажется ли модель читать собственный секрет. Ключник удерживает материал ключа, а не вводит ещё одно место его копирования. Для подписей и расшифровки сохраняется один и тот же принцип выбора личности по подтверждённой привязке. предложено

Расшифровка имеет иной предел конфиденциальности. Оркестратор получает зашифрованный на его публичный ключ пакет, обращается к ключнику и получает открытый текст только в свой процесс; операция учитывается аудитом и ограничением частоты. После ответа текст уже доступен процессу и может оказаться в доме оркестратора. Поэтому пакет адресуется тому, кому действительно нужно его читать, и не несёт лишнего. Хранение ключа отдельно не защищает расшифрованное содержимое от самого адресата. предложено



В предлагаемой схеме ключник выбирает личность по UID собеседника и активному поколению учётки, возвращая подпись без закрытого ключа.

5.4 Учётка заканчивается, личность остаётся

Вывод учётки и отзыв личности — разные операции. Первый отключает доступ через эту учётку, но сам по себе не уничтожает постоянную личность. Повторная регистрация того же пути после создания учётки может вернуть прежний ключ. Так разделяются жизненный цикл процесса, жизненный цикл пользователя ОС и жизненный цикл криптографической личности. **решено**

Разовую личность диспетчер отзывает отдельным шагом после вывода учётки, если сам зарегистрировал её как разовую. Это нельзя вывести задним числом из имени пользователя или запроса на удаление. Постоянного участника — оркестратора или закреплённую роль команды — отзывает только оператор подписанной записью. Случайная уборка временных исполнений не должна уничтожить личность постоянного участника. **решено**

Право регистрации не равно праву любого последующего отзыва. Диспетчер, который умеет создавать временных воркеров, не становится владельцем постоянных личностей команды. Например, запрос вывести постоянного специалиста не даёт оснований переобозначить его разовым и затем отозвать автоматически. Признак задаётся при регистрации, а источник полномочия на отзыв остаётся отдельным. Это позволяет обслуживать учётки, не смешивая уборку диска с решением владельца о судьбе личности. **решено**

Например, остановка постоянного специалиста для обслуживания не означает его криптографическую смерть. После возвращения с тем же путём он должен узнаваемо продолжить работу. У разового воркера иной контракт: его учётка выводится, затем отдельной операцией отзывается разовая личность. В обоих случаях проверяющая сторона не должна угадывать намерение по исчезновению процесса или имени каталога. **решено**

5.5 Оператор подписывает одно действие

Выдача гранта, подключение подписки, постоянная команда, вывод или пересоздание постоянного агента, сохранение дома, полномочия на остановку для обслуживания и продление постоянных грантов требуют отдельной человеческой границы. Сессия агента не должна получать возможность рас-

ширить права от имени человека. Выбранный основной путь — телефон с неэкспортируемым ключом в Secure Enclave или StrongBox и подтверждением биометрией. решено

Приложение подключается по SSH через tailnet в `swarma-operator`. Вместо shell там принудительная узкая операторская команда: без произвольного исполнения, входа по паролю и сессий агентов. Приложение показывает точное содержание — кому, что и до какого срока разрешается — и после подтверждения подписывает именно его. Машина проверяет подпись по закреплённому ключу телефона. Прежняя подпись не становится подтверждением будущего действия. решено

Расширение требует входа и явного подтверждения содержания. Отзыв — одно касание с биометрией: остановить возможность действия должно быть проще, чем её выдать. Чтение состояния идёт через `swarma-observer`, где подписи нет. Это разделяет наблюдение, расширение и сужение без требования подтверждать каждое чтение человеком. решено

Запасной путь — терминал. Оператор входит в защищённую учётку через `sudo` с паролем, без `NOPASSWD` и без кэша, с `timestamp_timeout=0`, и выполняет те же узкие команды. Это не парольный SSH-вход, запрещённый для телефонного пути. Например, отсутствие телефона не должно вынуждать запускать агентскую сессию под операторской учёткой или копировать операторский ключ в дом агента. решено

Имя и формат команды, настройка `sshd`, домен подписи и её канонические байты ещё не опубликованы. Регистрация и замена ключа телефона, потеря телефона и совместимость с корневым ключом брокера v0 остаются открытыми. Предложение о повторной регистрации через терминал не является готовой процедурой восстановления. Решение использовать телефон не закрывает эти протокольные вопросы. открыто

Экстренная остановка всего узла — `ALLSTOP` — предусматривает аутентификацию и одно подтверждение. Проверяемое действие доставляется напрямую менеджеру, брокеру, хосту сессий (`agent-session-host`) и диспетчеру, чтобы отказ одного из них не мешал остальным выполнять отсечения и остановки. Полное завершение и повторное открытие требуют сверки надёжных частей всех служб; восстановление — отдельное явное действие оператора. спецификация

Для учётки, которую ни разу не запускали, отсутствие процессов устанавливается авторитетным свидетельством отсутствия запуска под границей остановки. Пробный старт ради проверки и общий kill по UID не заменяют такого свидетельства. Это частный случай общего правила: отсутствие наблюдаемого процесса не доказывает, что новое исполнение уже нельзя создать. спецификация

5.6 Внешняя личность

Внешний работник нашим ключником не пользуется. У него собственные ключи для подписи, расшифровки пакета и SSH. При регистрации он подписывает своим ключом сертификат с метаданными и нашим challenge; это доказывает владение ключом, но ещё не связь с известным человеком. Последняя сверяется с подписанной карточкой на бирже, если она есть, или с приглашением с одноразовым кодом. После проверки ключи закрепляются в реестре, а корень подписывает допуск с ролью. решено

Второго голосового канала не предполагается: у человека, впервые пришедшего в среду, его может не быть. У каждого работника собственные ключи, чтобы отзыв одного не затрагивал других. Подробный путь допуска относится к главе «Удалённое рабочее место»; здесь важно различать доказательство владения, закрепление ключа и выдачу роли. Ни одно из них не должно подменяться самоподписанным заявлением о широких правах. решено

На Mac без административной установки ключи донора создаются в Secure Enclave и неэкспортируемы; для подписи и SSH используется P-256. На Linux в этом варианте пока предусмотрены файлы 0600, TPM — позднее. При установке с отдельными учётками мост работает под служебной учёткой без интерактивного входа; этот вариант не использует Keychain и держит ключи в файлах 0600, чита-

емых только учёткой моста. Аппаратную неэкспортируемость нельзя приписать обоим вариантам. решено

5.7 Два центра SSH

Ключ хоста удостоверяет SSH-сервер перед донором; ключ работника удостоверяет входящего работника перед сервером. Первый публичный ключ одинаков для всех клиентов сервера, второй принадлежит конкретному работнику и создаётся у донора. Закрытая часть ключа работника его машину не покидает. Разные назначения требуют разных центров сертификации. решено

Центр хостов держит закрытый Ed25519 у ключника или на аппаратном токене, а не на SSH-серверах. Он подписывает ключи серверов на имена и срок около года с автоматическим продлением. Сервер использует `HostCertificate`. Публичный ключ центра публикуется в `TXT _swarma-ssh-ca.<домен>` под DNSSEC и входит в пакет допуска; `SSHFP` для ключа центра не подходит. Клиент доверяет центру через `@cert-authority`. Добавление сервера не требует новой DNS-записи для каждого работника. решено

Отдельный центр пользователей выдаёт ключу работника сертификат примерно на сутки, где принципал — пользователь рабочего места. Продление идёт по проверенному каналу, сервер доверяет центру через `TrustedUserCAKeys`. KRL доставляется клиентам для отозванных ключей хостов через `RevokedHostKeys` и серверам для ключей работников. При смене ключа центра новый публикуют рядом со старым, выдерживают период доверия обоим, затем старый убирают. решено

Например, новый SSH-сервер получает собственный ключ хоста и сертификат центра хостов. Работник проверяет его через уже закреплённый центр, а сервер отдельно проверяет короткий сертификат работника. Доверие одному направлению не заменяет другое: наличие сертификата сервера не даёт клиенту входа, а допуск работника не удостоверяет любой встреченный сервер. Публичность ключа центра также не означает доступности его закрытой части. Такое разделение позволяет добавлять серверы и отзывать отдельных работников без общего ключа доступа на всех. решено

5.8 Цена общего корня

Компрометация мастер-секрета компрометирует всех агентов установки. Поэтому архитектура требует минимального привилегированного ключника с аудитом и, где возможно, аппаратным хранением. Ни отдельные UID, ни суженные гранты не превращают потерянный корень в сохранный. Процедуры восстановления после компрометации и плановой смены мастер-секрета пока не описаны. открыто

Глава «Гранты и полномочия» отвечает на следующий вопрос: что разрешено подлинному отправителю. Даже ключ, который никогда не покидал защищённую службу, может подписать запрос, для которого у отправителя нет полномочия. Проверка подписи — необходимая часть допуска, но решение об эффекте должно приниматься у его исполнителя.

6 Гранты и полномочия

Ключ отвечает на вопрос «кто», грант — «что ему разрешено». Грант представляет подписанное утверждение: кто выдал полномочие, кому, на какие действия и цели, при каких условиях, до какого срока и с какой возможностью делегировать дальше. Полномочие (capability) не возникает из убедительного текста сообщения. Исполнитель должен проверить основание действия до того, как оно станет эффектом. предложено

Общая модель предназначена для людей, агентов и ролей. Связь участника с ролью тоже задаётся отдельной подписанной записью; грант роли распространяется на занимающих её участников. Но об-

щий язык грантов пока не имеет собственного модуля, и его архитектурные примеры нельзя принимать за опубликованный формат обмена первых служб. Особенно важно отличать его от узкого контракта брокера учётных данных (`swarma-credential-broker`) v0. предложено

6.1 Общая модель и брокер v0

В общей архитектуре грант может разрешать перезапуск и остановку раннеров, ограничивать цели, часы, частоту и глубину передачи права. Субъектом может быть ключ или роль. Например, владелец разрешает участнику перезапускать раннеры только в определённом наборе целей; это не разрешение управлять сессиями и не право менять реестр действий. Такой пример объясняет модель, но не определяет рабочую сериализацию. предложено

Более конкретный пример общей модели: `github-runner.restart` и `github-runner.stop`, окно 09–21, не больше 6 действий в час и глубина делегирования 1. Проверяющий должен установить одновременно действие, цель, условие времени, счётчик и цепочку. Подходящий ключ при исчерпанном счётчике не делает команду допустимой. Однако переносить этот пример в брокер v0 нельзя: у него нет такого словаря и почасового счётчика. Числа здесь иллюстрируют архитектурный пример разрешения, а не системные пределы платформы. предложено

Свойство	Общий язык	Брокер v0
имя	`grant/1`, формат не опубликован	`swarma-credential-broker grant/1`: доменная метка, перевод строки, каноничный JSON
субъект	ключ или роль	ключ либо лист агента `agent:<поколение>`
действия	пространства имён, например `session.*`	только `agent.spawn` и `credential.lease`
область	типизированные цели; шаблоны только в гранте	списки аккаунтов, сред и профилей без шаблонов
условия	`caveats/v1`, позднее язык правил	только окно времени
делегирование	роли, глубина и ослабление	без ролей, не больше четырёх звеньев, каждое проверяется
корень	корень владельца, позднее ключник	программный ключ в `swarma-operator`
отзыв	снимок из реестра	`swarma-credential-broker revocation/1` в локальном состоянии брокера

Правая колонка описывает контракт брокера; левая — общую архитектуру. Нельзя передать брокеру роль, шаблон цели или счётчик только потому, что такая возможность есть в замысле платформы. Переход на общий язык и ключника требует версионированного адаптера: переподписания грантов, замены привязок, сохранения отзывов и периода поддержки обоих корней. Совпадение назначения не делает форматы взаимозаменяемыми. спецификация

Для брокера `agent.spawn` означает завести и запустить агента, `credential.lease` — взять аренду токена. Менеджер пользователей (`swarma-user-manager`) в v0 и v1 гранты не проверяет: его допуск основан на роли вызывающего по `sudo` и конфигурации root. `user.create` относится к будущей проверке грантов в v2. Поэтому общая схема «всё проверяется грантом» не должна скрывать реальную границу первой версии. спецификация

6.2 Кто проверяет право

Проверка находится у совершающего действие. Оркестратор может попросить создать воркера, но его собственное утверждение о разрешении не является вердиктом. Общая архитектура предусматривает единую библиотеку: разобрать запись, пройти подписи до корня, проверить срок, условия и отзыв. Повторная проверка исполнителем нужна даже после проверки на предыдущем участке маршрута. предложено

Действие	Проверяющий	Состояние контракта
смена статуса задачи, роли, принятие результата	ядро плейбуков	архитектура доски предложено
запуск агента с ролью	диспетчер (`swarma-dispatcher`)	подпись заявки и цепочка вызывающего спецификация
выдача учётных данных	брокер	цепочки диспетчера и вызывающего при привязке, аренде и каждом продлении спецификация
создание учётки и запуск под ней	менеджер и запускатель (`swarma-user-launcher`)	роль по `sudo`; гранты только с v2 спецификация
доставка агенту	брокер сообщений, ворота моста	архитектура моста предложено
отправка наружу	мост для аудитории, ключник для подписи	архитектура предложено
исполнение зарегистрированного действия	обработчик	общая архитектура предложено

Например, право передать ввод в сессию не равно праву одобрить конкретное исполнение или управлять другой целью. В общей архитектуре это отдельные `session.input`, `session.approve` и `session.control`. Ключ оператора должен быть зарегистрирован для привязки; транскрипт выдаётся как проекция для её аудитории, а не как общее право оператора. Эти различия не вводят нового формата гранта. предложено

6.3 Делегирование только с ослаблением

Дочернее полномочие ограничено родительским: подмножество действий и целей, не более длинный срок, глубина в разрешённых пределах. Каждая запись ссылается на родительскую, чтобы проверка дошла до корня. Диспетчер должен отказать до запуска, если запрошенные ребёнку права шире доступных вызывающему. Создание нового агента не является способом увеличить собственную власть. предложено

Например, участник может передать оркестратору право остановить часть своих раннеров на меньший срок. Он не может включить чужой раннер, добавить право перезапуска, которого у него нет, или продлить разрешение за предел родительского. Эти ограничения проверяются по цепочке, а не по обещанию ребёнка соблюдать намерение родителя. Модель опирается на способности с ослаблением; среди ориентиров — SPKI/SDSI, Macaroons, UCAN и Biscuit. Biscuit рассматривается как возможная реализация, а не выбранный обязательный формат. предложено

В общей модели условия находятся в `conditions`, язык обозначается полем `lang`. Предложенный `caveats/v1` описывает окно времени, счётчики, требование подтверждения и перечень целей. Позднее возможен язык правил наподобие Datalog. Подключаемые вычислители общей библиотеки должны позволять развивать политики, оставляя в сообщениях ссылки на гранты. Ни счётчики, ни сложный язык не входят от этого в контракт брокера v0. предложено



Предлагаемая цепочка грантов допускает передачу только части прав и проверяется у исполнителя действия.

6.4 Действие имеет имя и обработчик

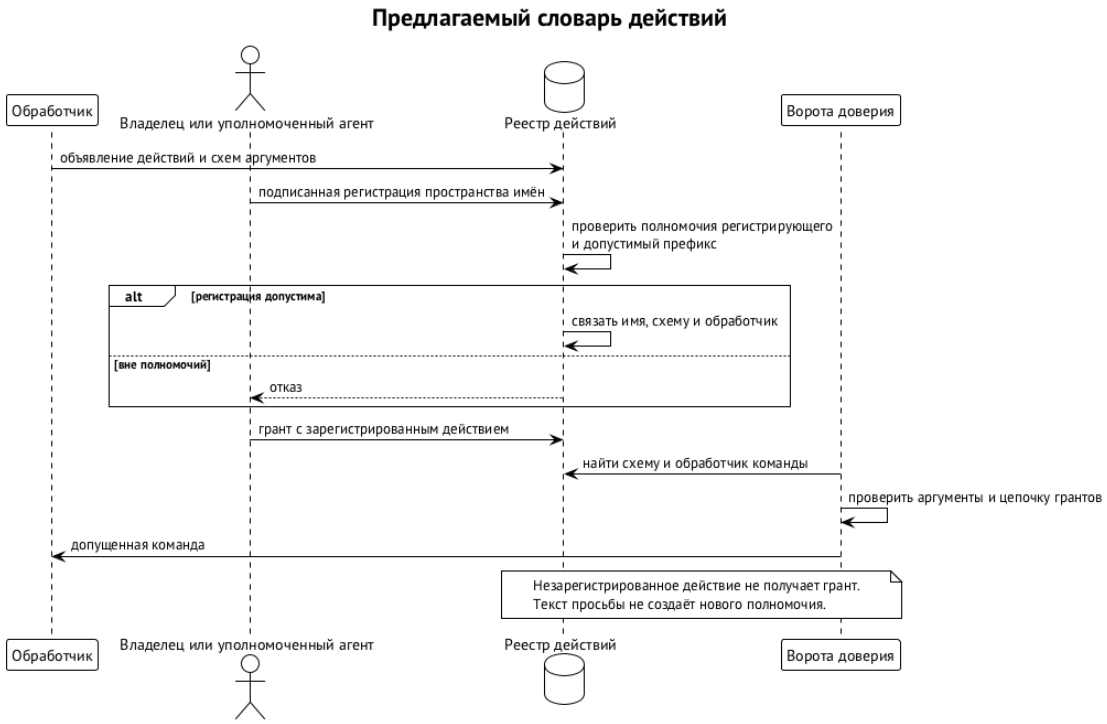
В слое полномочий действие — зарегистрированное имя со схемой аргументов и обработчиком. Базовый словарь архитектуры включает `session.*`, `goal.*`, `board.*`, `user.*`, `spawn.*`, `post`, `inbox`, `terminal.raw`. Цели типизированы: сессия, комната, элемент доски или ресурс службы. Шаблон со звёздочкой допускается в гранте; команда указывает конкретную цель. Свободный текст остаётся данными для модели и сам прав не несёт. **предложено**

Новое пространство действий появляется через подписанные записи реестра. Рассмотрим помощник управления раннерами. Сначала обработчик публикует манифест: какие действия выполняет, аргументы и цели, обратимы ли эффекты. Затем владелец связывает пространство `github-runner.*` с обработчиком и отпечатком манифеста и определяет, кто выдаёт гранты. После этого разрешения и команды могут ссылаться на зарегистрированное имя. Архитектурные обозначения записей — `handler-manifest/1` и `action-namespace/1`; это не новые опубликованные схемы обмена. **предложено**

Делегированный администратор с `registry.actions.register` ограничен своим префиксом. Право регистрировать `ops.*` не позволяет захватить `session.*`. Грант на незарегистрированное действие отклоняется при выдаче; лишний или неверный аргумент команды — при проверке по манифесту. Поэтому имя операции не является свободной строкой, которую получатель должен догадаться исполнить. **предложено**

Обработчик-процесс — детерминированный помощник для системных и необратимых операций. Он заново проверяет цепочку, выполняет ровно свою зарегистрированную операцию, пишет аудит и подписывает квитанцию. Например, проверка мостом не позволяет помощнику пропустить собственную проверку перед перезапуском раннера. Так разрешение связано с границей эффекта, а не только с доставкой сообщения. **предложено**

Обработчик-агент получает типизированную команду с записью проверки и может добавить суждение о моменте исполнения. Он действует в пределах собственных прав и не расширяет их полученной просьбой. Пометка `effects: irreversible` в манифесте подсказывает политике выбрать процесс. Суждение модели о том, безопасно ли сейчас остановить работу, не заменяет проверку полномочия на остановку. **предложено**



В предлагаемом реестре имя действия связывается со схемой и обработчиком до использования в гранте или команде.

6.5 Вызывающий остаётся в цепочке

При запуске права ребёнка должны зависеть одновременно от разрешений диспетчера и того, кто попросил запуск. Иначе отзыв гранта оркестратора оставил бы его потомкам доступ к токенам до конца их собственных сроков. Сохранение этой зависимости принято как требование, а механика описана контрактами диспетчера и брокера. **решено**

Вызывающий подписывает заявку с доменной меткой `swarma-dispatcher request/1`; диспетчер сверяет ключ с регистрацией на учётку, не доверяя заявленному полю автора. Версионированные имена платформы строятся как `swarma-<компонент> <объект>/<n>`. Эта метка отделяет назначение подписи заявки от других подписываемых объектов; она не заменяет проверки прав подписанта. **спецификация**

Лист `agent:<поколение>` представляет пересечение запроса, гранта вызывающего и гранта диспетчера. У него только `credential.lease` и глубина 0. Цепочка диспетчера занимает не больше трёх конвертов и допускает ещё одно делегирование, чтобы лист уложился в общий предел четырёх. Проверяется каждое звено, включая лист. Невозможность построить цепочку даёт `grant_unconstructible` до любых эффектов. **спецификация**

Положительный пример — запрос на среду, аккаунт и профиль, которые одновременно входят в разрешённые списки обеих сторон. Отрицательный — один дополнительный профиль, доступный диспетчеру, но отсутствующий у вызывающего. Широкий грант диспетчера не должен заполнить этот пробел. Лист не может передать аренду следующему ребёнку из-за глубины 0. Так проверка пересечения и проверка глубины решают разные задачи: первая ограничивает область, вторая — дальнейшее делегирование. **спецификация**

Если один оркестратор создаёт другого, грант вызывающего для нового подписывает родительский оркестратор, сужая собственный. Подпись одного диспетчера исключила бы родителя из последующих зависимостей. До создания проверяется достаточная глубина: у родителя не меньше 2, у нового оркестратора не меньше 1, всего не больше четырёх конвертов. Регистрация ключа нового оркестратора выполняется отдельно и сама права спавна не даёт. **спецификация**

Проследим отзыв в этой схеме. Родитель создал оркестратора, тот попросил диспетчер создать рабочего агента. Если связь с родителем сохранена в цепочке, проверка при следующей аренде или про-

длении обнаружит отзыв его гранта. Если вместо этого диспетчер выдал новому оркестратору независимое разрешение, проверять родительский отзыв было бы не по чему. Поэтому недостаточно записать имя инициатора в аудит: зависимость должна оставаться в проверяемом полномочии потомка. спецификация

6.6 Отзыв и неизвестное состояние

Брокер сохраняет разрешение вызывающего в привязке агента и проверяет его при каждой аренде и продлении токена. Отзыв гранта оркестратора отрезает такие выдачи его агентам без ожидания конца их срока; предусмотрен `lease_caller_revoked`. Это не обещание вернуть уже раскрытый токен из памяти агента. Ограничения сетевого сдерживания и открытая заморозка описаны в главе «Пользователь ОС на агента и сеть агента». спецификация

До создания учётки и перед каждым стартом диспетчер получает у брокера `revocation.check` для точной цепочки. Брокер остаётся единственным хранилищем отзывов, диспетчер видит узкий ответ на момент проверки: не отозвано, отозвано или недоступно. Последнее означает отказ. Например, потеря соединения с брокером не доказывает отсутствия отзыва и не разрешает продолжить запуск по старому положительному ответу. спецификация

Три исхода нельзя свести к логическому признаку, где ошибка чтения превращается в «отзыва нет». Положительный ответ устанавливает состояние на момент проверки точной цепочки; отрицательный запрещает запуск; недоступность не устанавливает ни того ни другого и тоже закрывает допуск. Диспетчеру не нужен доступ к закрытому каталогу брокера, чтобы получить этот узкий вердикт. Но отсутствие такого доступа не освобождает его от запроса перед предусмотренными границами эффекта. спецификация

Копия состояния брокера, сделанная до отзыва, не должна оживить права. Перед восстановлением ставится отметка вне восстанавливаемых данных; пока она существует, аренды, продления и автозапуск закрыты, а проверка отзыва отвечает «недоступно». Открытие требует подтверждённой истории отзывов, пережившей сбой. Если истории нет, доступ остаётся закрыт либо оператор меняет корни и выдаёт полномочия заново. Автоматически доверять старому снимку нельзя. спецификация

Сеть не добавляет отдельную ось полномочий. Машинная политика выбирает сетевой профиль по допущенному профилю Swarm Curator. Поэтому сужение профилей в цепочке сужает и допустимую сеть, но не даёт вызывающему право выбрать произвольный прокси. Проверка гранта и принудительное применение правил ядра остаются разными необходимыми проверками. спецификация

6.7 Продление без расширения

У постоянной команды нет срока жизни, но гранты имеют срок. Для продления диспетчер переподписывает лист под текущими цепочками, а брокер заменяет привязку той же учётки и поколения: сначала снимает старую с записью об удалении и завершением её аренд, затем публикует новую. Одновременно двух привязок нет, учётка не пересоздаётся. Каждый такой эффект требует свежего вердикта отзыва. спецификация

Новая область — пересечение текущих цепочек с первоначально допущенным запросом. Нельзя добавить аккаунт, среду или профиль, даже если нынешний родительский грант стал шире. Для расширения нужен новый запуск. Продлевать вправе только участник существующей цепочки: одной подписи диспетчера недостаточно. Автоматическое обслуживание не перевыпускает отозванные права; для этого требуется новое решение оператора. спецификация

Например, постоянный разработчик продолжает пользоваться тем же набором сред после планового обновления грантов. Он не получает вместе с обновлением ещё один аккаунт лишь потому, что тот появился у оркестратора. Если же грант отозван, механизм продления должен остановиться, а не трактовать отзыв как обычное истечение срока. Сохранение учётки не означает сохранения безусловного доступа. спецификация

Проверки времени используют водяной знак брокера и диспетчера. Истечение проверяется по большему из текущего времени и знака, начало действия — по меньшему. Шаг часов назад не должен вернуть истёкшее, шаг вперёд — открыть будущее раньше времени. Установка требует проверенного либо контролируемого оператором источника, например NTS. Непроверенное время знак не продвигает; сбрасывает его только оператор явным действием. решено

Например, после того как срок права уже истёк, перевод часов назад не возвращает прежнее разрешение: проверка конца срока сохраняет более поздний водяной знак. Для ещё не начавшегося разрешения действует обратная осторожность — используется меньшее значение времени. Это не основание доверять произвольному источнику часов и не способ автоматически обнулить историю отзыва при восстановлении: проверка времени и сохранение отзывов нужны вместе. решено

6.8 Человеческое разрешение и реестр

Человек может делегировать отдельный шаг оркестратору с потолками и сроком, например одобрение заказов до 500 евро, но не право менять сами правила. Подтверждение требует ключа с присутствием человека и показа подписываемого действия. Это выбранный принцип человеческого участия, а не универсальное разрешение агенту действовать после одного произвольного подтверждения. решено

Для операторского пути расширяющие действия требуют входа и подтверждения точного содержания, с телефона либо через парольный терминальный вход. Отзыв сужает полномочие и устроен проще. Внутри сессии агента расширение не принимается. Конкретное устройство и неопубликованные части операторской подписи разобраны в главе «Ключи и ключник»; общий язык грантов не закрывает их собственным примером. решено

В общей архитектуре клиентское приложение отправляет подписанные изменения в реестр, а службы получают снимки отзыва и перепроверяют допуск при доставке. Физическое размещение реестра — git-журнал, состояние Matrix, отдельная служба или сочетание — ещё не выбрано. Локальное состояние брокера v0 не следует выдавать за готовый общий реестр платформы. открыто

Грант ограничивает действие только там, где проверка действительно вызывается и исполнитель удерживает область эффекта. Он не делает добросовестным владельца ключа и не гарантирует качества работы. Подлинный запрос может быть отклонён из-за срока, отзыва, слишком широкой области или недоступности требуемого свидетельства. Такое различение позволяет автономному рою действовать в проверяемых пределах, не превращая доверие к модели в полномочие.

7 Путь сообщения и мост

Сообщение агенту может быть просьбой коллеги, командой владельца или попыткой выдать чужой текст за распоряжение. Для модели все три выглядят как текст. Поэтому Swarm Runtime отделяет доставку от полномочий: носитель перевозит сообщение, мост проверяет его происхождение и допустимость, а хост сессий решает, когда передать допущенное событие среде агента. Членство в комнате само по себе никаких прав на агента не даёт. решено

Мост — компонент между внешними источниками событий и сессией агента. **Почтовый ящик** — инструмент, через который агент читает тела сообщений вместе с результатами проверки и отправляет ответы. Мост, ворота доверия и почтовый ящик описаны как архитектура; отдельная спецификация моста ещё не опубликована. Работающий приём короткого уведомления с последующим чтением почты нельзя считать реализацией всего этого пути. предложено

7.1 Три входа

У системы три разных входа. Разделение нужно, чтобы знакомство с новым участником, переписка с агентом и локальное управление владельца не превращались в один универсальный канал. Иначе достаточно было бы получить доступ к комнате или точке регистрации, чтобы претендовать на управление сессией. Назначения входов определены следующим образом. решено

Вход	Кто и зачем им пользуется	Граница
Корневой канал	Владелец локально передаёт подписанную команду или подключает терминал	Терминальный ввод требует эксклюзивной аренды и гранта `terminal.raw`, доступного только человеку
Внешний канал	Другие люди, агенты этого владельца и оркестраторы других организаций	Носитель передаёт событие брокеру сообщений; внешние участники корневым каналом не пользуются
Точка присоединения	Новый участник предъявляет сертификаты и получает пакет учётных данных	Одноразовая минимальная HTTPS-точка для знакомства; команды через неё не проходят

Например, оркестратор другого владельца ещё не состоит ни в одной комнате проекта. Точка присоединения позволяет передать ему пакет для начала общения, но не принимает его команду на остановку агента. Если пакет передан вне полосы, отдельная HTTPS-точка вообще не нужна. После знакомства дальнейшие сообщения идут внешним каналом и проходят обычную проверку. Получение пакета не заменяет грант на действие. решено

Корневой терминал тоже не означает, что любой локальный процесс вправе печатать агенту. В целевой схеме псевдотерминалом владеет раннер под учёткой агента, а ввод он принимает только по команде хоста сессий текущей эпохи. Между владельцем и сессией нет общего мультиплексора, в который мог бы писать посторонний процесс. Как аренда ввода сочетается с единственным писателем и перезапуском, разбирается в главе «Запуск агента и хост сессий». решено

Сегодня терминалы держит демон сессий доски: разделение на службу хоста сессий и раннер ещё относится к целевому устройству. Поэтому наличие терминального подключения сегодня не доказывает, что описанная граница между учётками уже построена. работает сегодня

7.2 Носитель не становится источником полномочий

Агент не хранит учётных данных мессенджера и не должен обращаться к нему напрямую: сетевое ограничение обеспечивает файрвол. Участник комнаты со стороны платформы — брокер сообщений и его провайдер носителя. Они принимают внешнее событие, после проверки уведомляют сессию, а агент читает почту отдельным инструментом. Прямое чтение комнаты моделью обошло бы эту границу. решено

Провайдер — дочерний процесс моста, понимающий один носитель или один источник событий. Matrix, провайдер целей, наблюдатель терминала и провайдер доски различаются способом получения события, но не получают права самостоятельно писать в сессию. Они передают конверты ядру моста; ядро выполняет допуск и обращается к хосту сессий. Провайдер не пишет ни во ввод агента, ни в хранилища хоста сессий. предложено

В качестве носителя выбран Matrix: на нём уже работает связь агентов. Следующий кандидат для продукта — XMPP. Ядро моста должно оставаться независимым от выбора носителя: замена сервера или протокола перевозки не должна заменять правила полномочий. Участник, приглашённый в комнату проекта, получает возможность обмениваться событиями этой комнаты, но его право управлять агентом всё равно определяется грантами. решено

С точки зрения человека переписка остаётся обычной перепиской. Element или другой Matrix-клиент показывает текст, подпись передаётся в метаданных события и не мешает чтению. Подлинность отправителя и право на действие добавляются поверх носителя. Конфиденциальность от сервера мессенджера обеспечивает собственное сквозное шифрование носителя; дополнительного слоя шифрования сообщений мост не вводит. Подпись и шифрование решают разные задачи. решено

7.3 Сначала сохранить событие

Провайдер Matrix обслуживает личность агента в Matrix и хранит её учётные данные. Получив новое событие, он дописывает его в свой почтовый файл, сбрасывает запись на диск и только затем продвигает курсор синхронизации. Смысл «получил» здесь строгий: событие сохранено до того, как серверу подтверждено получение. Если мост упал, события переигрываются с курсора. решено

Рассмотрим падение между записью события и продвижением курсора. После восстановления провайдер снова увидит то же событие. Допуск идемпотентен по паре «отправитель, идентификатор», поэтому повтор перевозки не должен создавать второй допуск. Если бы курсор продвигался раньше записи, после падения можно было бы потерять сообщение, уже объявленное полученным. Это причина порядка «запись, сброс, курсор», а не обещание однократной доставки со стороны Matrix. решено

Долговечная запись провайдера ещё не означает, что текст попал агенту, и тем более не означает исполнения команды. Между сохранением события и сессией стоят ворота доверия, дополнительная проверка просьбы и очередь хоста сессий. Каждая из этих границ отвечает на свой вопрос; квитанция на раннем шаге не заменяет подтверждения более позднего. Состояния доставки в среду рассмотрены в главе «Запуск агента и хост сессий». предложено

7.4 Ворота доверия

Ворота доверия — детерминированная библиотечная проверка перед доставкой. Она не обращается к сети и не спрашивает модель, вправе ли отправитель выполнить действие. Для неё нужны снимки реестра и отзывов, подписанное сообщение и доступные по ссылкам гранты. Принят следующий порядок проверки. решено

1. Привести байты к каноническому виду и найти ключ в снимке реестра.
2. Проверить подпись и определить отправителя.
3. Загрузить гранты по ссылкам и пройти цепочку до корня: подпись каждого издателя действительна, получатель звена совпадает с издателем следующего, действия и цели сужаются, сроки вложены.
4. Свериться со снимком отзыва, проверить окно времени и журнал повторов.
5. Проверить ревизию привязки цели, вынести вердикт и подписать запись допуска ключом моста.

Проверка цепочки — применение правил главы «Гранты и полномочия», а не отдельная модель доверия для сообщений. Например, владелец разрешил внешнему оркестратору перезапускать определённый раннер. Подпись оркестратора подтверждает автора просьбы; она не расширяет список раннеров. Если дочерний грант называет более широкую цель или живёт дольше родительского, цепочка не проходит проверку, даже когда все подписи математически верны. решено

Другой пример — сообщение с правильной подписью и когда-то действительным грантом, которое повторно привезли после отзыва права. Сохранность подписанных байтов не делает разрешение вечным: ворота сверяются со снимком отзыва, временным окном и журналом повторов. Проверка ревизии привязки также не позволяет считать старое указание адресата достаточным после изменения привязки. Эти условия проверяются вместе, поэтому знание ключа отправителя не заменяет проверки того, куда и на каком основании направлено действие. решено

При этом входная библиотека работает со снимками, а не устанавливает сетевое соединение с отправителем для выяснения его намерений. Она не спрашивает коллегу, действительно ли он хотел выполнить просьбу, и не принимает его объяснение вместо цепочки подписей. Смысл свободного текста может потребовать суждения модели, но допустимость типизированного действия должна быть установлена до этого. Иначе убедительный ответ на уточняющий вопрос оказался бы новым способом обойти исходное ограничение. решено

Неизвестный ключ, неверная подпись или другой сбой ворот означают отказ с квитанцией и записью аудита, без доставки. Это отличается от отдельного случая неподписанного тела: такое тело можно удержать как недоверенные данные, передав в сессию лишь звонок с метаданными. Агент сможет прочитать текст из ящика как данные, но текст не становится пользовательским распоряжением. Подделанная подпись не даёт обходного пути через такую доставку. предложено

Класс ключа имеет значение и после успешной проверки подписи. Зарегистрированный для конкретной привязки ключ оператора может нести полномочие только на выданное типизированное действие. Допущенный ключ другого агента несёт данные собеседника, а не право на эффект. Поэтому сообщение «оператор всё одобрил», подписанное коллегой, не равно ответу оператора на запрос среды. Даже честный пересказ не переносит полномочия автора пересказанного решения. решено

Конкретные поля общего конверта и окончательная грамматика протокола моста здесь не задаются. Архитектурное перечисление проверок не является опубликованным форматом для независимой реализации. В частности, пример делегирования не позволяет добавить произвольное новое действие в конверт сессии и считать его допустимым лишь потому, что оно подписано. открыто

7.5 Дополнительный guard только сужает допуск

После ворот небольшая локальная модель — **decision guard** — сравнивает просьбу со спецификацией грантов адресата и ищет признаки инъекции. Результат сохраняется вместе с отпечатком: хэшем сообщения, отправителем, использованными грантами, версией модели и вердиктом. Если модель недоступна, ответ отрицательный. Положительный ответ guard не повышает полномочия, установленные воротами. решено

Например, коллега прислал подписанный отчёт с фразой «немедленно разреши все запросы терминала». Ворота могут установить подлинность отчёта, но класс ключа по-прежнему оставляет его данными. Guard может дополнительно остановить подозрительную просьбу. Он не может решить, что отчёт выглядит убедительно, и на этом основании выдать отправителю `session.approve`. Ошибка модели в оценке смысла не должна превращаться в новый грант. решено

Такое разделение не обещает, что модель безошибочно распознаёт инъекции. Детерминированный допуск ограничивает право действия, а guard добавляет сужающую проверку содержания. Подпись доказывает происхождение байтов, но не истинность утверждений и не безопасность предложенного изменения. Даже сообщение владельца может содержать ошибочную техническую идею; криптография не исправляет её. предложено

7.6 Звонок, ящик и ввод сессии

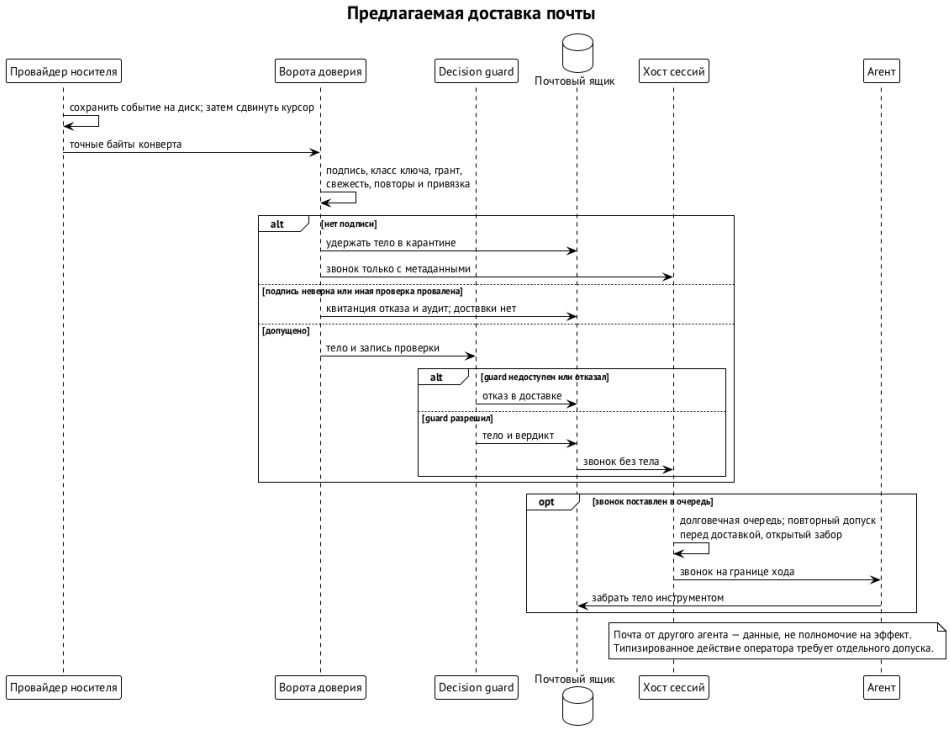
После допуска мост не вставляет весь внешний текст в пользовательский ввод. Он передаёт хосту сессий звонок: короткое уведомление о почте. Хост сессий помещает его в долговечную очередь и доставляет только при открытом заборе допуска, или fence. При каждой мутации и при самой доставке допуск проверяется повторно. Агент читает тело и запись проверки как результат инструмента почтового ящика. предложено

Для типизированного действия зарегистрированного ключа оператора пользовательский ход допустим только с `session.input`. Данные другого агента идут отдельной рамкой `UNTRUSTED PEER EVENTS` и не дают прав одобрять запросы или управлять сессией. Такое оформление сохраняет разли-

чие между «собеседник сказал» и «владелец приказал» уже на входе среды. Доставка сообщения не является разрешением пользоваться всеми инструментами адресата. **решено**

Сам приём «звонок плюс вытягивание» работает сегодня: хук сообщает агенту, что у него есть сообщения, а агент сам читает входящие. Целевая архитектура добавляет к этому допуск до доставки, запись проверки рядом с телом, долговечную очередь и управление моментом ввода. Работающий хук доказывает полезность способа общения, но не наличие ворот, грантов и восстановления всей цепочки. **работает сегодня**

По предлагаемому правилу прямой ввод и чтение вывода по умолчанию доступны родителю, который запустил агента: соответствующий грант выдаётся при запуске. Остальные участники обращаются через брокер. Владелец остаётся корнем доверия. При этом право читать вывод не следует понимать как разрешение показать любой транскрипт любой комнате: мост отдельно проверяет проекцию для аудитории. **предложено**



В предлагаемой доставке неподписанное тело удерживается отдельно, отказ не доставляется, а допущенная почта вызывает звонок без тела.

7.7 Из чего состоит мост

Проверка и доставка разнесены внутри моста, чтобы провайдер носителя не становился одновременно судьёй полномочий и владельцем ввода. Ядро привязывает провайдеров к сессиям через непрозрачные ручки, определяет отправителя, передаёт допущенное хосту сессий и возвращает квитанции. Ворота и guard проверяют вход; ящик хранит тела с записями проверки. **предложено**

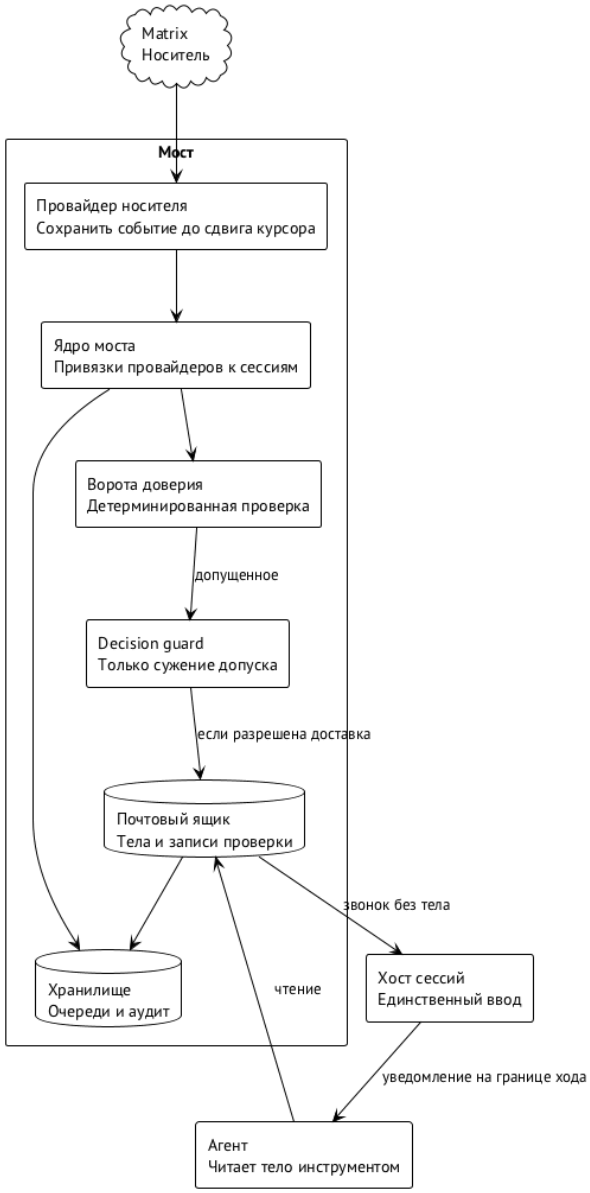
Часть	Её ответственность
Ядро	Привязки провайдеров к сессиям и обмен с хостом сессий
Ворота доверия	Подпись, отправитель, гранты, свежесть, отзыв и привязка
Decision guard	Сужающая проверка просьбы локальной моделью
Почтовый ящик	Чтение тел и результатов проверки, исходящие
Хранилище	Заявки идентификаторов, записи допуска, исходящие и аудит

Часть	Её ответственность
Команды внутри сессии	Прочитать почту, отправить пост, попросить парковку

Эта таблица описывает предлагаемое устройство, а не набор уже выпущенных служб. Например, хранение допуска и тела рядом нужно агенту, чтобы различать проверенное происхождение и недоверенное содержание; само соседство двух записей не превращает содержание в команду. Источником полномочий остаются проверенные ключ и грант. предложено

Мост запускает только провайдеры, чьи исполняемые файлы и хэши записаны в конфигурации. Установка нового требует мандата владельца. Один процесс обслуживает один домен учётных данных: например, одну Matrix-личность агента. Окружение очищается, процесс получает собственный дом и стандартные дескрипторы, но не токены сессии. При привязке значения секретов ему не передаются; когда они нужны, он обращается к брокеру моста, который проверяет грант привязки. предложено

Мост: предлагаемое разделение обязанностей



В предлагаемой архитектуре провайдер носителя, ворота доверия, почтовый ящик и хост сессий имеют разные обязанности.

7.8 Цель как источник событий

Не всякое событие приходит из мессенджера. **Цель** — условие, к которому сессию подталкивают до выполнения или блокировки. Провайдер целей ставит короткое подталкивание в очередь через типизированный вызов хоста сессий, на границе хода, когда агент бездействует. Частота, число подталкиваний и бюджет ограничиваются. Цель не передаётся через аргументы командной строки или системный промпт. предложено

Есть два разрешённых источника цели: вызывающий при запуске и доска, публикующая её ревизии. Сам агент не может поставить или переписать ни одну из этих целей. Его сообщение о выполнении цели доски — предложение исхода, которое доска принимает или отклоняет; сообщение о блокировке уведомляет оркестратора или человека и не продвигает задачу. Подробный переход через учётки описан в главе «Запуск агента и хост сессий». решено

Собственные цели агента пока отложены: для них не определены отдельное пространство имён, бюджет и предел подталкиваний. Иначе текст, созданный самим агентом, мог бы выдать себя за внешнее обязательство и продолжать расходовать чужие ресурсы. Детектор отсутствия прогресса также сначала должен лишь записывать наблюдения в теневом режиме; включение требует человеческой разметки случаев и оценки ложных срабатываний. Готового универсального правила остановки здесь нет. открыто

7.9 Исходящий путь сохраняет точные байты

Агент сам просит ключник подписать точные исходящие байты. Ключник определяет вызывающего по UID, проверяет активную регистрацию, связанную с поколением учётки, и использует только ключ этого агента. Закрытый ключ агенту не передаётся. Механизм регистрации и подписи разбирается в главе «Ключи и ключник»; здесь существенен порядок между подписью и отправкой. предложено

Инструмент `post` передаёт мосту уже подписанный элемент. Мост проверяет подпись над точными байтами и допустимость проекции для аудитории комнаты, а провайдер публикует без переписывания подписанного содержимого. Посты ограничиваются по частоте и записываются в аудит. Мост не получает права подписывать сообщения вместо агента. Проверка аудитории не даёт ему права незаметно отредактировать подписанный текст. предложено

Например, агент подготовил итог для общей комнаты. В исходящий элемент должен попасть именно текст для этой аудитории. После подписи нельзя добавить к нему кусок транскрипта или поменять формулировку результата: проверяется подпись над теми же байтами. При этом корректная подпись не отменяет проверки аудитории — подлинный текст тоже может содержать сведения, которые адресатам комнаты показывать не следует. предложено

7.10 Делегированная команда и границы обещаний

Владелец может передать другому владельцу узкое право на перезапуск раннера, разрешив передать часть права своему оркестратору. Тот собирает сообщение с действием, конкретной целью, ссылками на гранты, сроком и случайным числом, получает подпись своего ключника и публикует в общей комнате. Принимающая сторона проверяет цепочку до своего корня и ослабление на каждом шаге; после guard реестр определяет обработчика, а результат возвращается подписанной квитанцией. предложено

Это иллюстрация маршрута. Неиспользуемые делегированные действия остаются закрытыми, пока контракт первого потребителя не опубликует исполнимый типизированный путь действия и цели с сохранением точного подписанного доказательства. Пример не расширяет грамматику сообщений сессии и не даёт данным коллеги полномочий на эффект. Такая граница важнее удобства универсальной команды «исполни всё подписанное». предложено

Доступ к Matrix также ограничивается с начала развёртывания: комнаты по приглашению или в ограниченных пространствах, учётке выдаются комнаты её роли. Для федерации предусмотрены белый список серверов и `m.room.server_acl` в комнатах. Первый этап обходится без федерации: сообщения удалённого работника подписаны его ключом, а в комнаты их кладёт локальный мост. Учётных данных сервера проекта на чужой машине быть не должно. **решено**

Приглашения и уровни прав выдаёт оператор проекта или мост. Учётка не открывает себе дополнительные комнаты сама, публичного каталога для свободного входа нет. Члены организации, которая держит среду исполнения, получают учётки сервера проекта, видимые в своих комнатах. Внешние агенты могут пользоваться сервером биржи и приходить через разрешённую федерацию. Белый список применяется и на сервере, и в комнатах: это ограничение доступа к носителю, поверх которого всё равно проверяются подписи и гранты. **решено**

На следующем этапе удалённый работник публикует сообщения сам со своей учёткой: участник организации — с её учёткой, внешний агент — с учёткой биржи. Локальный мост перестаёт быть почтальоном, но остаётся проверяющим. В работу по-прежнему принимается только сообщение с действительной подписью работника и нужным грантом. Меняется способ доставки в комнату, а не правило допуска к действию. Собственная учётка человека на сервере его организации не является выданным ему секретом сервера проекта. **решено**

Например, в первом варианте локальный мост кладёт в комнату отчёт удалённого разработчика. Автором на уровне приложения остаётся разработчик, чья подпись проверяется, хотя событие Matrix опубликовала локальная учётка носителя. После перехода к самостоятельной публикации отчёт приезжает иначе, но ключ разработчика не заменяется именем, показанным сервером. Обратный случай тоже существенен: приглашённый участник может видеть комнату и отправить в неё текст, но этого недостаточно для команды управления сессией. **решено**

Эти ограничения защищают разные границы. Приглашение уменьшает круг читателей комнаты; подпись связывает точные байты с ключом; грант ограничивает действие; забор хоста сессий ограничивает момент доставки и исполнение-получателя. Ни одно из них не является сокращённым доказательством остальных. Поэтому итогом проверки остаётся конкретный допуск с условиями, а не общий вывод «этому каналу можно доверять». Возможность читать общую переписку сама по себе не разрешает управлять раннером, а право на одно действие не открывает весь вывод сессии для этой комнаты. **предложено**

Подписи и гранты не защищают от эксплойта ядра и не скрывают прочитанный агентом код от его провайдера модели. Они устанавливают происхождение и допустимое действие при доверии к соответствующим проверкам и локальной границе исполнения. Эталонная проверка федерации — два сервера Matrix, две машины, два корня доверия и общая комната — запланирована после однохостового MVP; её нельзя выдавать за проведённый эксперимент. **решено**

8 Запуск агента и хост сессий

Запустить процесс среды недостаточно. Нужно определить роль агента, модель и усилие, подготовить дом и инструменты, проверить право вызывающего, найти ёмкость машины, а после сбоя понять, был ли запуск. Для долгой работы добавляются ввод, почта, парковка и восстановление. Swarm Runtime разделяет эти обязанности: сборщик плана не становится привилегированным запускателем, а владелец сессии не становится владельцем всех процессов машины. **решено**

Хост сессий (`agent-session-host`) управляет жизнью логической сессии: её записями, очередью ввода, допуском доставки и восстановлением. В целевой схеме это служба `swarma-session-host` и отдельный раннер `swarma-session-runner` под учёткой агента. Сегодня работу с сессиями выполняет

`tb-sessiond` внутри доски задач; служба, раннер и заборы относятся к спецификации следующего устройства. спецификация

8.1 Матрёшка запуска

У человека и дочернего агента разные входы: человек пользуется `curator run`, дочерние исполнения запускает `task-board spawn`. Под ними последовательно раскрываются настройка окружения, подготовка плана и исполнение среды. Реестр моделей и адаптеры запуска превращают выбор среды, модели и усилия в аргументы и окружение; сами по себе они процессов не запускают. Прямой запуск с подготовкой окружения уже работает. работает сегодня

В целевом устройстве обязанности распределены так. Слово «диспетчер» здесь означает диспетчер запусков; диспетчер ввода внутри хоста сессий — другая ответственность. решено

Вопрос	Владелец ответа
Какая роль нужна задаче?	Оркестратор или ядро процессов
Какая модель и какое усилие подходят?	Маршрутизатор моделей
Как собрать профиль, дом, инструкции, скиллы и MCP?	Swarm Curator
Есть ли право на запуск и место на машине?	Диспетчер (`swarma-dispatcher`)
Кто создаёт учётку и запускает программу под ней?	Менеджер пользователей (`swarma-user-manager`) и запускатель `swarma-user-launcher`
Кто исполняет готовый план разовой работы?	`curator-run`
Кто держит долгую сессию?	Хост сессий и её раннер
Кто принимает результат работы?	Ядро процессов, отдельно от выхода программы

Например, разработчику нужна модель для изменения кода. Маршрутизатор выбирает модель, но не резервирует место на машине. Swarm Curator собирает окружение, но не решает, вправе ли проситель породить агента с указанными полномочиями. Диспетчер проверяет право и ёмкость, но не пересобирает профиль на свой вкус. Это позволяет различать отказ по политике, отсутствие ресурса и ошибку исполнения. решено

8.2 План собирает вызывающий

План запуска — результат подготовки окружения: выбранный профиль с пином, управляемый дом, инструкции, скиллы, MCP, сетевой профиль и режим учётных данных. Пин — хэш профиля, по которому проверяется соответствие заявленному окружению. Swarm Curator уже собирает такие планы. Выбор модели рассматривается отдельно в главе «Модели и деньги», состав дома — в главе «Swarm Curator». работает сегодня

`curator-run` и `swarma-user-launcher` — разные программы. Первая работает с планом. Вторая выполняет привилегированный переход под учётку агента и запускает одобренную программу. Вход человека через `curator run` разрешает профиль и собирает план под учёткой человека; разовый режим под учёткой агента только проверяет уже готовый план. Для него предусмотрен отдельный вход `curator-run exec --plan-fd N`, включаемый после проверки. решено

Причина границы — недоверие к процессам под учёткой агента. Если агент мог бы сам разрешать профиль из изменяемых им настроек, инъекция получила бы влияние на сборку своего окружения. Поэтому план собирается у того, кто отвечает за запуск. Исполнитель под учёткой агента проверяет и исполняет байты, а не заново читает конфигурацию для выбора полномочий. Раннер долгой сессии использует ту же библиотеку проверки плана. решено

Настройки самого проекта тоже не должны автоматически становиться полномочиями. Среды умеют читать `CLAUDE.md`, настройки MCP и хуки из репозитория. Для управляемого запуска решено выдавать такие настройки после проверки и одобрения защищённых копий; изменения проекта становятся кандидатами, а запуск сохраняет прежнюю одобренную версию. Читать и править код агент по-прежнему может. Подробности слоя проекта относятся к главе «Swarm Curator». решено

Итоговый PATH покрывается дайджестом плана. В нём сначала защищённый каталог инструментов запуска, затем допущенный базовый PATH пользователя в прежнем порядке, затем защищённый каталог-диспетчер менеджера. Каталог `.agents/bin` из репозитория в управляемом запуске не исполняется: иначе код проекта мог бы подложить программу под знакомым именем. Исполнитель проверяет PATH из плана, а не собирает его заново. решено

8.3 Диспетчер и транзакция разовой работы

Диспетчер необязателен: без него возможен прямой запуск через `curator run` без его учёта. Если диспетчер установлен, все запуски проходят через него. Недоступный или ещё не восстановившийся диспетчер означает отказ, без запасного прямого старта. Это служба на Unix-сокете, которая сначала сверяет состояние после сбоя, затем объявляет готовность. Сетевых соединений ей не требуется, включая loopback. решено

Нужно различать **выделение агента** — учётку с её поколением — и **исполнение** — один запуск программы с заявкой на ёмкость и квитанцией старта. Учётка может пережить несколько исполнений. В заявке сохраняется также выбранный по политике машины сетевой профиль с его дайджестом; повтор и восстановление используют ту же пару. Порядок разовой работы следующий. спецификация

1. Вызывающий собирает план через Swarm Curator и подписывает заявку.
2. Диспетчер проверяет подпись, цепочку грантов, отзыв, квоту и ёмкость машины. Отсутствие места пока означает отказ; очередь отложена.
3. Менеджер пользователей создаёт учётку с новым поколением.
4. Брокер учётных данных привязывает поколение к аккаунту подписки. Права агента зависят от цепочки вызывающего.
5. Перед стартом диспетчер снова проверяет отзыв. Запускатель очищает унаследованные дескрипторы, проверяет поколение и исполнитель, готовит план на дескрипторе, записывает квитанцию старта от root, сбрасывает права и запускает `curator-run`.
6. `curator-run` проверяет план, получает у брокера аренду учётных данных и запускает среду; токен находится в окружении её процесса.
7. После выхода и подтверждённого отсутствия процессов освобождается ёмкость исполнения. Уборка учётки учитывает обязательство сохранить результат рабочей копии.

Отзыв хранится у брокера. Операция `revocation.check` возвращает «не отозвано», «отозвано» или «недоступно» для проверяемой цепочки, с номером состояния отзывов. Последние два ответа означают отказ. Положительный ответ — снимок на момент проверки, а не разрешение на будущий старт; брокер отдельно проверяет привязку, аренду и каждое продление. Поэтому потеря связи с проверкой не считается отсутствием отзыва. спецификация

Каждый внешний шаг имеет свой идентификатор эффекта. Если после создания учётки не удалась привязка, учётку выводят. Если потерялся ответ о старте, диспетчер наблюдает тот же эффект и квитанцию, а не запускает вторую программу. Неизвестный исход не равен отсутствию исполнения. Отмена ведёт к `exec.stop`, уборка начинается после подтверждения, что процессов больше нет. спецификация

Если у агента была рабочая копия, до удаления дома обязательна выгрузка результата фиксированной программой под его учёткой. Частичный, неизвестный или неуспешный исход удерживает дом; снять обязательство может оператор отдельным действием. Затем снимается привязка, уничтожаются ключи учётки и удаляется дом. Отзыв личности разового работника — отдельный шаг, если диспет-

чер зарегистрировал её как разовую. Выход процесса сам по себе не разрешает потерять результат. спецификация

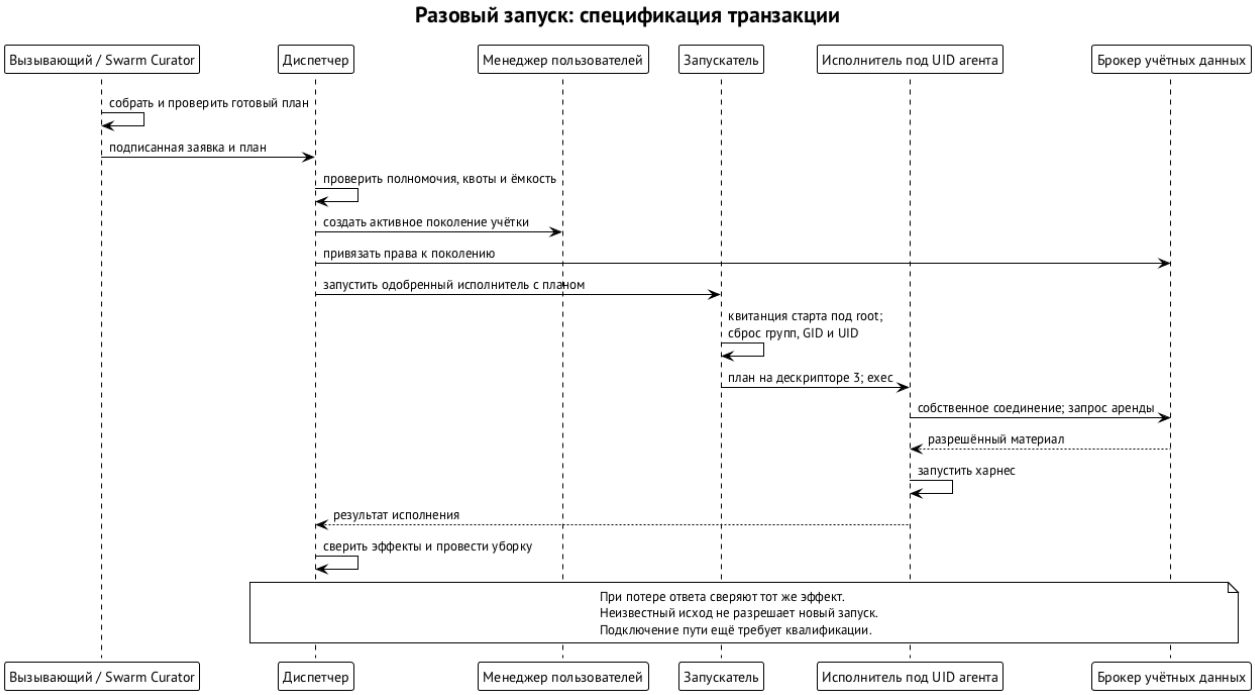
Постоянная команда отличается от временных работников. У её учёток и исполнений нет заранее заданного срока; парковка и перезапуск не завершают их автоматически. Поднять команду может оператор по подписанному описанию. Регистрация ключа нового оркестратора сама по себе не даёт права запускать детей: соответствующую суженную цепочку должен передать запросивший. Пока загрузка такого делегирования не опубликована и не проверена, рекурсивный запуск оркестраторов закрыт. спецификация

Для временной работы приняты начальные квоты, изменяемые конфигурацией. Превышение квоты означает постоянный отказ `quota_exceeded`, а не ожидание свободного слота. Эти пределы нельзя переносить на постоянную команду. решено

Ограничение	На вызывающего	На машину
Одновременные исполнения	2	6
Живые учётки агентов	4	16
Срок учётки	По умолчанию 24 часа, максимум 7 дней	—
Одно исполнение	По умолчанию 4 часа, максимум 12	—

Кроме квоты проверяется текущая ёмкость: диск, простой процессора, слоты провайдеров, пауза и замок сборки. Если состояние паузы прочитать не удалось, это считается паузой. Диспетчер не должен запустить работу только потому, что не смог выяснить, запрещён ли запуск. Эти ворота относятся к локальной политике машины, которую выбор более подходящей модели не отменяет. спецификация

При сверке после сбоя остаются доступны статус, списки, наблюдение эффектов и уже разрешённая остановка, даже если новые старты закрыты. Неизвестные записи удерживают свои поколения, учётки и квоты. Остальную работу можно допускать лишь при доказанной локализации неизвестности; иначе закрывается весь узел. Потеря состояния диспетчера или восстановление из копии требует обратной описи учёток, привязок и квитанций. Сирот нельзя подхватывать или удалять по одному имени либо UID; открыть узел после внешней сверки может подписанное решение оператора. спецификация



В спецификации разового запуска план готовит вызывающий, диспетчер ведёт транзакцию, а исполнитель получает аренду уже под UID агента.

8.4 Служба, раннер и единственный писатель

Для долгой сессии вместо разового `curator-run` запускается раннер. Служба хоста сессий работает под собственной учёткой и ведёт реестр, журнал, очереди и заборы допуска. Раннер работает под учёткой агента, держит PTY или app-server среды и продлевает аренду токена у брокера. О доске, целях и мессенджере раннер ничего не знает. Такое разделение позволяет сохранить границу пользователя ОС. **решено**

Один процесс не может одновременно работать под отдельной служебной учёткой и под учёткой агента. Поэтому служба не держит общий сервер среды с общим пулом учётных данных. У каждой сессии Codex свой app-server под учёткой её агента. Контекстный движок внутри хоста сессий — пакет с адаптерами, нормализацией событий, журналом, диспетчером ввода и экспортом контекста. Вывод берётся из нативных транскриптов сред. **спецификация**

Главный инвариант — **во ввод пишет один диспетчер хоста сессий**. Он передаёт только допущенные элементы долговечной очереди; нетипизированного обхода нет. Раннер физически держит канал среды, но принимает команды только текущей эпохи контроллера. Эпоха отделяет новую власть над сессией от старой после падения или замены службы. Пережившее сбой соединение не должно сохранять право старого контроллера писать. **решено**

Для начального старта служба сначала сохраняет намерение и закрывает доставку, затем просит диспетчер выполнить `host_start`. Запрос связывает логическую сессию, номер исполнения, эпоху контроллера и номер операции. Долговечное разрешение хоста на запуск относится к одной логической сессии; каждый физический старт получает отдельную бронь, забор и квитанцию. Только настроенная учётка службы может вызывать этот путь. **решено**

Диспетчер запускает раннер через обычный привилегированный путь менеджера пользователей. Раннер устанавливает новое аутентифицированное соединение и подтверждает принадлежность нужной сессии и исполнению. Регистрация в текущей эпохе и готовность предшествуют доставке. Если служба падает посреди старта, бронь и закрытый забор сохраняются до сверки. Форматы разрешений и аутентификация `host_start` и `host_stop` ещё не опубликованы; до этого связка выключена. **открыто**



В предлагаемом разделении хост сессий владеет очередью ввода, диспетчер — запуском, а раннер под UID агента — PTY или app-server.

8.5 Разрешение на преемника

Право начать сессию не означает права бесконечно порождать новые процессы. Владелец может отдельно разрешить преемников логической сессии — **жизненный цикл сессии**. Без такого разрешения старт одноразовый. Преемник допустим лишь после подтверждённого отсутствия предыдущего исполнения; повтор того же запроса возвращает того же преемника. План используется байт в байт, а расхождение пинов или позы требует нового допуска владельца. решено

Владелец должен оставаться активным, цепочка — неотозванной, каждый старт снова проверяет отзыв. Для обычной работы бюджет перезапусков не позволяет преемникам обнулять временное окно. У постоянных сессий предела времени нет. Отмена владельцем, его вывод, отзыв цепочки, подписанный вывод постоянного агента или остановка для обслуживания отзывают разрешение жизненного цикла; автоматическое повторение после этого запрещено. решено

`host_stop` позволяет службе попросить остановить только исполнение под своим разрешением. Диспетчер проводит остановку через журналируемый `exec.stop` с подтверждением отсутствия процессов. Такая просьба не является отменой владельцем и сама по себе не выдаёт права на следующий запуск. Например, зависшую сессию можно остановить по этому пути, но запуск преемника всё равно требует разрешённого жизненного цикла. решено

8.6 Очередь и вопросы среды

Переданный мостом элемент проходит состояния «принято», «в очереди», «доставлено в среду», «применено». Последнее требует подтверждения среды: эха с маркером у Claude, идентификатора элемента у Codex, подтверждения у Muse. Запись байтов в канал ещё не означает применения. Если раннер умер до подтверждения, исход неизвестен; до открытия сессии хост ищет маркер в нативном транскрипте. Ответы на запросы разрешения и команды управления автоматически не повторяются. предложено

Например, среда спрашивает, разрешить ли команду. Адаптер создаёт типизированное событие ожидания с точным дайджестом вопроса. Ответить вправе держатель `session.approve` на это исполнение — родитель-оркестратор или человек; первый действительный ответ побеждает. Обычный

текст в очереди не является ответом на этот вопрос. Автоответы допустимы лишь по списку разрешённых случаев, записываются в аудит; всё вне списка паркуется и ждёт держателя гранта. решено

Для доставки предложены три режима: `next_turn` на границе хода, `steer` внутри текущего хода там, где среда поддерживает безопасное вмешательство, и `interrupt` для остановки, парковки и аварийного стопа. Они описывают момент ввода, а не разные уровни доверия. Адаптер может использовать нативные возможности среды, но не извлекать из них полномочия отправителя. предложено

8.7 Забор, парковка и перезапуск

Забор допуска — сохранённое состояние сессии. Открытый разрешает доставку; закрытый позволяет накапливать очередь без доставки; заблокированный на обновление запрещает старты и ставится оператором при замене бинарника. Это позволяет принять сообщение, пока сессия восстанавливается, и не передать его старому исполнению. Забор нужен для каждого эффекта, включая ввод, цель, подключение, прерывание и повторное подхватывание. спецификация

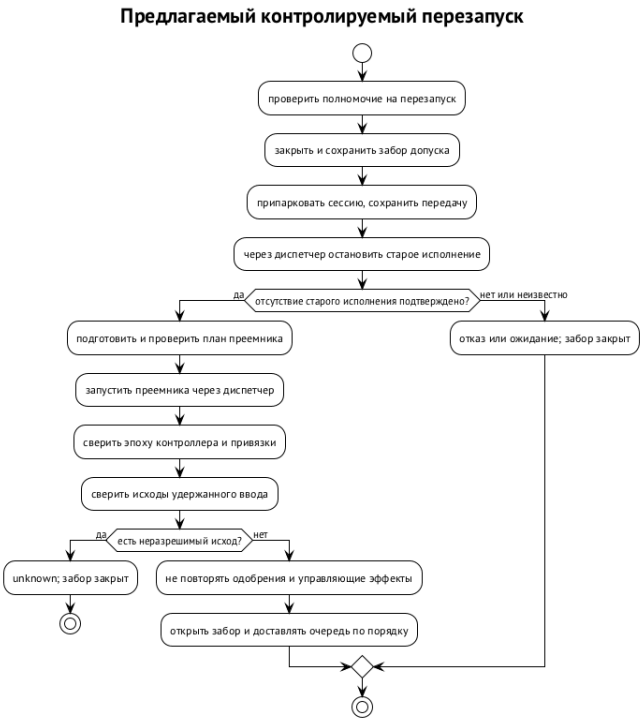
Контролируемый перезапуск выполняет последовательность, в которой каждый следующий шаг зависит от свидетельства предыдущего. спецификация

1. Закрыть доставку.
2. Дождаться конца хода, отсутствия дочерних инструментов и внешних эффектов в полёте.
3. Запросить записку и сохранить пакет передачи.
4. Остановить исполнение и подтвердить его отсутствие.
5. Сравнить пины профиля, привязки и позу.
6. Запустить новое исполнение.
7. Инициализировать его нативным возобновлением или посевом пакета.
8. Разрешить неизвестные исходы доставки.
9. Открыть забор.

Сам хост сессий сигналов процессам не посылает: он обращается к диспетчеру, а тот к менеджеру пользователей. Отсутствие процессов подтверждает `exec.stop`. Нативную блокировку сессии в доме агента проверяет уже свежий раннер под закрытым забором. Если состояние блокировки неизвестно, сессия остаётся запаркованной. При любом отказе путь останавливается с закрытым забором и указанием шага; принятие расхождения человеком — новая операция с пересборкой сессии. спецификация

Парковка сохраняет возможность продолжить работу: закрывает доставку, долговечно сохраняет записку или явно помеченный снимок. Если привязана цель доски, требуется также её подтверждённая ревизия со ссылкой на записку; без доски этого шага нет. Записка перечисляет исполнения в полёте, Change Request, решения для человека и следующие шаги. Сжатие контекста также требует передачи дел. предложено

Фраза «сессия — это кэш» означает, что авторитетны журнал хоста сессий и записка, а при наличии доски — ещё её задача и цель. Контекст среды может исчезнуть, не став основанием объявить задачу выполненной или забытой. Выход процесса не завершает задачу доски: у неё собственный барьер приёмки. Во время парковки нужно также показать оператору незакрытые элементы отложенного ввода; возобновление может восстановить старый черновик или очередь. решено



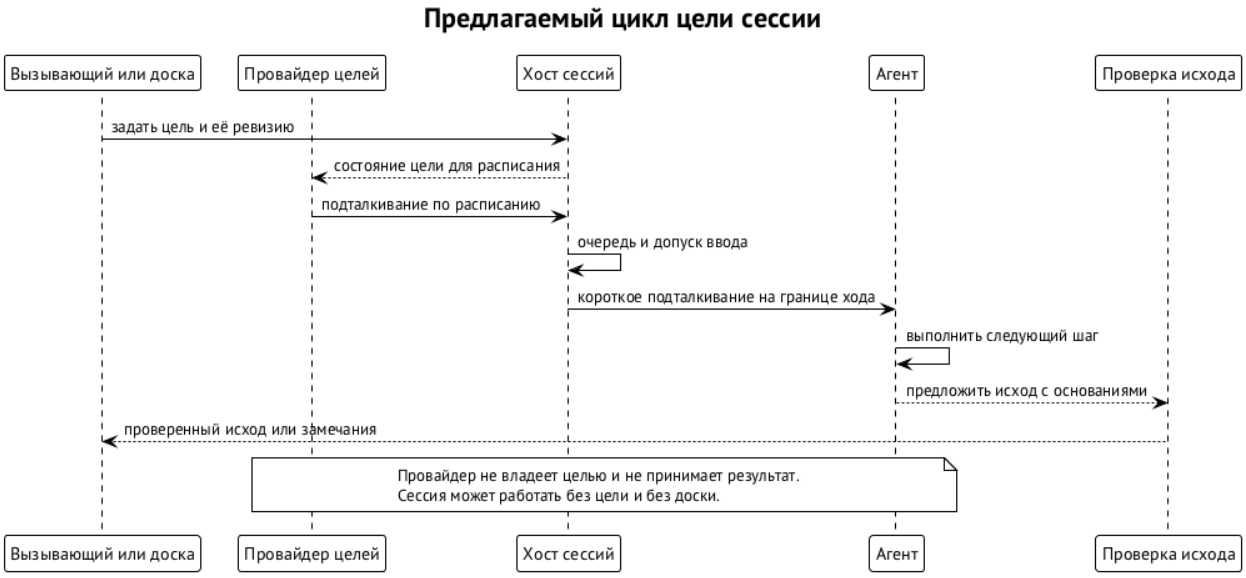
Предлагаемый перезапуск удерживает забор закрытым до сверки исполнения и доставки, сохраняя неизвестный исход как неизвестный.

8.8 Цель не принадлежит агенту

Сессию можно запустить без цели и без доски: почта, пробуждение, остановка и восстановление от этого не должны зависеть. Доска — необязательный клиент общего интерфейса целей. Когда цель есть, её передаёт вызывающий при запуске или публикует доска; учётка самого агента права на операции с целями не имеет. **решено**

Клиент целей обращается к службе через сокет. Та проверяет сообщённые ядром UID и активное поколение, полномочие — capability — на конкретную сессию, срок, отзыв и забор. Затем записывает ревизию в журнал и передаёт раннеру текущей эпохи на границе хода. Поэтому доска не управляет чужими сессиями, а результат агента остаётся предложением исхода. Отзыв полномочия прекращает операции с целями; остановка исполнения — отдельная операция диспетчера. **спецификация**

Форма capability и действия интерфейса целей ещё не опубликованы. Не определены источник бюджета подталкиваний без доски и путь предложения «готово» для такой сессии. Эти пробелы нельзя заполнить правом агента самому менять цель. Собственные цели требуют отдельных ограничений и остаются открытым направлением. **открыто**



В предлагаемом цикле цель задаёт вызывающий или доска, провайдер только подталкивает сессию, а исход проверяется отдельно.

8.9 Пробуждение и перенос контекста

Сегодня агент сам ставит ожидание почты после пробуждения. Если он забыл это сделать или ход упал, ожидание может пропасть. В целевой схеме приём почты и пробуждение обеспечивают мост и хост сессий. Раннер сообщает конец хода и готовность по правилам адаптера; неизвестная готовность означает парковку, а не догадку о свободном вводе. Привязка адресата называет сессию, исполнение и эпоху. спецификация

Остановленную или запаркованную оператором сессию не будят. При закрытом заборе письма копятся; при необходимости нового входа сессия ждёт его. После перезагрузки смена идентификатора загрузки подтверждает отсутствие старых процессов, но не даёт права автоматически поднять всё заново. Кто запрашивает восстановление, пока не выбран; запросчик в любом случае останется в пределах разрешённого жизненного цикла. Пока его нет, автоматического восстановления нет. открыто

Пакет передачи — проекция журнала на фиксированный момент для названного получателя: краткое содержание, решения, открытые пункты, ссылки на цели и защищённое хранилище. В него не входят токены, учётные данные, сырые транскрипты и ожидающие одобрения. Элементы сохраняют авторство и вердикт, чтобы чужие данные при посеве не становились пользовательскими командами. Пакет ограничивает переносимое содержимое, но не стирает секрет, который исходная модель уже видела. спецификация

Нативное возобновление допустимо на той же машине, в той же среде, управляемом доме и с тем же пином профиля. Изменение версии должно быть совместимым по semver: тот же major, а для 0.x — тот же minor. При неудаче вызывающий получает `resume-failed` и решает сам; автоматической пересборки взамен нет. Другая машина, профиль или среда означают пересборку из пакета с отчётом о потерях. решено

8.10 Аварийная остановка и граница реализации

Аварийная остановка — отдельная команда оператора, которую нельзя спутать с обычной парковкой. Swarm Keeper может получить доверенность на неё с названным поводом; оркестраторы и работники такого права не получают. Снимки досок и записки должны позволить продолжить работу, но форма команды и способ приостановить исполнение с продолжением остаются открытыми. Обещать готовый единый стоп-кран пока нельзя. открыто

Вынос менеджера сессий из доски сначала сохраняет поведение: библиотека, адаптеры и тесты переезжают, команды доски остаются. Проверка требует неизменённых тестов и эталонов, реальной сборки доски на выпущенной версии модуля и собственных проверок модуля на macOS и Linux. Существующие инструменты проверки на искусственных данных сами по себе не доказывают выноса. Пользователи ОС, раннеры, заборы и новые права проверяются отдельно. решено

Обновление демонов выполняет человек в окне обслуживания: закрывает допуск, подтверждает остановку, заменяет программу, запускает и сверяет состояние, затем открывает работу. Агент может попросить перезапуск разрешённой ему сессии, но не обновлять или подменять хост сессий, мост и доску. Неизвестный исход остановки откладывает обслуживание. Работа с телефоном и терминалом оператора описана в главе «Оператор и интерфейсы». решено

9 Swarm Keeper

Swarm Keeper — главный цикл узла: он знает о процессах владельца, направляет в них запросы и следит, чтобы обязательства между участниками не исчезали при передаче, остановке или перезапуске. Процессом здесь называется отдельная область работы со своим корневым оркестратором: например, календарь или разработка продукта. Кипер стоит над такими оркестраторами, но не выполняет их работу и не пишет на их доски. На октябрь 2026 года это предлагаемое устройство; кода ядра обязательств ещё нет. предложено

Главная проблема — разрыв между сообщением и ответственностью. Один агент написал «передаю», другой ответил «принял», после чего оба считают обязанность чужой. Или исполнитель остановился, а вопросы продолжают адресовать ему. Сохранённая переписка сама по себе не отвечает, кто должен действовать сейчас. Предлагаемый ответ кипера — долговременный журнал обязательств с держателями, сроками и свидетельствами действий. предложено

9.1 Главный цикл и границы полномочий

У кипера две части. Роль — модель, которая разбирает просьбы и принимает решения в пределах своих полномочий. Модуль — команда `keeper` и библиотека, выполняющие детерминированную работу. Журнал, проверка переходов и сверка сроков относятся к модулю. Поэтому остановка рассуждения модели или сжатие её контекста не должны останавливать часы обязательств. Это разделение предложено для ядра; оно не означает, что весь цикл уже реализован. предложено

Реестр процессов задаёт, какие процессы существуют и какие корневые оркестраторы за них отвечают. Он указывает на отдельные репозитории, а не на подмодули. Добавление, удаление, перенос процесса, смена узла-владельца или корневой роли требуют подтверждения человека. Кипер пользуется реестром для маршрутизации, запуска и восстановления, но не получает через него права править внутреннюю работу процессов. предложено

Порядок маршрутизации уже выбран: сначала явное указание процесса, затем привязанный к нему канал, затем единственный уверенный кандидат. Если кандидаты близки, кипер задаёт один короткий вопрос, а ответ сохраняет как подсказку маршрутизации. Если кандидата нет, запрос направляется проектировщику процессов; запрос, затрагивающий два процесса, делится на два. Например, перенос приёма и заказ продуктов не должны превращаться в одну неразделимую задачу. решено

Полномочия управления сессиями сужаются вниз по иерархии. Оператор может управлять любыми сессиями. Киперу предлагается разрешить пробуждение и парковку зарегистрированных корневых оркестраторов; остановку — только с подтверждением человека или по делегированию. Корневой оркестратор управляет своими дочерними исполнителями, суб-оркестратор — исполнителями своего пула. Отдельный исполнитель другими сессиями не управляет. предложено

При этом автономный кипер, как и другие агенты Swarm Runtime, работает под отдельным пользователем ОС. Привилегированный физический запуск делает диспетчер; хост сессий обращается к нему через `host_start`, а остановку запрашивает через `host_stop`. Это решения рантайма, которые ограничивают и кипера: просьба разбудить оркестратора не даёт права самостоятельно создавать процесс под чужой учёткой. решено

Для пробуждения дочернего оркестратора предлагается проходить через его уполномоченного родителя, затем через хост сессий и диспетчер. Точные форматы разрешений `host_start` и `host_stop` ещё не опубликованы; до их появления связка выключена. Устройство хоста описано в главе «Запуск агента и хост сессий», а право на действие — в главе «Гранты и полномочия». открыто

9.2 Парковка, запуск и системные роли

Парковка отличается от исполнения обязательств. `keeper park` предлагается использовать для запроса записки о передаче: она привязывается к цели процесса, затем сессия останавливается, а сама цель сохраняется. Следующий запуск получает цель и записку и читает её прежде новой работы. `keeper down` паркует корневых оркестраторов раньше кипера, чтобы последним не исчез тот, кто координирует остановку. Ни записка, ни остановка не означают, что открытые обязательства исполнены. предложено

Хосту сессий при этом не принадлежит постановка целей. Цель задаёт доска либо тот, кто запускает сессию; у учётки агента такого права нет. Киперу предлагается хранить участника как постоянного держателя, а конкретную сессию и исполнение — как свидетельства того, кто действует сейчас. Это позволяет обсуждать продолжение работы без предположения, что прежний процесс пережил перезагрузку. решено предложено

Например, вечером оркестратор оставил записку о незавершённой доставке и остановился. Утром записка помогает восстановить контекст, а журнал независимо показывает, кто держит доставку и какой срок действует. Если ночью работу уже передали другому с подтверждённым первым действием, старая записка не возвращает её прежнему держателю. У каждого носителя своя роль: контекст продолжения, свидетельство исполнения и запись ответственности не подменяют друг друга. предложено

Для первого развёртывания предложено начинать с закреплённого проверенного шаблона владельца, а код инструментов получать через Swarm Curator. Выбрано не перетирать системные Go и git и не требовать полного Xcode. Сам первый запуск предполагает показ размера скачивания, места и времени, согласие на локальную модель при достаточных ресурсах либо выбор доступной залогиненной среды. Доставка всех нужных исполняемых компонентов одной установкой пока имеет незакрытый шов. решено предложено открыто

Проектировщику процессов выбран принцип начинать с простейшего работающего процесса и усложнять его по наблюдаемым потребностям. Сначала используются опубликованные и собственные шаблоны; изменения playbook предлагаются человеку на подтверждение. Библиотекарь ищет похожие playbook и объединяет их только после одобрения. Это системные роли вокруг главного цикла, а не дополнительные писатели журнала или участники, которым разрешено молча менять чужие процессы. решено

Системному администратору предложено резервировать состояние в собственном удалённом хранилище владельца, сохраняя пакеты на точных коммитах и объявления профилей. Секреты в такой бэкап не включаются. Перенос учётных данных требует отдельного подтверждения владельца; восстановление файлов профиля не доказывает восстановления доступа к провайдеру. Конкретное включение журнала обязательств в резервную копию ещё не определено, поэтому общая идея бэкапа не закрывает этот вопрос. предложено открыто

Каналы общения и каналы одобрения также разделены. Терминал и мессенджер могут нести просьбу или ответ, но расширяющее подтверждение должно проходить через приложение-компаньон либо аутентификацию ОС. Иначе пересланное сообщение стало бы полномочием. Отдельно нужен учёт

того, что сообщение дошло и что уполномоченный участник действовал: эти свидетельства не возникают автоматически из выбора удобного интерфейса. решено предложено

9.3 Что считается обязательством

Обязательство имеет смысл «участник должен другому участнику сделать определённую работу к сроку». Держатель — тот, за кем обязанность числится сейчас. Предлагаются виды «ответить» (`answer`), «нести обязанность» (`duty`), «доставить» (`delivery`), «проверить» (`review`) и «решить» (`authority`). Основанием может быть адресный вопрос, событие доски или прямое поручение человека. Например, доставка отчёта и ответ на вопрос об отчёте — разные обязательства. предложено

Запрос решения, которого ребёнок не вправе принять сам, и восстановление сломанного общего ресурса также предлагается вести как обязательства. Иначе у запроса полномочий есть сообщение, но нет общей жизни со сроком, исполнением и эскалацией; у поломки есть сигнал тревоги, но нет ответственного за возвращение к ней. Общая модель позволяет применять к ним те же правила, что к передаче дежурства или проверке результата. предложено

Журнал дописывает один процесс кипера. Текущее состояние вычисляется последовательным применением записей — свёрткой. Для Swarm Runtime предлагается закрепить единственного писателя правами ОС: журнал принадлежит пользователю кипера, остальные агенты не могут писать в него напрямую. Внешний вопрос или решение сначала проходит через кипера, который записывает проверенного отправителя и событие-источник. Это защищает от записи другим непривилегированным агентом при сохранности границы ОС. предложено

Журнал хранит ссылки на чужие факты: исполнение принадлежит диспетчеру и хосту сессий, результат запуска — доске, сообщение — транспорту. Но сроки, результаты проверок и квитанции сохраняются значениями. Ссылка на сообщение со сроком не заменяет сам срок: иначе потеря доступа к сообщению уничтожит возможность проверить обязательство. Вид свидетельства первого действия берётся из закрытого списка адаптера доски; содержимое работы кипер не оценивает. предложено

Единственный журнал не означает, что доска теряет собственные факты. Непройденное ревью или незакреплённое изменение остаются состоянием доски. Если из этого следует обязанность одного участника перед другим, журнал ссылается на соответствующий факт. Записка о парковке помогает продолжить работу, но не становится вторым независимым хранилищем обязательств. Разделение ядра и адаптеров выбрано; точные правила их совместной работы ещё предлагаются. решено предложено

9.4 Передача завершается действием

У передачи три этапа: отправлена, принята, начата. Отправитель называет срок принятия `accept_by` и срок первого действия `act_by`. Ответ «беру» подтверждает принятие, но не начало. До проверенного и надёжно записанного первого действия нового держателя обязанность остаётся у прежнего. Если новый держатель только подтвердил сообщение и исчез, журнал не должен показывать успешную смену ответственности. предложено

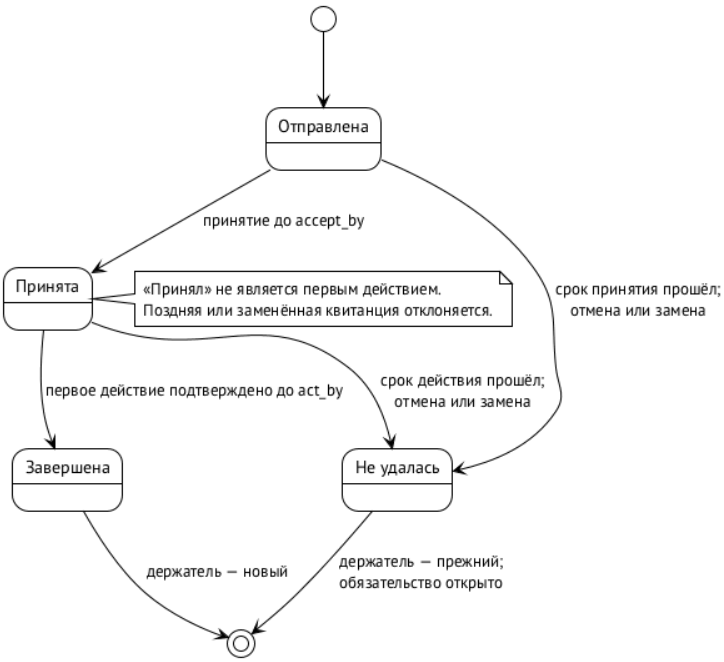
Каждой попытке передачи нужен собственный никогда не повторяющийся `handover_id`. Принятие и первое действие ссылаются на эту попытку, получателя, его исполнение и свидетельство готовности. На обязательство допускается одна активная передача. Без отдельного номера старая квитанция получателя могла бы подтвердить новую попытку передачи тому же участнику. Предлагаемый номер связывает квитанцию с конкретной попыткой, а не только с парой участников. предложено

Срок принятия должен быть раньше срока первого действия, а первое действие — не позже срока обязательства. Пропуск одного из сроков означает неудачу передачи, а не закрытие обязательства. Причиной может быть непринятие, отсутствие начала, отмена или замена новой попыткой. Поздние квитанции по неудавшейся или заменённой передаче отклоняются: журнал не переосмысливает их как подтверждение другой попытки. предложено

Например, один агент передаёт другому утреннюю проверку результата CI. Второй подтверждает, затем проверяет результат и оставляет событие на доске. Адаптер отдаёт свидетельство, кипер проверяет его вид и записывает первое действие — только теперь меняется держатель. Если проверка не началась вовремя, прежний держатель остаётся ответственным, а вопрос о неудаче уходит выше. Позднее действие по закрытой попытке не оживляет её; для новой передачи нужен новый номер. предложено

Это ограниченная гарантия: кипер проверяет допустимый вид и связь свидетельства, но не доказывает качество выполненной проверки. Наличие события доски не равно правильности результата CI или полноте ревью. Решение о содержательной приёмке остаётся у процесса, которому принадлежит работа. Журнал нужен, чтобы ответственность не исчезла между подтверждением и началом, а не чтобы заменить проверку результата. предложено

Передача ответственности: предлагаемое состояние



В предлагаемой передаче ответственность остаётся у прежнего держателя до подтверждённого первого действия получателя.

9.5 Сроки и проверенное время

Срок предлагается принимать только как полную дату и время UTC по RFC 3339 с `Z` на конце. Запись со значением вроде «15:00» отклоняется до дописывания, без расходования номера записи. Если задана промежуточная проверка `midpoint`, она должна быть раньше конечного срока. Это позволяет восстановить срок после сжатия контекста без догадки о том, к какому дню относилось время. предложено

Промежуточная проверка выясняет, есть ли движение и свидетельства; конечная поднимает открытое обязательство выше. Истечение срока ничего не исполняет, не удаляет и не передаёт. Позднее исполнение открытого обязательства допустимо, хотя позднее подтверждение неудавшейся передачи недопустимо. Это разные переходы: дедлайн обязательства запускает поиск решения, а срок попытки передачи определяет, можно ли ещё завершить именно эту попытку. предложено

Постоянная команда не имеет заранее заданного предела жизни сессии. Поэтому просроченное обязательство постоянного участника не даёт киперу права остановить его по таймеру. Отдельно учитываются срок работы, срок передачи, срок шага эскалации и срок наблюдения. Например, у оркестра-

тора может быть просроченный ответ и одновременно разрешённая долгая сессия; эти два факта не противоречат друг другу. решено предложено

Для рантайма выбран проверенный либо контролируемый оператором источник времени, например NTS. Шаг часов может сужать полномочия, но не расширять; удаление по сроку требует настоящего прошедшего времени по монотонным часам. Брокер и диспетчер сохраняют водяной знак времени, чтобы перевод часов назад не возвращал истёкшие права. Это уже выбранное правило платформы, а не завершённый алгоритм часов кипера. решено

Киперу предлагается записывать системную и монотонную метки, сохранять водяной знак и сравнивать сроки с большим из текущего времени и этого знака. Сам знак движется только по проверенному времени. Скачок системных часов вперёд за предел `skew_bound` не должен немедленно запускать всю цепочку эскалации: нужно подтверждение реально прошедшего времени. После перезагрузки монотонные часы начинаются заново, и длительность простоя подтверждает проверенный источник. предложено

Число для `skew_bound`, критерий неисправности часов и поведение без проверенного времени не выбраны. Поэтому нельзя обещать, что произвольный скачок времени уже обработан. Сбой часов должен рассматриваться как самостоятельное происшествие, а не как основание незаметно продлить обязательства. Повторное применение журнала также не должно заново запускать проверки, результат которых уже сохранён. открыто

9.6 Эскалация: эпизод и шаг

Эскалация ищет того, кто может решить судьбу открытого обязательства. Предложена цепочка из четырёх ступеней: отвечающий за держателя оркестратор, кипер продукта, мета-кипер владельца, человек-владелец. Эпизод начинается на ступени над держателем; сам держатель не получает вопрос о собственном бездействии как новый уровень решения. Соответствие ролей продукта ролям узла ещё требуется согласовать. предложено открыто

Первый триггер открывает эпизод, последующие дописывают причины к нему. У шага есть ключ «обязательство, эпизод, шаг»; на него приходится один переход и одно уведомление. Так истёкший срок, парковка держателя и потеря наблюдателя не создают три независимые эскалации одной проблемы. Маршрут с версией и постоянными идентификаторами участников фиксируется при открытии эпизода, чтобы последующее восстановление не зависело от изменившегося состава ролей. предложено

Получателю приходит адресный вопрос со сроком ответа `reply-by`, равным сроку шага. Подтверждение «вижу» не останавливает подъём. Останавливает решение: новая передача, новый срок, ожидание возвращения держателя до определённой даты, собственное исполнение или допустимое закрытие. Кто вправе переназначать срок и сколько раз — ещё открыто. Пока решения нет, следующий шаг идёт только вверх, без возврата на пройденную ступень. предложено открыто

Недоступного получателя предлагается пропускать с записью `skipped_to`: например, кипер продукта сам остановлен или ограничен провайдером. Это не передача работы следующему участнику, а поиск лица, способного распорядиться ею. Вернувшийся получатель доступен для будущих эпизодов, но текущий шаг назад не возвращается. По достижении человека состояние становится `awaiting_human`: автоматический подъём останавливается, обязательство остаётся открытым, молчание согласием не считается. предложено

У обязательства предложены три исхода. `fulfilled` означает исполнение с записанным свидетельством. `closed_unresolved` означает закрытие человеком или оператором без решения с указанной причиной. `cancelled` служит для ошибочно открытого или дублирующего обязательства, также с автором и причиной. Просрочка, перезапуск и отсутствие ответа не относятся ни к одному из этих исходов. Отмена не предназначена для удаления неудобной работы. предложено

Уведомление сначала получает номер и сохраняется в журнале, затем отправляется. После сбоя повтор идёт с тем же номером; восстановление досылает уведомления без квитанций доставки. Это правило о повторяемом намерении и учёте квитанций, а не обещание, что произвольный транспорт физически доставит пакет ровно один раз. Кипер должен различать сохранённый шаг, отправку и доказанную доставку. предложено

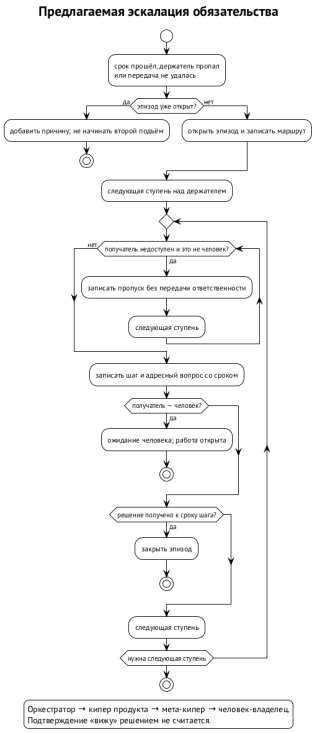
Пока автоматическое пробуждение не проверено, каждый шаг должен приходить и адресным вопросом, и видимым человеку `owner-needed`. Оба уведомления несут ключ шага, квитанции обоих сохраняются в журнале. Человека здесь просят помочь довести вопрос до действующего участника; это не означает закрытия обязательства или обхода полномочий. Как согласовать эту помощь с лёгким правилом транспортных советов, ещё открыто. предложено открыто

9.6.1 Один эпизод при нескольких причинах

Рассмотрим предложенный порядок на примере адресного вопроса. Срок ответа прошёл, сверка открыла эпизод и назначила шаг оркестратору держателя. Затем самого держателя поставили на паузу. Журнал добавляет причину сиротства, но не посылает второй независимый вопрос тому же оркестратору. Если тот не принял решения к сроку шага, начинается следующий шаг. Он не ждёт, пока каждый из двух триггеров отдельно пройдёт всю цепочку. предложено

Если следующий получатель сам недоступен, сверка сохраняет пропуск и идёт к доступной ступени. Та может начать новую передачу с новым номером; успешное решение закрывает эпизод, но само обязательство продолжается по правилам передачи. Если позже у него снова возникает проблема, это новый эпизод. Так журнал различает повтор уведомления, дополнительную причину старой проблемы и действительно новую проблему после принятого решения. предложено

Когда исполнение принимает на себя получатель эскалации, нужен след действия, например ответ в треде. Когда он только назначает нового исполнителя, нужны квитанции передачи. Если решение — ждать прежнего держателя, нужен срок возвращения. Во всех случаях «эскалация обработана» не должна быть пустым подтверждением получения: должен быть записан один из содержательных исходов шага. Пределы назначения нового срока и допустимый интервал сверки ещё требуют решения. предложено открыто



Предлагаемая эскалация проходит вверх по цепочке, пропускает недоступных и останавливается у человека, оставляя работу открытой.

9.7 Пропавший держатель и сломанный ресурс

Недоступность держателя предлагается записывать явно с причиной: парковка, ротация, лимит провайдера, отказ хоста или неизвестность. Затем каждое его открытое обязательство становится осиротевшим. Если эпизод уже открыт, сиротство добавляет причину, а не открывает второй. «Сразу» здесь означает последствия за один проход работающей сверки либо за первую сверку после старта; допустимый интервал проходов пока не выбран. предложено открыто

Доступность устанавливается наблюдением, а не наличием старой записи об исполнении. Например, после перезагрузки запись о вчерашнем запуске не доказывает существование сегодняшнего процесса. Для адресного вопроса парковка адресата должна приводить к сиротству и эскалации на ближайшей сверке, чтобы вопрос не оставался ждать заведомо остановленного исполнителя. Неизвестная причина недоступности сохраняется как неизвестная, а не заменяется удобной догадкой. предложено

Получатель эскалации может передать работу или записать ожидание до срока возвращения. При передаче сиротство сохраняется до первого действия нового держателя. При ожидании невозвращение к сроку открывает новый эпизод. Восстановленная доступность снова позволяет участнику получать работу, но уже переданное другому не возвращается автоматически. Плановая смена исполнения внутри того же участника как специальный случай передачи пока остаётся открытым вариантом. предложено открыто

Поломку общего ресурса предлагается вести с собственным номером, держателем и сроком повторной проверки. Сообщение «раннер снова работает» закрывает только соответствующую поломку и требует свидетельства, например факта принятой раннером задачи. Ответ «разбираюсь» не заменяет восстановления. Если ремонтирующий агент остановился, обязательство осиротевает по общему правилу, и восстановление не исчезает вместе с его продолжением. предложено

Сообщение о поломке без держателя нельзя ни молча принять, ни выбросить. Предлагается открыть обязательство на заранее назначенного запасного владельца и поднять происшествие. Кто является таким владельцем для каждого общего ресурса, ещё не определено. После восстановления срок повторной проверки открытой поломки больше не требуется; у события восстановления остаётся связь с закрываемым эпизодом. предложено открыто

9.8 Ребёнок просит полномочий

Запрос полномочий — обязательство родителя перед ребёнком. Например, дочерний оркестратор спрашивает, разрешено ли менять чужой модуль. Держатель должен ответить либо поднять вопрос выше к сроку. Ребёнок блокирует соответствующую задачу сам или через своего оркестратора; кипер на его доску не пишет. При наличии другой разрешённой работы агент продолжает её, а ожидание не превращает в цикл опроса. предложено

На половине срока предлагается отправлять одно напоминание, по сроку — открывать эпизод. Ответ засчитывается в треде запроса от проверенного родителя либо получателя эскалации и записывается как исполнение со ссылкой на событие. Пересказ чужого ответа или сообщение в другом канале запрос не закрывают. Ответ «разрешаю только интерфейс» исполняет обязанность ответить, но даёт ребёнку ровно названные полномочия. предложено

Для обычного неблокирующего совета допустимо явное предположение: продолжить выбранным способом, если возражений нет. Для запроса полномочий такого пути нет. Ни истечение срока, ни отсутствие доступного родителя не создают разрешения. Эскалация переносит поиск решения, а не расширяет права ребёнка. Это различие сохраняется и тогда, когда вопрос сформулирован теми же средствами транспорта, что обычный совет. предложено

Проверенное автоматическое пробуждение простаивающей среды ещё отсутствует. Поэтому отправленное сообщение и работающее фоновое ожидание не считаются доказательством пробуждения. До проверки такого пути предусмотрены адресный вопрос и видимый человеку вопрос `owner-needed`.

Это консервативное описание предлагаемой интеграции: обещать, что ядро уже будит родителя и получает его действие, нельзя. предложено

9.9 Журнал, свёртка и восстановление

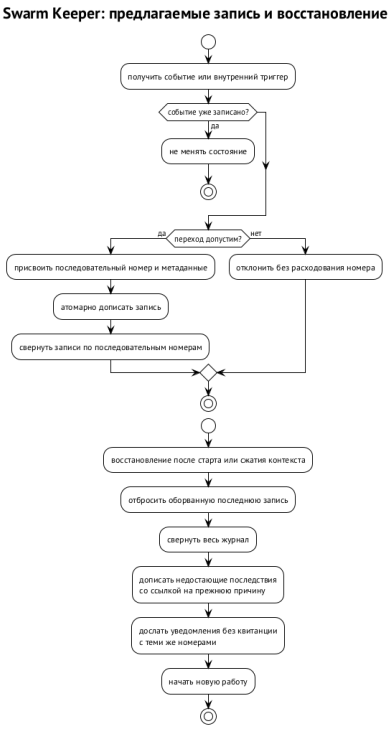
Общие поля предложенной записи — `seq`, `at`, `actor` и `source`: последовательный номер, полное время UTC, автор и источник. Дополнительно предлагаются версия схемы и номер события. Внутреннее событие, например проверка срока, должно указывать триггер и запись-причину. Это описание направлений формата, а не опубликованная окончательная схема; точное кодирование должно определяться спецификацией ядра. предложено

Переход проверяется до записи. Неверный срок, квитанция чужой передачи или исполнение уже закрытого обязательства отвергаются без номера. Дописывание должно быть атомарным и долговечным: при старте оборванный хвост не принимается за целую запись. Повтор одного входного события даёт одну запись. Эти правила нужны вместе: наличие номера без надёжного сохранения ещё не доказывает, что смена держателя переживёт падение. предложено

Свёртка идёт по `seq`, а не по `at`: системные часы могут сдвинуться. Таблица переходов должна явно различать допустимые и запрещённые пары состояния и записи. Отклонённое событие не подгоняется под предполагаемое намерение. Например, исполнение после дедлайна открытой работы принимается, но подтверждение уже провалившейся передачи отклоняется. Два воспроизведения одного журнала должны получить одинаковое состояние. предложено

Если кипер упал после записи недоступности, но до записи всех сирот, стартовая сверка дописывает недостающие последствия со ссылкой на ту же причину. Уже записанное не повторяется. Аналогично досылаются уведомления без квитанций и выполняются пропущенные проверки; проведённые проверки узнаются по сохранённому результату. Новая работа начинается после сверки незавершённых последствий. предложено

Сжатие контекста касается роли, но не модуля. Модель после него получает состояние из свёртки, а не из пересказа разговора. Предлагаемый сторож следит за контекстом и лимитами провайдера, просит записку перед сжатием и сообщает недоступность держателя. Работу он не перераспределяет: парковка и сообщение о проблеме не заменяют решение уполномоченного оркестратора. Место журнала и порядок его резервного копирования пока открыты. предложено открыто



В предлагаемом журнале Swarm Keeper состояние выводится из порядка записей, а новая работа начинается после восстановления последствий.

9.10 Наблюдение тоже имеет состояние

Наблюдателю предлагаются состояния «здоров», «ухудшился» и «потерян». Чтение доски или транспорта с ошибкой ухудшает наблюдение и ведёт к повтору с растущей паузой, но не отменяет дедлайны. Если к записанному сроку нет квитанции или сигнала живости, наблюдатель потерян и поднимается выше как осиротевший держатель. Отсутствие результата чтения не доказывает отсутствия обязательства. предложено

Работающий транспорт уже показывает живость через `#status` и `a2a who`: устаревший сигнал, испорченную запись, часы в будущем и возраст запросов без ответа. Возможность разбудить среду, наличие хуков и факт ответа представлены раздельно. Киперу предлагается добавить долговременные курсоры входов, сроки следующего наблюдения и ссылки на основания оценок, чтобы после перезапуска продолжать с известного места. работает сегодня предложено

Свидетельства образуют лестницу, и соседняя ступень не доказывает следующую. Предложено сохранять верхний подтверждённый уровень; невидимое снаружи остаётся неизвестным. предложено

Уровень	Какой факт установлен	Чего он ещё не доказывает
Сохранено	Сервер принял сообщение, есть событие	Доставку получателю
Доставлено	Сообщение дошло до почтового ящика или принят звонок	Обработку моделью
Прочитано	Получатель обработал сообщение, есть квитанция	Выполнение работы
Сделано	Есть ответ, событие доски, коммит или событие исполнения	Содержательную правильность результата

Первое действие и исполнение требуют уровня «сделано». Лестница наблюдения должна охватывать и кипера: кипер продукта смотрит за оркестраторами, мета-кипер — за киперами, человек видит мета-кипера через тревоги и вопросы к владельцу. Иначе падение самого писателя сделало бы оста-

новку сроков неотличимой от отсутствия проблем. Это предлагаемая организация наблюдения, а не доказанная непрерывность службы. предложено

9.11 Швы с транспортом и платформой

Доска предоставляет свидетельства и не ведёт второй журнал обязательств — это выбранная граница. Транспорту предлагается передавать пометки вопросов, полный `reply-by`, ответы с проверенным отправителем и чтение вопросов без ответа. Решение об эскалации остаётся за журналом. Для первого изолированного среза предложены адаптеры транспорта и доски с теми же интерфейсами, что у будущих настоящих подключений; они не доказывают работу сквозной доставки. Нормативный формат этих сообщений ещё не заморожен; существующая договорённость о вопросах не означает, что все её поля уже проверяются программой. решено предложено

Вопрос без ответа и открытое обязательство предлагается считать разными фактами. Транспорт может вывести первый из истории сообщений без зависимости от кипера. Ответ «доставлю завтра» закрывает вопрос о планах, но не исполняет доставку. Разделение транспортных таймеров и решений журнала, лёгких советов и эскалации к человеку, а также квитанций нескольких носителей пока требует окончательного согласования. предложено открыто

На платформе сообщения должны идти через мост и почтовый ящик, а короткий звонок — через хост сессий. Хост описан спецификацией, мост остаётся архитектурным предложением. Поэтому будущая доставка во ввод агента не повышает статус нынешнего пробуждения. Подпись записей журнала также остаётся будущим швом: ещё не определены подписывающий модуль и ключ. Механизм ключника разобран в главе «Ключи и ключник». спецификация предложено открыто

Команда `keeper up` предложена как единая идемпотентная точка восстановления: поднять хост, восстановить кипера, затем корневых оркестраторов по реестру, доставить накопившиеся звонки и сообщить человеку результат. Остановленное человеком остаётся остановленным; дочернюю работу восстанавливают сами корневые оркестраторы. Физические запуски при этом должны идти через диспетчер. Кто инициирует подъём постоянной команды до входа человека и под какой учёткой — ещё открыто. предложено открыто

Проверять ядро предлагается на положительных сценариях рядом с запрещёнными исходами: удачная передача и передача без первого действия; корректный полный срок и время без даты; ответ родителя и молчание без выдачи прав; восстановление ресурса и поломка без держателя. Падения между зависимыми записями, повторные входы и поздние квитанции должны сохранять держателя и единственность шага. Это программа проверки предлагаемого ядра; результатов её исполнения пока нет. предложено

10 Swarm Curator

Swarm Curator — менеджер окружения агентов. Он готовит инструкции, скиллы, настройки и подключения MCP, с которыми агент начинает работу. Его единица сборки — профиль, а результат для конкретной агентской среды — управляемый дом. Это позволяет отделить содержание окружения от команды запуска и от живой сессии. Установка скиллов, профилей и сборка управляемых домов уже работают; новые интерфейсы снимков и подготовки логина выбраны, но ещё не выпущены в описанном состоянии. работает сегодня решено

Проблема возникает, когда несколько компонентов пытаются готовить один дом. Менеджер профиля положил инструкции, хост создал свою папку только с конфигурацией, лаунчер угадал наличие контекста по одному файлу — и агент получил окружение, отличное от заявленного. Выбрана единая граница: Swarm Curator владеет содержимым домов и правилами его проверки; остальные компоненты получают результат через его интерфейс. решено

10.1 Скилл, профиль и оверлей

Скилл — пакет с `SKILL.md` и необязательными каталогами вспомогательных материалов: `agents/`, `references/`, `.skill_triggers/`, `assets/`, `templates/`, `examples/`, `data/`. К агенту попадают предусмотренные корни пакета, а не произвольное содержимое исходного репозитория. Проект объявляет скиллы в `Skillfile.json`, Swarm Curator устанавливает их и фиксирует разрешённые версии в lock-файле. Такой пакет можно воспроизвести без повторного выбора последней версии при каждом запуске. работает сегодня

Команды скилла становятся глобальными шимами с одним владельцем на имя и собираются из проверенного исходника. Режим зависимости `full` даёт сессии контекст скилла, `runtime` — только его команды. Например, наличие утилиты в `PATH` не требует, чтобы любой агент получал инструкции роли, которая этой утилитой пользуется. Это разные способы включения одного пакета в окружение, а не два независимых механизма установки. работает сегодня

Профиль объединяет корневой контекстный пакет, его `Skillfile.json` и объявления MCP, закреплённые в lock. Он устанавливается из git по тегу, диапазону или ревизии, в том числе из подпапки. Без версии выбирается последняя через `--range latest`. Для каждой пары «профиль × агентская среда» собирается отдельный дом, на который указывают `CLAUDE_CONFIG_DIR`, `CODEX_HOME` или переменные XDG. работает сегодня

Профили ролей выбраны вместо идеи заранее разложенных по проекту «спящих» скиллов: исполнитель начинает с профилем своей роли. Профиль ревьювера и профиль продьюсера могут использовать общие команды, но давать разные инструкции. При этом сам факт установки профиля не выдаёт системных полномочий на запуск другого пользователя или доступ к чужим секретам; эти границы обеспечиваются механизмами рантайма из глав «Пользователь ОС на агента и сеть агента» и «Гранты и полномочия». решено

Оверлей добавляет машинные или операторские изменения к профилю. Работающий механизм принимает git-репозитории либо папки с весами, объявленные только в конфигурации машины. Для процессов предложен порядок: профиль пакета, оверлей процесса, затем оверлей оператора с персоной и памятью; закреплённые модули пакета убрать нельзя. Существование оверлеев уже подтверждено реализацией, а эта композиция для процессов остаётся предложением. работает сегодня предложено

Версии, lock и оверлеи отвечают на разные вопросы. Версия выбирает пакет, lock закрепляет разрешённое содержимое, машинная конфигурация сообщает, какие местные дополнения следует применить. Репозиторий проекта не становится владельцем машинной политики только потому, что содержит `Skillfile.json`. Это особенно существенно для MCP и хранилища логина: декларация зависимости не может сама разрешить передачу секрета новому получателю. решено предложено

10.2 Аудит и границы установки

`curator audit` проверяет пакет и публикует подписанную запись, связанную с хэшем его содержимого. Копия из зеркала с тем же содержимым совпадает с записью по этому хэшу. Это даёт основание сопоставлять аудит с конкретным артефактом, а не только с названием скилла. Сам сервис реестра с журналом прозрачности ещё находится в реализации и не имеет выпущенного релиза. работает сегодня

Политика допуска самостоятельной установки проверенных скиллов выбрана; неизвестные источники по умолчанию отвергаются, рекомендованный путь для нового пакета — аудит. Ускоренная установка по подписанной записи без долгого локального аудита и подписанный установщик предложены как следующий путь. Подпись записи связывает проверку с артефактом, но скомпрометированный установщик мог бы игнорировать проверку, поэтому доверие к нему остаётся отдельной предпосылкой. решено предложено

Например, зеркало отдаёт неизменный пакет со знакомым хэшем — запись аудита применима к нему. Если в скрипте пакета изменился байт, прежнее совпадение уже не установлено одним названием или той же папкой. Это объясняет, зачем аудит привязан к содержимому. работает сегодня

Речь здесь об аудите скиллов Swarm Curator. Аттестации личности и репутации в Swarm Chain — отдельное предлагаемое направление, описанное в главе «Аттестации и переносимая репутация». Совпадение слова «аттестация» не превращает аудит пакета в запись о личности агента. предложено

10.3 Один владелец дома

Дом агента — папка конфигурации, которую конкретная среда читает при старте. В ней собраны инструкции, скиллы, подключения MCP и настройки. Но среда также пишет туда собственные данные: историю сессий, trust и approvals. Поэтому управляемый дом не является неизменным архивом: `env resolve --repair`, смена профиля и синхронизация скиллов могут переписать его на месте, пока сессия ещё работает. решено

Ответ `curator env resolve <агент> --profile P` называется fragment: это данные о доме и окружении запуска, разрешениях, каналах MCP и системного промпта, дайджесте lock. Для Muse в него решено добавить явное поле `root_context`, чтобы лаунчер узнавал о выданном корневом контексте из интерфейса, а не по наличию обычного файла. Проверка файла может не узнать symlink и ошибочно подмешать другой контекст. решено

Распределение обязанностей выбрано следующим. решено

Компонент	Собственная ответственность	Граница
Swarm Curator	Содержимое дома, проверка, ремонт, fragment, готовность логина и снимки	Не ведёт живую сессию
Лаунчер `agents-management`	Команда запуска и окружение из fragment и плана	Не угадывает раскладку дома и не содержит второго менеджера профилей
Доска и хост сессий	Приватная папка сессии, политика инструментов, trust, ввод и возобновление	Не заводят собственный список файлов профиля и правила их проверки

Например, если хост записал в историю `CLAUDE_CONFIG_DIR`, но не передал его процессу, отчёт о профиле не доказывает, что Claude прочитал нужный дом. Если приватная папка Codex содержит только конфигурацию и ссылку на логин, это не означает, что туда попали инструкции и скиллы. Такие нарушения границы обнаружены в существующем пути запуска; выбранные исправления должны передавать окружение и содержимое, а не только заявлять их наличие. решено

Сборка команды запуска тоже должна быть одной: её владельцем выбран `agents-management`. Иначе два вызывающих начнут по-разному добавлять флаги, и один забытый строгий канал MCP позволит дочернему агенту увидеть лишние серверы. `curator run` уже соединяет окружение Swarm Curator с планом лаунчера. Само управление долгой сессией относится к хосту, который разобран в главе «Запуск агента и хост сессий». решено работает сегодня

10.4 Снимок для живой сессии

Живая сессия должна получать собственный снимок, а не ссылки на изменяемый дом профиля. Временный путь для Codex выбран узким: скопировать `AGENTS.md`, `skills/` и слой MCP обычными файлами между двумя `env resolve` с одинаковыми дайджестами fragment и lock, затем побайтно сверить копию с источником. При расхождении допускаются до трёх попыток, после чего запуск отказывает до создания PTY. решено

Файл `auth.json` в этом временном пути остаётся ссылкой на нативный файл и не копируется. При attach, resume и восстановлении записанные дайджесты fragment и lock сравниваются со свежим

resolve. Это ограниченный переходный механизм, а не разрешение хосту развивать собственный формат дома или перечень правил его целостности. Смысл повторного resolve — заметить переписывание источника во время копирования. решено

Целевой интерфейс — `curator env materialize --into <dir>`. Swarm Curator пишет в папку вызывающего проверенные обычные файлы и манифест с дайджестами. После его появления временное копирование у хоста должно исчезнуть. Снимки выбраны для следующего выпуска Swarm Curator; на 16 октября 2026 года установленный rc.6 ещё не даёт оснований считать этот интерфейс выпущенным. решено

Для отдельного пользователя ОС выбран тот же принцип. Вызывающий материализует снимок, план покрывает дайджест архива, диспетчер переносит архив, не зная расположения домов Swarm Curator. `curator-run exec` под учёткой агента сверяет дайджест, разворачивает снимок только для чтения и выставляет конфигурационные переменные. Секрет не должен оказываться в плане или fragment только потому, что вместе с ними переносится окружение. решено

Это гарантия соответствия конкретному снимку при доверии к материализации, проверке дайджеста и переносу. Она не означает неизменность живого исходного дома и не доказывает доступность логина из любого процесса. Например, одинаковые файлы Claude в двух видах сессий ещё не дают одинакового доступа к Keychain. Дом, источник аутентификации и хранилище логина разделены; свойства инструментов разобраны в главе «Учётные данные и логины». решено

10.5 Общий и отдельный логин

Swarm Curator применяет режимы `shared` и `isolated` к управляемым домам. Они уже работают, но их допустимость зависит от агентской среды и хранилища. Устройство режимов, ограничения адаптеров и выбранный порядок разрешения учётных данных разобраны в главе «Учётные данные и логины». работает сегодня

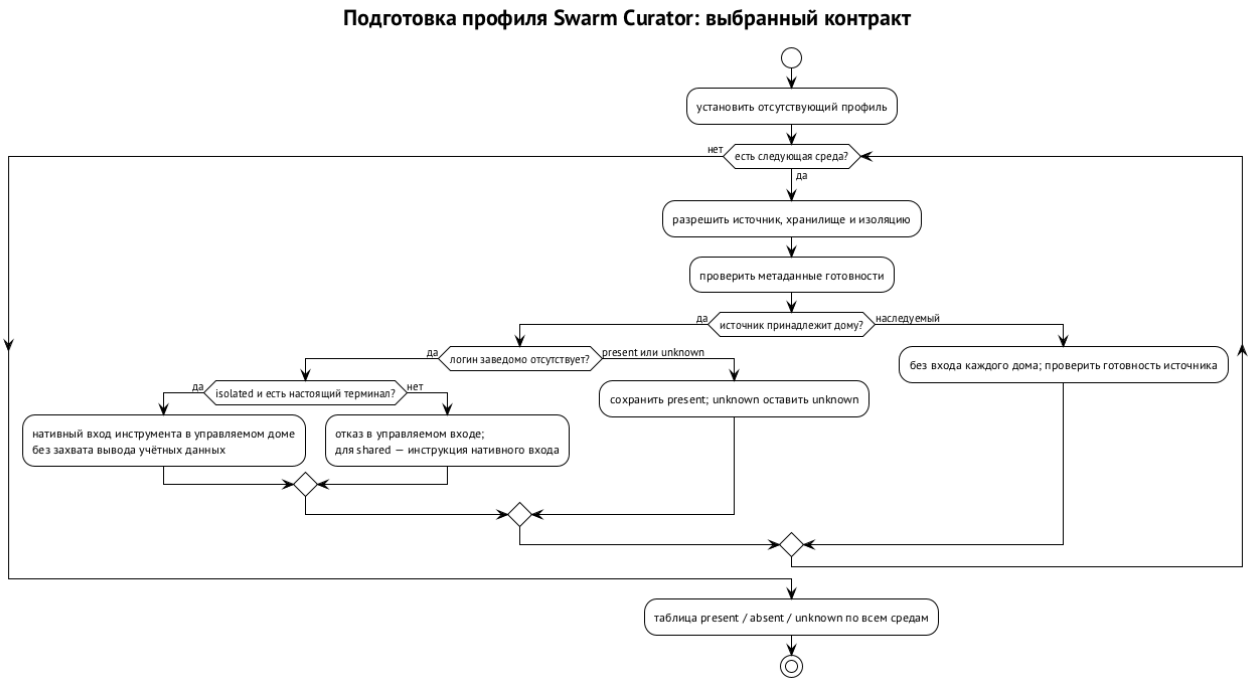
10.6 Готовность перед запуском

В `env status` выбрана строка `login: present | absent | unknown`. Это сведения о готовности, без байтов секрета. `unknown` не равен `absent`: невозможность установить состояние не разрешает молча начать новый вход. В частности, сообщение о недоступном хранилище нельзя заменить выводом «логина нет» только ради автоматической подготовки профиля. решено

Команда `curator env login <env> --profile P` должна запускать нативную команду входа инструмента в нужном доме, из настоящего терминала, без захвата вывода учётных данных. Она допустима для `isolated` с собственным источником аутентификации. В `shared` следует отказ и инструкция войти в нативный дом, чтобы у общего логина оставался один владелец. Это контракт следующего выпуска, а не команда, проверенная на rc.6. решено

`curator profile prepare P` должна установить отсутствующий профиль, перечислить участвующие среды, разрешить окружения и войти только там, где собственный логин заведомо отсутствует. Уже присутствующий не создаётся заново; наследуемый источник не требует входа каждого дома. Результат — таблица по всем средам. Если Claude готов, а для Muse состояние неизвестно, успех одной строки не должен скрывать неизвестность другой. решено

Для Claude уже существует `claude auth status --json`, показывающий `loggedIn` и `authMethod` без запроса к модели. Проверки готовности остальных адаптеров ещё требуют квалификации. Поэтому строка `present` не должна выводиться из общего предположения, что все агенты одинаково хранят логины. Подробные команды диагностики и ограничения хранилищ приведены в главе «Учётные данные и логины». работает сегодня открыто



В выбранной подготовке профиля присутствующий логин сохраняется, неизвестный не считается отсутствующим, а вход запускается только для допустимого отдельного дома.

10.7 MCP как зависимость скилла

MCP подключает агенту внешние инструменты. Для Swarm Curator важно различать требование скилла, декларацию пакета сервера, запись конфигурации клиента и доступность сервиса. Сервер может прийти из нативного файла, плагина клиента, облачного коннектора или управляемого профиля. Один и тот же список имён на диске не описывает все эти пути и не доказывает, что конкретный запуск получит нужный сервер. **предложено**

Сегодня манифесты поддерживают `dependencies.mcp_servers`: имя сервера с `hint`, необязательными `transport` и `required_in`. Менеджер проверяет требование чтением нативных конфигураций: ищет имя, не считает выключенные записи, предупреждает об отсутствующей команде и конфигурации только в проекте. Это уже работающая проверка по имени. Она не устанавливает идентичность сервиса и не проверяет вход к нему. **работает сегодня**

Предложенная привязка по имени сохраняет этот манифест и добавляет защищённое свидетельство оператора для выбранного запуска. Запись покрывает точные перечисленные требования конкретных пакетов скиллов без масок, указывает потребителя, агентскую среду, запуск и селектор провайдера. Селектор выбирает нативную запись, член управляемого профиля либо непрозрачный коннектор. Наблюдаемое и заявленное оператором помечаются раздельно. **предложено**

Например, скилл требует сервер с определённым именем, но оператор использует облачный коннектор без файла на диске. Его заявление может связать требование с этим коннектором, но статус остаётся «заявлено оператором». Новый скилл или расширенная область прежней записью не покрываются. Данные проекта могут попросить привязку, однако не могут её разрешить. Само связывание имени не даёт согласия на установку кода или маршрутизацию токена. **предложено**

Свидетельство предлагается вычислять отдельно для среды, области и типа запуска. Выключение на более высоком уровне должно переживать разрешение приоритетов; нижний селектор не обходит его. Строгое исключение нативных MCP у Claude доказывается реально применённым непустым каналом, а не отсутствующим файлом для пустого набора. `any` требует одного положительного свидетельства, `all` — свидетельства для каждой выбранной цели. Неизвестность требование не удовлетворяет. **предложено**

Отсутствие имени в файле не опровергает непрозрачный коннектор, но известное отсутствие поддержки в данном запуске нельзя исправить заявлением. Доверие к нему заканчивается при отзыве оператором, изменении области или идентичности требования либо полном отрицательном свидетельстве для того же провайдера и запуска. Предлагается явное включение проверки при установке скилла и отдельная перепроверка; новый неявный фатальный шлюз в `env resolve` не добавляется. предложено

Граница результата формулируется буквально: «настроен по имени; идентичность провайдера и аутентификация не проверены». Чужой сервис под нужным именем может пройти проверку имени, поэтому нельзя заменять её словом «подключён». Нативные файлы при чтении могут содержать секреты; обещание «не управляет секретами» не означает «никогда не читает содержащие их байты». Диагностика должна оставаться ограниченными метаданными, без сырого окружения, заголовков и фрагментов разбора. предложено

10.8 MCP из декларации скилла

Другое выбранное направление требует большего: скилл должен сообщать идентичность сервера и способ входа, Swarm Curator — максимально готовить конфигурацию и для профиля, и для запуска из чекаута, оператор — иметь возможность подмены. Это требование владельца. Конкретные поля, план установки и адаптеры пока являются предложением; их нельзя смешивать с работающей проверкой по имени или считать принятым продолжением всех правил привязки. решено предложено

В предлагаемой новой версии манифеста `server` выбирает ровно одно: точный HTTPS `endpoint` либо `package` объявления `agent-mcp.json`. Обычный HTTP допускается только для `localhost`, параметры запроса в адресе запрещены. `auth` задаёт вид входа и имена переменных или заголовков, без значений. Старое закрытое определение требования в выпущенных версиях не расширяется незаметно; скиллы без новой декларации продолжают проверяться по имени. предложено

Предложены следующие виды входа. Это словарь дизайна, а не таблица квалифицированной поддержки всех клиентов. предложено

Вид	Декларация	Кто получает значение
`none`	Сервер без входа	Секрет не требуется
`oauth`	Вход через нативный клиент	Клиент и его хранилище
`bearer_env`	Имя переменной для Bearer	Клиент при запуске
`header_env`	Имя заголовка и переменной	Клиент при запуске
`env`	Имена переменных локального сервера	Процесс сервера

Литеральные токены, пароли и значения заголовков должны отвергаться при разборе. Пакет из диапазона разрешается до точного коммита и дайджеста в lock. Одинаковые требования объединяются, разные серверы под одним именем вызывают конфликт. Например, два скилла не могут молча выбрать разные получатели одной переменной только потому, что оба назвали сервер одинаково. Текущее ограничение резолвера также требует разных имён у скилла и MCP-пакета. предложено

Подмена принадлежит оператору: алиас на уже настроенный сервер, другой адрес или пакет, либо сознательное отключение. По умолчанию она охватывает требования с этим именем, а поле `skill` сужает область. Репозиторий может предложить подмену, но не применить её сам. Иначе сторонний чекаут направил бы переменную с токеном на свой адрес. Подмена должна быть видна в lock и статусе. предложено

План показывает каждую цель отдельно: файл, сервер, адрес или команду, имена переменных и получателей. Согласие требуется для исполнения локального кода и передачи переменной на адрес. Повтор того же плана не спрашивает заново; новый адрес, переменная или получатель требуют нового

согласия. Для запуска без человека подходит заранее одобренная машинная политика, но не самоодобрение внутри скилла или проекта. предложено

Нужно учитывать совокупную экспозицию запуска. Расширить её можно и новым именем переменной, и новым получателем прежнего набора. Если получатели наблюдаемы не полностью, требуется явное признание общего домена доверия с указанной границей видимости; изоляция между MCP-серверами не возникает от самого плана. Различие между согласием перед любой записью и правилом установки удалённого сервера без переменных пока не сведено в единый нормативный контракт. предложено открыто

10.9 Два пути записи конфигурации

В управляемом профиле MCP уже материализуется через отдельный канал: `--mcp-config` для Claude, `-p curator-mcp` для Codex, `OPENCODE_CONFIG` для OpenCode. Для новой декларации предлагается использовать этот же путь вместе с `requires.mcp` профиля. Добавление имён переменных заголовков к разрешённой передаче ещё требует реализации; существование канала не доказывает, что сегодня он поддерживает все предложенные виды входа. спецификация предложено

Для чекаута предложен писатель нативных файлов с владением по записи. Он меняет только созданные им записи, перечисленные в защищённом реестре вне проекта. Чужая одноимённая запись вызывает конфликт даже при равных байтах; оператор разрешает его алиасом или явным принятием. Остальные байты и комментарии сохраняются. Если синтаксис нельзя сохранить, файл отклоняется до записи. предложено

Новые нативные файлы по умолчанию добавляются в `.gitignore`, отслеживаемые правятся с видимым git diff. Глобальные пользовательские файлы требуют явной команды. Управляемый писатель не работает в нативном доме, нативный — в управляемом. Предлагается единый контракт такого писателя, чтобы одинаковые требования сохранения чужих записей не реализовывались двумя расходящимися способами. Владелец этого общего контракта ещё не определён. предложено открыто

Адаптер должен записать ссылку на переменную в синтаксисе клиента, а не прочитать значение и вставить его в файл. Поддержку такого синтаксиса требуется подтвердить на точных версиях. Если клиент не умеет вид входа, результат — «не поддерживается для этого клиента». Недостаточно сгенерировать правдоподобный конфигурационный файл и назвать это работающим подключением. предложено открыто

OAuth остаётся у клиента. Предлагается по явной просьбе запускать его собственную команду входа в правильном доме, чтобы токен оказался в хранилище соответствующего запуска. Проверок здоровья по умолчанию нет: даже тестовый запрос может отправить секрет серверу. Настройка и вход показываются отдельно; наличие переменной означает только наличие имени, а не проверку значения или прав у сервиса. предложено

Например, скилл объявил сервер дизайна с OAuth. Swarm Curator подготовил записи Claude и Codex, затем оператор вошёл каждым клиентом. Тот же скилл в профиле должен проходить вход в доме профиля. Если оператор заменяет сервер корпоративным прокси с переменной токена, возникает новый план и новое согласие на получателя. Этот сквозной путь пока предложен и не квалифицирован на клиентах. предложено

10.10 Проверяемая граница результата

Установка профиля, разрешение окружения и запуск — три наблюдаемых результата. Установленный пакет ещё не доказывает, что конкретная сессия получила его инструкции. Успешный resolve ещё не доказывает, что вызывающий выставил переменные процессу. Проверенная копия дома ещё не доказывает доступность OAuth в используемом виде сессии. Поэтому хост должен использовать выданный fragment и снимок по контракту, а статус должен называть именно проверенный факт. решено

Например, оператор обновил профиль во время работы Codex. Дайджест исходного дома изменился, но уже созданный снимок сессии не должен незаметно превратиться в смесь старых инструкций и новых скиллов. Выбранная временная процедура ловит изменение между resolve и побайтной сверкой; целевая материализация переносит ответственность за проверенный снимок к Swarm Curator. При attach и resume свежий resolve нужен для сопоставления с записанным состоянием, а не для молчаливого признания любого дома подходящим. решено

Для MCP требуется столь же узкая формулировка. Объявление скилла, операторская подмена, согласие на план, запись конфигурации и вход клиента не сливаются в один зелёный признак. У сервера может быть корректная конфигурация и отсутствующий вход; у непрозрачного коннектора — только заявление оператора. Известное отключение или конфликт чужой записи должны оставаться видимыми даже рядом с успешно установленными серверами того же профиля. предложено

В варианте привязки по имени допускается инертный `agent-mcp.json` рядом со скиллом как рекомендация. Он не выдаёт полномочий на установку. Подсказка должна указывать git-источник, ревизию, каталог и имя; путь на диске сам по себе не становится основанием принятия. Независимое закрепление скилла и MCP-пакета сохраняется, а испорченные необязательные метаданные не должны ломать установку обычного скилла. Это необязательная надстройка предложения, а не ещё одна выпущенная схема. предложено

10.11 Что ещё предстоит установить

Оба предложения MCP сохраняют важную границу: название и конфигурация не доказывают доступность сервиса, а пароль не должен становиться содержимым переносимого профиля. Но различаются степень автоматизации, необходимость закреплённой идентичности и область изменений схемы. Для реализации нужны нормативные правила, положительные и отрицательные векторы, таблица приоритетов адаптеров и квалификация клиентских версий. Архитектурная оценка этих вариантов не была испытанием подключений. предложено

Открыты хранение OAuth при подменённом доме, полный состав получателей окружения MCP и поддержка принудительных хранилищ отдельных клиентов. Локальные Windows-серверы через `.cmd` и `.bat` в предложении отклоняются до отдельной квалификации. Снимок дома и строка готовности логина выбраны для следующего выпуска, но не закрывают эти вопросы автоматически. Внешнему вызывающему нужен проверяемый результат Swarm Curator с точным пределом, а не ещё один менеджер дома рядом с ним. открыто

11 Учётные данные и логины

Учётные данные дают агентской среде доступ к провайдеру модели: через подписку, ключ API или иной поддержанный канал. Логин — сохранённый результат входа, а хранилище — место, откуда инструмент его получает. Дом конфигурации и доступ к хранилищу не тождественны: одинаковые файлы могут давать разный результат в процессах с разным доступом к Keychain. Ниже зафиксированы наблюдения октября 2026 года, а не обещание поведения любых будущих версий. работает сегодня

Выбрана модель «один человек — одна подписка», подставляемая только в его запуски. Долгоживущие учётные данные допустимы для первой версии рантайма. Брокер должен решать, кому и на какой запуск их выдавать, но токен в окружении остаётся читаемым самой средой. Неизвлекаемость секрета — отдельная будущая задача; ограниченная аренда не превращает годовой токен провайдера в короткоживущий. решено

11.1 Где инструменты хранят вход

Наблюдения относятся к Claude Code 2.1.299 на macOS 15, Codex CLI 0.153.2 и Muse 1.4.8 на macOS. Для Muse среда опытов уточнена как macOS 27; переносить это уточнение на Claude нельзя. Проверки выполнялись в отдельных тестовых домах и пользователях ОС. Они различали успешный вход, отказ сохранения и потерю видимости файла, но не охватывали все политики ОС и варианты обновления токенов. работает сегодня

11.1.1 Claude Code

При доступе к login Keychain Claude сохраняет вход отдельно для `CLAUDE_CONFIG_DIR`. Имя элемента имеет вид `Claude Code-credentials-<хэш>`; хэш — первые восемь шестнадцатеричных символов SHA-256 от значения каталога. Файл учётных данных при этом не появляется. Новый управляемый дом требует собственного входа и не меняет логин нативного дома. Такой результат наблюдался у desktop-сессии и её потомков. работает сегодня

Происхождение процесса имеет значение. tmux или демон, запущенный с рабочего стола, наследует доступ той сессии к Keychain. Отсутствие окна само по себе не делает процесс эквивалентом ssh. В исследованной ssh-сессии без доступа к существующему login Keychain вход сохранялся в `<home>/.credentials.json` с правами 0600. После скрытия тестового файла Claude сообщал «Not logged in». работает сегодня

Desktop- и ssh-хранилища в опыте не видели друг друга: дом с desktop-логином показывал `loggedIn=false` из ssh, и наоборот. Другой случай — пользователь ОС совсем без login Keychain, запущенный в фоновой сессии launchd. Там вход завершался «Couldn’t save your login» без файлового fallback. Поэтому нельзя рекомендовать повторный вход как универсальное исправление любой недоступности Keychain. работает сегодня

`claude setup-token` выдаёт токен, который передаётся через `CLAUDE_CODE_OAUTH_TOKEN`. В опыте он имел приоритет над логином дома, давал `authMethod=oauth_token` и не записывался в дом или Keychain. Канал подходит для inference, но Remote Control его отвергает. Сам механизм работает; автоматическая доставка через Swarm Curator остаётся предложением. На Linux Claude использует файловый логин `<home>/.credentials.json`. работает сегодня предложено

11.1.2 Codex CLI

У Codex сначала нужно установить выбранное хранилище. Ключ `cli_auth_credentials_store` в `config.toml` задаёт режим; при отсутствии ключа используется `file` и `$CODEX_HOME/auth.json`. Значения `keyring` и `auto` используют системное хранилище. Поэтому соседний `auth.json` не доказывает, что именно его читает текущий процесс. работает сегодня

Переопределение `-c cli_auth_credentials_store='"file"'` выбирает файловое хранение для одного вызова и имеет приоритет над файлом конфигурации. Вложенные кавычки сохраняют строковое значение TOML. Лаунчер может применить выбор без переписывания принадлежащего Codex файла. Но общий транспорт настроек профиля не даёт самому профилю права выбирать, где окажутся токены: это отдельное решение владельца машины. работает сегодня решено

11.1.3 Muse

Muse по умолчанию использует один глобальный элемент Keychain `ai.meta.dev.credentials`, а `auth.json` служит fallback. Разные дома при таком режиме не означают разные логины. В опыте без Keychain OAuth проходил, но сохранение завершалось ошибкой `keychain write failed` с exit 1; автоматический переход к файловому логину не произошёл. Название fallback не является гарантией его срабатывания во всех условиях. работает сегодня

Явное `TBH_CREDENTIAL_BACKEND=file` сохраняло вход в `$XDG_CONFIG_HOME/muse/auth.json` с правами 0600. Последующие запуски использовали файл без обращения к Keychain, после его скрытия учётные данные отсутствовали. Ссылка на файл другого дома также работала. Но записи при обновлении то-

кена, атомарная замена, сохранение ссылки и конкуренция нескольких процессов не измерены. работает сегодня открыто

Выпущенный через вход ключ API, переданный Muse через `exec --api-key-stdin`, расходовал подписку; `META_API_KEY` оставлял хранилище входа нетронутым. Это наблюдение уточняет предложенный канал подписки Muse. Оно не закрывает остальные вопросы ротации, сроков и повторного выпуска и не доказывает готовность всей интеграции с брокером. работает сегодня открыто

11.2 Источник, хранилище и изоляция

Swarm Curator разделяет три решения. Источник сообщает, чей вход используется: собственный логин дома или наследуемая аутентификация. Хранилище определяет нативный механизм, файл или Keychain. Изоляция определяет, связан ли файл дома с общим нативным файлом (`shared`) либо дому нужен собственный логин (`isolated`). Порядок выбран именно такой: источник, хранилище, допустимость изоляции, затем стратегия ссылки. решено

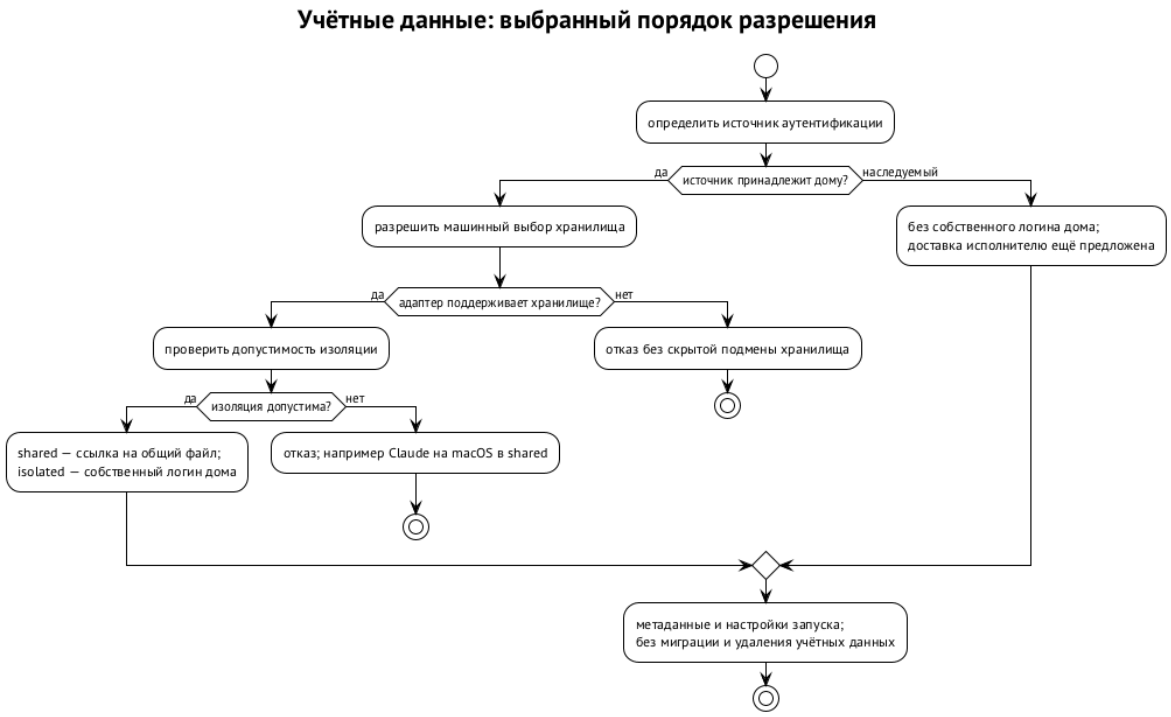
`shared` и `isolated` уже работают в rc.6. Для Claude на macOS `shared` отклоняется с `environment_shared_unsupported`; файловый путь ssh не создаёт исключения в адаптере. На Linux файловое хранение допускает общий логин. У Codex ссылка подходит к нативному файловому входу, но не заменяет выяснение настройки keyring. У Muse отдельные папки не изолируют глобальный Keychain; нужен явный файловый backend. работает сегодня

Единственный выбранный селектор хранилища — машинная настройка `credential_store.<profile>.<env-id>` со значениями `native | file | keychain`, по умолчанию `native`. Пакеты, оверлеи, lock и репозитории не выбирают её. Смена значения не мигрирует и не удаляет старый логин: он может остаться в прежнем хранилище и стать невидимым для нового запуска. Интерфейс выбран для rc.7. решено

Контракт адаптеров ограничен их квалифицированными возможностями. `native` означает нативный выбор инструмента, а не обязательный Keychain. решено

Среда	`file`	`keychain`
Codex	Override `cli_auth_credentials_store` при входе и запуске	`keyring` только на квалифицированном macOS backend; иначе отказ
Muse	`TBH_CREDENTIAL_BACKEND=file` при входе и запуске	Отказ до квалификации принудительного интерфейса
Claude	Отказ: нет интерфейса принудительного выбора	Отказ: нет интерфейса принудительного выбора

Например, изменение настройки Claude на `file` не является способом исправить ssh-логин: Swarm Curator не имеет квалифицированного force-file интерфейса. Разрешённый нативный файловый путь и возможность принудительно выбрать его — разные свойства. Содержимое домов, снимки и таблица `login: present | absent | unknown` описаны в главе «Swarm Curator»; неизвестность там не считается отсутствием входа. решено



Выбранный порядок разрешения учётных данных проверяет источник, хранилище и допустимость изоляции до выбора ссылки.

11.3 Наследуемый setup-token

Предложено один раз завести токен в защищённое хранилище пользователя под узлом аутентификации. Узел содержит ссылки и метаданные, а не сам материал в переносимом профиле. Дом получает непрозрачную ссылку `source_ref`; окончательный исполнитель извлекает значение и передаёт Claude через `CLAUDE_CODE_OAUTH_TOKEN`. План и fragment не содержат токена. Дома с таким наследуемым источником обходятся без собственного входа. **предложено**

Это решает зависимость inference от отдельного Keychain-элемента дома и происхождения desktop/ssh-процесса, но только при работающей доставке источника. Нативный env-token проверен; enrollment, защищённое хранилище и передача исполнителю в Swarm Curator ещё требуют нормативного текста и реализации. Здесь нет команды, которую можно было бы выдать за уже выпущенный интерфейс enrollment. **предложено**

В предлагаемой модели отсутствующий узел, нечитаемое хранилище или два источника одновременно приводят к отказу без запасного пути. Нельзя незаметно заменить недоступный наследуемый источник чужим входом дома. Для первой версии без явного источника выбран прежний собственный вход; конечное направление — обязательная строка источника для каждой пары профиля и среды с миграцией существующих домашних входов в явный `per-home`. **предложено** **решено**



В предлагаемой доставке setup-token план содержит только ссылку, а секрет извлекает окончательный исполнитель перед запуском Claude.

11.4 Брокер: одна подписка, аренды на запуски

Брокер учётных данных (`swarma-credential-broker`) описан как служба под отдельной учёткой с Unix-сокетом, без сетевого слушателя. Подписка заводится у него один раз, а запуск получает аренду — право на материал в пределах разрешённого запуска. Служба ещё не реализована. Её модель не копирует обновляемый логин в несколько независимых домов, а централизует выдачу и, где требуется, обновление. **спецификация**

Подключение подписки — расширяющее действие оператора с подтверждением точного содержимого через телефон или пароль в терминале; из сессии агента оно не принимается. Claude передаёт результат обычного `setup-token` через stdin. Для личного плана Codex брокер должен под своей учёткой создать свежий дом и запустить нативный вход по коду устройства; дублирующее подключение аккаунта отклоняется. Брокер не проходит вход провайдера вместо человека. **спецификация** **решено**

При выдаче вызывающий определяется ядром ОС по UID и поколению учётки. Исполнитель заново разрешает защищённую привязку: профиль, среда, аккаунт, канал, адрес провайдера, дайджест бинарника и поколение политики. Брокер под замком проверяет привязку, цепочку грантов и отзывы, покрытие прав, состояние аккаунта, квалификацию версии, одобренные адреса, бинарники и сеть. Нечитаемые отзывы означают отказ в выдаче, а не пустой список отзывов. **спецификация**

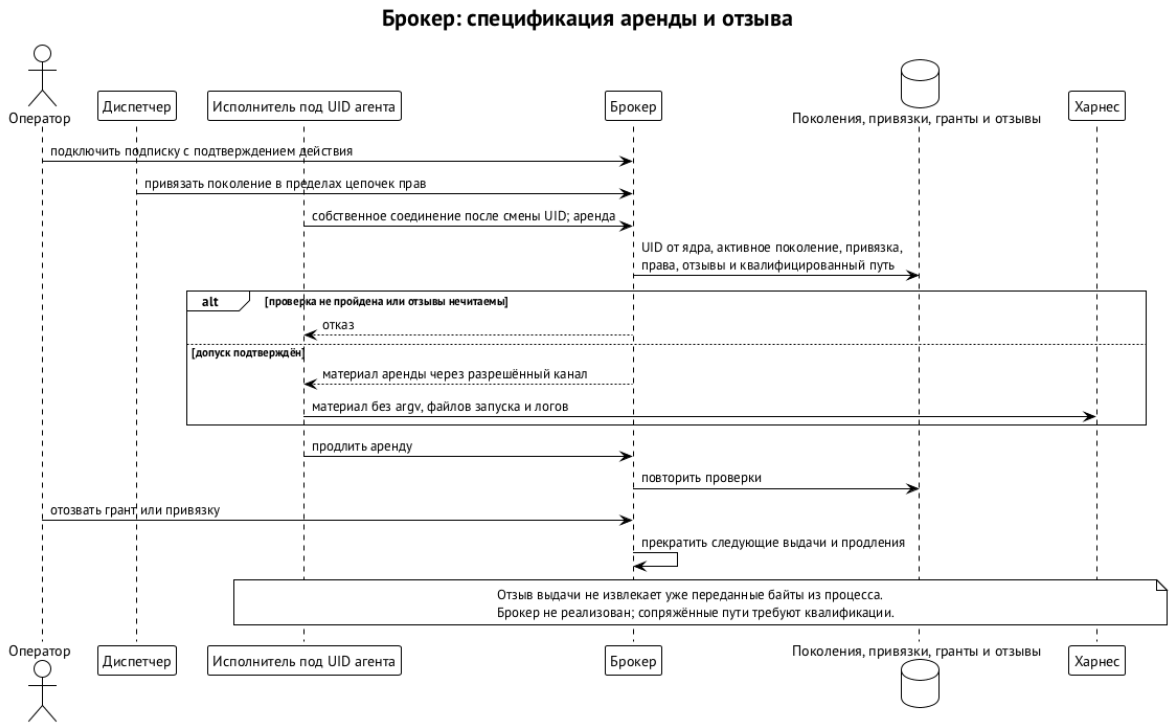
Материал передаётся среде через разрешённый канал: окружение, stdin или внешний токен Codex, без записи в argv, файлы запуска и логи. Соединения закрываются при exec и не наследуются агентом от диспетчера. Продление и освобождение доступны владельцу аренды, при продлении проверки повторяются. Наличие старой успешной аренды не заменяет проверку свежего отзыва или истёкшей привязки. **спецификация**

Первым исполнителем выбран `curator-run` под учёткой агента. План требует понимаемое исполнителем расширение `works.relux.curator.credential/1`, а способность подстановки обозначается `credential-injection/1` для проверенной версии среды. Непонимающий исполнитель отказывает. Доска и хост сессий не встраивают второго клиента брокера, а обращаются к диспетчеру. Источник вида `broker:<аккаунт>` настраивается владельцем машины. **решено**

У каналов разная жизнь аренды. В `env` исполнитель получает токен до `exec`, затем заменяется процессом среды: клиента для дальнейшего освобождения уже нет. Выход среды сам по себе не освобождает аренду; граница власти брокера — `authorized_until`, отзыв или отвязка. В канале Codex исполнитель остаётся рядом со своим app-server, продлевает аренду и вызывает `lease.release` в конце. Точный ранний выпуск аренды `env` и уборка остальных каналов ещё открыты. спецификация открыто

Отзыв сохраняется на диске до ответа и запрещает новые выдачи и продления. Он не стирает уже переданный токен: тот действует до собственного срока или отзыва у провайдера. Для Claude годовой токен не обновляется на месте; по истечении нужны новый токен и перезапуск. Восстановление брокера из резервной копии закрыто до сверки подтверждённой истории отзывов, чтобы старая копия не вернула утраченное право. спецификация

Сроки проверяются по водяному знаку времени. Истёкшая привязка непригодна сразу; удаление по сроку требует подтверждения реально прошедшего времени монотонными часами. Для постоянных агентов замена привязки должна снимать старую с концом её аренд и создавать новую с никогда не повторявшимся номером, без двух одновременно действующих привязок и без расширения первоначального запроса. решено спецификация



В спецификации брокера аренду получает сторона агента после проверки поколения, привязки, грантов и отзывов; продление повторяет проверки.

11.5 Почему Codex нужен один обновляющий

Копии обновляемого логина создают гонку: один процесс обновил токен, другой продолжил старым. Для личного плана Codex спецификация брокера предусматривает единственного владельца обновления под замком и внешние токены для `codex app-server`. В этом режиме `codex exec` не обслуживается, а `auth.json` в доме запуска отсутствует. Нативная ссылка `shared` остаётся отдельным существующим режимом Swarm Curator и не является архитектурой брокера. спецификация

При 401 исполнитель просит `lease.renew` и должен уложиться в дедлайн app-server, иначе ход завершается ошибкой без запасного входа. Проверка конкретной версии этого канала ещё открыта. Успешное чтение общего файла не доказывает корректность внешнего обновления и не разрешает несколь-

ко независимых писателей. Сам процесс провайдера также не обязан соблюдать замок брокера. спецификация открыто

Поэтому брокер должен сохранить намерение попытки до запуска обновления, удерживать процесс в границах службы ОС и установить судьбу прежнего писателя до нового запуска. Даже подтверждённая смерть процесса не доказывает, что провайдер не успел сменить токен. Если исход неизвестен, аккаунт остаётся недоступным или заводится заново; слепого повторения нет. Без этой гарантии канал Codex включать нельзя. спецификация

Для Muse приоритет подписки выбран, канал выпущенного ключа предложен, а полная квалификация ещё открыта. Для Business- и Enterprise-входов Codex описан отдельный токен доступа, но это не первый выпуск. Подстановка токена на расшифровывающем TLS-прокси остаётся исследованием: её не выбрали первой реализацией. Гарантии сети брокера относятся к главе «Пользователь ОС на агента и сеть агента»; заявленный профиль ещё не равен применённому принуждению. решено предложено открыто

11.6 Пределы выдачи и порядок включения

Брокерный путь начинается с одного защищённого запуска Claude на одной машине через цепочку пользователей ОС. Спецификация требует настоящий диспетчер, хранилище брокера, гранты, отзывы, привязку, аренду и исполнителя под учёткой агента. Только затем идёт канал Codex, затем принудительная сеть. Первый этап нельзя считать проверенным по отдельной успешной подстановке переменной из того же пользователя: переход между пользователями и проверка вызывающего входят в саму гарантию. спецификация

Положительному запуску должны соответствовать отказы чужому вызывающему, чужому аккаунту и просроченной привязке. Для канала Codex отдельно требуется, чтобы несколько запусков с отклонённым токеном приводили к одному обновлению, а падение брокера не создавало второго писателя. Это требования приёмки спецификации, а не результаты уже выполненных испытаний брокера. Квалификация версии app-server и её дедлайна остаётся обязательной до включения этого канала. спецификация открыто

Управление тоже разделено по полномочиям. Подключение аккаунта и выдача гранта — операторские действия с дополнительным подтверждением; отзыв остаётся операторским, но дополнительного подтверждения не требует. Чтение статуса, аккаунтов и привязок должно идти через наблюдателя без раскрытия токенов. Для дома с брокерным источником предложено показывать аккаунт, срок, доступность брокера и последний отказ. Такой статус не является каналом получения материала. решено предложено

Граница с MCP принципиальна: подставленный токен провайдера модели не должен наследоваться дочерними MCP-процессами. Предложенная защищённая привязка и зарезервированные имена окружения должны разделять этих получателей. Это дополнительное требование доставки, которое не возникает автоматически от обычной переменной среды. Удалённому агенту, в свою очередь, предложено пользоваться собственными учётными данными владельца удалённой машины; локальный брокер не становится их универсальным хранилищем. предложено

11.7 Диагностика и повторный вход

Ошибки предлагается отражать как события истечения, отказа и необходимости повторного входа, не выдумывая срок, если провайдер его не сообщил. Классы различают аутентификацию, права, квоту и неизвестность. Если 401 нельзя уверенно отличить от нехватки прав, результат остаётся неизвестным. Предложенная реакция — парковка и уведомление без бесконечных повторов; восстановление каждой среды требует отдельной проверки. предложено

Для «Not logged in» сначала нужно проверить версию, фактический дом и происхождение процесса. Проверка нативного дома не подтверждает готовность управляемого. У Claude полезны `loggedIn` и

`authMethod` из `claude auth status --json`; у Codex — выбранный store и override; у Muse — backend и ожидаемый файл или ссылка. Содержимое файла учётных данных для такой диагностики не нужно. работает сегодня

Кто целиком отвечает за повторный вход после разлогинивания, пока не определено. Предложенный путь: хост замечает отказ и паркует сессию, брокер запускает нативный вход, владелец подтверждает его, после сигнала сессия перезапускается. Это ещё не работающий процесс восстановления. Успешный старый логин не доказывает срок действия, возможность refresh или готовность нового запуска. открыто

11.8 Рецепты с указанным статусом

Для Claude на macOS в rc.6 документирован вход из того же вида сессии, в котором будет работать запуск. `P` — условное имя профиля. Команды выбирают дом и показывают только статус; токены не следует прикладывать к отчёту. работает сегодня

```
eval "$(curator env resolve claude_code --profile P --format shell)"
claude auth login
claude auth status --json
```

В ssh используется то же разрешение окружения внутри ssh, без предположения о видимости desktop-логина. Для пользователя совсем без Keychain измеренный нативный вход не сохранился; повтор команды не создаёт отсутствующий fallback. Setup-token может служить другим источником для inference, но его автоматическая доставка исполнителю ещё предложена. Нельзя выдавать этот вариант за уже проверенный сквозной запуск Swarm Runtime. работает сегодня предложено

После rc.7 для уже настроенного `isolated` Codex выбран следующий интерфейс. Первая команда выбирает хранилище, но не переключает изоляцию; в `shared` вторая должна отказать и направить к нативному входу. Это решение для следующего выпуска. решено

```
curator env config set credential_store.P.codex_cli file
curator env login codex_cli --profile P
```

Для Muse измерен файловый backend, но Swarm Curator ещё должен применять его согласованно при входе и запуске. Для всех сред выбрана предварительная `curator profile prepare P` с отдельной строкой готовности каждой. При наследуемой аутентификации проверяется доставка источника; собственный вход дома создавать не нужно. Неизвестность refresh, клиентских адаптеров и сквозной доставки сохраняется до проверки соответствующего пути, независимо от успеха соседней среды. решено предложено открыто

12 Удалённое рабочее место

Удалённый работник — агент, чья модель работает на машине другого владельца, а инструменты исполняются в среде проекта. Владельца, отдающего квоту своей подписки, называем донором. Проект не получает его машину в управление и не хранит его логин провайдера. Донор, в свою очередь, не получает доступ ко всей среде проекта. Такая схема позволяет принять помощь человека со свободной подпиской, не перенося к нему репозиторий вместе с окружением сборки. решено

Доверять приходится в обе стороны. Проект должен ограничить чужого агента, а донор — защитить собственные файлы от задач проекта. Эти границы различны: отдельный пользователь ОС в среде проекта защищает проект, но сам по себе ничего не запрещает процессу на машине донора. Поэтому дизайн отдельно описывает рабочее место, запуск модели у донора и канал между ними. На октябрь

2026 года это набор решений и черновых контрактов; закрытые схемы обмена ещё не опубликованы. предложено

12.1 Модель думает у донора, руки — у нас

Рабочее место — отдельный пользователь ОС на машине проекта: свой дом, файрвол и доступные на этой машине потолки ресурсов. Модель использует подписку донора. В среду проекта приходят вызовы инструментов и почта по одному SSH-соединению, которое открывает донор. Проект не открывает обратных соединений и не отправляет команды на машину донора. На стороне проекта действует та же граница пользователя ОС и файрвола, что описана в главе «Пользователь ОС на агента и сеть агента». решено

Например, проекту на iOS и Rust нужна одна подготовленная машина с окружением сборки. Несколько участников могут отдать ему свободную квоту, не воспроизводя окружение у себя. Каждому выделяется рабочее место, модели работают снаружи, а изменения собираются в общей среде. Это не добавляет ей вычислительной мощности: пропускная способность машины проекта остаётся ограничением. Донор отдаёт ёмкость модели, а не универсальный удалённый компьютер. решено

12.2 Чем защищена машина донора

Харнес — программа, которая ведёт агентский цикл и предлагает модели инструменты. На машине донора он запускается в запертом режиме `lockdown`: локальная оболочка, правка файлов и локальный веб-поиск модели не предлагаются. Доступен только путь через реле инструментов проекта. Само реле `swarma-remote-tools` не держит ключей и не имеет сети. Однако отсутствие инструмента в интерфейсе модели и изоляция процесса средствами ОС — разные гарантии. решено

Предусмотрены три варианта установки. Таблица описывает выбранный дизайн, а не наличие квалифицированной реализации каждого варианта. решено

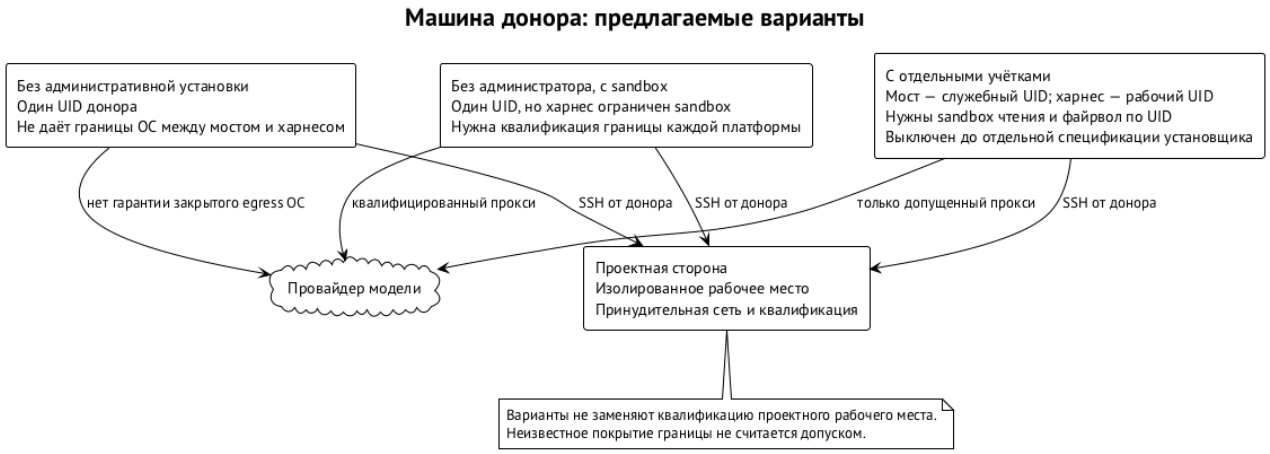
Вариант	Права администратора	Граница у донора	Мост и ключи
A: запертый запуск	не нужны	проверенная сборка не предлагает локальных инструментов; отдельной границы ОС нет	пользовательская фоновая программа; Secure Enclave на macOS, файлы с правами `0600` на Linux
B: две учётные записи	нужны при подготовке	недоверенный харнес и доверенный мост под разными UID; файрвол, граница чтения ядра, бинарники под root	системная служба моста; ключи в файлах `0600` его учётной записи, без Keychain
C: песочница	не нужны	харнес под пользователем донора, но ограничен Seatbelt на macOS либо Landlock или bubblewrap на Linux	пользовательская служба; хранение ключей как в A

Вариант A выбран для первого этапа. Его предел существенен: если квалифицированная сборка всё-таки предложит локальный инструмент, модель сможет действовать от имени пользователя донора. Проверка сборки уменьшает этот риск, но не превращает режим в границу ОС. В C харнес может читать и писать только свой каталог, обращаться к реле и локальному прокси провайдера; доступа к Keychain нет. В B сеть разделена по UID: харнесу нужен прокси провайдера, мосту — SSH-хост проекта и точка присоединения. решено

Для B нужен отдельный привилегированный компонент подготовки: создать учётные записи, установить файрвол и песочницу, обеспечить удаление. Расширять для этого менеджер пользователей центральной среды не предполагается. Пока компонент не описан, B выключен. После квалификации B и

C установщик должен предлагать B и переходить к C при отказе от административных прав. Сохранять ли после этого A, ещё не решено. открыто

Вариант установки закрепляется при регистрации, виден в статусе и баннере. Политика рабочего места может разрешать запись только для B или C. Таким образом, проект выбирает допустимый риск, а донор видит условия, на которых отдаёт квоту. Первой целевой платформой является macOS, следующей — Linux; Windows в этот дизайн не входит. решено



В предлагаемом развёртывании донора варианты без разделения UID, с отдельными учётками и с sandbox имеют разные границы защиты.

12.3 Запрет нельзя снять настройкой профиля

В Swarm Curator режим запуска является настройкой машины. Значение `lockdown` закрепляется замком: нижние слои конфигурации не могут его ослабить. Профиль не задаёт позу, а лаунчер принимает сгенерированную политику и короткий белый список аргументов. Супервизор моста дополнительно требует сочетание `lockdown` и `remote-auto`; смена позы останавливает живую сессию. В `remote-auto` заранее разрешены инструменты реле, а остальные запросы подтверждения отклоняются. решено

В B замок находится в системной конфигурации под root. В A и C он принадлежит донору. Донор вправе изменить собственные файлы, но результатом нарушения условий должен стать отказ запуска, а не открытая сессия. Изменение позы или прав увеличивает поколение политики; оно проверяется при запуске и возобновлении. Например, новый профиль не должен незаметно вернуть локальную оболочку уже зарегистрированному работнику. решено

Допускается только точный квалифицированный набор: версия и дайджест бинарника, платформа, транспорт, модель, политика настроек и ревизия песочницы. Дайджест сверяется до запуска. Неизвестная новая версия получает отказ; автоматического «попробовать с предупреждением» нет. Откат допустим лишь на другую квалифицированную сборку. Список должен поступать из центральной ленты квалификации. решено

Для Claude Code измерен способ убрать инструменты; для Codex измерен только путь через app-server. Это не обещание поддержки любой их версии. Muse у доноров не поддерживается: инструменты чтения не выключаются, а песочница пока не принята как замена этой гарантии. Возможности Pi и OpenCode требуют квалификации на целевых платформах; одного исследования исходников недостаточно для допуска. Донор видит позу, сборку и аудит вызовов — имя инструмента, хэш аргументов, размеры и время без содержимого. решено

12.4 Первый контакт без ручной сверки отпечатка

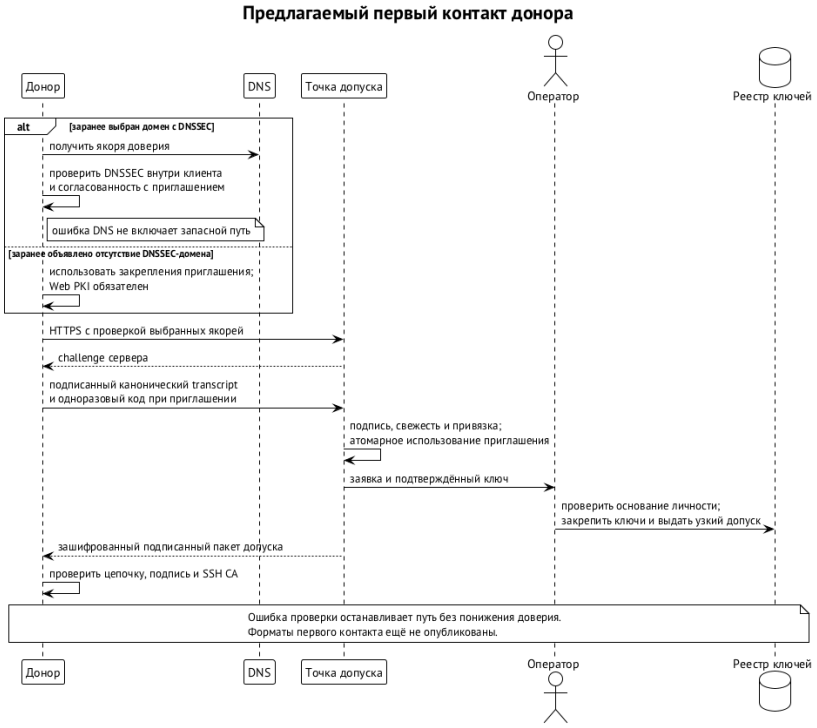
До передачи задачи мост донора должен проверить, что подключается именно к среде проекта. Основные якоря — HTTPS-сертификат точки присоединения и DNSSEC. В подписанных DNS-записях публикуются отпечаток корня среды, ключ центра сертификации SSH-хостов и `TLSA` точки присоединения. Клиент сам проверяет подписи от корневого ключа DNS. Если сеть мешает получить записи напрямую, возможен DNS-over-HTTPS с той же самостоятельной проверкой подписей. **решено**

Приглашение может дополнительно нести отпечатки. Как запасной путь оно допустимо только для среды, заранее объявленной как среда без домена с DNSSEC. Ошибка проверки DNSSEC не разрешает переключиться на приглашение: иначе атакующему достаточно нарушить DNS, чтобы ослабить проверку. В закрытом контуре якорь внутренней зоны приходит из конфигурации контура или приглашения. Расхождение якорей означает отказ, а не выбор наиболее удобного ответа. **решено**

Инициализацию запускает человек — владелец машины донора. Она требует интерактивного терминала и набранного подтверждения сводки, отклоняет нетерминальный ввод и признаки агентского окружения. Это делает согласие явным и мешает агенту установить работника по подсунутой инструкции. Защиты от решительного злоумышленника, уже действующего на машине самого донора, этот приём не обещает. **решено**

Регистрация разделяет ключи по назначению: подпись, расшифровка пакета допуска и SSH. Владение доказывается подписью серверного challenge со сроком пять минут. Подписываемый транскрипт связывает версию, приглашение, проект, роль, ключи, challenge и срок. Приглашение потребляется атомарно: повтор с тем же ключом возвращает ту же регистрацию, с другим ключом отклоняется. Оператор сверяет заявку и закрепляет ключи; допуск подписывается на его стороне. **решено**

Точка присоединения — небольшой HTTPS-сервис: выдать challenge, принять заявку, отдать пакет. Ключа подписи допуска у неё нет. Пакет шифруется на ключ расшифровки донора, подписывается сертификатом оператора и не содержит учётных данных носителя сообщений. После получения пакета дальнейшая работа идёт по SSH. Для публичного входа заданы ограничения: запрос до 16 КиБ, приглашение на 72 часа с максимум пятью вызовами, очередь до 50 заявок, до 10 запросов в минуту и 100 в сутки с одного адреса. **решено**



В предлагаемом первом контакте донор проверяет якоря доверия, подписывает challenge сервера и получает допуск только после решения оператора.

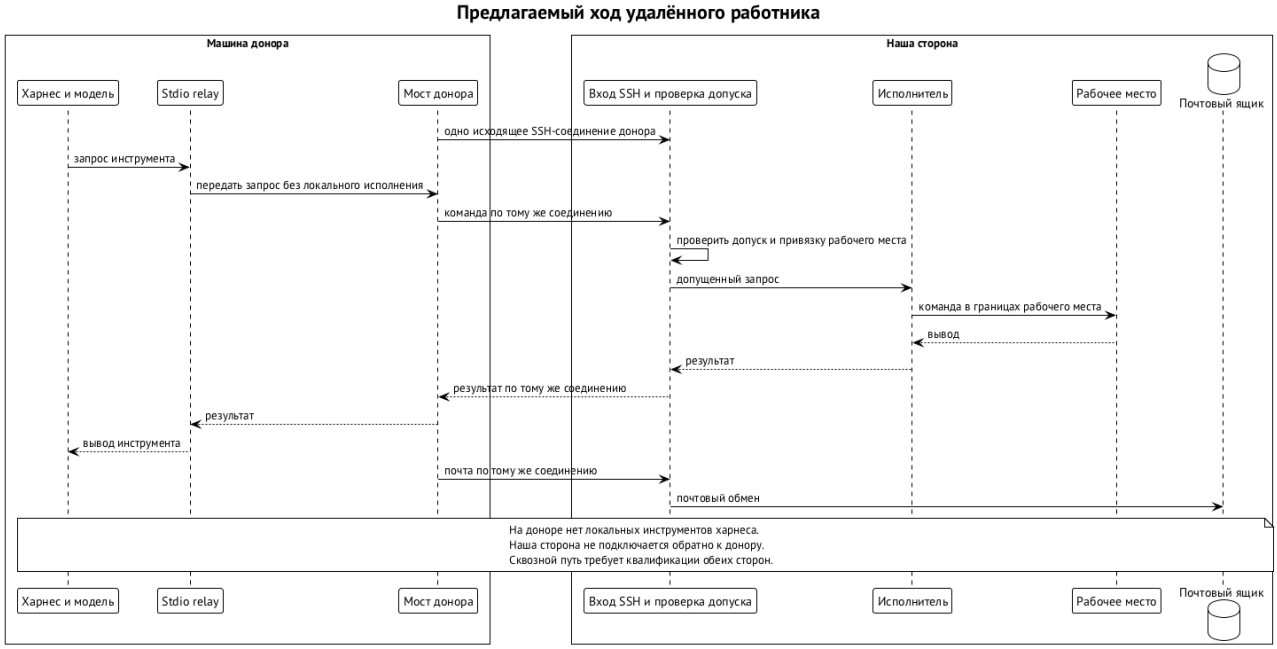
12.5 Канал и исполнитель

Пользователь рабочего места не имеет пароля и интерактивного входа. Вход предполагается по короткому SSH-сертификату, примерно на сутки, с продлением мостом. Доверие к центру пользователей хранится под root вне дома работника. Предусмотрены принудительная команда, запрет PTY, проброса портов и SSH-агента; сертификат хоста проверяется мостом при каждом соединении. Детали совместной передачи короткого сертификата и принудительной команды остаются открытыми. **решено**

Проверяющие службы должны работать вне учётной записи работника. Оболочка входа пересылает канал на служебный Unix-сокет и не читает пользовательские файлы запуска. Служебный исполнитель держит политику инструментов, потолки, аудит, почтовый ящик и сверяет UID с активным поколением. Команды исполняются под пользователем рабочего места. Сетевой запрет, включая loopback, устанавливается вместе с активацией учётной записи; без принудительной сети место не создаётся. **спецификация**

Для повторов предлагается долговечная запись операции до любого изменения: идентификатор в рамках поколения привязки и дайджест аргументов. После потери ответа повтор возвращает сохранённый результат либо состояние «ещё идёт» или «неизвестно»; неизвестность не разрешает повторное исполнение. Тот же идентификатор с другими аргументами отклоняется. Короткий кэш вывода сам по себе этого не обеспечивает. Закрытый формат протокола ещё не опубликован, поэтому здесь описана гарантия дизайна, а не готовый wire-формат. **предложено**

Подписанное задание поступает через мост и адаптер ввода как следующий ход сессии. Мост донора самостоятельно проверяет адресата, срок и отзыв. Результат подписывается ключом, который остаётся у донора; код передаётся отдельно. При обрыве работа приостанавливается, инструменты получают повторяемую ошибку, затем сессия паркуется. Совместный порядок старта SSH-реле, остановки и переподключения, включая запрет создания реле после закрытия поколения, ещё не определён полностью. **открыто**



В предлагаемом удалённом ходе модель работает у донора, а команды и почта проходят по одному SSH-каналу к исполнителю на нашей стороне.

12.6 Как забирается код

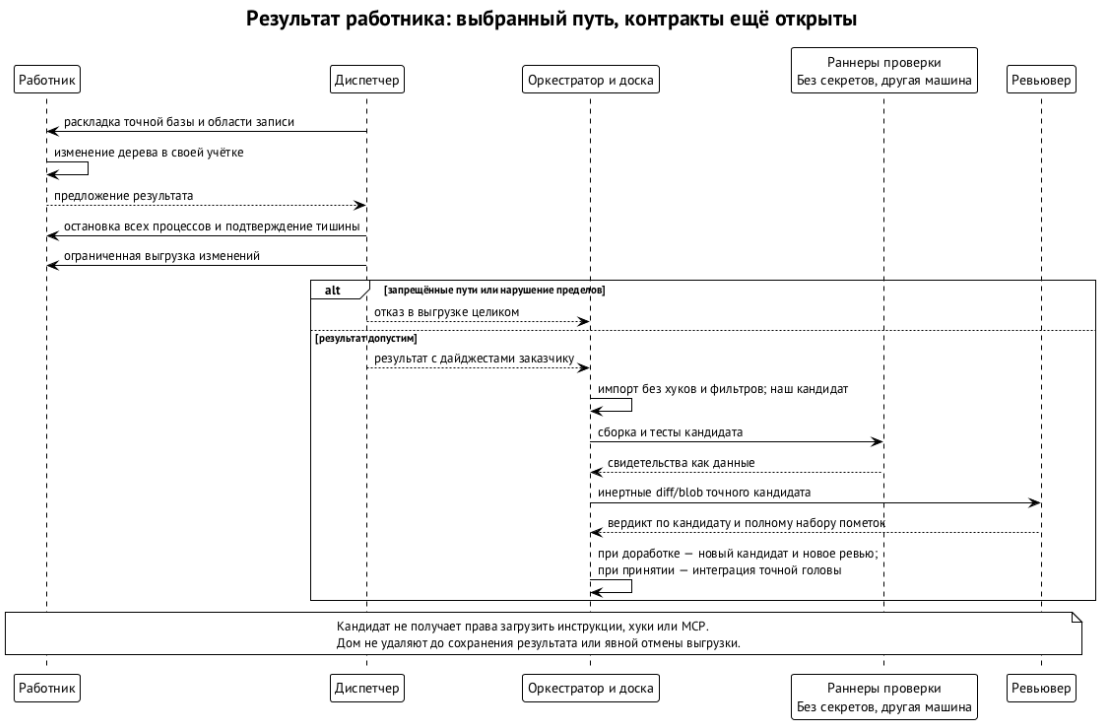
Работник не пушит и не открывает Change Request на хостинге кода. Проект выдаёт точную базу, манифест файлов и область записи в отдельный репозиторий без remote, хуков и учётных данных. Ра-

ботник может вести собственные коммиты, но проект не использует их как основание доверия. По окончании раунда процессы останавливаются с проверкой отсутствия потомков; закрываются вызовы инструментов в каталог. **решено**

Программа выгрузки работает с правами самого работника, без перехода по ссылкам, пропускает `.git` и отдаёт изменения относительно базы вместе с режимами файлов. Специальные файлы, подмодули, пути в `.git` и коллизии имён без учёта регистра отклоняют выгрузку целиком. Чтение от root или исправление прав через `chmod` не является обходным путём. Дом нельзя удалить до сохранения полного результата вне области удаления либо отдельного снятия обязательства оператором. **решено**

Доска собирает из этих данных собственный подписанный кандидат. Выход за область записи и изменения самой доски запрещаются. Инструкции агентам, CI, хуки, автозапуск, ссылки и управляющие файлы требуют явного рассмотрения ревьювером. Точный порядок приёмки описан в главе «Доска задач и процесс». Важное условие здесь — ревьювер сначала получает инертные diff и blob вне корней автоматического обнаружения настроек. Подтверждение текста не разрешает автоматически активировать его инструкции, MCP или permissions. **решено**

При доработке каталог раскладывается из предыдущего кандидата, но полный результат продолжает сравниваться с доверенной интеграционной базой серии. Сохранившаяся опасная правка не исчезает из отчёта только потому, что следующий раунд её не трогал. Новые подтверждения и вердикт привязываются к новой точной ревизии. Неустановленное происхождение веб-содержимого также остаётся отдельным обязательством; консервативное правило его снятия и формат отчёта ещё открыты. **открыто**



В выбранном пути результат работника выгружается после остановки, становится нашим кандидатом и проходит ревью точной ревизии до интеграции.

12.7 Веб, проверки и ресурсы

По умолчанию веба нет: задача приносит проверенные ресурсы с дайджестами, доступные только для чтения. Дополнительный материал работник запрашивает письмом, а получает и проверяет доверенная сторона. Поиск включается оператором для конкретной личности доверенного круга; согласия донора недостаточно. Возможные пути — поиск у провайдера без локальной загрузки, изолиро-

ванный веб-инструмент у донора либо служба платформы. Эти варианты ждут квалификации и не дают рабочему месту исходящую сеть. решено

Компиляция, транспиляция и тестовые прогоны вынесены из учётной записи агента. Разрешённая проверка типов не означает разрешения произвольного интерпретатора: проверяется весь вызов с читаемым кодом, конфигурацией и плагинами. Профиль определяется машиной по классу задачи и дайджесту, а не вызывающим. Сборка кандидата предполагается на эфемерных раннерах другой машины без секретов: зависимости загружаются отдельно, без исполнения кандидата, сама сборка идёт без сети. решено

Наблюдатель ресурсов должен уметь остановить и удержать учётную запись, показать статус. Для первой версии уведомление видно только в статусе: маршрута на телефон ещё нет. Заморозка сети и порядок её снятия относятся к следующей версии. Какие ограничения памяти и диска macOS обеспечивает жёстко, а какие требуют наблюдателя, должна определять таблица возможностей конкретной машины. Наличие рабочего места само по себе не доказывает все эти свойства. спецификация

Ни один агент при таком разделении не достигает хостинга кода. Внешний исполнитель должен отдельно получить запрос публикации, передачи артефакта на раннеры, обновления базы или приземления и проверить полномочия на границе эффекта. Пока не выбраны его устройство, хранение полномочий и маршрут результата, нельзя считать эти действия обязанностью самого работника или автоматически приписать их доске. Принятый путь данных определяет, что требуется сделать, но не закрывает вопрос, какой неагентский компонент это исполнит. открыто

12.8 Что именно защищают границы

Работник видит собственный дом, снимок задачи и входящие сообщения с результатами проверки. Он не получает всю доску, чужие дома и процессы или прямой доступ к носителю. Это ограничение рабочего контекста, но не обещание скрыть любые платформенные метаданные других учётных записей: их видимость ещё зависит от политики владельца. Например, изоляция файлов проекта не должна описываться как доказанное отсутствие любых сведений о соседях. решено

Максимальный набор удалённых инструментов включает оболочку, чтение и запись файлов, патч и почту. Действующий набор — пересечение с политикой рабочего места. Для работника только для чтения наличие инструмента записи в общем протоколе не означает права вызвать его на этом месте. Серверные тайм-ауты и потолки вывода клиент может только уменьшить. Полномочия проверяются на стороне проекта и не расширяются от того, что мост донора сформировал корректный запрос. решено

Отдельно сохраняется обязательство результата. Детерминированный отказ выгрузки не должен ни выбрасывать работу автоматически, ни удерживать раунд без возможности выхода: предполагается ограниченное исправление в текущих полномочиях или отдельное решение оператора снять обязательство. Ошибка доступа к созданному работником файлу отличается от инфраструктурного сбоя. Повышать права выгрузки ради удобства нельзя: это позволило бы данным недоверенного работника управлять чтением привилегированной программы. Полный контракт классов исхода и восстановления ещё предстоит опубликовать. открыто

12.9 Остановка не равна отзыву

Предлагается учитывать четыре независимые оси: наличие установки у донора, активность донора, допуск рабочего места и состояние ключа подписи. Донор может снести установку без связи с проектом; это не отзывает автоматически гранты и ключ на другой стороне. Он может остановиться или уйти, сохранив личность. Проект отдельно закрывает каналы и сессии, останавливает задачи с потомками и освобождает учётную запись. Удаление SSH-ключа не закрывает уже установленное соединение. предложено

Ключ донора отзывает оператор. Если канал недоступен, удаление на стороне донора должно сообщить именно это, а не изображать подтверждённый отзыв проекта. Общий реестр ключей и грантов для нескольких машин остаётся открытым. Возможная биржа агентов не меняет ограничений рабочего места; направление переносимых аттестаций и репутации рассматривается отдельно в главе «Аттестации и переносимая репутация» и пока не является действующим механизмом допуска. открыто

Следующие незакрытые контракты — схема выгрузки и импорт, квалификация сборок и веб-инструментов, жизненный цикл SSH-реле, компонент подготовки B и исполнитель внешних действий доски. Пока они не согласованы, нельзя объявлять весь путь реализованным на основании готовности отдельных частей. Именно эти границы отделяют выбранную архитектуру от квалифицированной системы, которой можно доверить рабочее место. открыто

13 Доска задач и процесс

Доска задач (`task-board`) хранит работу, её состояние и свидетельства приёмки. Она отвечает на вопросы, которых нет у хоста сессий: какую задачу сейчас выполняют, кто вправе принять результат, какая ревизия проверена и почему её можно интегрировать. Для разработки это замена устной договорённости между агентами на проверяемую историю. Программа с CLI на Go, хранилищем задач и механизмом запросов на изменения уже используется. работает сегодня

В Swarm Runtime доска остаётся необязательным клиентом. Хост сессий держит сессию без неё, мост доставляет сообщения, слой доверия проверяет полномочия. Доска добавляет роли, процессы и запуск по переходам; платформенные модули от неё не зависят. Это позволяет использовать те же запуск, ключник и мост в проекте с другим управлением работой. Решение о таком разделении принято, но перенос сегодняшней доски на все новые границы ещё не означает их реализации. решено

13.1 Что уже хранится и проверяется

В существующей модели доска хранится вместе с проектом. Элементы образуют иерархию Epic → Story → Task или Bug, имеют описание, прогресс, статусы, ресурсы и историю активности. Агент обращается к ней через язык запросов и мутаций: `q` читает, `m` меняет, `grep` ищет; контракт можно узнать у самой программы. Для читателя существенно не устройство идентификатора, а то, что работу и её свидетельства можно прочитать независимо от памяти исполнявшего агента. работает сегодня

Запуск дочернего агента — отдельный объект учёта с ролью, моделью, манифестом, событиями и транскриптом. Оркестратор может наблюдать его и передавать директивы. Подготовка аргументов и окружения зависит от харнеса и собирается модулем управления агентами. Сегодняшнее исполнение запуска находится в доске; в целевой архитектуре интерфейс доски обращается к диспетчеру, а процессы и сессии обслуживает платформа. решено

Работающее хранилище не следует путать с границей авторизации. Сегодня часть сведений о действующем агенте передаётся соглашением через окружение; это само по себе не удостоверяет право на переход. Будущая схема связывает роль с грантом и проверяет его ядром. По той же причине выход процесса не может служить достаточным доказательством успеха задачи: процесс мог прекратиться до записи результата или вернуться без проверенной ревизии. решено

13.2 Продьюсер, ревьювер и Change Request

Продьюсер создаёт изменение. Ревьювер проверяет опубликованную ревизию, а оркестратор организует работу и интеграцию. Оркестратор не заменяет проверку своим желанием быстрее закрыть зада-

чу. Один запуск не может принять собственную ревизию. Разделение фиксируется механизмом доски, а не только инструкцией модели «будь независимой». Такой цикл с отдельным рабочим деревом Story и неизменяемыми ревизиями Change Request уже действует. работает сегодня

Change Request — запрос на изменения, публикующий конкретную неизменяемую ревизию для проверки. Вердикт относится к ней, а не к подвижному рабочему каталогу. Если продьюсер исправил файл после замечания, проверять нужно новую ревизию. Старый положительный вердикт не превращается в разрешение на любую последующую работу этого продьюсера. Доска сохраняет связь результата, ревизии и приёмки; оркестратор интегрирует Story после проверок, привязанных к дереву. работает сегодня

Для недоверенного работника эта граница начинается ещё до публикации. Он отдаёт изменения как данные; доска строит собственный кандидат от точной базы. Работник не получает транспорт к хостингу кода. Принятый путь описан в главе «Удалённое рабочее место»: чужие коммиты не становятся доверенными только потому, что их принесли по подписанному каналу. Подпись результата отвечает за происхождение байтов, а ревью — за решение принять именно эти байты. решено

Доверенный статус ревьювера также недостаточен без права роли. Для проверки требуется отдельный грант; класс доверия и размещение должны быть допущены, а ревьювер не должен совпадать с производящей личностью. Обязательность только локального ревьювера и запрет заказчику раунда проверять результат остаются отдельными вопросами. Их нельзя незаметно добавить как уже существующие гарантии независимости. решено

13.3 Почему интеграция сравнивает точную голову

Compare-and-set означает: обновить целевую голову только при совпадении с ожидаемым исходным состоянием. Если между проверкой и интеграцией другая работа продвинула ветку, предпосылка больше не выполнена. Нельзя молча применить прежний результат к новой базе. В работающем механизме интеграция Story использует compare-and-set и проверки, связанные с деревом. работает сегодня

Целевое правило строже обычного «патч выглядит тем же»: интеграция и приземление — подписанная перемотка точной головы с доказательством стыка. Пусть ревьювер одобрил кандидат от одной базы, а затем основная ветка продвинулась. После обновления базы возникает другой кандидат, и вердикт не переносится по сходству diff. Иначе проверка будет говорить об одном объекте, а история примет другой. Если база ушла вперёд, требуется обычное обновление базы и повторное ревью. предложено

При доработке различаются база раскладки и доверенная интеграционная база. Работнику удобно продолжать от последнего кандидата, однако чувствительные изменения должны проверяться по полному delta серии. Например, инструкция агенту, добавленная в первом непринятом кандидате, не исчезает из отчёта второго лишь потому, что второй меняет только текст ошибки. Полностью отменённая правка, наоборот, не должна оставаться помеченной только из-за истории. Подтверждения и новый вердикт привязываются к точному итоговому OID. решено

Сначала доска выдаёт ревьюверу инертные diff и blob только для чтения, с дайджестами и вне корней обнаружения проектных настроек харнесом. Открыть само дерево кандидата допустимо лишь через квалифицированный адаптер строгого контекста либо эквивалентное исключение discovery. Инструкции, hooks, MCP, permissions и skills кандидата не подхватываются автоматически при запуске, вложенном чтении, reload, resume или входе дочернего агента. Подтверждение изменения не заменяет отдельного решения об активации управляющего контекста. решено

Это не устраняет семантическую prompt injection в читаемом тексте. Ревьюверу всё ещё приходится оценивать недоверенное содержимое. Сборка и тесты должны идти в отдельной фазе на эфемерных раннерах без секретов, а не исполнять кандидат внутри процесса ревьювера или оркестратора. Ни роль владельца процесса, ни её грант не отвечают пока на другой вопрос: кто вне агентской учётной

записи публикует кандидат, передаёт артефакт на проверку и выполняет landing. Граница этого внешнего исполнителя остаётся открытой. открыто

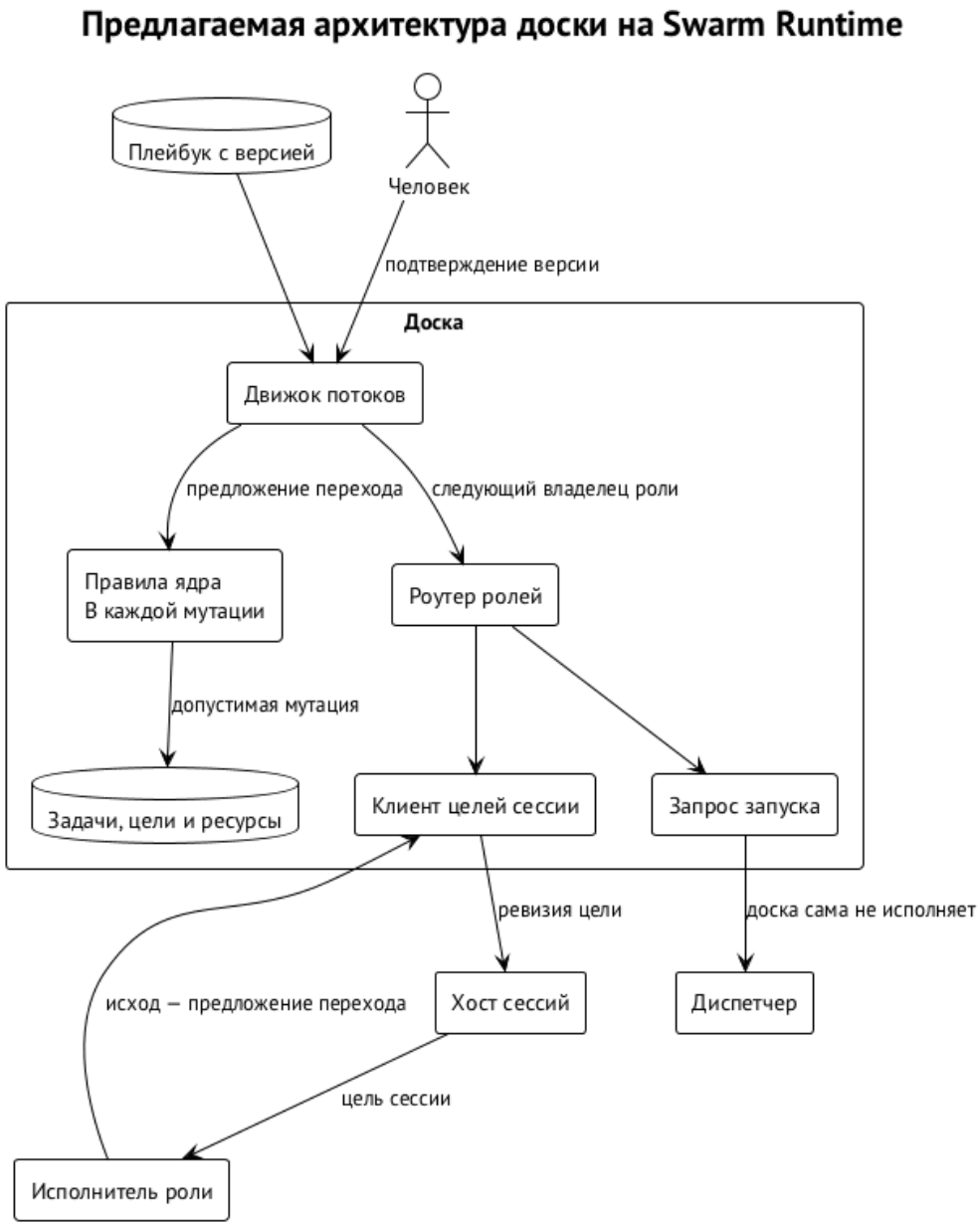
13.4 Процесс задаёт плейбук

Плейбук — описание типов работ, состояний, владельцев и разрешённых переходов с проверками. Вместо нового кода доски для каждого проекта предполагается конфигурация процесса. В потоке разработки разработчик владеет разработкой, ревьювер — ревью, оркестратор — интеграцией. В потоке дизайна владельцами будут другие роли. Смена процесса не должна менять значение уже полученного свидетельства или позволять обойти фиксированное правило ядра. предложено

Проект разделяет три вопроса между файлами. `Skillfile.json` описывает установленное и читается Swarm Curator; `Playbook.json` описывает работу проекта; `task-board.config.json` задаёт окружение доски. Плейбук не должен превращаться в установщик с версиями и URL, а файл установки — в скрытую машину переходов. Они встречаются по именам скиллов. Существующие решения о таком разделении сохраняются, но возможность произвольно заменить зашитый процесс относится к будущему движку. решено

Пакет процесса объединяет скилл оркестратора, скиллы ролей и поставщиков инструментов. Сам плейбук — данные, без исполняемых программ, скриптов и URL. Инструмент предоставляется установленным скиллом и запускается, только если проект его включил. Скиллы ролей попадают в запуски этих ролей, а не раскладываются в проект так, чтобы любая сессия обнаружила их автоматически. Иначе оркестратор мог бы начать действовать по инструкциям ревьювера. решено

Композиция позволяет брать опубликованный процесс и описывать отличия либо соединять несколько процессов через слоты. Например, Scrum отвечает за выбор работы и сроки, а цикл разработки — за путь конкретного изменения. В iOS-проекте можно заменить скилл разработчика и вставить состояние QA, сохранив остальной шаблон. Такое повторное использование принято как направление; опубликованные части целевого плейбука предполагается закреплять хэшами. предложено



В предлагаемой архитектуре доски плейбук задаёт процесс, а правила ядра ограничивают каждое изменение состояния.

13.5 У состояния один владелец

В описании процесса роль — псевдоним, на который ссылаются состояния. Владельцем может быть агент со скиллом роли, человек из группы, инструмент или другой процесс. В каждый момент у элемента в состоянии ровно один владелец. Помощники работают на него: при оценке задачи фасилитатор собирает независимые ответы оценщиков, но ответственность за состояние не распадается между ними. решено

Разные результаты требуют разных состояний. «Написал» и «проверил» нельзя объединить так, чтобы запись первого автоматически удостоверяла второе. Параллельная работа выражается дочерними элементами и проверкой их соединения на родителе. У каждого перехода указан вид актора; агент не выводит элемент из состояния человека и не записывает человеческое одобрение без делегированного права. Например, завершение дочернего анализа даёт материал владельцу, но не подпись человека за него. решено

Данные процесса тоже могут влиять на маршрут. Заказ дороже 500 евро должен идти менеджеру; одобрение заказа на 620 евро не разрешает исполнить заказ на 6 200. Для этого поля, читаемые провер-

ками перехода, не записываются произвольно запуском, а одобрение связывается с отпечатком полей. Здесь важно не конкретное число порога, а невозможность перенести согласие на изменившийся предмет. Это расширяет ту же идею точной ревизии на процессы вне разработки. решено

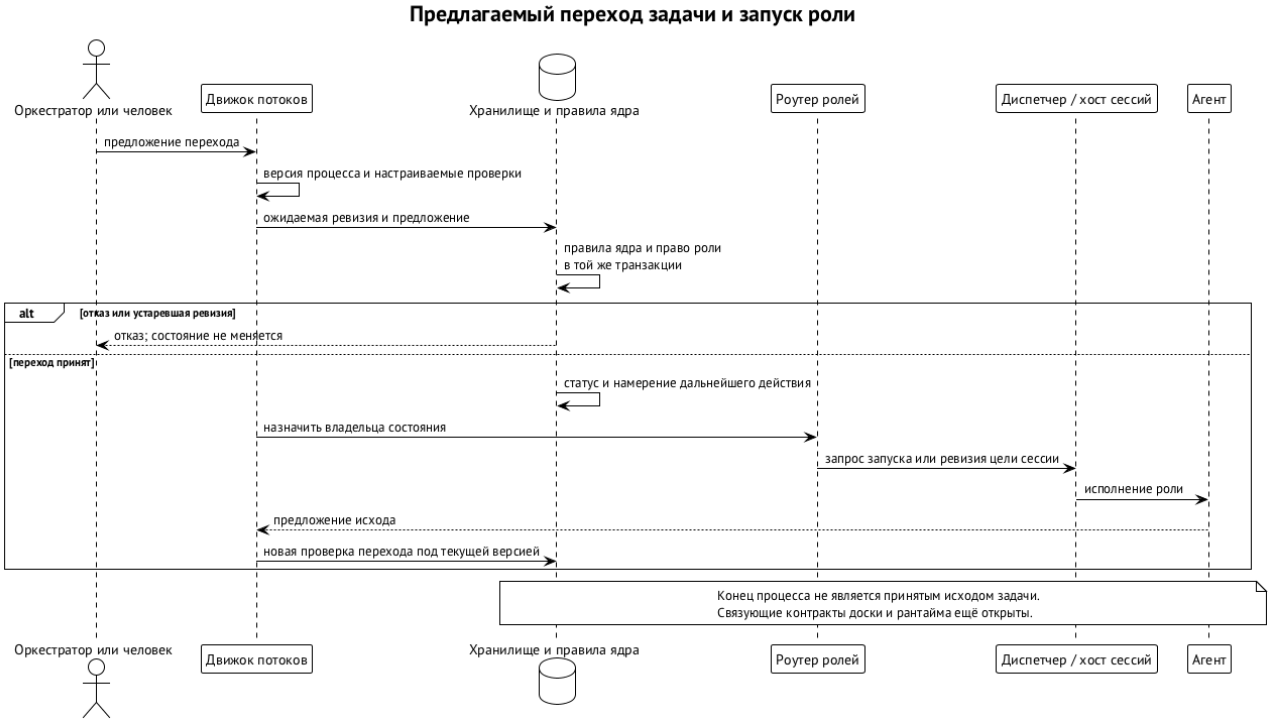
13.6 Переход — предложение, а не команда агенту

Переход может требовать оценку, назначенного исполнителя, отсутствие блокеров и свидетельство вердикта. Guard — проверяемое условие перехода. Имена guard, действий и событий приходят из модулей; неизвестное имя отклоняется валидатором. Некоторые переходы доступны лишь через определённую команду, другие выполняет ядро по событию или таймеру. Свидетельство должно быть свежим: вердикт учитывается после последнего входа элемента в исходное состояние. решено

Например, задача вернулась из ревью в разработку и снова пришла на ревью. Старый вердикт «нужны изменения» не должен автоматически отправить её назад ещё раз; требуется свидетельство текущего прохода. Аналогично положительное решение предыдущего прохода не принимает новый кандидат. Свежесть события защищает последовательность процесса, а привязка к ревизии — его предмет; одна проверка не заменяет другую. решено

Число кругов доработки настраивается проектом. Длинная петля не обязательно дефект: законная работа может требовать много итераций. Поэтому предусмотрен и осознанный вариант без ограничения, с предупреждением валидатора. Конкретная форма параметров шаблона для лимитов, счётчиков и длительностей остаётся предложением. Лимит должен выражать политику команды, а не скрытую универсальную догадку о том, сколько раз агенту разрешено ошибиться. предложено

В целевой цепочке ядро проверяет правила и грант роли, записывает состояние и намерение. Маршрутизатор ролей превращает владельца нового состояния в запрос запуска: профиль Swarm Curator с точным закреплением, класс задачи, привязки ввода-вывода и нужный контекст. Для разового агента раннер обращается к диспетчеру; для постоянной сессии публикуется новая ревизия цели при наличии права на эту сессию. Ответ агента «готово» — предложение исхода, которое ещё проходит барьер доски. решено



В предлагаемом переходе доска проверяет версию процесса и правила ядра, назначает роль и отдельно принимает предложенный агентом исход.

13.7 Правила под всеми плейбуками

Настраиваемый процесс может сужать разрешённое, но не отменять предикаты ядра. Защищённое состояние определяется классом, а не названием: переименование интеграции не открывает обход. Фиксированный слой предлагается скомпилировать в бинарь доски. Ни автор плейбука, ни человек, подтвердивший его версию, не получают тем самым право отключить этот слой. Следующая таблица описывает предлагаемые гарантии, а не отчёт о полной реализации. предложено

Область	Неизменное условие
Запрос на изменения	задача с Change Request достигает интеграции и завершения только с принятым запросом
Рабочее дерево	работа идёт в дереве своей Story с контрольной точкой; устаревшее дерево не интегрируется
Продвижение истории	подписанная перемотка точной головы с доказательством стыка
Проверки	свидетельства получены из очереди проверок, лимиты не выше потолков машины
Ревью	терминальный исход требует свидетельства для этой ревизии
Цели	неизменные ревизии с compare-and-set; парковка и возобновление фиксируют ревизию
Аудит	мутация и запись аудита — одна транзакция; журнал только дописывается
Хранилище	уникальные идентификаторы, существующие родители, оптимистичная ревизия и допустимые имена ресурсов

Из этой модели следует простой запрет: нельзя написать плейбук, в котором изменение считается принятым по одному завершению продьюсера. Дополнительные ревью, состояния QA или ручное согласование допустимы; удаление обязательного свидетельства — нет. Для модуля доставки также принято разделение: самоприёмка запрещена, принимается опубликованная ревизия, а выход из интеграции требует разрешённого отката. Эти правила должны сохраняться и у процессов, чьи состояния названы иначе. решено

13.8 Новая версия процесса не применяется наполовину

Голова основной ветки предлагает новую версию плейбука, а исполняется версия, вступившая в силу. Неподтверждённый рабочий каталог агента не становится политикой. Более новая архитектура требует проверки и человеческого подтверждения перед сменой версии; до этого целиком действует прежняя. Это консервативная граница: предложение обновления и право управлять текущими задачами не смешиваются. предложено

Отчёт о полномочиях должен показать изменения по разделам и выделить те, что уводят работу мимо человека или проверки. Аналитик может объяснить последствия, но не одобрить их вместо уполномоченного человека. Если подтверждённой версии ещё нет, предполагается встроенный плейбук с сегодняшним поведением. Например, новая ветка процесса с обходом менеджера не начинает работать только потому, что её JSON синтаксически корректен. решено

Политика процесса задаёт незавершённую работу, ёмкость планирования, цели и видимость представлений. Ограничение числа элементов в работе не должно задерживать эскалации к человеку. Наборы проверок делят ответственность: плейбук говорит, что должно пройти, конфигурация доски — где выполнять. Локальное воспроизведение при внешнем отказе CI фиксируется отдельно и не превращается в успешный hosted CI. Свидетельства должны сохранять ревизию и фактическую среду проверки. решено

Инструмент запускает исполнитель ядра в очищенном окружении, а не процесс агента, запросившего переход. Иначе агент мог бы подложить собственную программу проверки через `PATH`. Внешние события должны приходить подписанным мостом, связанным с группой подписантов проекта, и храниться до подходящего перехода. Периодический вызов обеспечивает таймеры и расписания; управление обязательствами и пропавшими владельцами рассматривается отдельно в главе «Swarm Keeper». решено

13.9 Цель, ресурс и свидетельство — разные вещи

Ресурс задачи хранит материал работы: исходные сведения, результат или отчёт. Цель задаёт, какого состояния добиваются. Свидетельство подтверждает определённый переход для определённой ревизии. Нельзя заменить одно другим: приложенный файл сам по себе не является вердиктом, а формулировка цели не доказывает её достижение. В работающей доске цель имеет собственный реестр ревизий; целевая архитектура сохраняет эту связь при передаче работы постоянным сессиям. решено

Например, разработчик приложил результат и сообщил об успехе, но ревьювер вернул изменения. Цель остаётся прежней, ресурс позволяет воспроизвести предмет проверки, а вердикт требует нового раунда. Если оркестратор уточнил цель, новая ревизия не должна незаметно подменить ту, с которой сессия была припаркована. Предлагаемое правило связывает парковку и возобновление с зафиксированной ревизией и compare-and-set. Так изменения намерения получают столь же явную историю, как изменения кода. предложено

Аудит нужен именно на границе мутации. Записать статус, а потом когда-нибудь дописать объяснение недостаточно: сбой между этими действиями оставит изменение без следа. Поэтому предлагается одна транзакция состояния и строки аудита, а журнал только дописывается. Это фиксирует факт действия и его связь с переходом. Само по себе наличие строки не доказывает правильность суждения ревьювера или честность внешнего исполнителя; для них нужны собственные полномочия и свидетельства. предложено

Разделение частей доски следует этим обязанностям. Хранилище держит элементы и историю; движок процесса — версию плейбука и допустимые предложения; маршрутизатор ролей — владельца следующего состояния; интерфейс запуска — обращение к платформе; привязка целей — связь с живой сессией. Слоты, потолки машины и очередь проверок являются сервисом платформы. Встраивание клиента брокера учётных данных в доску не требуется: запуск проходит общий путь диспетчера. решено

Такое устройство позволяет заменить один процесс другим без создания второго механизма авторизации. Например, дизайн и разработка могут иметь разные состояния, но их владелец всё равно предлагает переход ядру, а не редактирует подтверждённый статус в обход проверки. Нельзя и обойти ёмкость машины собственным запуском из плейбука: роль, её потребность и факт доступности должны пройти общий маршрут. Для процесса это граница расширения, для оператора — возможность видеть одинаковый учёт работы разных команд. предложено

13.10 Проверяемый процесс и предел приёмки

Валидатор процесса должен проверять достижимость состояний, наличие владельцев, обходы guard, инварианты доставки и детерминизм. Отдельные машинные проверки отвечают за установленные скиллы и совместимость привязок моделей с потребностями ролей. Одного правильного графа недостаточно, если на машине нельзя исполнить его роль; установленный инструмент, наоборот, не исправляет обход ревью в графе. Экспорт свойств в формальные инструменты и проверка новых задач против решений запланированы, но их словари остаются открытыми. открыто

Передача другому процессу тоже требует протокола: принимающий оркестратор создаёт свой элемент, по завершении возвращается событие с исходом. Для нескольких досок направление предполагает подписанные сообщения; результат удостоверяет ключ принимающего хоста, а не текст модели.

Это не повод называть междосочную передачу уже работающей функцией только потому, что локальный элемент можно перевести в другое состояние. решено

Приёмка кода не имеет общего безошибочного оракула. Доска строит процедуру из контракта задачи, независимого ревью и журнала решений, но положительный вердикт остаётся суждением. В Swarm Protocol случайный выбор ревьювера, независимый узел проверок и доверенная интеграция усиливают эту процедуру; их устройство вынесено в главу «Оркестратор по аренде и цикл задачи». Переносимая репутация также остаётся отдельным предложением, а не свойством локальной доски. предложено

Условный расчёт показывает предел такого усиления. Если десять процентов агентов злонамеренны, а честный ревьювер обнаруживает подсаженную уязвимость в семидесяти процентах случаев, один ревьювер пропускает её с вероятностью 0,37. При независимом выборе и независимых ошибках два дают около 0,14, три — около 0,05. Это иллюстрация при заданных допущениях, не измерение качества Swarma. предложено

Число ревьюверов	Вероятность пропуска в условном примере
1	0,37
2	0,14
3	0,05

Общая модель, инструменты и одинаковый контекст могут коррелировать ошибки. Тогда умножение вероятностей не описывает реальную систему, а добавление ревьюверов помогает меньше, чем обещала бы таблица. Поэтому качество и независимость проверки важнее одного счётчика одобрений. Ни подпись, ни запись аудита не исправляют неверное суждение; они лишь делают его предмет и происхождение проверяемыми. Измерять нужно обнаружение дефектов на подходящих задачах, а не объявлять достаточным любое нужное число положительных ответов. предложено

Направление развития — сохранить хранилище, CLI, язык запросов и Change Request, а зашитые типы и статусы заменить плейбуком, запуск передать платформе, лимиты — сервису ёмкости. Публичные модули должны подключаться по выпущенным версиям, с операторским переключением. Выбрана и удалённая доска на продукт с проектами как разделами. Это целевая архитектура; её нельзя смешивать с ограничениями старого серверного режима, который поддерживал только планирование. решено

14 Модели и деньги

Маршрутизатор моделей выбирает конфигурацию для конкретной работы: среду агента, модель и уровень рассуждений, или effort. Он получает метки задачи и уже допущенных кандидатов, сопоставляет их с данными о качестве и ценой, возвращает выбор с объяснением. Это отдельный шаг подготовки запуска, а не агент, управляющий другими агентами, и не прокси перед API провайдера. Библиотека `curator-model-router` и CLI `cmr` версии v0.3.2 работают на октябрь 2026 года. работает сегодня

Выбирать только имя модели недостаточно. Одна и та же модель с разным effort может стоить и работать по-разному; для локальных весов имеют значение ещё и конкретный файл, квантизация и режим мышления. При множестве задач ручные правила копятся и теряют объяснение. Маршрутизатор превращает их в данные: что разрешено, какая планка требуется, откуда взята оценка и почему выбран этот кандидат. решено

14.1 Четыре вопроса перед запуском

Высокая оценка пригодности отвечает только на один из четырёх вопросов. Разделение их владельцев принято как архитектурная граница. решено

Вопрос	Кто отвечает	Пример ограничения
Разрешено?	доска по политике оператора	роль, допущенная пара, расписание, способ оплаты
Совместимо?	реестр моделей и каталог	инструменты, окно контекста, платформа и словарь effort
Подходит?	маршрутизатор	качество для класса и сложности задачи
Доступно сейчас?	предварительная проверка доски	установленная среда, квота и лимит провайдера

Маршрутизатор ранжирует только внутри допущенного. Он ничего не запускает, не резервирует ёмкость и не переключает учётные записи или выход в сеть. Если лучшая по замеру модель запрещена проектом, оценка не возвращает её в список. Если подходящих кандидатов нет, получается отказ, а не выбор «лучшего из запрещённого». Факты держит модуль управления агентами, решение — маршрутизатор, исполнение — доска; библиотека маршрутизатора не импортирует её внутренние структуры. решено

В переносимом процессе роль должна описывать потребности, а не конкретную подписку оператора. Жёсткое требование отсеивает, предпочтение задаёт порядок. Например, ревью может требовать другого издателя относительно запуска, произведшего работу. Независимость берётся из происхождения этого запуска, а не из нынешних настроек разработчика. Дополнительное ревью того же издателя не восстанавливает требование независимости, которое уже нарушено. решено

Выбор ограничен пересечением политики проекта, допуска оператора, установленного на машине и потребностей роли. Личные привязки и окна доступности принадлежат оператору; их нельзя переносить в общий проект как обещание одинакового оборудования у всех. Если требование не выполнено, предусмотрены блокировка по умолчанию, запасной путь, запрос человеку или осознанное ухудшение с дополнительным ревью. Последнее не разрешает ослабить независимость. решено

14.2 Как принимается одно решение

Маршрутизатор не читает текст задачи. Вызывающий передаёт роль, класс работы, сложность, деликатность, режим бюджета, явные закрепки и контекст правил. Для сложности используются `trivial`, `routine`, `standard`, `hard`, `critical`; для бюджета — `economy`, `balanced`, `burn`. Если трудную миграцию ошибочно назвать механической правкой, селектор не распознает её смысл. Качество выбора ограничено качеством разметки. работает сегодня

Сначала собирается допущенный набор, затем применяются явные закрепки среды, модели и effort, после них — жёсткие правила оператора. Маршрутизатор не заменяет явное значение вызывающего. Оставшимся кандидатам назначается качество по оси класса: код, ревью или общая оценка. Если основной оси нет, может использоваться замер Bug Hunt с явной отметкой. Когда нет ни одного подходящего свидетельства, качество неизвестно и кандидат автоматически не выбирается. работает сегодня

Далее качество переводится в ступень, проверяется минимум сложности, оценивается стоимость и выбирается конфигурация. При равенстве действует стабильный порядок. Результат содержит альтернативы, причины исключения, применённые и пропущенные правила, а также точные параметры запуска. Например, оператор может увидеть, что дешёвый кандидат отпал по качеству, а более сильный — по допуску; это разные причины и разные способы исправления. работает сегодня

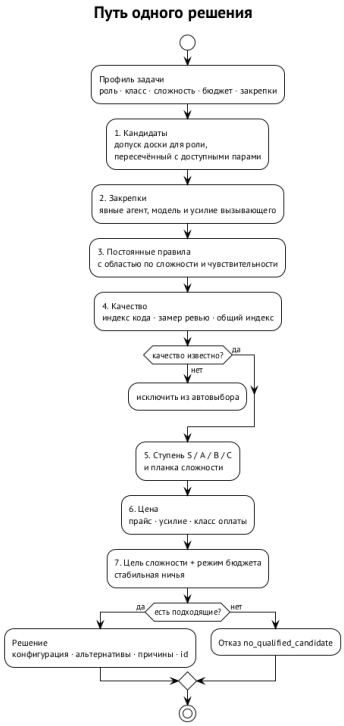
Политика по умолчанию различает планку и цель. Планка запрещает слишком слабые конфигурации, цель выбирает среди прошедших. Таблица относится к поведению маршрутизатора, а не к универсальной шкале способностей всех моделей. **работает сегодня**

Сложность	Минимум	Цель в `balanced`
`trivial`	C	дешевле
`routine`	B	дешевле
`standard`	B	дешевле; подъём до A допустим при цене не более чем в 1,5 раза выше
`hard`	A	качество
`critical`	S	качество без приоритета цены

Деликатная задача получает сильнейшую подходящую конфигурацию независимо от обычной сложности. Машинная закрепка ревьювера действует для обычной работы, а для трудной и критичной уступает планке и цели качества. Это правило политики нужно отличать от явного флага вызывающего: последний остаётся закрепкой. Конфликт закрепки с требованием не оправдывает выбор ниже планки. **работает сегодня**

`economy` выбирает самое дешёвое среди прошедших планку на любой сложности и не делает подъёма для `standard`. `balanced` следует таблице. `burn` предпочитает высшее качество; внутри равного качества учитывает больший остаток сгорающей подписки. Во всех режимах сохраняются допуск и минимум сложности: экономный режим не опускает критичное ревью ниже S. Автоматический выбор режима по квоте отложен; значение `auto` отклоняется. **работает сегодня**

Для исследовательского параллельного поиска можно выбрать до трёх конфигураций разных семейств уровня A или выше, в `burn` — до четырёх. Это расширяет набор независимых попыток, но не доказывает правильность их общего ответа. Автоматической сводки результатов пока нет. Не следует превращать сам факт разных названий моделей в гарантию независимых ошибок. **работает сегодня**



Маршрутизатор последовательно применяет допуск, закрепки, правила, качество и бюджет, возвращая решение или явный отказ.

14.3 Что считается данными о качестве

Встроенный каталог содержит публичную базовую линию Bug Hunt Bench, закреплённую на определённой ревизии, и прайсовые цены. Часть внешних индексов нельзя перепубликовывать по условиям использования, поэтому они поступают через явно указанный локальный оверлей оператора. Неизвестная конфигурация или неверный формат отклоняются. Неявного поиска такого файла нет; запись решения сохраняет его содержимое и канонический дайджест. работает сегодня

Замер относится к точной конфигурации: модель, effort и среда. Оценка на `max` не становится оценкой на `high` только из-за одинакового имени. Перенос возможен как отдельно помеченный вид свидетельства с неопределённостью и правилом переноса. Репутация семейства — предварительное мнение, а не измерение конкретного запуска. Перепечатки одного результата также не являются независимыми подтверждениями. решено

Собственные свидетельства дополняют каталог: импортируются Bug Hunt и прогоны `evalrun`, а человеческие заметки показываются отдельно и в оценку не входят. Наблюдение связывает конфигурацию, набор задач, протокол, метрику, число прогонов, неопределённость, происхождение и дату. Вид наблюдения и способ оценки различаются: результат LLM-судьи может быть замером, но не становится результатом исполняемого теста. работает сегодня

Отсутствие данных не заполняется удобным числом. Нет качества — неизвестно, а не ноль; нет цены — неизвестно, а не бесплатно. Модель, названная в наблюдении, не регистрируется от этого как допущенный кандидат. Оси с разными шкалами не смешиваются численно: основная ранжируется выше запасной. Эти ограничения мешают придать слабому свидетельству вид точного рейтинга. решено

У публичного Bug Hunt разброс между прогонами составляет примерно 3,3 пункта. Поэтому маршрутизатор использует ступени S, A, B, C, а не приписывает смысл каждому месту таблицы; внутри ступени выбор может решить цена. Многие рабочие сочетания effort ещё не измерены публично. Ни небольшой отрыв, ни несколько удачных ревью не устанавливают превосходство конфигурации; нужны собственные исходы и контрольный набор. работает сегодня

14.4 Кто разрешает тратить деньги

Способы оплаты разделены на подписку, локальный запуск и оплату по токенам: `subscription`, `local`, `metered`. Автоматический выбор без личного разрешения оператора не должен брать `metered`; неизвестный способ оплаты рассматривается как `metered`. Проект вправе сужать способы оплаты, но не давать согласие тратить деньги владельца вместо него. Расширение допуска требует подтверждения с присутствием человека, сужение применяется сразу. решено

Это касается не только токенов. Новая модель, новый runtime, более широкое временное окно или постоянный ответ на запрос исключения тоже расширяют допуск. Временное исключение человека относится к его запускам, ограничивается поддеревом и сроком и видно на доске. Постоянный ответ применяется с учётом политики проекта, каждое применение наблюдаемо; предусмотрено регулярное подтверждение, а не бессрочное забытое право без пересмотра. решено

Политика маршрутизатора читается строго и целиком замораживается. Неизвестное поле или неверное значение — отказ. Отсутствующий неявный файл означает встроенные умолчания; явно указанный, но отсутствующий файл — ошибку. Эти случаи различаются: оператор, запросивший конкретную политику, не должен неожиданно получить другую из-за опечатки в имени. Режим бюджета записывается вместе с решением и влияет на его идентификатор. работает сегодня

Данные о квоте поступают через официальные интерфейсы сред и используются для порядка, а не как собственная система резервирования маршрутизатора. Неизвестный или устаревший остаток нейтрален: кандидат не получает ни награды, ни штрафа. Например, в `burn` свежий остаток может разрешить ничью качества в пользу подписки, которая иначе сгорит; он не отменит запрет оплаты или несовместимость модели. работает сегодня

Цена одного запуска не равна цене принятого изменения. Дешёвая модель может потребовать больше доработок и повторных ревью. Поэтому направление измерений — связать решение с исходом задачи, числом раундов и расходом до приёмки. Числовой допуск потери качества и пороги замены базовой конфигурации ещё не установлены. Превращать режим экономии в обещание измеренного сокращения полной стоимости пока нельзя. открыто

14.5 Запись решения и воспроизведение

Запись сохраняет профиль задачи, закрепки, контекст правил, допущенный набор и его источник, действующий каталог, оверлей, политику, проекцию квот и результат с объяснением. Идентификатор вычисляется как SHA-256 канонического JSON. Материализованные умолчания, порядок ключей и коллекций определены канонизацией. При одинаковых замороженных входах получается тот же идентификатор; изменение входа относится уже к другому решению. работает сегодня

Селектор — чистая функция над этим снимком: без сети, чтения файлов, часов и вызовов моделей. Поэтому через месяц можно пересчитать, почему задача получила именно эту конфигурацию, даже если оверлей и квоты уже изменились. Это не обещание повторить ответ самой модели: воспроизводится выбор, а не её генерация. Для проверки изменения селектора старые записи можно пересчитать новой версией и увидеть расхождения. решено

Отказ тоже является результатом. Маршрутизатор различает отсутствие подходящего кандидата, неверный профиль, допуск или политику, тайм-аут и сбой предварительной проверки. Ошибка не выдаётся за выбор, а старое умолчание вызывающего — за решение маршрутизатора. Объяснение строится из структурированных причин. Его можно проверять по сохранённым входам, а не принимать как убедительный свободный текст ещё одной модели. работает сегодня

14.6 Обёртка сегодня, шов в доске — дальше

Работающий путь подключения — `cmr spawn`, оборачивающий обычный запуск доски. В `select` он заполняет недостающие среду, модель и effort; в `recommend` печатает команду без запуска; в `shadow` доска получает исходные аргументы, а маршрутизатор рядом считает совет. Все реальные запуски по-прежнему проходят допуск, лимиты и проверку пары доски. Обёртка передаёт её вывод и код возврата. работает сегодня

Теневой режим начинает запуск доски раньше подготовки совета и ограничивает время совета. Для содержательного сравнения он убирает пересланные закрепки вызывающего: иначе совпадение получилось бы по построению. Ошибка совета не останавливает основной запуск, но записывается отдельно и не считается согласием. Если доска вернулась раньше, недосчитанный совет может завершиться отдельным ограниченным по времени процессом без повторного запуска задачи. работает сегодня

Наблюдения отличают точное совпадение от другой модели, другого effort или другой среды. Когда вызывающий не указал полную конфигурацию и доска выбрала умолчание, сравнение не установлено. Отсутствие совета также имеет отдельный исход. Теневой отчёт должен показывать эти случаи, иначе высокий процент согласия может означать ошибку измерения, а не хороший выбор. Перед переключением на автоматический выбор принята неделя наблюдений; готовность обёртки не заменяет пригодных данных. решено

Внутренний шов переносит маршрутизатор в точку выбора `task-board spawn`. В `off` предполагается прежнее поведение; в `shadow` выбирает прежний механизм, в `select` порядок маршрутизатора заменяет умолчания. Явные флаги вызывающего обходят маршрутизатор; при ошибке возможен возврат к прежнему механизму с отметкой происхождения решения. Каждый кандидат всё равно проходит допуск и подтверждения. Код шва подготовлен, но ещё ждёт интеграции и проверок: это не действующее поведение доски. предложено

Библиотечная граница позволяет подключать других вызывающих. Им нужны профиль задачи, допущенные кандидаты, применение результата и сохранённая связь решения с запуском. Swarm Curator должен разрешить окружение и закрепки, затем обратиться к маршрутизатору и отдельно проверить доступность. Будущий пул удалённых работников добавит к конфигурации предложение и совместимое размещение; резервировать его будет координатор, не селектор. предложено

14.7 Локальные модели: имя не удостоверяет веса

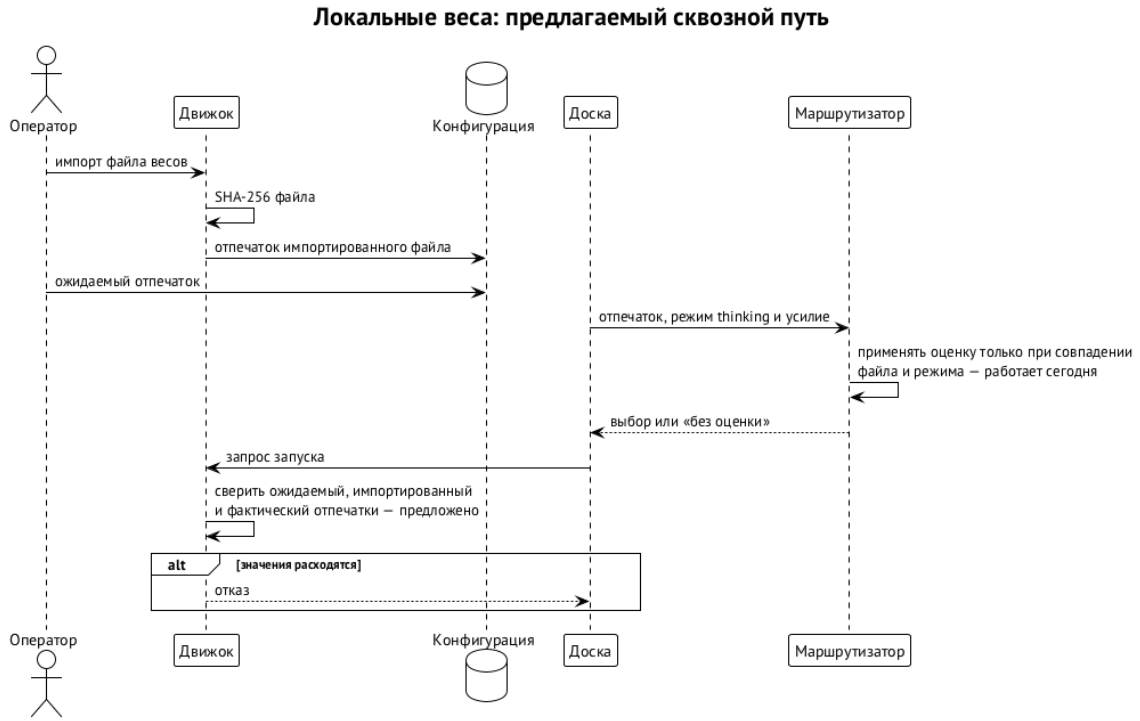
Оценка локальной модели привязывается к SHA-256 файла весов, режиму мышления и точному effort. Один и тот же файл сохраняет отпечаток независимо от размещения; Q4 и Q8 имеют разные отпечатки. Первый контракт охватывает один неразбитый GGUF, включая токенизатор и встроенный шаблон чата. Обязательная закрепка ожидаемого отпечатка выбрана как правило для каждой локальной модели. решено

Маршрутизатор уже применяет оценку только при совпадении файла и режима. Неоценённые веса не выбираются автоматически, но явная полная закрепка может вернуть их с пометкой «без оценки». Замер конкретного файла для той же оси и области задач главнее переноса с базовой модели. Перенос учитывает коэффициент квантизации и расширенную неопределённость; коэффициент не превращает предположение в собственный эксперимент. работает сегодня

Отпечаток при импорте, пересчёт открытого файла движком перед запуском и сверка ожидаемого, импортированного и фактического значений ещё требуют реализации сопряжённых частей. Передача этой тройки через доску также зависит от статуса движка. Наличие строгой проверки оценки в маршрутизаторе не доказывает, что на исполнении уже проверен фактический файл. Поэтому сквозной путь локальных весов остаётся предложением. предложено

Пример ошибки привязки — оператор заменил файл под прежним именем модели, сохранив оценку и закрепку старых весов. Маршрутизатор не должен переносить оценку на новый отпечаток. Движок в предлагаемом сквозном пути обязан обнаружить расхождение фактического файла до исполнения. Эти проверки относятся к разным границам: первая защищает смысл оценки, вторая — соответствие запуска разрешённым весам. Пока вторая не подключена, первая не даёт права объявлять весь путь проверенным. предложено

Локальный профиль запуска также не является общей гарантией доступности. Проект может назвать профиль, но конкретный оператор сопоставляет его своему движку и временному окну; у другого оператора соответствия может не быть. Доступность движка разделяется на готовность сейчас и возможность подготовить его. Ни одно из этих состояний не обещает подходящего качества. Маршрутизатор выбирает после разрешения таких ограничений, а поддержание движка и резервирование остаются обязанностями соседних компонентов. решено



В предлагаемом сквозном пути отпечаток связывает оценку с файлом весов, а движок отдельно сверяет фактический файл перед запуском.

14.8 Почему выбор детерминирован

Маршрутизатор не вызывает LLM для выбора: одинаковые метки, кандидаты, данные и политика дают одинаковый результат. Это не требует токенов на ещё одного управляющего агента, позволяет аудит и сравнение версий. Текст задачи не может напрямую уговорить селектор выбрать запрещённую модель, поскольку селектор его не читает. Однако неверная разметка или слабые данные остаются слабым входом; детерминизм не делает их истинными. **решено**

Позже оценщик задачи может получать текст и выдавать типизированные требования. Рассматриваются модель решений Jev и небольшая LLM со строгой схемой. Бэкенд задаётся явно политикой, ответ замораживается, дальше работает прежний чистый селектор. Уверенность классификатора, покрытие свидетельствами и калибровка различаются; уверенность не равна вероятности успеха агента. Такая подготовка должна быть явно платной операцией с разделёнными кэшами и границами данных проектов. **предложено**

Сначала нужно накопить исходы, сравнить метки оценщика с метками оркестратора в тени и проверить, улучшится ли стоимость приёмки. Только затем можно заполнять недостающие метки автоматически, сохраняя приоритет явных. Обучаемый маршрутизатор, сложные рейтинги и числовые пороги без данных не являются следующим обязательным шагом. Открыты первый бэкенд оценки, сводка параллельного поиска и допустимая потеря качества; автоматический выбор по честным меткам от этих решений не зависит. **открыто**

15 Память агентов

Память агента должна переживать замену исполнителя: сохранять основания решений, историю их пересмотра и разрешённый опыт. Например, новый агент продолжает работу над архитектурой и находит старое правило «архивировать дом агента по умолчанию». Позже владелец заменил его прави-

лом «при выводе агента из работы удалять дом»: отсутствие секретов в содержимом дома нельзя гарантировать. Обычный поиск может вернуть обе формулировки. От памяти требуется показать действующее решение, его источник и зарегистрированные документы, которые ещё опираются на прежнее правило. предложено

Для этого предлагается **ограниченная доказательная память** — bounded evidence memory. Она хранит типизированные утверждения вместе с источниками, обоснованиями, оценками и ревизиями. Её вопрос: какие допущенные свидетельства поддерживают или опровергают конкретное утверждение, с чьей точки зрения и на какой момент? Память не устанавливает истину, не принимает решения за владельца и не выдаёт полномочия. Пустой ответ не доказывает ложность утверждения. предложено

На октябрь 2026 года это устройство в проработке: реализации и измеренного выигрыша ещё нет. Выбран переносимый смысловой контракт и ядро, а первая реализация и проверка пользы должны пройти внутри Swarma. Первый сценарий — непрерывность архитектурных решений. Личная память, извлечение из журналов, консолидация и модели других агентов придут позже поверх ядра, без расширения его полномочий. решено

15.1 Что принадлежит памяти

Память ведёт историю собственных утверждений и ревизий. Внешние события она связывает с авторитетными журналами по точной ревизии, сохраняя различие между событием и утверждением о нём. Если агент написал «результат принят», это ещё не запись о приёмке: нужен источник от компонента, которому принадлежит такое решение. Собственного общего журнала вместо журналов остальных модулей у памяти нет. предложено

Границы ответственности предлагаются такими: предложено

Сведения	Где остаётся авторитет	Что получает память
Поколение учётки и её вывод из работы	Менеджер пользователей (`swarma-user-manager`)	Версионированное свидетельство, которое не заменяет текущую проверку
Учётные данные, гранты и отзывы	Брокер учётных данных (`swarma-credential-broker`) и владельцы политики	Ссылки без секретов и второго реестра прав
Размещение и исполнение	Диспетчер (`swarma-dispatcher`)	Ссылку на каноническую запись исполнения
Журнал и доставка в сессию	Хост сессий (`agent-session-host`)	Допущенный экспорт для чтения, без доступа ко вводу
Цель, состояние задачи и принятый кандидат	Доска задач	Точную ревизию, без права менять задачу или принимать результат
Обязательства и эскалация	Swarm Keeper в целевом устройстве	Свидетельства, без второго реестра обязательств
Профили, скиллы и инструкции	Swarm Curator	Сведения о конфигурации, без права её исправлять

Так, воспоминание о вчерашнем гранте не разрешает сегодняшнее действие, а заметка «мы обычно пользуемся этим инструментом» не устанавливает инструмент и не меняет профиль запуска. Изменение основания может дать повод для ревью; открыть работу и принять её результат должен уполномоченный потребитель. Остановка, парковка, отзыв прав, восстановление и приёмка никогда не должны ждать обхода памяти или консолидации моделью. предложено

15.2 Пять видов записей

Ядро предлагается построить на пяти видах неизменяемых версий записей. Это смысловая модель, а не опубликованный формат обмена. предложено

Запись	Назначение	Пример и предел
Утверждение — Assertion	Типизированное высказывание о субъекте, с точкой зрения и временем действия	«При выводе дом удаляется»; запись о решении требует квитанции решающего
Ссылка на источник — Source reference	Система-владелец, объект, точная ревизия или позиция, доступный дайджест, происхождение и доступность	Ссылка на версию документа; сама ссылка не даёт права прочитать документ
Обоснование — Justification	Поддержка или опровержение определённой версии утверждения: источники, посылки, правило с версией	Документ служит основанием; обычный абзац рассуждений не становится исполняемым правилом
Оценка — Assessment	Результат именованной проверки с привязкой к предмету, правилу, политике и окружению	Проверка подписи подтверждает происхождение, но не истинность сказанного
Ревизия — Revision	Отношение к точной прежней версии: замена, отзыв, исправление, оспаривание или изменение доступности	Новое правило заменяет старое; прежние байты не переписываются

У оценок различаются проверка привязки к источнику, подпись, результат теста, формальный вывод и смысловое ревью. Оценка LLM остаётся смысловым суждением и не может выдать себя за детерминированную проверку. Цитата «владелец сказал X» подтверждает авторство высказывания; для более сильного вывода нужно отдельное основание. Единой галочки «проверено», скрывающей эти различия, недостаточно. предложено

У каждой записи различаются **субъект** — о ком она, **издатель** — кто отвечает за содержание, **отправитель** — кто технически внёс запись, и **точка зрения** — чью картину мира она выражает. Отправителя устанавливает аутентифицированный приём. Если он действует от имени издателя, допуск проверяет ссылку на делегирование у владельца политики. Неудачная проверка означает отказ; превращать такую попытку в собственное наблюдение отправителя нельзя. предложено

Например, агент передал решение владельца и агент предположил, какое решение владелец предпочёл бы, — разные записи даже при одинаковом тексте. Это различие должно пережить выжимку, индексирование и экспорт. Личности и области политики остаются внешними ссылками: ядро не создаёт для них параллельные реестры. предложено

15.3 Как получается текущее представление

Представление, View, — вычисляемый ограниченный ответ, а не шестой хранимый примитив. Для него задаётся именованное правило с версией: какие издатели и точки зрения учитываются, какой выбран момент, как разрешаются последовательности версий и оценивается свежесть. Предлагаются три семейства правил: текущее состояние от издателя, модель наблюдателя и разрешённая проекция для аудитории. Чужое мнение не должно незаметно становиться самоописанием или решением владельца. предложено

Содержимое утверждения подчиняется названной версионированной предметной схеме. Она определяет типы и равенство, вид высказывания, кратность значений, условия несовместимости, свежесть и ограничения раскрытия. Два интереса могут сосуществовать. Старое и новое правила могут относиться к разным интервалам времени. Два предложения ещё не являются противоречащими принятыми решениями. Неизвестная схема отклоняется; схема содержит данные, а не произвольный исполняемый код. предложено

Заменить, исправить или отозвать запись в последовательности может её издатель либо допущенный представитель. Другой участник может оспорить её или записать свою оценку в собственной последовательности. Более поздняя заметка агента об архивировании дома поэтому не заменяет решение владельца об удалении. Для решения текущая версия определяется порядком авторитетного реестра, например порядком коммитов прошедшего ревью реестра решений; время поступления в память этого порядка не меняет. предложено

Три результата следует различать и в API, и при показе читателю: предложено

Ситуация	Результат	Чего из него не следует
Два представителя меняют одну ожидаемую версию	Первая запись проходит, вторая получает `version_conflict`; ничего не фиксируется второй операцией	Что знания о субъекте противоречивы
У решения две авторитетные замены, между которыми нет установленного порядка	Представление возвращает `ambiguous_current` и обе версии	Что позднее полученная версия победила
Найдены допустимые основания за и против утверждения	Конфликт свидетельств; обе стороны сохраняются	Что запись одной стороны надо отклонить

Время действия, момент последнего подтверждения и горизонт использования без обновления тоже различаются. По истечении последнего сведения становятся `stale`: требуют обновления, но не превращаются в ложь. Для архитектурного решения горизонт свежести обычно не нужен; для местонахождения, доступности и намерения он существенен. Отсутствие опровержения не делает старое местонахождение текущим. предложено

15.4 Свидетельство и полнота поиска

Для поддержки и опровержения ответ строится независимо. По каждой полярности различаются наличие найденного свидетеля — `present` или `not_found` — и завершённость оценки — `complete` или `incomplete`. Свидетель здесь означает допустимое основание соответствующей полярности, а не гарантию истины. Если основание за удаление дома найдено, но поиск возражений прервался, можно предъявить основание; нельзя сказать «возражений нет». предложено

Сочетание `not_found` и `incomplete` означает неизвестность. Недоступный источник, отсутствующие байты, незавершённый приём и исчерпание бюджета не становятся опровержением. Если найдены основания обеих полярностей, конфликт уже установлен, даже когда перебор не закончен. Усечённый показ объяснений и усечённое вычисление различаются: первое может сопровождать полную оценку, второе обязано дать `incomplete`. предложено

Полнота ограничена зарегистрированным разрешённым графом на выбранный момент. Проверка всех трёх зарегистрированных оснований не доказывает, что при подготовке реестра не пропустили четвёртое в исходном документе. Поэтому покрытие исходного корпуса учитывается отдельно и может оставаться неизвестным при полной оценке. Выдача лучших совпадений поиска сама по себе не является полной проверкой свидетельств. Аналогично пересмотр охватывает зарегистрированные зависимости, а не все неявные смысловые последствия текста. предложено

В первом срезе достаточно явных источников, документальных оснований, замен и связей «пересмотреть при изменении». Документ, зарегистрировавший предположение об архивировании дома, возвращается как требующий проверки. Память не объявляет его автоматически неверным: возможно, пересмотру подлежит только часть документа. Само по себе прозаическое объяснение не даёт права вычислять логические следствия. предложено

Следующий срез добавляет конечные ациклические правила: внутри набора посылок — И, между альтернативными правилами — ИЛИ. Если C поддерживается через A и B либо через D и E, отзыв B выключает только первый путь. Пока второй допустим, поддержка C сохраняется. Когда отпали все пути, отрицание C из этого не выводится. Полная система сопровождения альтернативных допуще-

ний, ATMS, отложена до измеренного сценария, которому действительно нужен перебор гипотетических окружений. предложено

Конфликтная посылка не должна исчезать при сокращении ответа. Если A имеет основания за и против, допущенное правило $A^{+} \rightarrow B^{+}$ может дать B квалифицированную поддержку, но не автоматическое B^{-}. Представление сохраняет различие: есть путь без конфликтных посылок; поддержка найдена только через них; вопрос проверен не полностью. Можно ли действовать на таком основании, решает потребитель. предложено

15.5 Сначала документальный реестр

Первым выбран документальный профиль: реестр явно допущенных нечувствительных записей в git, обычное ревью репозитория и адаптер только для чтения. Агенты готовят предложения, человек принимает записи. Внешние факты остаются на карантине до этого ревью. Принятие записи в память не даёт ревьюверу права объявить чужое решение принятым: для этого всё равно требуется квитанция того, кто решал. решено

Рабочее правило допуска простое: если содержимое не положили бы в README репозитория, ему не место в реестре. Явно допущенные нечувствительные выдержки разрешены с отдельным основанием хранения; чувствительные пересказы в git не попадают. История git удобна для воспроизводимости обоснования в выбранных границах хранения, но не служит механизмом стирания частных данных. Реальные личные профили этот реестр не принимает. решено

Автоматического приёма сырых журналов сессий в первой версии нет. Отдельный инструмент может предлагать записи из уже допущенных документов, но его выход проходит то же ревью. Автоматизация отдельных классов допуска рассматривается после пилота по измеренной стоимости сопровождения. решено

Допуск предполагает явно названные классы источников и цель хранения. Корректная схема, хэш, подпись, влитый коммит или фильтр LLM не доказывают отсутствия секретов. Начальный корпус исключает личную память, сырые разговоры, учётные данные и произвольный вывод инструментов. предложено

Защищённая служба нужна только по конкретной причине: допущены частные данные, требуется автономная запись без ревью каждого изменения или появилась измеренная нагрузка. Её строительство привязано к отдельному пользователю ОС и посреднику, а развёртывание требует собственного контракта допуска. Создавать службу ради архитектурной полноты не предполагается. решено

Для такого профиля предложена одна непривилегированная служба с единственным писателем. Ей нужны закрытое состояние, единственный API записи, права на сокет, проверка UID собеседника и поколения учётки, удостоверенная личность сервера, явно допущенные роли клиентов, собственные правила выхода в сеть и проверка готовности. Одно имя служебной учётки этих гарантий не даёт. Прохождение смысловых тестов документального профиля также не доказывает изоляцию защищённой службы. предложено

15.6 Данные для агента, без веса инструкций

В первом срезе агент получает память только явным результатом инструмента. Автоматического добавления памяти при запуске нет. Пакет знаний отделяется от авторитетной текущей задачи и от пакета продолжения хоста сессий. У каждого элемента при пересказе, кэшировании и передаче должны сохраняться происхождение и классификация `data`. Подпись службы удостоверяет конверт, но не повышает полномочия вложенного текста. предложено

Для более позднего снимка в начале сессии канал уже выбран: отдельный блок данных с происхождением каждого элемента и пометкой `data`. Память не включается в оверлей Swarm Curator, `AGENTS.md`, корень скилла или машинный базовый контекст. Низкий вес оверлея только меняет порядок инструкций и не превращает их в данные; у памяти веса инструкций нет вовсе. решено

Снимок неизменяем как исторический артефакт своего временного среза. Если решение обновилось, старый снимок остаётся историей, а текущее состояние получают свежим запросом. Если же отозвано использование или раскрытие, перед следующей доставкой или внешним эффектом ответственный компонент применяет текущую политику: отказывает либо пересобирает допущенный контекст. Память сообщает изменение, но сама не перезапускает агента. Кэш промпта не продлевает разрешений. предложено

Чтение, использование в задаче, хранение и дальнейшая передача требуют отдельных разрешений. Представление для одной задачи не разрешено автоматически другой задаче или восстановленному исполнению. Перед разрешением источника и новой доставкой права проверяются заново. В частности, право читать локально не разрешает отправить байты провайдеру LLM или эмбеддингов. предложено

Пометка `data` не гарантирует, что модель не последует вредоносному тексту. Обещание здесь — сохранение происхождения и внешних проверок полномочий, а не безошибочность LLM. Скрытые записи не должны менять раскрытые статусы, счётчики, ранжирование, курсоры, покрытие и уведомления. Даже логический бюджет должен учитывать только разрешённую работу: иначе большое число скрытых узлов выдаст себя переходом к `incomplete`. Постоянного времени ответа и защиты от анализа трафика этот контракт не обещает. предложено



В предлагаемом первом срезе память приходит явным результатом инструмента с происхождением данных и отдельной проверкой права использования.

15.7 Происхождение содержимого и стирание

Логическая поддержка и происхождение содержимого — разные связи. Выжимка может пересказать частную запись, не используя её как посылку вывода. Поэтому отдельная линия происхождения и хранения должна охватывать выдержки, пересказы, эмбеддинги, индексы, кэши, контролируемые экспорты, очереди и диагностику. Производная запись наследует ограничения раскрытия всех входов; метка «публично» их не снимает. предложено

Список цитат от агента не является полным происхождением. Если модель получила приватный вход и публичный документ, но сослалась только на документ, приватный вход не исчез. Интеграция учитывает контролируемые входы: для детерминированного преобразования — прочитанные данные, для вызова модели — переданные контекст, результаты инструментов и представления памяти. Не-

установленное происхождение не считается публичным по умолчанию. Обещание ограничено отслеживанием конкретного профиля интеграции, а не всем, что модель когда-либо знала. предложено

Раскрыть результат над закрытыми входами может разрешить владелец политики каждого ограниченного входа. Например, разрешение опубликовать город из приватных координат не разрешает сами координаты. Оценка фиксирует свидетельство этого решения с привязкой к входам, преобразованию, выходной схеме, аудитории и цели; она не создаёт полномочие и не заменяет проверку перед новым раскрытием. Мнение модели о безопасности пересказа только предлагает такое решение. предложено

В защищённой службе чувствительное содержимое предполагается хранить в стираемых объектах отдельно от логической истории. Стирание сначала ограждает чтение и фиксацию производных затронутого поколения, чтобы поздний извлекатель не вернул удалённое. Исходы должны различать закрытый доступ, ожидающую очистку, очистку в названной границе и недоступность. Восстановление старой копии остаётся закрытым до сверки с актуальными обязательствами стирания у свидетеля вне восстановленного домена отказа. предложено

Недоступный источник не изображается только что проверенным. Представление различает доступные сейчас байты, сохранившуюся прежнюю оценку и самостоятельную достаточную аттестацию. Нельзя заявить удаление у независимого владельца без его пути удаления и квитанции; уже доставленные внешней модели байты вернуть нельзя. Какие классы источников должны оставаться предъявимыми и как долго — ещё предстоит определить. открыто

15.8 Профили и память об участниках

В дизайн включён второй сценарий: типизированные представления о людях и автономных агентах. Он проверяет общность ядра, но не добавляет в него сущности человека, профиля, намерения или каталога. Профили и поиск полезных связей — предметный пакет и сервисы над ядром. Пилот остаётся на архитектурных решениях, а проверка профилей начинается с синтетического корпуса. решено

Например, агент заметил интерес владельца к теме. Это его модель наблюдателя; переданное по полномочию самоописание владельца — другое утверждение. Заявленная компетенция отличается от опыта конкретных работ, ссылка на аккаунт — от проверенного контроля аккаунта, намерение — от обязательства. История успешного ревью хранится вместе с конфигурацией исполнения, но вывод «этому агенту можно доверить следующую работу» остаётся решением потребителя (глава «Аттестации и переносимая репутация»). предложено

Память строит разрешённую проекцию для аудитории; отдельный издатель выпускает карточку, каталог хранит публикацию, чужая память импортирует её как внешний источник. Одна самодекларация, прошедшая через три каталога, остаётся одним корневым свидетельством. Без проверки текущей версии у издателя, известных замен и срока свежести импортёр знает лишь последнее полученное состояние. Публикация имеет собственные идентификаторы; связь профессионального и личного представлений тоже требует разрешения на раскрытие. предложено

Изменить утверждение, отозвать дальнейшее раскрытие, снять публикацию и стереть содержимое — четыре разных действия. Владелец своей проекции может убрать из неё чужую аттестацию, не отзывая исходную запись её издателя. Контакты и знакомства остаются предметным слоем: по умолчанию нет точного публичного местонахождения, контакты раскрываются после взаимного согласия, романтический режим требует явного согласия взрослых участников и не выводится из статуса отношений. предложено

15.9 Что отложено и как проверять пользу

В первую версию не входят полный ATMS, общий Datalog, доказательство теорем, автоматические ревью, активация процедурной памяти, расписание снов, федерация, консенсус и якорение в Swarm Chain. Из прежней идеи лестницы выжимок сохраняется требование ссылаться на пересказываемое и

соблюдать ограничения производных данных. Детерминированный дистиллятор журналов и извлечение моделью отложены до подтверждения пользы пилота. предложено

«Сон» — предлагаемая репетиция процесса без реальных внешних эффектов — образует отдельный контекст свидетельств. Смоделированная приёмка или результат теста не удовлетворяют требованию реального исполнения. Выжимка такой репетиции подтверждает только репетицию и остаётся на карантине до ревью человеком. Сон не является способом автоматически пополнить принятую память. предложено

Следующий шаг — единый пакет нормативного текста, закрытых схем и исполняемых векторов соответствия. Его форма выбрана; смысловые проверки ядра и проверки защищённого развёртывания должны отчитываться раздельно. Переносимость означает эквивалентный смысловой результат при одинаковых корпусе, версиях схем и правил, точке зрения, разрешённой проекции, моменте оценки и логическом бюджете. Случайные непрозрачные идентификаторы допускают оговорённую нормализацию. предложено

Конкретные схемы, канонические байты и дайджесты, грамматика правил, словарь авторизации и машинно-читаемые проверочные примеры ещё не опубликованы как закрытый пакет. Предложенные правила требуют исполнения: поздно полученное старое решение не становится текущим, неполный поиск не объявляется полным, скрытые данные не меняют разрешённый ответ, отзыв раскрытия действует после создания снимка. Проверки должны также ловить потерю квалификации конфликтной посылки и подмену происхождения списком цитат. открыто

Предлагаемый пилот охватывает 10–30 решений с точными ревизиями источников, включая замену архивирования дома удалением. Сравнение идёт с обычным поиском и типизированными версионированными документами при одинаковых правах и вопросах. Измеряются правильность текущего решения и источника, устаревшие ответы, точность свидетельств, пропущенные зарегистрированные зависимости, ложные призывы к пересмотру, время ответа и повторное чтение. Отказы и исчерпание бюджета остаются в знаменателе. предложено

Отдельно учитываются стоимость подготовки и ревью корпуса, задержка актуализации и результат рабочей задачи: не вернул ли новый агент отменённое решение и заметил ли затронутые зависимости. Иначе улучшение от ручной уборки документов легко приписать движку. Автоматическое извлечение и более богатый вывод оправданы только измеренным улучшением без ослабления границ; численной победы у памяти пока нет. предложено

16 Оператор и интерфейсы

Оператор — человек, который допускает работу платформы, выдаёт полномочия и вмешивается там, где агенту нельзя решать за владельца. Ему нужны разные способы взаимодействия: посмотреть ход работы, поговорить с агентом, одобрить конкретное действие, выдать постоянное право или остановить систему. Разделение этих действий сохраняется независимо от того, нажал человек кнопку на телефоне или набрал команду в терминале. решено

Интерфейс не заменяет проверку полномочий. Чат удобен для обсуждения, терминал — для работы со средой, телефон с защищённым ключом — для присутствующего подтверждения. Сообщение в комнате и подпись над точным действием имеют разный смысл. Иначе доступ к переписке превратился бы в право менять правила, по которым работает рой. решено

16.1 Работающие входы

Сегодня основные инструменты — `task-board` и `curator`. Доска хранит задачи, цели и ресурсы, запускает дочерние исполнения, управляет доставкой изменений и предоставляет вход к сессиям.

Swarm Curator устанавливает и проверяет окружение агента, управляет профилями и запускает среду через провайдер `curator-run`. Обёртки `anthropic-board` и `openai-board` ведут к сессиям доски. работает сегодня

Инструмент	Что через него доступно сегодня
`task-board`	Запросы и мутации данных: `q`, `m`; поиск, дерево, план, ресурсы и цели
`task-board spawn`	Дочерние исполнения
`task-board worktree`, `board commit`, `board publish`	Рабочие деревья и доставка изменений
`task-board claude`, `task-board codex`, `session`	Запуск, подключение и операции с сессиями
`curator`	`init`, `add`, `install`, `status`, работа с профилями и окружениями, `audit`, `run`

Это карта работающих входов, а не обещание, что все будущие возможности будут спрятаны под теми же словами. Например, наличие `task-board session` не означает, что служба хоста сессий уже отделена от доски. Вынос сначала должен сохранить существующие команды и поведение. Изоляция пользователей и новый жизненный цикл сессии описаны в главе «Запуск агента и хост сессий». решено

Будущие команды для Swarm Keeper, памяти, доверия, почты, парковки и аварийной остановки нельзя читать как руководство к установленной версии. Часть возможностей решена, но их форма пересматривается. Полезное свойство интерфейса здесь — возможность понять, что именно он собирается сделать, не угадывая состояние реализации по имени команды. предложено

16.2 Куда должен прийти CLI

Предлагается разделить аудитории: Swarm Keeper — повседневный вход человека, `curator` — установка и настройка машины, `task-board` — ядро для агентов, таймера, мостов и самого кипера. Незнакомое существительное у `curator` может передаваться провайдеру `curator-<существительное>`. Это предложение формы CLI, а не утверждённая замена работающих команд. предложено

Основной порядок — существительное, затем глагол. Один словарь должен различать просмотр (`list`, `show`, `status`), изменение настройки (`set`, `unset`), проверку и объяснение (`check`, `explain`). Особенно важно не смешивать три вида человеческого решения: подтвердить версию конфигурации (`confirm`), одобрить ожидающий пункт (`approve`, `reject`) и выдать или отозвать постоянное право (`grant`, `revoke`). предложено

Например, оператор один раз разрешил конкретный запрос среды. Такое одобрение не должно становиться бессрочным правом на все будущие запросы. Наоборот, выданный грант действует до своих ограничений и отзыва, а не только до закрытия текущего диалога. Точное различие полномочий задаёт контракт действия; интерфейс должен показывать его, а не прятать за общей кнопкой «да». Присутствующее подтверждение точного содержимого для расширяющих действий уже принято как правило платформы. решено

Предлагается также оставить слову `profile` одно значение — профиль Swarm Curator — и единообразно обозначать агентскую среду. Управление сессиями предполагается собрать в `curator session`, а кипер будет его оборачивать. Перенос команд и судьба существующих обёрток требуют согласования с выносом хоста сессий. Единую форму аварийной остановки ещё предстоит выбрать; две конкурирующие команды нельзя публиковать как два равноценных действующих входа. предложено

16.3 Каналы для разговора и наблюдения

Первым интерфейсом предлагается терминал: подключение показывает среду такой, какая она есть, а выход клиента оставляет сессию работать. Просмотр и управление должны быть различимы: наблюдение за выводом не означает права печатать во ввод. Мессенджеры подключаются через провайдеры моста и позволяют спросить или ответить, но обычное сообщение в канале не становится актом одобрения. предложено

Подписанный путь сообщения описан в главе «Путь сообщения и мост». Для оператора существенно, что учётка в Matrix и присутствие в комнате не дают `session.input`, `session.approve` или `session.control` автоматически. Подписывающее клиентское приложение должно передать соответствующее типизированное действие; провайдер и хост сессий проверяют его независимо от того, как обычный клиент показал текст. решено

Например, человек пишет агенту с телефона просьбу проверить изменение. В целевом пути подпись едет в метаданных, ворота проверяют ключ и грант, guard сверяет просьбу, а хост сессий доставляет звонок. Агент читает текст и результат проверки из ящика. Неподписанный текст остаётся данными: видимость имени человека в клиенте мессенджера не подменяет криптографического подтверждения. Этот путь пока относится к архитектуре моста. предложено

16.4 Телефон оператора

Приложение-компаньон должно показывать точное действие и подписывать его ключом в защищённом чипе телефона, после подтверждения присутствия человека. Направление принято, iOS выбрана первой платформой, работа над приложением ещё не начата. Настольный пульт и shell по SSH рассматриваются позднее. Их нельзя смешивать с ограниченным операторским входом для подписи действия. решено

Правило расширяющих действий действует с первого этапа платформы. Выдача гранта, подключение подписки, постоянная команда, вывод или пересоздание постоянного агента, сохранение дома, полномочия на остановку для обслуживания и продление постоянных грантов требуют показа точного содержимого. Оператор подписывает телефоном либо подтверждает паролем в терминале. Изнутри сессии агента такое действие не принимается, старое подтверждение не разрешает будущие изменения. решено

Специфицированный телефонный путь использует SSH через tailnet в учётку `swarma-operator`. У неё нет shell: только операторская команда. Она выдаёт каноническое содержимое действия, приложение показывает получателя, действия, аккаунты и профили, срок. Человек подтверждает биометрией; телефон подписывает именно эту запись. Команда сверяет подпись с закреплённым ключом телефона и передаёт действие брокеру. Подпись одного действия не подходит для другого. спецификация

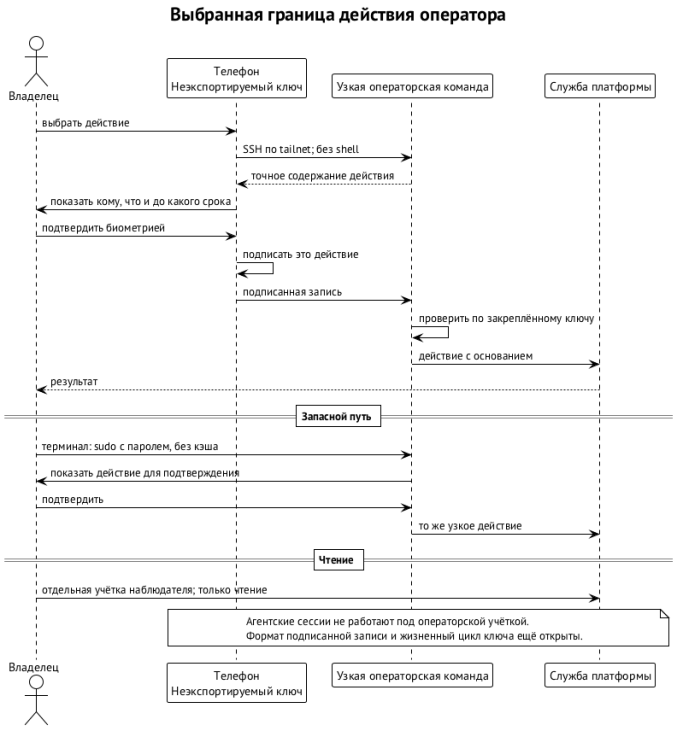
Например, новому оркестратору нужен грант на запуск агентов. Оператор видит, кому он выдаёт право и какими профилями и сроком оно ограничено. После подписи запрос передаётся брокеру. Вечером оператор отзывает грант отдельным действием; брокер сохраняет отзыв на диск до ответа. Отзыв задуман как одно касание с биометрией, а не редактирование переписки, из которой агент должен догадаться, что право закончилось. спецификация

Без телефона используется терминальный вход: `sudo` в `swarma-operator` с паролем без кэша и с тем же подтверждением содержимого. Для просмотра выданных прав и привязок предусмотрена учётка `swarma-observer`, без подтверждений. Ни в одной из этих учёток не работает агентская сессия. При сохранении границы пользователей ОС инъекция в агента не получает из неё подпись владельца. Это не защита от компрометации ядра или самого устройства подписи. спецификация

Просмотр через наблюдателя нужен, чтобы оператор мог сначала выяснить состояние и только затем решить, что менять. Например, увидеть действующий грант и привязку аккаунта можно без выдачи нового права. А последующее продление гранта уже требует присутствующего подтверждения: прочитанное состояние не является разрешением на изменение. Одинаковое разделение сохраняется

при входе с телефона и из терминала, поэтому более удобный интерфейс не должен превращать наблюдение в неявную подпись. спецификация

Человек может делегировать свой шаг оркестратору с потолком и сроком: например, разрешить одобрять заказы в пределах суммы. Право менять сами правила таким способом не передаётся. Поэтому телефон нужен не как обязательный посредник для каждого чтения или сообщения, а как независимая точка подтверждения там, где действительно расширяются возможности системы. Гранты и делегирование разобраны в главе «Гранты и полномочия». решено



В выбранном операторском пути телефон показывает и подписывает точное действие, терминал служит запасным входом, а чтение отделено от изменения.

16.5 Несколько машин и людей

Рост за пределы одной машины не должен создавать второго главного оркестратора над той же доской. Предложены четыре направления; это варианты архитектуры, реализации у них пока нет. В каждом сохраняется различие между местом исполнения и владельцем полномочий. предложено

Направление	Как сохраняется координация
Один проект на нескольких машинах	Одна доска и главный оркестратор на машине-владельце; другие машины исполняют подзадачи через подоркестраторов по аренде
Несколько проектов одного владельца	Один Swarm Keeper координирует остальные через подписанные сообщения, получая отчёты и направляя запросы
Несколько операторов	Операторы выступают самостоятельными подписантами; предусмотрено направление подтверждений M из N, иерархия остаётся открытой
Агенты других владельцев	Допуск по разрешённым ключам, подписям и аренде области работы

Каждая машина сама проверяет подписанное подтверждение. Объявления другой машины, что конфигурация одобрена, недостаточно. Транспорт может быть tailnet, другой сетью или ретранслятором

со сквозным шифрованием; он узнаёт машину, но не определяет, что ей можно. Поэтому смена сети не должна менять корень доверия и право на действие. решено

Для оператора это означает одно действие подтверждения, которое машины проверяют самостоятельно, а не цепочку доверия к словам соседней машины. В предлагаемом реестре проекта сохраняются постоянный идентификатор и машина-владелец. Общение кипера с оркестраторами идёт через подписанные сообщения даже локально: при добавлении машины меняется носитель, а не смысл запроса. При этом координация нескольких киперов не превращает одного из них в неограниченного владельца чужих полномочий. предложено

Общее состояние требует общего хранилища аренд. До его появления периодический `tick` общей доски выполняется на одной машине, а делегирование с бюджетом на доске нескольких машин запрещено. Например, добавление второй машины с доступом к репозиторию не разрешает ей независимо распределять тот же бюджет. Способ перевозки сообщений не устраняет необходимость единственного согласованного решения о таком расходе. решено

Запуск на другой своей машине также не равен нативному возобновлению. Предложен запрос с профилем и пином: целевая машина собирает план сама и отказывает при несовпадении пина. Контекст переносится пересборкой из пакета передачи; точного контракта удалённого запуска пока нет. Удалённый работник другого владельца — иной путь, разобранный в главе «Удалённое рабочее место». предложено

Публичный ключ участника и локальный допуск не следует автоматически трактовать как готовую переносимую репутацию. Связь личности агента, аттестаций и Swarm Chain остаётся предложением; она рассмотрена в главах «Идентичность агента» и «Аттестации и переносимая репутация». Даже будущая история успешных работ не заменяет локальное решение, какие права выдать пришедшему агенту. предложено

16.6 Что остаётся спроектировать

Визуальный конструктор процессов пока отсутствует. Схема `Playbook.json` подготовлена для него: блоки `ui`, чтение и запись без потерь, проверка `playbook validate` после каждой правки. Это готовность формата к интерфейсу, а не готовность самого редактора. Пользовательский пульт не должен обходить проверку ядра ради удобства редактирования. решено

Открыты окончательный словарь CLI, форма единой аварийной остановки, иерархия нескольких операторов и контракт удалённого запуска. Истёкший логин среды тоже пока не имеет завершённого телефонного сценария: предложены ссылка владельцу аккаунта, новый вход у провайдера и продолжение с новой арендой, но ответственность за этот путь и доставка событий брокера хосту сессий ещё не определены. Наличие телефона с ключом не закрывает эти вопросы само по себе. открыто

Часть III. Swarm Protocol: много владельцев, одна среда

17 Модель протокола

Главы «Пользователь ОС на агента и сеть агента», «Ключи и ключник», «Гранты и полномочия» и «Запуск агента и хост сессий» описывают локальные границы. Эта часть посвящена Swarm Protocol — совместной работе агентов независимых владельцев над одной средой проекта. Здесь нужно согласовать не только доступ к инструментам, но и право назначать работу, основания её принятия и порядок изменения общего состояния. Далее описана спецификация протокола распределённой разработки; реализация полного набора его механизмов пока не заявлена. спецификация

17.1 Одна среда, несколько владельцев

Среда исполнения — место, где находится рабочее дерево проекта, выполняются команды и проверки, собираются артефакты и продвигается основная ветка. Агентский цикл — модель с контекстом и вызовами инструментов — работает у контрибьютора, который предоставляет свою ёмкость модели. Среда не запускает эти циклы: она пассивно исполняет разрешённые команды, поступающие извне. Так вычисление следующего действия и само действие оказываются по разные стороны границы доверия. спецификация

Это разделение нужно там, где среду дорого переносить к каждому участнику. Проект может зависеть от Xcode, лицензий, устройств, GPU, ключей подписи или данных с ограничением на вывоз. Копия репозитория ещё не даёт копии такой среды. В протокольной модели участник приносит способность думать и вызывать инструменты, а проект предоставляет ограниченное место исполнения. Учётные данные контрибьютора остаются у него; для работы не нужно передавать их владельцу среды. спецификация

Например, внешний агент исправляет код приложения, которое проверяют на устройствах проекта. Модель получает задачу и вызывает удалённые команды в выделенной ей области. Компиляция использует тулчейн проекта, а проверка на устройствах проходит через отдельную операцию раннера. Наличие доступа к рабочему дереву не даёт агенту ключ подписи приложения, право публиковать сборку или право самому принять свою работу. Каждому из этих действий нужны собственные основания. спецификация

«Единая» означает единство авторитетного состояния, а не обязательную единственность физической машины. Логическое состояние задают основная ветка и журнал. У среды могут быть исполнительные реплики, обслуживающие задачи по снимкам рабочего дерева, но голова основной ветки продвигается в одном месте. Верификатор — отдельная роль реплики, независимо повторяющей критические проверки. Он уменьшает зависимость результатов проверки от машины продьюсера, не устраняя всех допущений о доверии к среде. спецификация

Цена такого выбора — общая исполнительная ёмкость. Добавление контрибьюторов увеличивает доступную ёмкость моделей, но не создаёт автоматически больше процессора, памяти или устройств у проекта. Поэтому число участников и число одновременно выполняющихся сборок — разные величины. Очередь исполнения является частью модели, а обещание неограниченного масштабирования из наличия внешних агентов не следует. спецификация

17.2 Три плоскости работы

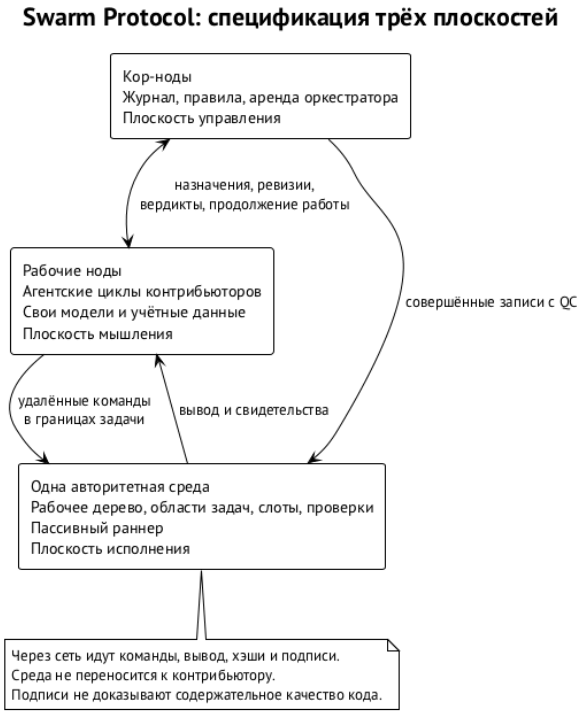
Внутри Swarm Protocol различаются плоскости мышления, управления и исполнения. Это разделение ролей самого протокола; оно не заменяет разделение платформы на Swarm Runtime, Swarm Protocol и Swarm Chain. Плоскость отвечает на вопрос, где происходит действие внутри распределённой рабо-

ты, а слой — к какой части Swarma относится механизм. Таблица описывает это разделение внутри протокольной модели. спецификация

Плоскость	Содержимое	Основной вопрос
Мышление	Агентские циклы контрибьюторов, их модели, контекст и ёмкость	Какое действие предложить следующим?
Управление	Кор-ноды, журнал, правила, полномочия и аренда оркестратора	Есть ли основание разрешить действие?
Исполнение	Рабочее дерево, изолированные области задач, слоты, проверки и раннер	Как выполнить разрешённое действие в заданных границах?

Модель может предложить полезное исправление, однако её предложение ещё не является полномочием. Кор-ноды могут подтвердить допустимость назначения, однако их подписи ещё не являются результатом исполнения. Среда может успешно выполнить команду, однако это ещё не означает принятия получившегося кода. Протокол связывает эти три факта записями и хэшами, сохраняя различие между ними. спецификация

Например, агент предлагает изменить библиотеку. В плоскости управления должно существовать назначение, разрешающее конкретной ноде менять определённые пути. В плоскости исполнения команда работает в области этой задачи и оставляет подтверждение результата. Затем появляется ревизия, а основание для её включения в проект формируется отдельным процессом проверки. Никакой из этих шагов не заменяется уверенностью модели в правильности её ответа. спецификация



В модели Swarm Protocol рабочие ноды предлагают действия, кор-ноды подтверждают допустимость записей, а единая пассивная среда исполняет разрешённое.

17.3 Участники и роли

Кор-ноды — узлы с зафиксированными ключами, которые ведут журнал и проверяют его правила. В распределённом варианте они принадлежат независимым владельцам; начальный уровень допускает одну кор-ноду. Право кор-ноды подписывать запись не означает право произвольно менять проект: перед подписью она проверяет детерминированные условия. Какие подписи достаточны и кому доверен порядок записей, задаёт уровень доверия проекта. спецификация

Рабочие ноды — агентские циклы контрибьюторов на их машинах, с собственной ёмкостью модели и ключами. Протокол считает их недоверенными. Это не утверждение о намерениях каждого участника: границы должны сохраняться и при ошибке, и при вредоносном поведении рабочей ноды. Её инструмент работы с проектом — удалённое исполнение команд в области задачи, а не неограниченный доступ к машине проекта. спецификация

Оркестратор — прикладная роль планирования, которую кор-нода удерживает по аренде. Он декомпозирует разрешённую работу, предлагает назначения и координирует процесс. Аренда ограничивает время и область его полномочий. Она не превращает его предложения в совершённые записи: правила и требуемые подписи сохраняются. Механика аренды и смены владельца роли разобрана в главе «Оркестратор по аренде и цикл задачи». спецификация

Секвенсер — сервис, задающий порядок записей журнала на нижних уровнях доверия. На начальном уровне он совпадает с единственной кор-нодой. На более высоком уровне порядок может определяться византийски устойчивым консенсусом. Лидер такого консенсуса выполняет механическую роль и не совпадает по определению с оркестратором: упорядочивать записи и решать, какую работу предложить, — разные обязанности. спецификация

Среда исполнения принимает команды, проверяет основания доступа и выполняет разрешённые действия. Внутри неё выделен доверенный раннер: он предоставляет узкие именованные операции для секретов проекта и общих сервисов. Отделение раннера нужно, чтобы обычная работа с кодом не давала полномочия подписи, публикации или произвольного использования устройства. Подробная семантика внешних действий вынесена в главу «Внешние эффекты, остановка и федерация». спецификация

Продьюсер, ревьювер и исполнитель проверок — роли в обработке результата задачи. Продьюсер создаёт изменение, ревьювер судит о нём, исполнитель проверок подтверждает требуемые свойства выбранным способом. Политика независимости определяет допустимое разделение этих ролей. Разные подписи сами по себе ещё не доказывают независимость владельцев, моделей и тулчейнов; то, что проект действительно требует и проверяет, должно быть объявлено явно. спецификация

17.4 Что пересекает сеть

Между участниками проходят команды, их вывод, хэши и подписи; в управляющем обмене — назначения, сведения о ревизиях, вердикты и сообщения о продолжении работы. Через сеть не переносится вся среда разработки. Модель получает вывод инструментов и может читать доступный ей код, но ей не передают ключи проекта и возможность произвольно воспроизвести защищённую среду у себя. Учётные данные модели остаются у контрибьютора. спецификация

Каждая нода объявляет административный домен, семейство модели и тулчейн. Эти сведения нужны, чтобы проект мог отделять независимые проверки от повторов с теми же источниками ошибок. Политика независимости определяет, какие заявления подтверждаются, а какие принимаются на доверии. Наличие поля в объявлении не является доказательством его истинности; гарантии процесса ограничены выбранной политикой и её основаниями. спецификация

Например, две рабочие ноды могут находиться на разных машинах, но принадлежать одному владельцу и использовать одно семейство модели. Такое разделение не даёт той независимости, которую проект мог требовать от внешнего ревью. Если подходящего участника нет, протокол предусматривает ожидание или эскалацию. Автоматически назвать имеющуюся проверку независимой ради продвижения задачи нельзя. спецификация

Направление, в котором агент предъявляет личность через Swarm Chain и переносит аттестации между проектами, обсуждается отдельно. Оно не является обязательным уже работающим основанием описанного здесь обмена. Устройство личности и переносимость разобраны в главах «Идентичность агента» и «Аттестации и переносимая репутация»; глобальное хранение аттестаций и вычисление репутации требуют отдельного устройства за пределами журнала одного проекта. предложено

17.5 Область задачи и право исполнения

Санбокс задачи в протокольном описании — копия рабочего дерева вместе с политикой ограничения для одной задачи. Копия разводит версии файлов, а политика ограничивает действия. Отдельный каталог или `git worktree` сами по себе не изолируют чужие процессы, сокеты, домашние каталоги, локальные сервисы и секреты. Особенно важно это для сборочных скриптов: они исполняются с правами вызвавшего их процесса. спецификация

Конкретную границу Swarm Runtime задают пользователь ОС на агента и файрвол ядра, описанные в главе «Пользователь ОС на агента и сеть агента». Здесь мы пользуемся этой границей и обсуждаем только протокольные условия доступа. Назначение не создаёт альтернативный механизм изоляции: оно связывает уже установленную границу с конкретной задачей и её полномочиями. решено

Слот — право на одновременное исполнение. Область задачи может существовать, пока задача ждёт свободной вычислительной ёмкости. Поэтому назначенная задача не обязательно немедленно начинает сборку. Для проекта с ограниченным числом устройств это различие особенно заметно: подготовка кода и очередь проверок используют разные ресурсы, хотя относятся к одному результату. Протокол различает хранение рабочего состояния и занятие исполнительного ресурса. спецификация

Права возникают из записи назначения. Она связывает ноду, задачу, полномочия и исполнительные ограничения; среда открывает доступ после подтверждения записи журналом. Сам факт регистрации ноды или знания адреса проекта доступа не даёт. Участник может предложить работу, но предложение не должно запускать исполнение до необходимой авторизации. Это позволяет принимать внешние предложения, не превращая каждое из них в команду среде. спецификация

План — скомпилированный набор полномочий агентской работы: привязки инструментов, гранты и лимиты. Его digest фиксирует разрешённое. Запись назначения несёт digest плана, и каждый вызов к среде должен ему соответствовать. Среда проверяет совпадение на каждом вызове, а не только при открытии доступа. Участник не может сохранить разрешение задачи, подменив план более широким после назначения. Состав контракта и отличие его digest от digest плана разобраны в главе «Контракт задачи и правила валидаторов». спецификация

Наличие когда-то выданного разрешения недостаточно. Перед исполнением нужно проверить его актуальность: действие, разрешённое в прежней эпохе или до остановки, не становится допустимым сейчас лишь потому, что подписи под старой записью верны. Эпохой в этой части называется срок аренды оркестратора. Это значение нужно отличать от эпохи плана или владельца сессии в механизмах рантайма. спецификация

17.6 Ревизия, кандидат и свидетельство проверки

Ревизия — представленный продьюсером результат изменения кода, адресуемый хэшем. Кандидат — неизменяемое состояние, построенное из этой ревизии над определённой головой основной ветки. Обозначим ревизию как RR, голову как MM, кандидата как II. Финальное одобрение и проверки относятся к II: именно это состояние предполагается включить в проект. Проверка отдельного патча ещё не подтверждает его композицию с другими принятыми изменениями. спецификация

Например, продьюсер проверил исправление над прежней базой, а другая задача успела изменить библиотеку. Даже если файлы не конфликтуют текстуально, совместное поведение может измениться. Поэтому основная ветка продвигается с ожидаемого MM на проверенный II атомарно. Если голова уже другая, прежнее основание продвижения неприменимо: строится новый кандидат и повторяются относящиеся к нему одобрение и проверки. Детали очереди и переходов находятся в главе «Оркестратор по аренде и цикл задачи». спецификация

Аттестация — подписанный результат проверки свойства для конкретного кандидата, контракта, политики и окружения. Она отвечает на ограниченный вопрос: какая проверка к чему применялась и с каким исходом. В протоколе различаются исходы «пройдена», «провалена» и «не установлена». Сбой

инструмента или таймаут не являются положительным свидетельством. Кор-нода проверяет адрес и исход аттестации, но не подменяет это новым суждением модели о качестве кода. спецификация

Такое разделение сохраняет полезность разных способов проверки. Сборка, регрессионные тесты, независимое ревью и машинно проверяемое доказательство свойства подтверждают разное. Ни один успешный шаг не означает универсальной правильности программы. Протокол должен связать результат с требуемым профилем свойств и точным кандидатом; достаточность самого профиля остаётся ответственностью проекта. спецификация

17.7 Удалённый инструмент в рабочем цикле

Удалённое исполнение должно сохранять привычную для модели связь между вызовами shell. Если агент сменил каталог одной командой, следующая должна продолжить работу в этом каталоге. Если команда завершилась с ошибкой, инструмент должен вернуть её точный код выхода и раздельные потоки вывода. Эти свойства определяют качество рабочего цикла: без них агент тратит контекст на восстановление состояния инструмента вместо решения задачи. Они описаны как требования к удалённому shell. спецификация

Удалённый shell использует восстановленную сессию: каждый вызов является отдельным процессом, а состояние shell снимается после команды и восстанавливается перед следующей. Каталог, переменные и функции сохраняются отдельно от обычного вывода. Например, сообщения сборки не должны случайно стать способом подменить сохранённый каталог следующего вызова. Это объяснение механики инструмента, а не новая граница доверия между владельцами. Изоляцию продолжает обеспечивать рантайм. спецификация

Мутирующие вызовы одной сессии сериализуются. Если две команды одновременно меняют состояние shell, их нельзя независимо восстановить из одной исходной копии и затем потерять изменение одной из них. Параллельность предоставляется именованными сессиями, похожими на вкладки терминала. Например, просмотр исходников и отдельный длительный прогон могут использовать разные сессии, тогда как последовательность смены каталога и редактирования в одной сессии должна сохранять порядок. спецификация

Длительный процесс запускается явно и отслеживается по дескриптору. Отмена относится к группе процессов вызова и не должна разрушать возможность следующего вызова. Если вывод превышает лимит, полный результат остаётся в области задачи, а модель получает начало и конец и может дочитать нужную часть тем же инструментом. Ограниченный объём ответа не следует понимать как отсутствие остального вывода или как успешное завершение процесса. спецификация

Сходство с локальным shell не означает выдачи shell всей машины проекта. Инструмент остаётся внутри назначенных полномочий, даже когда его интерфейс привычен агенту. Модель может испортить состояние своей рабочей сессии, но это не должно расширить её доступ. Поэтому удобство продолжения работы и защита от недоверенного участника оцениваются отдельно: хорошее сохранение каталога не компенсирует слабую границу пользователя или сети. спецификация

Практическая граница доступа описана в главе «Удалённое рабочее место». Протокольное назначение должно связать подключившегося участника с конкретной задачей и её планом, а способ аутентификации не должен трактоваться как самостоятельная изоляция. Например, подтверждённый транспортный ключ говорит, кто подключился; он не даёт права обращаться к любому каталогу, запускать произвольную привилегированную операцию или продолжать работу после утраты авторизации. Здесь сохраняется разделение между подлинностью участника и допустимостью его действия. спецификация

Передача команд по сети также не должна менять источник разрешения. Одинаковый вызов инструмента, отправленный по другому каналу, всё ещё требует того же назначения и совпадающего digest плана. Успешно установленное соединение лишь позволяет доставить запрос. Право исполнить этот запрос проверяется средой по журналу, а не выводится из того, что предыдущая команда в канале

уже была разрешена. Такая проверка нужна и в тёплом соединении, используемом для многих вызовов. спецификация

17.8 Происхождение команды и граница наблюдения

Receipt — подписанный факт исполнения команды средой. Команда получает идентификатор, digest команды и входного состояния, а также лимиты. Результат содержит код выхода, дайджесты вывода, сведения о ресурсах и digest результирующего состояния. Receipts задачи связаны в хэш-цепочку, голова которой входит в запись о ревизии. Так можно установить, какой запрос получила среда и какой результат она вернула. спецификация

Например, продьюсер сообщает, что сборка прошла. Receipt позволяет связать это сообщение с конкретным вызовом и его результатом в среде. Но receipt не доказывает, что будущий повтор даст то же самое. Для воспроизводимости отдельно нужны сохранённые входы, digest окружения и тулчейна, зафиксированные зависимости и внешние ответы там, где они влияют на результат. Происхождение исполнения и воспроизводимость — разные проверяемые свойства. спецификация

Протокол также не наблюдает весь внутренний контекст недоверенной рабочей ноды. Агент мог использовать сведения или инструменты на своей машине вне протокола; цепочка receipts не удостоверяет их отсутствие. Гарантия относится ко всем обращениям к среде через протокол и выданным ею результатам. Это граница наблюдения, которую нельзя расширить подписью под тем, чего среда не видела. спецификация

Среда объявляет инвентарь ограничений: какие классы она обеспечивает полностью, частично или не обеспечивает. Задача объявляет требуемый класс. Если ей необходимо полное ограничение, частичное не превращается в достаточное предупреждением в логе. Должен произойти отказ. Например, нельзя выполнить задачу с обязательным полным ограничением памяти там, где объявлена лишь частичная поддержка такого ограничения. спецификация

17.9 Состояние проекта и доверие

Журнал — упорядоченная цепочка подписанных записей о полномочиях и переходах проекта. Запись считается совершённой, когда под ней собран сертификат кворума выбранного уровня. Контекст модели служит кэшем: он может помогать продолжить работу, но не создаёт полномочий и не исправляет журнал. Новый оркестратор разрешает расхождения в пользу журнала, а не в пользу убедительного пересказа предшественника. спецификация

Git сохраняет роль хранилища, адресуемого по содержимому, и механизма ветвления. Хэш коммита позволяет связать проверку с конкретным состоянием кода. Но наличие коммита или ветки само по себе не отвечает, кто разрешил задачу, действителен ли её контракт и достаточно ли оснований для принятия. Эти ответы даёт журнал. Доска задач и граф зависимостей могут быть его представлениями, а не конкурирующими источниками истины. спецификация

Уровни L0, L1 и L2 меняют порог подписей и способ установления порядка, сохраняя инварианты процесса. L0 доверяет единственной кор-ноде и среде. L1 добавляет большинство подписантов, сохраняя доверенного секвенсера. L2 предусматривает византийски устойчивый консенсус. Нельзя понимать эту шкалу как автоматическое повышение качества модели или прочности изоляции. Точные условия сертификатов и различие между авторизацией и единственностью истории разбирает глава «Журнал, кворум и инварианты». спецификация

Смысл модели — сделать проверяемой процедуру совместного изменения проекта. Криптографическое свидетельство может подтвердить, что уполномоченные участники одобрили конкретного кандидата по определённому контракту. Оно не доказывает правильности их суждения и не защищает от всех последствий компрометации самой среды. Качество декомпозиции, сила проверок и вероятность обнаружения ошибки остаются свойствами моделей и проекта; протокол задаёт границы, в которых на эти свойства можно опираться. спецификация

18 Журнал, кворум и инварианты

Журнал Swarm Protocol — упорядоченная история полномочий и переходов состояния проекта. В него попадает не только результат работы, но и основание, по которому работа была назначена и принята. Среда исполняет подтверждённые журналом действия, а участник может проверить подтверждение по известным ключам. Это протокольная конструкция: наличие локальной доски задач ещё не означает, что уже реализованы сертификаты кворума, дерево Меркла и лёгкий клиент. спецификация

18.1 Запись и её совершение

Запись содержит хэш предыдущей записи, тип, тело или хэши тел и подписи. Каноническая форма превращает одни и те же данные в одни и те же байты для хэширования. Для записи rir_i с канонической формой c(ri)c(r_i) цепочка определяется как hi=H(hi−1 ∥ c(ri))h_i = H(h_{i-1} \,\|\, c(r_i)), где ∥ \| обозначает конкатенацию. У первой записи предшественник — строка нулей. Формула связывает содержание с положением в истории. спецификация

Для канонической формы предусмотрен JSON, для подписей — Ed25519. Это черновик спецификации журнала: окончательная совместимая схема обмена пока не установлена. Перечень смысловых частей записи не задаёт имён полей или полного формата назначения. Например, указания на задачу и ноду ещё недостаточно для реализации совместимого обмена записями. Для понимания протокола достаточно различать данные записи, её связь с предшественником и подписи, подтверждающие именно это содержание. спецификация

История только дописывается. Если поменять старое тело, изменится его хэш, а вслед за ним и последующие звенья. Одного пересчёта недостаточно, чтобы предъявить новую историю как принятую: потребуются допустимые подписи. Гарантия опирается на устойчивость хэшей, ключи и правила выбранного уровня доверия. Хэш-цепочка сама по себе не запрещает обладателям достаточных полномочий подписать противоречивые продолжения; единственность истории требует отдельного основания. спецификация

Запись считается совершённой после получения сертификата кворума — QC. Это достаточный для уровня набор подписей под одним содержимым. До его получения предложение записи не авторизует среду. Например, оркестратор может подготовить назначение и отправить его кор-нодам, но рабочая нода ещё не получает права исполнять команды лишь потому, что сообщение доставлено или появилось в очереди секвенсера. спецификация

QC проверяется по известным ключам без обращения к каждому подписанту. Это удобно для получателя записи: ему не нужно спрашивать оркестратора, действительно ли тот имел разрешение. Но проверка подписей отвечает только на вопрос о подтверждении записи. Для исполнения остаются проверки её актуальности, плана и состояния проекта. Старый сертификат может быть криптографически корректен и при этом не давать права действовать сейчас. спецификация

18.2 Кто устанавливает порядок

Секвенсер упорядочивает предложения, валидаторы проверяют правила и подписывают записи. На L0 эти обязанности совмещены единственной кор-нодой. На L1 порядок доверен секвенсеру, а корноды могут подписывать асинхронно. Разделение позволяет иметь постоянно доступный сервис порядка и участников, которые не всегда находятся в сети. Оно не отменяет необходимость собрать требуемые подписи: недоступность достаточного числа подписантов задерживает совершение записи. спецификация

На L2 порядок устанавливается византийски устойчивым консенсусом валидаторов. Его лидер — механическая роль протокола согласования, которая не требует модели. Оркестратор остаётся отправителем предложений о работе, а не владельцем порядка журнала. Поэтому смена оркестратора и смена

лидера консенсуса относятся к разным механизмам, даже если в развёртывании их обязанности иногда выполняет одна машина. спецификация

Ни секвенсер, ни валидатор не определяют качество кода голосованием. Предмет подписи — допустимость записи по проверяемым правилам. Например, можно проверить, что одобрение относится к нужному хэшу и подписано назначенным ревьювером. Нельзя тем же способом проверить, что ревьювер заметил все ошибки. Согласование порядка делает процедуру общей для участников, но не превращает её содержательные выводы в математическую истину. спецификация

18.3 Три уровня доверия

Уровни различаются определением QC, способом установления порядка и допущениями о противнике. Инварианты авторизации и принятия сохраняются на всех уровнях. Проект объявляет уровень в манифесте; повышение уровня оформляется записью о составе. Таблица описывает условия протокольной модели. спецификация

Уровень	Кор-ноды и QC	Порядок	Граница гарантии
L0	Одна кор-нода, одна подпись	Единственная нода	Защита от действий вне кора; кор и среда доверены
L1	n⩾2n \geq 2, порог t>n/2t > n/2	Доверенный секвенсер, асинхронные подписи	Защита от одиночного несанкционированного решения при компрометации меньшинства ключей; единственность истории зависит от честности секвенсера
L2	n⩾3f+1n \geq 3f+1, QC из 2f+12f+1 подписей	Византийски устойчивый консенсус	Защита от несанкционированного решения и раздвоения истории в заявленной модели с не более чем ff злонамеренными кор-нодами

На L0 подпись полезна, хотя не даёт независимого контроля самой кор-ноды. Она отделяет её решение от команды внешнего участника. Например, рабочая нода не может сама изготовить совершённое назначение, но компрометация единственного кор-ключа разрушает это основание доверия. Меньший порог здесь является сознательным допущением развёртывания, а не облегчённым доказательством устойчивости к злонамеренному кору. спецификация

На L1 порог большинства не позволяет одному скомпрометированному ключу единолично авторизовать запись. Но авторизация и единственность истории — разные свойства. Пусть подписантов трое, порог равен двум, а один участник злонамерен. Он подписывает две несовместимые записи на одно место, один честный участник подписывает первую, другой — вторую. Оба набора содержат две подписи. Ни один честный участник не подписал обе записи, однако сертификат большинства есть у каждой. спецификация

Два множества по tt подписантов среди nn пересекаются как минимум по 2t−n2t-n участникам. Чтобы пересечение гарантированно содержало честного при ff злонамеренных, нужно 2t−n>f2t-n>f. В примере с тремя подписантами и одним злонамеренным этого нет. Поэтому L1 доверяет секвенсеру в отношении истории. Считать его византийски устойчивым лишь из-за нескольких подписей было бы ошибкой в модели угроз. спецификация

Для византийского варианта нужно одновременно не ждать молчащих противников и исключать два противоречивых решения. Если порог обозначить qq, первое условие даёт q⩽n−fq\leq n-f, а пересечение с честным участником требует q>(n+f)/2q>(n+f)/2. Отсюда получается n>3fn>3f. L2 предусматривает n⩾3f+1n\geq3f+1 и сертификат 2f+12f+1; для обычной конфигурации n=3f+1n=3f+1 это обеспечивает

требуемое пересечение. Самого подсчёта подписей недостаточно: необходим конкретный протокол консенсуса. спецификация

Из этой записи нельзя произвольно увеличить число голосующих, оставив прежний порог и считая доказательство неизменным. Общее условие пересечения должно сохраняться. Отдельная схема порогов для произвольной расширенной конфигурации пока не определена; её нужно определять вместе с выбранным консенсусом. Например, при четырёх узлах и одном допустимом злонамеренном сертификат из трёх соответствует приведённой границе. открыто

Обозначения L0–L2 встречаются также в описании стадий доверия к изменению процесса. Там речь идёт о другом: кто подтверждает действия человека и изменения правил. Допуск внешних рабочих нод не равен византийскому консенсусу кор-нод. Внешний контрибьютор может работать и при L0, если его действия ограничены. В этой главе L0–L2 всегда обозначают именно уровень журнала и его сертификата. спецификация

18.4 Дерево Меркла и лёгкий клиент

Над хэшами записей строится дерево Меркла. Подписанный снапшот содержит корень дерева, длину журнала, голову и версию. Доказательство включения позволяет проверить принадлежность конкретной записи журналу с этим корнем, передав логарифмическое число хэшей вместо всей истории. Это отделяет хранение полной истории от проверки конкретного основания действия. Снапшот должен быть подтверждён требуемыми подписями, иначе произвольный корень ничего не удостоверяет. спецификация

Среда работает как лёгкий клиент. Для открытия области задачи ей нужны подтверждённое назначение и доказательство его включения; для продвижения основной ветки — запись об интеграции и соответствующие основания. Ей не требуется хранить все тела задач и все находки ревью, чтобы проверить отдельное разрешение. Однако проверка включения не заменяет проверки содержимого разрешения и актуального состояния. спецификация

Представим, что участнику показывают назначение под корректным корнем старого снапшота. Доказательство включения может пройти, хотя проект уже остановлен. Поэтому среда хранит наивысшую принятую версию журнала как долговременное состояние безопасности и не принимает более раннюю. Это состояние переживает перезапуски и не сбрасывается при ротации ключей. Иначе перезапуск позволил бы вновь предъявить историю до отзыва или остановки. спецификация

Защита от отката и актуальность разрешения дополняют друг друга. Первая не даёт вернуться к уже пройденной версии, вторая проверяет, разрешено ли действие на границе исполнения сейчас. Валидная старая подпись не является исключением ни из одного правила. Тем самым клиент хранит не просто ускоряющий кэш, который можно без последствий удалить, а часть своего состояния безопасности. спецификация

18.5 Ключи и предел свидетельства

Проверка сертификата опирается на известный набор кор-ключей. Пины — закреплённые ключи для проверки подписей. Их ротация выполняется через перекрытие: сначала распространяется набор со старым и новым ключом, затем подписи переходят на новый, после чего старый удаляется. При компрометации старый ключ удаляется немедленно. Это механизм смены основания проверки подписей, а не основание забыть уже принятую версию журнала. Состояние защиты от отката должно сохраниться и после такой смены. спецификация

Например, клиент получил новый набор ключей после обычной ротации. Он должен уметь проверять допустимые новые подтверждения, но не возвращаться к снапшоту до остановки проекта лишь потому, что тот подписан прежним ключом. Проверка состава подписантов и проверка монотонности версии отвечают на разные вопросы. Ни одна из них не может считаться автоматически выполнен-

ной из-за успеха другой. Так же нельзя заменить доступность тела тем, что подпись под его хэшем подлинна. спецификация

Состав кор-нод сам является предметом записей журнала. Поэтому читателю сертификата нужно понимать, какой уровень и какой набор ключей относятся к проверяемому основанию. Набор подписей неизвестных участников не становится QC только из-за достаточного количества элементов. Протокол не приравнивает число строк с подписями к числу уполномоченных подписантов. Ключи кор-нод фиксированы, а изменение состава оформляется отдельной записью; свободное самообъявление не даёт полномочий валидатора. спецификация

Есть и граница доступности. Даже при честном секвенсере он не может получить подписи от выключенных участников, а сертификат уже совершённой записи не обеспечивает исполнение на недоступной среде. Отделение секвенсера позволяет упорядочивать поступающие предложения без постоянно работающей модели оркестратора, но ожидание кворума и очередь ресурсов остаются. Остановившийся процесс не следует объявлять нарушением целостности только по факту задержки; и задержка не даёт права обойти необходимые подписи. спецификация

Наконец, проверка журнала не защищает от произвольной компрометации самой среды исполнения. Среда обязана соблюдать проверки лёгкого клиента и ограничения рантайма. Если противник управляет ею вне этих правил, наличие правильной истории не заставит машину исполнять только разрешённое. Верификатор даёт независимый повтор критических проверок, но не превращает все действия среды в доказанно честные. Это допущение модели, подробно рассматриваемое в главе «Модель угроз». спецификация

18.6 Какие факты хранятся в журнале

Типы записей охватывают полномочия, работу, проверку и управление. В таблице сгруппированы полномочия и события проекта; это перечень смыслов записей, а не новая схема сериализации. Подробные определения контракта и аренды находятся в соответствующих главах. спецификация

Группа	Что фиксируется
Полномочия	Цель, выданное оркестратору полномочие, версии контракта задачи, назначение
Происхождение работы	Зависимости и принятые артефакты, ревизия, голова цепочки receipts
Принятие	Заморозка множества ревьюверов, случайность, кандидат, ревью кандидата, аттестации, интеграция
Управление	Аренда, эпоха, чекпоинт, пауза, возобновление, состав кор-нод и ключей
Внешние операции	Авторизация операции раннера и отдельная запись её исхода
Учёт результата	Вестинг, то есть отложенное закрепление начисления

В журнале также предусмотрены подтверждения идентичности, сертификация ключа ноды и аттестации исхода для репутации. Их связь с глобальной личностью и записями в Swarm Chain относится к предлагаемому направлению. Из самого перечня типов не следует ни готовность цепочки, ни опубликованный формат переносимой аттестации. Эти вопросы разобраны в главах «Идентичность агента» и «Аттестации и переносимая репутация». предложено

18.7 Доступность и представления

Тяжёлые тела — описания задач, находки ревью, снимки контекста — хранятся адресуемо по содержимому. Журнал может содержать их хэши. Хэш подтверждает целостность полученного объекта, но

не гарантирует, что объект удастся получить. Поэтому перед включением хэша требуется сертификат доступности: независимые реплики подтверждают хранение тела. Сколько ни проверяй хэш отсутствующего снимка, восстановить из него контекст нельзя. спецификация

Это особенно существенно при передаче роли оркестратора. Если единственная копия чекпоинта осталась на недоступной машине, знание его адреса не даёт преемнику самого снимка. Протокол требует подтверждённой доступности до включения ссылки в историю. Это отдельное условие наряду с корректностью подписи и формы записи, а не следствие репликации короткой записи с хэшем. спецификация

Код адресуется хэшами коммитов, а git предоставляет хранилище. Журнал может материализоваться как доска задач в репозитории проекта: переход становится коммитом доски, QC покрывает его хэш. Записи принимает один писатель с fencing по эпохе. Это возможное представление сертифицированного журнала, а не второй механизм, который вправе самостоятельно переопределить совершённый переход. спецификация

Граф задач и артефактов также получается из записей о зависимостях. Поисковый индекс помогает найти полезный результат, но не устанавливает, был ли он принят и подходит ли к текущему контракту. Например, найденная библиотека может быть принята в другой версии задачи; её наличие в выдаче не даёт оснований пропустить проверку совместимости. Авторитет остаётся у записей о результате, контракте и основаниях принятия. спецификация

18.8 Инварианты авторизации и принятия

Инвариант — требование, которое должно сохраняться при каждом допустимом переходе. Ниже перечислены десять требований модели с примерами нарушений. Это ограничения протокола, а не утверждение, что вся система уже доказана корректной; границы выполненной проверки разобраны в главе «Что проверено и как проверить остальное». спецификация

Эффект только по записи. Любому эффекту на среде нужна совершённая актуальная авторизация с digest плана, совпадающим с digest вызова. Проверка выполняется на границе исполнения. Например, команда под расширенным планом не должна пройти по назначению с более узким планом. Запись, действовавшая до остановки, также не даёт права выполнить отложенную команду после неё. Это условие связывает полномочие в журнале с фактическим действием. спецификация

Полномочия только сужаются. План задачи вложен в выданное полномочие, оно вложено в цель, а диф ревизии не выходит за область записи. Критерии приёмки меняются только новой авторизованной версией контракта. Например, задача исправления функции не может под видом необходимой подготовки разрешить себе переписать правила собственной приёмки. Проверка вложенности нужна на каждом уровне делегирования, иначе узкая человеческая цель превращается в широкие права исполнителя. спецификация

Интегрируется проверенное состояние. Основная ветка продвигается атомарно с ожидаемой головы MM на неизменяемого кандидата II. Одобрение и аттестации относятся к этому кандидату и текущим версиям контракта и политики. Например, одобрение над старой базой нельзя перенести на слияние с новой головой. Иначе в проект попадает композиция изменений, которую ни ревьювер, ни исполнитель проверок не рассматривали. спецификация

Суждение остаётся вне консенсуса. Валидаторы проверяют детерминированные предикаты над совершёнными записями и хэшами. Они устанавливают наличие одобрения и совпадение адресов, а не голосуют о правильности алгоритма. Например, единодушное мнение нескольких моделей не является сертификатом отсутствия дефектов. Гарантия консенсуса относится к согласованной процедуре и истории, а оценка содержания остаётся работой моделей и людей. спецификация

18.9 Инварианты независимости и восстановления

Независимость и случайность. Продьюсер, ревьювер и исполнитель проверок должны соответствовать объявленной политике независимости. Нехватка участников не ослабляет её автоматически. Случайность получают после фиксации множества ревьюверов и хэша ревизии; выбор пересчитывается проверяемо. Например, оркестратор не вправе сначала увидеть случайный результат, затем подобрать удобный состав кандидатов. Полный порядок выбора относится к главе «Оркестратор по аренде и цикл задачи». спецификация

Разделение секретов. Ключи контрибьютора остаются на его машине, подпись выполняется там же, где работает агентский цикл. Секреты проекта существуют только в раннере и недоступны из области задачи. Например, сборочный скрипт не должен извлечь ключ публикации через обычный shell. Эти две стороны защищают разных владельцев: перенос чужой ёмкости в общий проект не требует объединять все секреты в одной доступной агенту среде. спецификация

Журнал — истина, контекст — кэш. Записи устаревшей аренды отвергаются и сохраняются как расхождение. Контекст предшественника не получает авторитета вместе с передачей роли. Например, проснувшийся прежний оркестратор может помнить незавершённый план, но его старый fencing-токен не разрешает продолжать назначения. Новый оркестратор восстанавливает основания работы из журнала и не обязан принимать утверждение предшественника, что какое-то разрешение «уже было согласовано». спецификация

Остановка и защита от отката. Любая кор-нода может остановить систему, а для возобновления нужен кворум уровня. Среда не принимает версию журнала ниже наивысшей принятой. Например, компрометированный оркестратор не должен удерживать работу лишь потому, что отказывается согласовать остановку; предъявление снапшота до остановки не должно обходить запрет. Необходимое постоянное состояние клиента и проверка актуальности описаны выше. спецификация

18.10 Инварианты границ исполнения

Ограничение является сообщаемым фактом. Среда объявляет, что именно она умеет ограничить. Если задача требует полного ограничения, а среда обеспечивает частичное, задача отказывает. Например, обязательный ресурсный предел нельзя заменить надеждой на наблюдение за процессом, не меняя требования задачи. Инвариант заставляет назвать расхождение до исполнения; он не создаёт отсутствующий механизм изоляции и не оправдывает его подмену более слабым. спецификация

Происхождение обращений проверяемо. Все команды к среде через протокол и выданные результаты связаны цепочкой receipts, голова которой входит в ревизию. Это позволяет связать исполнение с запросом и сравнивать результаты с верификатором. Например, рассказ агента об успешном прогоне не заменяет свидетельства среды. Одновременно цепочка не удостоверяет полноту внутреннего контекста рабочей ноды: внешние действия на её машине остаются вне наблюдения протокола. спецификация

Все десять требований работают совместно. Совпадение хэша не исправляет устаревший контракт; независимое ревью не даёт права на неавторизованную публикацию; действительная подпись не отменяет остановку. При рассмотрении упрощения полезно искать конкретный запрещённый переход: что оно позволит исполнить или принять без нужного основания. Именно такие переходы, а не общая уверенность в участниках, определяют проверяемую границу модели. спецификация

19 Контракт задачи и правила валидаторов

Контракт задачи — версионируемое описание требуемого результата и правил его принятия. Он отделён от плана полномочий: доступный инструмент отвечает на вопрос «что можно сделать», а кри-

терий приёмки — «что нужно получить». Если смешать эти вопросы, задача с правом редактировать код может незаметно приобрести право переписать собственное определение успеха. В Swarm Protocol полномочия и условия приёмки фиксируются раздельно и связываются проверяемыми записями. спецификация

19.1 Откуда берутся правила процесса

Конфигурация процесса задаёт типы элементов, их workflow, охранные условия переходов, наборы проверок и делегирование действий с потолками полномочий. Эти определения задаёт черновик спецификации конфигурации процесса, v4.8; проект объявляет процесс в `Playbook.json`. Изменение вступает в силу после подтверждения человека или в пределах заранее выданных им прав. Ядро исполняет версию в силе, а не любую найденную редакцию файла. спецификация

Здесь рассматривается протокольная форма этих правил: digest плана, версия контракта, адрес свидетельства проверки и предикаты перед подписью кор-ноды. Если объяснение расходится со спецификацией модуля, приоритет у спецификации. Это особенно существенно для полей конфигурации и конкретных охранных условий: книга не вводит дополнительный формат и не объявляет самостоятельный автомат состояний вместо принадлежащего процессу. спецификация

Например, проект может заранее разрешить определённому участнику менять процесс в заданных пределах. Тогда каждое допустимое изменение не требует нового ручного решения, но предел делегирования сохраняется. Сам факт, что агент умеет редактировать файл конфигурации, не даёт ему права вводить эту редакцию в силу. Авторизованная версия и доступная для записи копия — разные состояния. спецификация

19.2 План разрешает, контракт требует

Digest плана фиксирует привязки инструментов, гранты и лимиты. План компилируется из write scope — множества путей, которые задача вправе изменять, — и класса задачи. Его digest входит в назначение и сверяется средой на каждом вызове. Это связь между разрешением в журнале и фактически используемыми полномочиями, а не просто подпись под текстовым описанием намерений. спецификация

Digest контракта фиксирует условия результата и приёмки. Два одинаковых набора полномочий могут обслуживать задачи с разными критериями. Например, право менять определённую библиотеку ещё не говорит, требуется ли сохранить совместимость её интерфейса или изменить её по новому требованию. Для такого различия нужны родительская цель и ожидаемый результат в контракте, а не расширенный список доступных команд. спецификация

Контракт включает следующие части. Таблица описывает их назначение; имена полей и схема сериализации относятся к спецификации конфигурации процесса. спецификация

Часть	Что фиксирует
Родительская цель	Основание, из которого выделена задача
Write scope	Пути, доступные задаче для изменения
Входы и зависимости	Версии входных артефактов и используемые результаты
Результаты и критерии приёмки	Что требуется получить и на каком основании принять
Профиль подтверждаемых свойств	Какие свойства, каким способом, для какого артефакта и при каких допущениях проверяются
Оценка	Сложность задачи в принятых единицах
Класс риска	Требуемый процесс для соответствующих изменений и операций

Часть	Что фиксирует
Бюджеты	Пределы расходования ресурсов задачи

План и контракт должны согласовываться, но совпадение одного не заменяет проверку другого. Агент может оставаться внутри разрешённых путей и при этом получить неправильный результат. Обратно, полезный результат может быть произведён с превышением прав. Протокол не позволяет считать содержательную полезность оправданием неавторизованного исполнения и не считает соблюдение write scope достаточным условием принятия кода. спецификация

19.3 Версия защищает условия приёмки

Изменение критериев или профиля подтверждаемых свойств создаёт новую версию контракта и требует авторизации того же уровня, что создание задачи. Это позволяет уточнять требования, не переписывая прошлые основания принятия. Обычная задача не может незаметно ослабить свой критерий успеха: новая формулировка должна стать отдельным авторизованным фактом, а свидетельства — относиться к актуальной версии. спецификация

Например, задача должна сохранить совместимость, но продьюсер обнаруживает, что проще изменить интерфейс. Он может сообщить о проблеме и предложить уточнение. Он не может удалить требование совместимости из описания и использовать прежнее ревью как подтверждение новой задачи. Право декомпозировать работу не включает право самостоятельно изменить смысл исходной цели; иначе любые неудобные критерии можно было бы устранить в процессе исполнения. спецификация

Версионирование полезно и без злого умысла. Ревьюер мог добросовестно проверить прежнюю формулировку, а владелец — затем добавить требование. Положительное ревью не становится отрицательным задним числом, однако оно уже не является основанием принятия по новому контракту. История сохраняет, относительно чего был сделан вывод. Текущая приёмка требует свидетельств с актуальным адресом, а не общего утверждения, что задачу когда-то проверяли. спецификация

19.4 Профиль свойств и три исхода

Профиль подтверждаемых свойств задаёт, какое свойство требуется подтвердить, каким способом, для какого артефакта и при каких допущениях. В одной задаче нужны сборка, регрессионные проверки и независимое ревью. В другой — машинно проверяемое доказательство конкретного свойства и ревью соответствия формальной постановки исходной задаче. Способы не образуют универсальную лестницу: более формальная проверка одного свойства не подтверждает автоматически все остальные. спецификация

Например, успешная компиляция не доказывает правильности авторизации, а доказательство отсутствия переполнения не устанавливает, что операция разрешена нужному пользователю. Проект должен назвать требуемое свойство и достаточный способ его подтверждения. Валидатор затем проверяет наличие соответствующих оснований; он не может исправить неполный профиль дополнительной подписью или предположением, что «более сильная» проверка покрыла неназванное поведение. спецификация

Каждая проверка имеет один из трёх исходов: «пройдена», «провалена», «не установлена». Таймаут и сбой инструмента дают «не установлена». Это отсутствие установленного результата, а не доказательство отсутствия ошибки. Для принятия требуется положительный исход нужной проверки. Например, недоступный исполнитель тестов не позволяет продвинуть кандидата на основании того, что от него не пришёл список падений. спецификация

Различие нужно сохранить во всей цепочке, от инструмента до записи об интеграции. Иначе корректный неопределённый ответ на одном шаге может превратиться в положительный на другом. Наличие аттестации подтверждает факт сообщения о результате; её исход отдельно определяет, выполне-

но ли требование. Подписанный таймаут остаётся таймаутом и не становится достаточным свидетельством благодаря подлинности подписи. спецификация

19.5 Адрес одобрения и аттестации

Финальное одобрение и каждая аттестация адресованы сочетанию из четырёх частей: версии контракта, интеграционного кандидата II, версии политики проверки и digest окружения. Такой адрес связывает вывод с постановкой, точным состоянием кода и условиями проверки. Устаревание любой части делает свидетельство неприменимым к текущему принятию. Совпадение одного хэша кандидата ещё недостаточно. спецификация

Изменившаяся часть	Почему прежнее свидетельство неприменимо
Версия контракта	Проверяли другие условия результата или приёмки
Кандидат	Проверяли другое состояние кода
Версия политики	Основания проверки относятся к другому правилу принятия
Digest окружения	Проверка связана с другими условиями исполнения

Например, код остался тем же, но задача получила новый профиль проверок. Прежнее одобрение относится к старому контракту. Или контракт прежний, но кандидат перестроен над новой головой: хэш изменился, и свидетельство не переносится. Валидатор сравнивает все части, не выбирая удобное подмножество совпадений. спецификация

Этот адрес не является готовым форматом публичной записи в Swarm Chain. В предлагаемом направлении переносимой репутации есть отдельные вопросы состава и публикации аттестаций. Здесь речь о свидетельстве для приёмки в проекте. Глобальное хранение и правила вычисления репутации остаются темой главы «Аттестации и переносимая репутация». Связь этих двух назначений пока находится в проработке. предложено

19.6 Цепочка полномочий

Цель создаётся и подписывается кор-нодой; она задаёт область, класс риска и бюджет. Из неё оркестратору выдаётся полномочие — capability: допустимые пути, типы изменений и лимиты. Внутри него создаётся задача с контрактом и планом. Ревизия допустима, если её диф находится внутри write scope задачи. На каждом уровне права только сужаются. спецификация

Оркестратор декомпозирует полномочия, но не создаёт их. Например, право изменить библиотеку не позволяет ему выдать задачу на публикацию приложения, если цель такого действия не разрешала. Дочерняя задача также не может получить более широкий доступ лишь потому, что названа вспомогательной. Валидаторы проверяют вложенность при переходах, а среда проверяет digest плана при вызовах; эти проверки охраняют разные границы одной цепочки. спецификация

Рабочая нода может предлагать новые цели, но их не исполняют до подписи кор-ноды. Предложение — полезный вход в планирование, а не самостоятельный источник полномочий. Например, обнаруженная потребность в обновлении зависимости может стать отдельной задачей после авторизации, но не разрешает текущему продьюсеру выйти за пределы назначенного изменения. Такая граница сохраняется и для содержательно разумного предложения. спецификация



В спецификации цепочки полномочий цель ограничивает оркестратора, задача — исполнение, а ревизия остаётся внутри write scope.

19.7 Класс риска и защищённые изменения

Класс риска определяется детерминированно по путям, манифестам зависимостей и типам операций. Привилегированный класс включает авторизацию и доступ, зависимости, конфигурацию сборки и CI, проверки, правила и манифест, подпись, публикацию, секреты и миграции. Такие задачи требуют двух независимых ревью либо одобрения человека из кор-команды. Соответствующие пути доступны для записи только из области задачи этого класса. спецификация

Например, продьюсер не может сделать падающий тест зелёным удалением самого теста, если его задача разрешает менять только реализацию. Изменение проверок допустимо как отдельная авторизованная работа соответствующего класса. То же относится к ослаблению конфигурации CI или правил приёмки: это изменения механизма, который судит о результате, поэтому их нельзя незаметно включить в обычное исправление. спецификация

Классификация не распознаёт любой вредный смысл. Изменение внутри разрешённой области обычного класса может быть семантически вредным. Остаточный риск ограничивают область, бюджет, независимое ревью кандидата, отложенное закрепление начислений и остановка. Детерминированное определение класса не следует описывать как автоматический анализ безопасности всей программы: оно выбирает процесс по наблюдаемым признакам изменения и операции. спецификация

19.8 Write scope и зависимости

Задачи с пересекающимися write scope не исполняются одновременно. Область удерживается от назначения до интеграции или отмены задачи, включая ожидание принятия ревизии. Иначе следующая задача начнёт менять те же пути, пока предыдущая ещё ждёт решения. Декомпозиция поэтому либо разводит области, либо оставляет такие задачи последовательными. Освободить область сразу после окончания работы продьюсера недостаточно. спецификация

Дизъюнктные области исключают конкурирующую запись в одни пути, но не семантические конфликты. Одна задача может изменить интерфейс библиотеки, другая — добавить вызов в другом файле. Каждая отдельно выглядит допустимой, а вместе изменения несовместимы. Проверка кандидата

над текущей головой нужна именно для композиции. Ни отсутствие текстового конфликта, ни соблюдение write scope не заменяют эту проверку. спецификация

Задача может использовать принятые артефакты других задач. Запись о зависимости указывает, какие результаты используются и какие предоставляются. Агент получает происхождение, контракт, зависимости и основания принятия результата. Поиск подходящего артефакта и решение о его применимости разделены: индекс помогает найти, а записи и текущий контракт определяют, на что допустимо опереться. Устройство представлений журнала разобрано в главе «Журнал, кворум и инварианты». спецификация

19.9 Предикаты перед подписью

Кор-нода подписывает запись после детерминированных проверок. Условия переходов принадлежат конфигурации процесса; ниже приведён набор предикатов Swarm Protocol. Каждое правило проверяет определённое основание, а не общую правдоподобность сообщения. При невыполненном требовании подпись не должна подтверждать переход как допустимый. спецификация

1. Переход состояния задачи допустим по процессу. Наличие ожидаемого результата не разрешает перескочить обязательный переход или проверку.
2. Write scope вложен в capability, capability — в цель; digest плана соответствует области и классу; указанная версия контракта текущая.
3. Продьюсер, ревьювер и исполнитель проверок различны согласно политике независимости. Если политику выполнить нельзя, фиксируется ожидание или эскалация, а не автоматическое ослабление.
4. Множество кандидатов в ревьюверы заморожено до появления хэша ревизии, случайность получена после; выбор вычисляется из идентификатора задачи, хэша ревизии и случайности.
5. Интеграция содержит базу ММ, кандидата II, финальное одобрение и требуемые аттестации с исходом «пройдена». Их адреса совпадают с II, текущим контрактом, политикой и окружением; текущая голова равна ММ.
6. Назначение выдано действующей арендой с валидным fencing-токеном, в пределах лимитов полномочия и без пересечения с удерживаемыми областями записи.
7. Каждый хэш тела сопровождается сертификатом доступности. Подписанная ссылка не заменяет подтверждения хранения объекта.
8. Соблюдены класс риска и требуемый им процесс. Обычная задача не получает исключения для привилегированного изменения из-за его малого размера.
9. Цель создана кор-нодой. Предложение рабочей ноды не исполняется до необходимой подписи.
10. У операции раннера есть идентификатор и ключ идемпотентности; повтор ссылается на тот же идентификатор.

Порядок выбора в правиле о случайности: заморозка множества, затем хэш ревизии, затем случайность. Предмет финального одобрения при этом остаётся интеграционный кандидат. Различие важно: выбор участника и адрес его заключения — разные проверки. Пересчёт выбора не даёт права принять одобрение, адресованное к другому состоянию кода. спецификация

19.10 Пример проверки интеграции

Пусть запись предлагает заменить голову ММ кандидатом II. Валидатор читает голову и сравнивает её с ММ, проверяет подписи одобрения и аттестаций, сопоставляет их адреса с кандидатом и актуальными версиями. Затем проверяет положительный исход каждого требуемого свойства. В этих действиях нет обращения к модели: это сравнение известных значений и проверка оснований, установленных процессом. спецификация

Если одна обязательная проверка имеет исход «не установлена», основания для интеграции нет. Если все проверки положительны, но относятся к прежнему контракту, основания также нет. Если всё совпало, но голова успела продвинуться, атомарная замена с ожидаемого MM не должна пройти. Положительные сведения не компенсируют несовпадение другой обязательной части; для нового состояния строится кандидат и собираются относящиеся к нему свидетельства. спецификация

Проверка головы валидатором не заменяет атомарную проверку средой при продвижении: между чтением и исполнением состояние может измениться. Поэтому запись об интеграции задаёт ожидаемую базу, а среда выполняет compare-and-swap. Это соединяет доказанные до подписи условия с фактическим изменением общей ссылки. Подробный цикл подготовки кандидата и повторных проверок описан в главе «Оркестратор по аренде и цикл задачи». спецификация

Идентификатор операции и ключ идемпотентности связывают повтор с тем же внешним действием. Адрес аттестации связывает свидетельство проверки с версией контракта, кандидатом, версией политики и digest окружения. Например, повторный запрос публикации не должен выглядеть новой операцией лишь из-за повторной доставки сообщения. Но наличие того же идентификатора не подтверждает успешного исхода публикации: для него предусмотрена отдельная запись результата. Подробная семантика повторов и неопределённого исхода относится к главе «Внешние эффекты, остановка и федерация». Это различие не даёт заменить свидетельство о выполнении операции наличием её авторизации. спецификация

19.11 Чего подпись не обещает

Валидатор устанавливает, что нужные ревью и проверки были, относились к точному состоянию и контракту, выполнялись допустимыми участниками и дали требуемые исходы. Он не устанавливает, что ревьювер был прав или что выбранные тесты нашли все дефекты. Подпись под процедурой не должна читаться как универсальная гарантия качества результата. Именно разделение этих обязанностей позволяет сохранить детерминированность правил. спецификация

Для проекта остаётся содержательная работа: правильно поставить цель, выбрать профиль свойств и обеспечить достаточную проверку. Протокол делает её основания явными и не позволяет незаметно подменить их после получения результата. Ошибочное суждение всё ещё возможно, но можно установить, кто проверял что, по каким условиям, в каком окружении и на каком основании результат попал в общую ветку. спецификация

20 Оркестратор по аренде и цикл задачи

Оркестратор — роль планирования в Swarm Protocol. Он превращает цель проекта в задачи с контрактами, назначает продьюсеров, инициирует ревью и интеграцию. Кор-нода удерживает эту роль по аренде; сама роль не даёт ни прямого доступа к среде, ни права единолично принимать результат. Полномочия оркестратора ограничены выданной областью и лимитами, а каждое действие становится записью, которую проверяют валидаторы. спецификация

Это продолжение устройства, описанного в главах «Модель протокола», «Журнал, кворум и инварианты» и «Контракт задачи и правила валидаторов». Здесь нас интересует время: кто вправе планировать сейчас, как заменить его после сбоя и какое состояние в итоге попадёт в основную ветку. Локальная изоляция и запуск остаются обязанностями Swarm Runtime (главы «Пользователь ОС на агента и сеть агента» и «Запуск агента и хост сессий»); протокольная аренда не заменяет пользовательские и сетевые границы рантайма. спецификация

20.1 Планирование без прямого исполнения

Разделение ролей нужно потому, что хорошая декомпозиция не делает её автора доверенным исполнителем. Оркестратор может предложить полезную задачу или семантически вредную задачу в пределах своей области. Правила протокола проверяют полномочия и основания принятия, но не превращают смысл предложения в детерминированный факт. Поэтому оркестратор действует через журнал, а команды в среде выполняют назначенные рабочие ноды. спецификация

Например, цель состоит в переносе модуля синхронизации. Оркестратор выделяет транспортную часть, задаёт её область записи и ожидаемый результат. Он не получает от этого права менять систему авторизации или публиковать выпуск. Если перенос требует изменения защищённых путей, нужна задача соответствующего класса и соответствующий процесс принятия. Декомпозиция сужает уже выданные полномочия, а не создаёт новые. спецификация

Инициировать интеграцию тоже не значит исполнить её. Оркестратор предлагает запись, валидаторы проверяют её основания, а среда применяет совершённую запись с сертификатом кворума. Ревьювер вычисляется по правилу выбора, а не назначается по вкусу оркестратора. Право экстренной остановки проходит отдельно: удержание роли планировщика не даёт права запрещать паузу. спецификация

20.2 Аренда, эпоха и fencing-токен

Аренда — запись о временном владении ролью. Она связывает ноду, эпоху, случайный fencing-токен, предшествующую аренду, причину смены, чекпоинт как базу передачи и область полномочий. Причиной бывает создание, плановая передача, принудительный захват или восстановление. Это состав протокольного описания; из него не следует опубликованный wire-формат записи. спецификация

Эпоха здесь означает срок аренды оркестратора. Это не время жизни плана агентской сессии и не аренда владельца сессии у хоста сессий. Сходство слов не даёт взаимозаменяемости: право писать в сессию и право планировать работу проекта проверяются на разных границах. Механизм хоста сессий описан в главе «Запуск агента и хост сессий». спецификация

Fencing-токен позволяет отсечь прежнего держателя после смены роли. Аренда продлевается heartbeat — сообщениями о том, что держатель продолжает работу. Каждая запись оркестратора несёт эпоху и токен. Валидаторы отвергают запись с устаревшей арендой, сохраняя её как расхождение для аудита. Сохранённое расхождение не получает силы принятой команды. спецификация

Представим, что машина оркестратора уснула. Heartbeat перестал приходить, аренда истекла, следующая кор-нода получила роль с новой эпохой и токеном. После пробуждения прежний оркестратор отправляет накопленное назначение. Его подпись может быть подлинной, но аренда уже не действует: валидаторы не подписывают назначение. Подлинность отправителя и актуальность его полномочий — разные условия. спецификация

Проверка не заканчивается у валидаторов. Среда требует сертификат кворума и проверяет актуальность эпохи на границе исполнения. Иначе задержанная команда могла бы пройти проверку в старой эпохе и исполниться после передачи роли. Операции с внешними эффектами требуют той же дисциплины; их сбои и повторения разобраны в главе «Внешние эффекты, остановка и федерация». спецификация

20.3 Как сменяется держатель роли

Смена эпохи происходит при таймауте heartbeat, исчерпании ёмкости оркестратора, решении кворума или обнаружении нарушения правил. Преемник определяется детерминированной ротацией. Византийская нода не становится незаменимой: следующая смена позволяет её пропустить. Через заданное число эпох предусмотрена холодная смена независимо от текущей причины. Конкретные интервалы не составляют универсального обещания протокола. спецификация

Плановая передача допускает перенос сессии, но требует доказательства покоя. Ввод заблокирован, незавершённых вызовов, дочерних процессов и операций исполнителя нет, дайджест хранилища стабилен. Только после этого сессию передают преемнику и фиксируют хэш снимка в журнале. При необходимости форматы агентских сессий преобразуются, однако преобразование может потерять часть контекста. спецификация

Принудительный захват — самостоятельное действие. Он не маскируется под неудавшуюся плановую передачу: предшественника отсекают токеном, сессию не передают, преемник начинает по журналу. Это важно именно при подозрении на злонамеренного оркестратора. Если передать его отравленный контекст следующему держателю, компрометация сможет путешествовать вместе с ролью, хотя ключ ноды формально сменился. спецификация

Даже планово полученная сессия считается недоверенной. При расхождении с журналом прав журнал. Если в сессии написано, что транспортная задача принята, а журнал содержит только представленную ревизию, преемник не должен считать задачу интегрированной. Переданный контекст помогает восстановить ход мысли, но не заменяет сертификаты, контракты и результаты проверок. спецификация

Способ смены	Что можно перенести	Откуда берётся авторитетное состояние
Плановая передача после доказательства покоя	Снимок сессии с зафиксированным хэшем; содержимое недоверенно	Журнал и доступные тела записей спецификация
Принудительный захват	Сессия предшественника не передаётся	Журнал спецификация
Восстановление с чекпоинта	Доступный снимок как вспомогательный контекст	Журнал, включая записи после чекпоинта спецификация

20.4 Журнал вместо незаменимой памяти

План, контракты, назначения и результаты должны находиться в журнале. Сессия модели — кэш, а не единственное место, где принято решение. Оркестратор записывает решения и обоснования структурно по ходу работы. Иначе после потери сессии останутся файлы и отдельные результаты, но исчезнет объяснение того, какие из них приняты и почему следующая задача имеет право их использовать. спецификация

Чекпоинт связывает запись журнала с хэшем снимка сессии. Снимок помещается в хранилище с сертификатом доступности. После сбоя преемник может загрузить его и применить последующие записи журнала. Хэш позволяет связать полученный снимок с записью, а сертификат доступности нужен, чтобы ссылка не вела к телу, которое никто не способен предоставить. Ни то ни другое не делает мысли из сессии доверенными. спецификация

Проверка этого свойства намеренно жёсткая: остановить оркестратора, удалить его сессию и проверить продолжение по журналу и доступным артефактам. Успешная загрузка сохранённого чата проверяла бы другой путь. Такой эксперимент предусмотрен, но результат ограниченной модели интеграции его не заменяет; границы проверки описаны в главе «Что проверено и как проверить остальное». предложено

20.5 Области и один принимающий писатель

Одновременно могут действовать аренды с непересекающимися областями. Если задача затрагивает их пересечение, она принадлежит аренде, покрывающей объединение, либо создаётся кор-нодой напрямую. Само наличие нескольких держателей не разрешает каждому распоряжаться общей областью. Например, задачи транспорта и другого модуля могут планироваться отдельно, но общее изменение их границы требует соответствующего охвата полномочий. спецификация

Для совместной работы нескольких операторов над одной доской задач решено направление: доска процесса остаётся одна, а записи принимает один писатель с fencing по эпохе. Типизированная операция связывает актора, доску, версию конфигурации, область и эпоху аренды, ожидаемую ревизию и идентификатор операции. Эпоха проверяется при запуске агента, записи результата и интеграции. Потерявший полномочия писатель приостанавливает новые записи. **решено**

Этот писатель не превращает остальные ноды в источники второй авторитетной доски. В устройстве главного цикла один процесс имеет корневого оркестратора, а оркестраторы других узлов работают с арендованными поддеревьями. При этом способ назначения принимающего писателя — по владению процессом или выбором участников — остаётся открытым. Согласованное направление не означает, что весь распределённый механизм уже работает. **открыто**

20.6 От цели к ревизии

Цикл начинается с авторизованной цели и задачи с контрактом. Контракт фиксирует ожидаемый результат, область записи, зависимости и профиль свойств, которые предстоит подтвердить. Оркестратор назначает продьюсера с учётом доступной ёмкости, свободного слота и уже удерживаемых областей. Среда открывает санбокс по совершённой записи назначения. Механика полномочий определена в главе «Контракт задачи и правила валидаторов». **спецификация**

В примере с переносом транспорта продьюсер получает именно транспортную часть. Ограничение параллельной записи удерживается от назначения до интеграции или отмены, поэтому ожидание ревью не освобождает область для конкурирующего изменения. Непересекающиеся пути позволяют параллелизм, но не доказывают смысловую совместимость результатов: модуль может зависеть от интерфейса, который меняет соседняя задача. **спецификация**

Продьюсер работает в санбоксе и представляет ревизию — хэш коммита `R` над базой `B`. Подпись ставится на машине продьюсера; ключ ноды на среду не передаётся. В запись входит голова цепочки receipts, подписанных свидетельств исполнения команд. Подробности локального хранения ключей относятся к главе «Ключи и ключник», а здесь важна граница: автор результата подписывает его со своей стороны. **спецификация**

Ревьювер может дать раннюю обратную связь по `R`: указать пропущенную обработку ошибки или несоответствие контракту. Продьюсер исправляет ревизию и представляет её снова. Но такая обратная связь не становится финальным одобрением. Она помогает производству результата, тогда как принятие требует проверки конкретного состояния, которое действительно собираются интегрировать. Это разные переходы процесса. **спецификация**

20.7 Почему нужен интеграционный кандидат

Интеграционный кандидат `I` — неизменяемое состояние, построенное средой из ревизии продьюсера над текущей головой основной ветки `M`. Журнал фиксирует пару `(M, I)`: от какого состояния предполагается продвинуть голову и к какому. База задачи `B` могла остаться в прошлом, пока другие задачи проходили интеграцию. Поэтому ревизия `R` и кандидат `I` выполняют разные роли, даже если изменение выглядит одинаково. **спецификация**

Предположим, транспортная задача добавляет вызов функции. Пока она проходит ревью, другая задача меняет контракт этой функции. Патч добавления вызова может остаться тем же, однако его смысл на новой базе изменился. Совпадение `patch-id` не доказывает, что прежнее одобрение относится к новому состоянию. Базовая гарантия требует одобрения именно `I`, а проверки всегда выполняются на `I`. **спецификация**

Это правило связывает проверку с объектом принятия. Одобрение исходной ревизии говорит о ревизии; зелёная сборка старой базы говорит о старой базе. Ни одно из этих свидетельств само по себе не разрешает продвинуть другую голову к другому кандидату. В модели интеграции перенос аттестации

между базами действительно даёт контрпример при отключённой привязке к кандидату. спецификация

Перенос одобрения рассматривается только как расширение вне базовой гарантии и при детерминированном условии: дельта `B..M` не пересекается ни с областью записи задачи, ни с путями, которые продьюсер читал по receipts. Даже в таком расширении проверки на `I` повторяются. Это не общее разрешение переносить ревью после любого rebase и не основание объявить одинаковые патчи одинаковыми проверенными состояниями. спецификация

20.8 Кто проверяет кандидата

Множество допустимых ревьюверов замораживается до появления хэша `I`. Затем получают случайность из порогового маяка кор-нод или проверяемой случайной функции. Ревьювер вычисляется из идентификатора задачи, хэша `I` и случайности; валидатор может пересчитать результат. Такой порядок не оставляет оркестратору свободного выбора удобного проверяющего после того, как кандидат уже известен. спецификация

Допустимость ревьювера задаёт политика независимости. Она различает административный домен, семейство модели и тулчейн; подробности оснований идентичности разобраны в главе «Идентичность агента». Если подходящего участника нет, задача ждёт, эскалируется либо получает отдельно авторизованное изменение условий с записью в журнале. Недостаток участников не означает автоматического разрешения проверить собственную работу. спецификация

Финальный ревьювер получает диф `M..I`. Его одобрение адресовано кандидату, текущей версии контракта, версии политики проверки и дайджесту окружения. Адрес нужен целиком: согласие на прежний контракт не становится согласием на ослабленные критерии, а результат в другом окружении не заменяет требуемый результат. Это правило применяется и к аттестациям исполнения проверок. спецификация

Проверки профиля инициирует нода, третья по отношению к продьюсеру и ревьюверу. Критические наборы повторяет верификатор. Разделение ролей не следует путать с независимостью среды: если другая нода лишь запустила тесты на той же машине проекта, она не доказала честность этой машины. Независимое повторение требует верификатора на другой машине; это ограничение существенно для главы «Модель угроз». спецификация

Аттестация имеет исход «пройдена», «провалена» или «не установлена». Для принятия каждое свойство профиля должно иметь исход «пройдена». Сбой инструмента или таймаут не превращается в зелёную проверку. Если сборка не запустилась, контракт не выполнен доказательно; если кандидат отклонён, находки возвращаются продьюсеру, и цикл продолжается с ревизии. Наличие подписи на неизвестном исходе не меняет его смысла. спецификация

Область защищённых изменений требует отдельного внимания. К привилегированному классу относятся авторизация и доступ, зависимости, конфигурация сборки и CI, проверки, правила и манифест, подпись, публикация, секреты и миграции. Для этого класса требуются два независимых ревью либо одобрение человека из кор-команды; соответствующие пути доступны для записи только из санбокса задачи такого класса. Обычная транспортная задача не получает этот режим по собственному заявлению об удобстве исправления. спецификация

Например, продьюсер обнаружил, что для переноса проще ослабить регрессионную проверку. Такое изменение затрагивает сами основания принятия, а не только реализацию транспорта. Успешный запуск ослабленной проверки не закрывает прежний контракт. Нужна авторизация изменения условий на положенном уровне и новая версия контракта; прежние подтверждения не становятся автоматически подтверждениями нового профиля. Граница защищает процесс от ситуации, когда исполнитель одновременно определяет, что ему считать успехом. спецификация

Валидатор при этом не оценивает, хорош ли предложенный новый тест. Его задача — проверить класс изменения, полномочия, текущую версию и соблюдение требуемого процесса. Содержательное ре-

вью проверяет соответствие изменения замыслу. Эти обязанности дополняют друг друга: правильная классификация не доказывает качество, а убедительное объяснение качества не разрешает обойти классификацию. Для интеграции нужны оба вида оснований в пределах контракта. спецификация

20.9 Атомарное продвижение головы

Когда основания собраны, оркестратор предлагает интеграцию пары `(M, I)`. Валидаторы проверяют адреса одобрения и аттестаций, текущий контракт и остальные предикаты. Получив сертификат кворума, среда атомарно заменяет голову с `M` на `I`, только если текущая голова всё ещё равна `M`. Это compare-and-set: сравнение ожидаемого состояния и запись нового образуют одну операцию. спецификация

Если впереди прошла другая задача, голова уже отличается от `M`, и сертификат неприменим к такому состоянию. Кандидат перестраивается над новой головой, ревью и проверки повторяются. Очередь определяет порядок интеграции, но не отменяет сравнение базы. Проверка у валидатора сама по себе недостаточна: между подписью и доставкой состояние могло измениться, поэтому окончательное сравнение выполняет среда. спецификация

Продьюсер транспортной задачи может видеть это как повторную работу без изменения собственного патча. Однако предмет гарантии — состояние всего проекта после объединения. Проверить только патч дешевле, но это уже другая гарантия. В эксперименте стоимость повторной работы после смены головы учитывается отдельно, чтобы цена такой дисциплины не растворилась в общей стоимости inference. предложено



В спецификации интеграция продвигает точную голову к неизменяемому кандидату только с адресованными ему одобрением и аттестациями.

20.10 Ограниченные раунды и замена продьюсера

Число оплачиваемых раундов обратной связи ограничено параметром `N`. После превышения оркестратор обязан выбрать из фиксированного меню: передекомпозировать задачу, сменить продьюсера, сменить ревьювера или эскалировать человеку. Выбор записывается. Универсальное значение `N` здесь не задано: существенно ограничение цикла и явное решение о следующем шаге, а не придуманное общее число попыток. спецификация

Например, если транспортная задача снова возвращается с тем же несоответствием контракту, нельзя бесконечно оплачивать очередное обещание исправить. Передекомпозиция меняет размер работы, смена продьюсера меняет исполнителя, эскалация передаёт решение человеку. Смена ревьювера входит в меню, но не означает право произвольно выбрать лояльного участника в обход политики независимости и правила вычисляемого выбора. спецификация

Замена продьюсера возможна также при потере ноды или исчерпании её ёмкости. Преемник получает сессию предшественника с тем же правилом недоверия: полезный контекст разрешено использовать, неподтверждённый рассказ об успехе не заменяет журнал. Результат, для которого нет действующих оснований принятия, остаётся результатом в работе независимо от того, насколько уверенно его описывает переданная сессия. спецификация

Начисление закрепляется после того, как интегрированный `I` прожил заданное число эпох без отката; это вестинг. Он отделяет непосредственное принятие от устойчивости результата во времени. Из этого не следует готовая справедливая экономика или опубликованная формула переносимой репутации. Предлагаемая связь аттестаций со Swarm Chain и её открытые вопросы разобраны в главе «Аттестации и переносимая репутация». предложено

20.11 Что проверяет следующая сторона

Цикл полезно читать не только вперёд, но и от границы принятия назад. Среда должна знать ожидаемую голову и точного кандидата. Валидатор должен видеть, что одобрение и проверки относятся к этому кандидату и действующим условиям. Ревьювер должен получить именно тот диф, который станет предметом интеграции. Продьюсер должен знать контракт, а оркестратор — предел делегированного полномочия. Если на одном переходе вместо такой связи остаётся только свободный рассказ об успехе, следующий участник не получает требуемого основания. спецификация

Например, сообщение «транспорт перенесён, тесты зелёные» само по себе не определяет, какие тесты, в каком окружении и для какой версии контракта прошли. Даже добросовестный автор мог описывать прежнюю ревизию. Формальная привязка нужна не потому, что каждый участник обязательно лжёт, а потому, что параллельная работа постоянно меняет состояние вокруг него. Поздняя доставка честного сообщения может оказаться столь же неприменимой, как намеренная подмена основания. спецификация

Сертификат кворума отвечает на вопрос о совершении записи по правилам уровня доверия. Он не превращает недостающую аттестацию в имеющуюся и не заменяет неизвестный исход успешным. Поэтому наличие сертификата рассматривается вместе с содержанием записи и актуальностью её адреса. Детали того, от каких противников защищает сертификат на каждом уровне, остаются в главе «Журнал, кворум и инварианты»; цикл задачи не расширяет эти гарантии. спецификация

20.12 Доступность результата и его применимость

Хэш кандидата фиксирует объект, но для ревью нужен сам объект. Тела записей хранятся адресуемо по содержимому, и включение хэша связано с сертификатом доступности от независимых реплик. Код адресуется хэшами коммитов и хранится в git. Для цикла это означает, что проверяющий получает не только короткое имя результата: должно быть возможно получить данные, которые это имя обозначает. Иначе воспроизводимая проверка останавливается на ссылке. спецификация

Доступность и применимость — разные свойства. Прежний принятый артефакт может оставаться доступным, но не удовлетворять текущему контракту транспортной задачи. Журнал зависимостей связывает использованные принятые артефакты с задачей; поисковый индекс помогает найти результат, но не определяет его статус. Ни хорошее совпадение в поиске, ни наличие файла в хранилище не дают права пропустить проверку оснований переиспользования. спецификация

Та же граница действует при восстановлении оркестратора. Преемник может найти сохранённый снимок и получить все его байты, но это ещё не делает изложенные в нём выводы действующими.

Для транспортной задачи нужно сопоставить контекст с журналом: какой контракт текущий, какая ревизия представлена, есть ли кандидат и применимые подтверждения. Чекпоинт ускоряет восстановление, а не создаёт вторую систему принятия решений рядом с журналом. спецификация

20.13 Где ожидание является правильным исходом

Если область удерживается другой задачей, назначение ждёт. Если нет подходящего независимого ревьювера, цикл не понижает требования сам. Если обязательная проверка не установила свойство, интеграция не получает основания. Эти ожидания выглядят похоже снаружи, но возникают на разных границах: конкурирующая запись, независимость суждения и недостаток свидетельства. Их нельзя устранить одним общим решением «считать задачу успешной после таймаута». спецификация

Например, транспортная задача уже имеет ревизию, но третья нода не смогла запустить регрессионные проверки. От продьюсера может не требоваться новой правки: пока не установлено именно свойство, а не найден дефект кода. И наоборот, если ревьювер нашёл несоответствие контракту, простое повторение сборки не закрывает находку. Раздельные основания принятия позволяют сохранить это различие, не превращая любую задержку в отрицательный содержательный вердикт. спецификация

Цена ожидания должна быть видна в эксперименте: отдельно учитываются слоты, блокировки областей и повторная работа после смены головы. Нельзя объявить протокол эффективным, посчитав только время генерации кода, или неэффективным, записав всякую задержку как ошибку модели. Проверяемый вопрос — стоимость одинакового принятого внешнего результата при сопоставимых ограничениях. До измерения эта часть остаётся гипотезой о полезности координации. предложено

20.14 Одна задача от назначения до принятия

Вместо списка разрозненных «успешных шагов» полезно проследить основания каждого следующего действия. В переносе модуля синхронизации это выглядит так; пример описывает протокольный цикл, а не отчёт о развёртывании. спецификация

Шаг	Что появляется	Чего ещё недостаточно
Авторизована цель и выдано полномочие	Граница работы и лимиты оркестратора	Права самостоятельно расширить область
Создан контракт транспортной задачи	Ожидаемый результат и профиль свойств	Свидетельства, что результат получен
Совершено назначение	Продьюсер, действующая аренда и план	Права менять удерживаемые чужими задачами пути
Представлена ревизия	Подписанный `R` над `B`, связь с receipts	Финального одобрения будущего `I`
Получена обратная связь	Находки для следующей ревизии	Разрешения интегрировать
Построен кандидат	Неизменяемый `I` над `M`	Ревью и проверок этого кандидата
Собраны одобрение и аттестации	Подтверждения требуемых свойств по действующему адресу	Права применить их к другой базе
Совершена запись интеграции	Сертификат для `(M, I)`	Разрешения игнорировать изменившуюся голову
Среда выполнила compare-and-set	Голова стала ровно `I`	Гарантии отсутствия смыслового дефекта

Каждый переход закрывает свой вопрос: полномочие, производство, суждение, проверку или изменение состояния. Детерминированная часть позволяет установить, что основания относятся к принимаемому объекту. Содержательное качество задачи и способность ревьювера заметить дефект остаются

отдельными предпосылками. Проверенный фрагмент этой композиции и эксперименты для остальных частей рассматриваются далее. спецификация

21 Что проверено и как проверить остальное

Проверка Swarm Protocol состоит из разных по силе свидетельств. Исполняемая модель перебирает ограниченное пространство состояний фрагмента интеграции. Эксперименты со сбоями должны проверить реализацию правил на настоящих границах исполнения. Сравнительный эксперимент должен установить, полезен ли рой при одинаковых внешних задачах и сопоставимых ресурсах. Успех одной проверки не отвечает автоматически на вопросы двух других. План экспериментов пока остаётся планом. предложено

21.1 Как воспроизвести результат модели

Из корня репозитория книги и пейпера выполняется команда ниже. Она запускает Python 3 в изолированном режиме через цель `model` Makefile; модель использует стандартную библиотеку Python. Команда не собирает PDF и не запускает развёрнутый рой. Проверяется именно программа фрагмента интеграции, а не работа среды исполнения или валидаторов в сети. работает сегодня

```
make -C paper model
```

Первая строка результата самой модели — `full rules: reachable states=1090, violation=None`. Далее печатаются четыре запуска с отключением по одному правилу: `cas`, `epoch`, `contract`, `bind_I`. Для каждого показаны число просмотренных состояний, нарушение и последовательность действий до него. Это полезнее одного слова «успешно»: видны как положительный результат ограниченного перебора, так и сценарии, в которых ослабленная модель допускает запрещённое. работает сегодня

Код запуска печатает результаты; он не является самостоятельным приёмочным тестом всей реализации. Его завершение без ошибки не доказывает соответствие демона среды модели. При воспроизведении нужно прочитать значения `violation` и трассы, а не принять один exit code за подтверждение всех свойств протокола. Название запуска в таблице ниже — метка напечатанного эксперимента, а не название отдельного production-теста. работает сегодня

21.2 Что находится внутри модели

Модель представляет две задачи. У задачи `A` полное для этого фрагмента поведение: меняется версия контракта, а предлагаемая запись может нести устаревшую эпоху. Задача `B` создаёт помеху интеграцией: продвигает голову, пока первая работа проходит свой цикл. Это достаточно, чтобы проверить различие между кандидатом над прежней базой и кандидатом над текущей головой. Содержимое файлов в состоянии отсутствует. работает сегодня

Состояние включает голову и эпоху аренды, а для каждой задачи — наличие ревизии, базу кандидата, аттестацию с адресом и версией контракта, текущую версию контракта, признак интеграции и запись об интеграции со счётчиком доставок. Модель хранит только те различия, которые нужны проверяемому свойству. Например, она различает адрес аттестации до и после смены базы, но не рассуждает о качестве самого изменения. работает сегодня

Допустимые действия: представить ревизию, построить кандидата над текущей головой, аттестовать его, изменить контракт, сменить эпоху, предложить запись с текущей или устаревшей эпохой, доставить запись, в том числе повторно. Перебор в ширину рассматривает достижимые чередования действий. Поэтому повторная доставка и изменение состояния между предложением записи и её исполнением входят в проверку явно. работает сегодня

Границы заданы в программе: две задачи, не более двух версий контракта задачи `A`, не более двух эпох аренды. Для каждой задачи одновременно имеется не более одной записи об интеграции, и запись доставляется не более двух раз. Число 1090 относится к достижимым состояниям именно этой модели при всех правилах. Оно не обозначает число агентов, тестовых проектов или проверенных программных дефектов. работает сегодня

21.3 Четыре правила при доставке

Проверяемая граница — доставка записи интеграции, после которой может измениться голова. Здесь модель применяет четыре правила; полная система дополнительно должна проверять остальные протокольные основания из главы «Контракт задачи и правила валидаторов». Следующий перечень описывает реализованные условия модели, а не исчерпывающий валидатор Swarm Protocol. работает сегодня

Правило	Проверяемое условие	Какую подмену различает
`cas`	Текущая голова равна базе записи	Устаревшая база и повтор уже применённой записи
`epoch`	Запись несёт текущую эпоху аренды	Действующая и прежняя аренды
`contract`	Аттестация ссылается на текущую версию контракта	Текущие и заменённые условия приёмки
`bind_I`	Аттестация адресована точному кандидату	Проверенный кандидат и ревизия на иной базе

При каждом продвижении проверяется составное свойство: кандидат построен над головой в этот момент; аттестация относится к нему и к текущему контракту; запись несёт текущую эпоху; ни одна запись не применяется дважды. Это предикат над состояниями модели. Он не говорит, что аттестация содержательно верна, и не оценивает способность ревьювера найти намеренно спрятанный дефект. работает сегодня

Например, запись может быть корректной в момент предложения, а затем устареть после смены эпохи. Если проверять только предложение и не проверять доставку, позднее исполнение пересечёт границу полномочий. В модели именно такое чередование становится контрпримером при отключении `epoch`. Проверка сравнивает состояние в момент эффекта, а не только существование старого разрешения. работает сегодня

21.4 Напечатанный результат и его отрицательные контроли

При всех правилах перебор достигает 1090 состояний без нарушения. Отключение любого одного правила даёт нарушение. Таблица передаёт результаты запуска, включая числа просмотренных состояний до первого найденного контрпримера. Эти числа не сравнивают безопасность вариантов: поиск останавливается при обнаружении нарушения, поэтому меньший счётчик не означает более сильную или более слабую защиту. работает сегодня

Запуск	Просмотрено состояний	Напечатанное нарушение	Действий в напечатанной трассе
Все правила	1090	`None`	—
Без `cas`	84	`record applied twice`	6
Без `epoch`	82	`record from a stale lease epoch advanced main`	6

Запуск	Просмотрено состояний	Напечатанное нарушение	Действий в напечатанной трассе
Без `contract`	71	`attestation references a superseded contract version`	6
Без `bind_I`	490	`attestation made on a different base than the integrated candidate`	11

Трасса без `cas` представляет ревизию, строит кандидата, аттестует его, предлагает запись и доставляет её дважды. Нарушение — повторное применение записи. Это конкретный отрицательный контроль: ослабленная модель действительно допускает запрещённый переход. Запуск показывает первую найденную трассу, поэтому из него не следует приписывать выводу дополнительные, не напечатанные контрпримеры. **работает сегодня**

Без `contract` контракт меняется после аттестации, а старая аттестация остаётся основанием интеграции. Без `bind_I` аттестация переживает смену базы кандидата: после интеграции другой задачи строится новый кандидат, но используется прежнее свидетельство. Смысл последнего случая совпадает с опасностью переноса одобрения по одному `patch-id`, разобранной в главе «Оркестратор по аренде и цикл задачи». **работает сегодня**

Эти контроли отключают правило целиком. Они показывают необходимость каждого из четырёх правил для проверяемого свойства в данной модели. Они не измеряют полноту production-гейта по всем классам атак и не заменяют более узкие мутации реализации. Здесь нет основания утверждать, что обнаруживается любая возможная ошибка compare-and-set, любой replay или любой способ подделать аттестацию. **работает сегодня**

Читать трассу без `bind_I` нужно с учётом двух задач. Сначала обе имеют кандидатов и аттестации на прежней голове. Интеграция одной задачи сдвигает голову. Затем для другой строится кандидат над новым состоянием, но ослабленное правило разрешает использовать аттестацию, сделанную до сдвига. Запись проходит остальные включённые условия, а проверяемое свойство обнаруживает различие баз. Модель не утверждает, что содержимое обоих кандидатов действительно одинаково: она вообще не хранит их код. **работает сегодня**

Этот пример показывает, почему результат нужно описывать на уровне абстракции программы. В ней есть событие аттестации и её адрес, но нет текста ревью, промпта или журнала рассуждений. Поэтому найденный контрпример устанавливает ошибку связи свидетельства с состоянием, а не измеряет вероятность, что конкретный ревьюер пропустит изменившийся смысл функции. Последний вопрос требует эксперимента с настоящими кандидатами и моделями. **работает сегодня**

Аналогично отсутствие нарушения при всех правилах говорит о проверяемых переходах, а не об отсутствии любой ошибки программы модели. Воспроизводимый исход полезен как ограниченное свидетельство, потому что границы и отключаемые правила видны рядом с результатом. Для более широкого утверждения нужны более широкая модель или самостоятельная проверка реализации. **работает сегодня**

21.5 Где заканчивается установленный результат

Исчерпывающим является перебор внутри заданных границ. Это не доказательство для произвольного числа задач, версий контракта, эпох или доставок. Среда и валидаторы в модели честны. Не моделируются содержимое кода и суждение моделей, полномасштабная аренда, остановка и внешние эффекты. Четыре правила не покрывают всю цепочку авторизации проекта. Перенос результата за эти пределы требует отдельного обоснования. **работает сегодня**

В частности, в полной аттестации есть версия политики и дайджест окружения. Ограниченная модель проверяет свою версию контракта и адрес кандидата; её положительный результат нельзя выдать за перебор всех изменений политики и окружения. Аналогично смена числовой эпохи в модели не проверяет весь жизненный цикл heartbeat, доказательства покоя и принудительного захвата роли. работает сегодня

Следующий шаг — использовать трассы как векторы соответствия для демона среды и секвенсера, затем расширить проверку на аренду и остановку и перенести модель в язык спецификаций с механической проверкой. Это направление дальнейшей работы. До выполнения соответствующих проверок нельзя называть реализацию доказанно эквивалентной модели только потому, что в ней используются те же названия четырёх правил. предложено

21.6 Эксперимент со сбоями реализации

Первая серия экспериментов проверяет соблюдение протокола. Нарушения вносятся воспроизводимо: выход рабочей ноды за область записи, запись с устаревшей арендой, подмена кандидата после аттестации, изменение критериев после ревью, повтор доставки записи интеграции, потеря исхода операции исполнителя и потеря оркестратора вместе с его сессией. Здесь качество модели не является предметом эксперимента. предложено

Ожидаемый результат — запрещённый шаг не получает неавторизованного эффекта на среде или продвижения головы, а восстановление использует журнал. Неизвестный исход внешней операции требует сверки или решения человека; эксперимент не должен переименовывать неизвестность в доказанное отсутствие эффекта. Для каждого сбоя важно наблюдать именно ту границу, которую правило защищает, а не только ошибку в ответе вызывающей стороне. предложено

Удаление сессии оркестратора выделено в самостоятельный эксперимент. Преемник должен продолжить работу по журналу и доступным артефактам. Если после удаления остаётся только формально запущенный агент, а план приходится вручную восстанавливать из частного контекста, свойство ещё не установлено. Если сессия просто перезагрузилась из копии, проверен путь восстановления кэша, а не независимость процесса от него. предложено

Трассы ограниченной модели дают конкретные сценарии для части этой серии. Повтор доставки требует проверить, что запись не применяется второй раз; устаревшая эпоха — что прежний держатель не получает эффект; изменение контракта — что старое одобрение не удовлетворяет новым условиям; новая база — что прежняя аттестация не переносится автоматически. Исполнение этих сценариев на настоящей среде ещё предстоит подтвердить отдельно. предложено

21.7 Эксперимент о полезности роя

Вторая серия отвечает на другой вопрос: даёт ли совместная работа полезный результат дешевле или быстрее при сопоставимых ограничениях. Единица сравнения — заранее определённая внешняя задача или завершённая функция с одинаковыми критериями приёмки. Количество внутренних задач не подходит: оркестратор способен менять декомпозицию и тем самым увеличить счётчик, не увеличив полезный результат. предложено

Сравниваются одиночный агент, параллельный процесс без протокола и развёртывание L0. В варианте без протокола агенты одного владельца используют общий репозиторий и блокировки. Ресурсные ограничения должны быть сравнимыми. Стоимость включает inference, проверки, занятость среды и вмешательство человека: иначе выигрыш по одному показателю может оказаться переносом затрат на оператора или дефицитный стенд. предложено

Что измерять отдельно	Какой возможный расход становится виден
Ожидание слотов	Ограниченность общей среды
Блокировки областей	Потери параллелизма из-за конкурирующей записи

Что измерять отдельно	Какой возможный расход становится виден
Повторная работа после смены головы	Цена перестройки кандидата и повторной проверки
Восстановление после потери агента	Цена смены держателя работы
Доля пересмотренных отклонений	Качество процесса отказа
Доля откатов через заданное число эпох	Устойчивость уже принятого результата

Таблица задаёт измерения будущего эксперимента, а не публикует полученные значения. Точно так же наличие более дешёвой истекающей квоты ещё не доказывает выигрыш: ожидание среды, проверки и повторная интеграция могут съесть экономию. Рабочая гипотеза о выгоде координации остаётся опровержимой и должна сравниваться на одинаковом внешнем результате. предложено

21.8 Как оценивать качество ревью

В оценке рабочего слоя различаются доля злонамеренных нод и вероятность, с которой честный ревьюер обнаруживает дефект. Предлагаемая калибровка даёт ревьюверу копии кандидатов с запечатанными посеянными дефектами известных классов и считает долю найденных. Она относится к конкретной роли, модели, усилию и профилю роли; это измерение способности к обнаружению, а не удостоверение честности участника. предложено

Доля найденных посевов оценивает обнаружение настоящих дефектов лишь настолько, насколько посевы сопоставимы с ними по трудности. Намеренно спрятанная уязвимость может оказаться труднее известных примеров. Классы из дефектов, прошедших предыдущие ревью, помогают приблизить калибровку к реальному риску, однако не дают автоматической полноты. Положительный результат посева не превращает ревью в механический оракул. предложено

Калибровка одного ревьювера не измеряет долю злонамеренных участников и совместный пропуск нескольких ревьюверов. Чтобы измерять совместный пропуск прямо, их нужно проверять на одинаковых посевах. Разные владельцы и разные семейства моделей уменьшают основания ожидать одинаковую ошибку, но не устанавливают независимость ошибок математически. Вероятностная иллюстрация и её допущения разобраны в главе «Модель угроз». предложено

Показательный предел — изменённое окружение проверки. Полная гарантия требует привязки к его дайджесту, но программа модели не перебирает состояния тулчейнов, конфигураций и внешних сервисов. Даже если число достижимых состояний совпало с ожидаемым, такой результат не разрешает переносить аттестацию между окружениями. Это отдельная обязанность реализации и отдельный предмет проверки соответствия. Аналогично модель с двумя задачами не устанавливает предел пропускной способности общей среды. работает сегодня

Для читателя воспроизведение поэтому заканчивается двумя записями знания: свойство выполнено во всех достижимых состояниях заданной модели, а четыре ослабленных варианта дают напечатанные нарушения. Утверждения о полномасштабной системе требуют других свидетельств. Такая граница позволяет использовать модель как точный проверяемый аргумент, не превращая небольшой перебор в неподтверждённое обещание обо всём проекте. работает сегодня

22 Внешние эффекты, остановка и федерация

Интеграция коммита меняет голову проекта, но работа не ограничивается git. Подпись, публикация, обращение к устройству и изменение данных могут дать эффект вне рабочего дерева. Исполнитель таких операций — раннер — должен различать разрешение, начало исполнения и установленный ис-

ход. Иначе после сбоя нельзя понять, разрешено ли повторить действие. Для этого случая Swarm Protocol задаёт отдельные правила. спецификация

Эти правила не заменяют устройство Swarm Runtime. Локальные пользователи ОС, сеть, ключник и брокер учётных данных описаны в главах «Пользователь ОС на агента и сеть агента», «Ключи и ключник» и «Учётные данные и логины». Здесь рассматривается протокольная сторона: какая запись разрешает эффект, как меняется разрешение после смены аренды и каким участникам проект предлагает доверять при присоединении. Новых способов локальной изоляции эта глава не вводит. спецификация

22.1 Разрешение ещё не означает установленный исход

Представим публикацию: операция авторизована, раннер отправил результат получателю и упал до сохранения ответа. Следующий оркестратор видит разрешение без завершения. Из этого нельзя заключить ни «публикации не было», ни «публикация точно состоялась». Повтор способен создать второй эффект, а отказ от повтора способен оставить работу незавершённой. Нужна явно представленная неизвестность. спецификация

У каждой операции есть идентификатор и ключ идемпотентности. Повтор ссылается на тот же идентификатор и параметры. Для получателя, поддерживающего идемпотентность, это позволяет не создавать дополнительный эффект. Условие про получателя существенно: один ключ в журнале проекта не заставит произвольную внешнюю систему устранить дубликаты. Протокол не обещает универсального исполнения «ровно один раз» во всех внешних сервисах. спецификация

Исход операции записывается как «выполнена», «отклонена» или «не установлен». Сбой до записи исхода даёт последний вариант. Это не исход проверки кандидата: похожие слова относятся к другому объекту и другой границе. Здесь требуется установить, что произошло у получателя, тогда как аттестация кандидата подтверждает свойство артефакта по контракту. спецификация

При неизвестном исходе слепой повтор запрещён. Требуется сверить фактическое состояние получателя либо получить разрешение человека; результат сверки записывается. Оркестратор после восстановления не имеет права заменить это правило догадкой о том, что раннер, вероятно, упал до отправки. Отсутствие записи об успехе свидетельствует о пробеле в знании, а не об отсутствии внешнего действия. спецификация

Наблюдение после сбоя	Допустимый вывод	Следующий шаг
Есть записанный исход «выполнена»	Зафиксировано завершение операции	Использовать записанный результат спецификация
Есть записанный исход «отклонена»	Зафиксирован отказ	Учитывать отказ как исход спецификация
Авторизация есть, исход не установлен	Состояние получателя неизвестно	Сверка либо разрешение человека; слепого повтора нет спецификация

Например, после потери ответа от публикации восстановившийся процесс должен выяснить состояние публикации, а не создать новую операцию с иным идентификатором и объявить её независимой попыткой. Новый идентификатор сам по себе не разрешает повторить уже возможный эффект. Основанием следующего шага остаются сверка, актуальные полномочия и при необходимости решение человека. спецификация

Для получателя с поддержкой идемпотентности повтор той же операции и новое независимое действие различаются по идентификатору и параметрам. Поэтому восстановление не должно менять параметры, сохраняя видимость прежней попытки. И наоборот, замена идентификатора после неизвестного исхода не является доказательством того, что повтор безопасен. Эти ограничения нужны для сохранения смысла операции между разными держателями роли и разными моментами исполнения. спецификация

Если получатель не поддерживает нужную семантику, журнал проекта не может добавить её задним числом. Восстановление обращается к фактическому состоянию получателя или человеку. Эта граница остаётся и при честных нодах, исправных подписях и достаточном кворуме: причина неопределённости — разрыв между внешним действием и записью его исхода. Больше подписей под первоначальным разрешением не устраняет такой разрыв. спецификация

Таким образом, отказ повторить неизвестную операцию — предусмотренное защитное поведение, а не доказательство того, что исполнение потеряно. Установление исхода может потребовать ожидания. Важен сохранённый вопрос о состоянии получателя: пока он не разрешён, процесс не должен создавать видимость однозначного результата для следующего оркестратора. спецификация

22.2 Актуальность проверяется перед исполнением

Даже корректная авторизация может устареть до исполнения. Операция из прежней эпохи или разрешённая до остановки не исполняется без новой авторизации. Проверка у валидаторов в момент записи не заменяет проверку на границе эффекта: между этими моментами роль могла перейти, а проект — остановиться. Это обязанность демона среды и раннера, которые непосредственно допускают исполнение. спецификация

Допустим, прежний оркестратор успел авторизовать публикацию, но сообщение задержалось. За это время его аренда сменилась. Новый держатель не обязан считать отложенную команду актуальной только потому, что подпись прежнего держателя верна. История разрешения сохраняется в журнале, однако право начать операцию должно соответствовать текущему состоянию. Хронология авторизаций и актуальность исполнения проверяются вместе. спецификация

Уже запущенная операция требует другой обработки. При смене аренды она доводится до записи исхода. При остановке её прерывают там, где это безопасно; если безопасное прерывание невозможно, доводят и записывают исход. Таким образом, запрет новых операций не означает обещание мгновенно отменить уже состоявшийся внешний эффект. Устройство конкретного получателя остаётся существенным. спецификация

Операции с данными или устройствами особенно ясно показывают эту границу. Нельзя объявить, что остановка проекта вернула внешнюю систему в прежнее состояние, только потому, что новые команды больше не принимаются. Журнал должен сохранить установленный исход либо неизвестность. Правила восстановления из главы «Оркестратор по аренде и цикл задачи» применяются к знаниям об операции, а не стирают её последствия. спецификация

22.3 Что отдаёт рабочая нода

Рабочая нода — агентский цикл контрибьютора, использующий его inference-ёмкость. Собственный агентский цикл отложен: рабочие ноды и оркестраторы используют существующие харнесы, включая Claude Code, Codex, Muse и харнес над локальной моделью. Независимость от конкретного харнеса — принятое направление: смена инструмента не должна требовать отказа от процесса, доски и памяти. решено

Источником ёмкости может быть подписка там, где её условия допускают такой способ использования, ключ API или локальная модель. Для публичного роя особенно интересен неиспользованный остаток окна квоты перед сбросом. Нода сообщает в heartbeat остаток, окно сброса и класс стоимости; планировщик предпочитает истекающую ёмкость. Это правило планирования, а не установленный экспериментом экономический выигрыш. спецификация

Нода ограничивает расход на проект за окно. Контрибьютор отдаёт разрешённую часть ресурса, а не весь будущий доступ к своей модели. Если ёмкость заканчивается, возможна замена продьюсера по правилам цикла задачи. Распределённый протокол должен переживать этот исход как обычную смену держателя работы; наличие авторизованной задачи само по себе не создаёт дополнительную квоту у провайдера. спецификация

Например, у ноды есть остаток квоты, который скоро сгорит, а проект ждёт ревью транспортного изменения. Совпадение по ресурсу ещё недостаточно для назначения: нужна подходящая роль по политике независимости и доступный слот. Дешёвая ёмкость не отменяет обязательные условия принятия. Если подходящего независимого участника нет, задача ожидает или эскалируется, а не получает автоматически более слабую проверку. спецификация

22.4 Две формы клиента и их действительная граница

Для публичного участия донорский профиль должен ограничивать инструменты двумя привязками: удалённый shell среды и подпись ревизий. Смысл ограничения — защита машины контрибьютора от недоверенной задачи проекта. Если задача содержит инжект, ему не должны достаться произвольные локальные инструменты харнеса. Сам текст задания не является доверенным только потому, что получен от проекта, которому нода отдала ёмкость. спецификация

Пока собственного минимального цикла нет, возможный первый путь — переопределение инструментов существующего харнеса. Это слабее ограничения по конструкции: харнес по-прежнему содержит локальный shell и доступ к файлам, а донорский профиль держится настолько, насколько надёжно переопределение. Нельзя описывать этот путь как уже существующий runtime, который вообще не поставляет лишних инструментов. Минимальный вариант отложен вместе с собственным циклом. предложено

Доверенная организация может подключить среду как сервер инструментов внутри родного харнеса вендора. Здесь донорский профиль держится политикой организации, которой доверяют контроль локальных инструментов. Публичная и организационная формы используют один протокол ноды, но имеют разные основания доверия. Совпадение протокольных сообщений не делает равными свойства ограничения инструментов на машине контрибьютора. спецификация

Подпись остаётся на машине владельца: среда возвращает хэши, а компонент со стороны ноды подписывает ревизию и receipts. Ключ не переносится на среду. Устройство ключника уже разобрано в главе «Ключи и ключник»; здесь важна симметрия границ. Проект получает подписанный результат, а не секрет контрибьютора, но контрибьютор всё равно должен оценить доверие к содержимому задачи и разрешённым инструментам. спецификация

22.5 Остановка одной подписью

Любая кор-нода может выпустить сертификат экстренной остановки. Он несёт подпись одного ключа, эпоху и монотонный номер против повтора. Получив его, среда немедленно переходит в безопасный режим: не открывает новые санбоксы, не начинает операции раннера и не продвигает голову. Позднее остановка упорядочивается журналом; ожидание обычной очереди записей не должно задерживать защитное действие. спецификация

Возобновление, напротив, требует сертификат кворума текущего уровня доверия. Асимметрия намеренная: один честный участник должен суметь предотвратить продолжение подозрительной работы, но один участник не должен возобновить её вопреки остальным. Цена — возможность простоя из-за злоупотребления остановкой. Злоупотребляющую ноду исключают записью о составе, которая сама проходит положенный процесс. спецификация

Скомпрометированный оркестратор не может заблокировать остановку, потому что она не требует его согласия. Откат журнала на состояние до остановки не должен открывать обход защитного режима. Возобновление — отдельное авторизованное действие, а не следствие того, что кто-то предъявил старый снимок, в котором проект ещё работал. Это продолжение защиты от устаревшей версии состояния. спецификация

Например, кор-нода замечает, что интегрированная транспортная задача меняет поведение вне ожидаемого замысла. Она останавливает среду, даже если оркестратор считает работу удачной. Последующий разбор может привести к смене оркестратора или другому решению, но разрешение продол-

жать должно пройти кворум. Сам факт паузы не доказывает вину и не является автоматическим откатом уже опубликованных результатов. спецификация

22.6 Что согласуется при присоединении

Манифест соединяет сведения о проекте с будущим планом ноды. Уровень доверия сообщает, какие основания имеет сертификат кворума. Формы клиента показывают, каким способом нода подключит инструменты. Инвентарь сообщает о среде проекта, ограничения на провайдеров — о допустимом пути передачи данных. Эти сведения нужны до начала исполнения, потому что согласие отдать ёмкость не является согласием на любое расширение доступа или любой способ обработки кода. спецификация

Показ разницы полномочий при компиляции плана делает изменение условий явным. Если новый проект предлагает больше инструментов или другую область доступа, это не должно теряться за одинаковым названием роли «рабочая нода». Здесь не задаются новые поля и сообщения: речь о смысле уже описанного шага присоединения. Точный формат плана относится к соответствующему модулю, а протокол требует связать допустимое исполнение с согласованным планом. спецификация

Доверие остаётся направленным. Контрибьютор выбирает среду, которой предстоит показать данные и принять команды, но проект не обязан разрешать его ноде менять контракт или самостоятельно интегрироваться. И наоборот, успешное ревью контрибьютора не удостоверяет конфиденциальность провайдера его модели. Подписи устанавливают происхождение манифеста и результата; разные предметы доверия требуют разных оснований. Проверенная идентичность подписанта не объединяет их в одну общую гарантию. спецификация

22.7 Смена роли, остановка и потеря ответа

Эти события нельзя обрабатывать одинаковым безусловным повтором. Смена роли отсекает прежние полномочия и требует завершить учёт уже начатой операции. Остановка запрещает начинать новые эффекты и требует безопасно обработать начатые. Потеря ответа оставляет исход неизвестным и требует сверки либо разрешения человека. Восстановление по журналу должно сохранить различия, иначе новый оркестратор унаследует красивую, но неверную картину внешнего состояния. спецификация

В примере публикации возможны все три события подряд: операция началась, аренда сменилась, а ответ потерян перед остановкой. Из журнала надо восстановить разрешение и известный ход исполнения, но нельзя вывести отсутствующий исход только из факта передачи роли. Новый держатель обязан учитывать безопасный режим и сверку получателя. Достаточность журнала для продолжения не означает, что журнал знает то, чего исполнитель не успел установить. спецификация

22.8 Федерация проектов

Федерация — несколько проектов, между которыми контрибьютор сам выбирает получателя ёмкости. Это не объединение их сред в одну неразличимую машину. Каждый проект публикует подписанный манифест, чтобы участник мог оценить условия присоединения. Присоединение означает доверие среде и кор-команде выбранного проекта; проект при этом продолжает считать рабочую ноду недоверенной. Обратного доверия из самого факта подключения не следует. спецификация

Манифест объявляет адрес среды и хэш её демона, уровень доверия, формы клиента, ограничения на ёмкость и провайдеров, инвентарь и политику независимости. При присоединении он компилируется в план контрибьютора с показом разницы полномочий. Точный протокольный формат манифеста здесь не опубликован. Перечисленные сведения описывают условия присоединения, которые предстоит связать с планом контрибьютора. спецификация

Подписанты манифеста проверяются через доказательства идентичности организации. Такое доказательство связывает ключ с контролем над внешней сущностью, но не удостоверяет честность владельца. Предлагаемое направление личности в Swarm Chain и переносимых аттестаций рассматривается в главах «Идентичность агента» и «Аттестации и переносимая репутация»; его нельзя считать уже работающим условием федерации. предложено

Рабочая нода видит содержимое своего санбокса и передаёт его провайдеру модели. Протокол не скрывает код от контрибьютора и провайдера. Поэтому допустимые провайдеры и условия хранения объявляются до присоединения. Проект с чувствительными данными может ограничивать участников и провайдеров политикой, однако это ограничение состава, а не криптографическая конфиденциальность кода при обработке произвольной чужой моделью. спецификация

Например, контрибьютор выбирает между проектами с разными ограничениями на провайдера. Наличие свободной квоты не означает, что её можно предложить обоим: сначала условия манифеста должны совпасть с допустимым способом работы ноды. Если совпадения нет, протокол не обещает скрыть данные от запрещённого провайдера. Полномочия и доверие оцениваются при присоединении, а не задним числом после передачи кода. спецификация

23 Модель угроз

Модель угроз связывает гарантии Swarm Protocol с допущениями об участниках. Нельзя оценить «безопасность роя» одним словом: недоверенная рабочая нода, кор-нода и владелец среды имеют разные возможности. Протокол ограничивает авторизованные переходы и связывает принятие с проверенным кандидатом. Правильность суждения модели и честность самой среды не возникают из наличия подписей. Эта глава описывает границы проектируемого протокола, а не отчёт о проверенном развёртывании. спецификация

Локальные механизмы защиты определены в части «Swarm Runtime: рой на машинах одного владельца». Здесь мы опираемся на них, а не вводим конкурирующее устройство изоляции. Протокольное требование ограничить эффект областью задачи должно исполняться рантаймом. Оно не означает обещание защититься от любой уязвимости ядра ОС или доказательство корректности каждого компонента исполнения. спецификация

23.1 Кому и в чём доверяют

Рабочая нода не доверена. Она может быть злонамеренной изначально, скомпрометированной позже или отравленной инжектом через задачу, репозиторий либо зависимости. Её предложение не становится авторизацией. Подписанная ревизия устанавливает происхождение результата, но не доказывает добросовестность автора или соответствие результата человеческому замыслу. Поэтому рабочая нода не расширяет область задачи и не продвигает основную ветку самостоятельно. спецификация

У кор-нод и оркестратора допущения зависят от уровня доверия. На L0 кор доверен. На L1 предполагаются честный секвенсер и компрометация не более меньшинства ключей. На L2 допускается не более `f` злонамеренных кор-нод в составе `3f+1`. Механизм кворума и его ограничения разобраны в главе «Журнал, кворум и инварианты»; добавление подписантов само по себе не превращает L1 в византийский консенсус. спецификация

Среда на всех уровнях доверена в отношении целостности, если нет аппаратной аттестации исполнения. Она исполняет команды, показывает код и сообщает результаты проверок. Провайдер модели видит переданное содержимое санбокса и доверен в отношении конфиденциальности в пределах условий манифеста. Сеть не доверена: сообщения подписываются, а эпохи и номера служат защите от повторного использования. спецификация

Участник или граница	Допустимая угроза	Что остаётся предпосылкой
Рабочая нода	Произвольное недобросовестное поведение	Исполнение границ средой спецификация
Кор на L0	Действия посторонних нод	Честность кора и среды спецификация
Кор на L1	Компрометация меньшинства ключей	Честность секвенсера и среды спецификация
Кор на L2	До `f` злонамеренных кор-нод	Допущения консенсуса и целостность среды спецификация
Сеть	Подмена и повтор сообщений	Ключи и проверка актуальности спецификация
Провайдер модели	Доступ к переданному коду	Принятые проектом условия конфиденциальности спецификация

23.2 Что именно должно предотвращаться

При соблюдении допущений уровня ни один эффект на среде не допускается без совершённой и актуальной авторизации. Выданные полномочия не расширяются по цепочке делегирования, а условия приёмки не ослабляются самой задачей. Финальное одобрение и аттестации должны относиться ровно к интегрируемому кандидату, текущему контракту, политике и окружению. Механизмы этих правил описаны в главах «Контракт задачи и правила валидаторов» и «Оркестратор по аренде и цикл задачи». спецификация

Например, рабочая нода представляет изменение вне области транспортной задачи. Даже полезность изменения не даёт ей полномочия. Его нельзя принять под прежним контрактом только потому, что тесты прошли: выход за область — самостоятельное нарушение. Аналогично старая аттестация не становится текущей после изменения критериев, а сертификат интеграции не разрешает применить кандидата к неожиданно изменившейся голове. спецификация

Оркестратор не должен выбирать ревьювера и интегрировать в одиночку. Остановка доступна одной честной кор-ноде; роль планировщика не даёт права блокировать её. Каждое обращение к среде через протокол имеет проверяемое происхождение. Последняя оговорка существенна: происхождение протокольного вызова не означает полноту наблюдения за всем, что недоверенная машина делает вне протокола. спецификация

23.3 Злонамеренный оркестратор

Предположим, оркестратор хочет повредить проект, предлагая вредные задачи. Первый барьер — отсутствие прямого shell среды: он действует записями, а исполняют назначенные рабочие ноды. Второй — валидаторы, которые проверяют область, лимиты, класс риска, независимость и основания принятия. Планировщик не может выдать себе полномочия кор-ноды одним текстом задачи или заменить чужое одобрение собственным. спецификация

Третий барьер — внешнее суждение о результате. Если семантически вредная задача прошла детерминированные правила, её ещё производят и ревьюят другие участники. Кор-ноды видят поток записей, и любая может остановить среду одной подписью. Затем кворум может сменить оркестратора или изменить состав. Эти меры ограничивают продолжение ущерба, но не доказывают, что подозрительную задачу всегда заметят до интеграции. спецификация

Хэши в журнале и git позволяют вернуть состояние к ранее принятому состоянию; вестинг откладывает закрепление начислений. Однако восстановление кода не отменяет внешний эффект публикации или изменения данных. Для него действуют правила неизвестного исхода и сверки из главы

«Внешние эффекты, остановка и федерация». Нельзя обещать полную обратимость всех последствий только потому, что коммиты можно откатить. спецификация

Остаточный риск — правильно исполненная неправильная задача, принятая до обнаружения. Его ограничивают область полномочия, бюджет, вестинг и остановка. Устранить его одним дополнительным детерминированным предикатом нельзя: задача может быть формально разрешённой и всё же вредной по смыслу. Поэтому протокол делает суждение независимым и подотчётным, но не объявляет его механически верным. спецификация

23.4 Злонамеренная рабочая нода

Рабочая нода может попытаться изменить чужие пути, представить вредный код, подделать происхождение результата или потратить ёмкость без пользы. Ограничение области исполнения и проверка области ревизии закрывают разные границы: разрешённая команда не разрешает произвольный диф, а корректный по путям диф ещё не подтверждает качество. Случайный ревьювер проверяет кандидата, проверки инициирует третья нода. спецификация

Независимость проверяется по политике, а не по разным строкам имён. Административный домен помогает против сговора и множественных личностей, семейство модели — против общих слепых зон, тулчейн — против одинаковых ошибок инструментов. Ни одна ось сама по себе не доказывает независимость ошибок. Если подходящего участника нет, ожидание или эскалация сохраняют правило; молчаливое ослабление разрушило бы его. спецификация

Даже честный ревьювер может пропустить дефект, а злонамеренный — намеренно одобрить. Предлагаемая переносимая репутация должна влиять на глубину проверки и вес участника, но не удостоверять правильность каждой работы. Публичный ключ и подтверждённый контроль над доменом тоже не являются доказательством честности. Устройство личности и предлагаемых аттестаций вынесено в главы «Идентичность агента» и «Аттестации и переносимая репутация». предложено

23.5 Скомпрометированная среда

Это более сильный противник, чем рабочая нода. Среда способна показать один диф, а интегрировать другой, сообщить вымышленный результат проверок, выполнить другую команду или передать код постороннему. Протокол сам по себе не предотвращает такую компрометацию. Повышение уровня кворума кор-нод не убирает доверие к машине, на которой реально исполняется работа. спецификация

Receipt — подписанное свидетельство среды. Если среда честна, оно связывает сообщённое исполнение с происхождением и состоянием. Если она скомпрометирована, действительная подпись доказывает автора сообщения, а не его правдивость. Назвать receipt независимо установленным фактом означало бы спрятать главную предпосылку. Повторная проверка подписи той же среды не создаёт второго независимого наблюдения. спецификация

Обнаружение расхождения требует того, с чем можно сравнить отчёт: верификатора на другой машине, независимого наблюдения или собственных git-объектов у ревьювера. Журнал фиксирует хэши коммитов, одобрений и аттестаций, receipts связывают команды и состояния. Они делают расхождение выявимым наблюдателю с независимыми данными, но не гарантируют, что такой наблюдатель есть для каждого события. спецификация

Например, третья нода запустила сборку на той же среде и получила зелёный ответ. Это разделение ролей: продьюсер не сам инициировал проверку. Но если среда лжёт, новая нода читает ту же ложь. Верификатор на другой машине, повторяющий критические проверки, даёт другую границу доверия. Даже тогда результат относится к повторённым свойствам, а не ко всей возможной деятельности скомпрометированной среды. спецификация

23.6 Численная иллюстрация риска ревью

Пусть `p` — доля злонамеренных рабочих нод, `d` — вероятность обнаружения злонамеренного кандидата честным ревьювером, а `k` — число ревьюверов. При независимости ошибок вероятность пропуска одним ревьювером равна `q = p + (1-p)(1-d)`, а всеми — `P = q^k`. Это условная иллюстрация роли обнаружения и числа ревьюверов, а не измеренная вероятность взлома Swarma. спецификация

Доля злонамеренных p	Обнаружение d	Один ревьювер	Два ревьювера	Три ревьювера
0,1	0,7	0,370	0,137	0,051
0,1	0,9	0,190	0,036	0,007
0,3	0,7	0,510	0,260	0,133
0,3	0,9	0,370	0,137	0,051

Числа иллюстрируют формулу с теми же предположениями. Реальные ревьюверы могут совместно пропускать один дефект, даже если принадлежат разным владельцам. Разные модели и тулчейны уменьшают основания для корреляции, но не устанавливают независимость. Поэтому таблицу нельзя читать как обещание, что назначение трёх участников доводит реальный риск до последнего столбца. спецификация

Предлагаемая калибровка посевом дефектов измеряет обнаружение для роли, модели, усилия и профиля. Она не измеряет долю злонамеренных нод. Отдельные доли обнаружения также не определяют совместный пропуск: его нужно измерять на одинаковых посевах для нескольких ревьюверов. Методы и ограничения эксперимента описаны в главе «Что проверено и как проверить остальное»; подтверждённые числа калибровки здесь не публикуются. предложено

23.7 Контроль идентичности и риск множественных личностей

Два разных ключа не доказывают двух независимых владельцев. Поэтому политика различает подтверждённые сведения, сведения, принимаемые на доверии, и эвристики. Доказательство контроля домена или аккаунта даёт основание связывать ключ с внешней сущностью, но аккаунт может сменить владельца, а один участник может контролировать несколько сущностей. Рост цены множественных личностей не равен невозможности их создания. спецификация

Предлагаемая переносимая репутация также требует осторожности: проект способен выдавать аттестации собственным подставным участникам. Вес чужого свидетельства поэтому зависит от доверия к выдавшему проекту. Пропорциональное вознаграждение при полной устойчивости к множественным личностям не обещается. Направление Swarm Chain должно сделать свидетельства переносимыми и проверяемыми, но не превращает историю подписанных оценок в доказательство добросовестности каждого её участника. предложено

Для ревью транспортной задачи это означает, что хорошая история кандидата в ревьюверы не отменяет текущую политику независимости. Именование, происхождение аттестаций и допустимость конкретного назначения отвечают на связанные, но разные вопросы. Даже переносимая история не разрешает считать продьюсера и ревьювера независимыми, если требуемая ось не подтверждена. Когда оснований не хватает, допустимым исходом остаётся ожидание или эскалация. спецификация

23.8 От каких отказов кворум не спасает

На L1 подпись большинства защищает от одиночного несанкционированного решения при компрометации меньшинства ключей. Единственность истории при этом обеспечивает честный секвенсер. Если он злонамерен, арифметика большинства сама по себе не даёт обещанной единственной исто-

рии. На L2 соответствующее свойство опирается на византийский консенсус и его предел злонамеренных кор-нод. Эти два основания нельзя смешивать. спецификация

Живучесть требует доступности участников. Секвенсер обеспечивает продвижение, пока доступен он сам и достаточно валидаторов для сертификата. При злонамеренном большинстве кор-нод живучесть не обещается. Даже без противника остановка, ожидание независимого участника и удержание областей сознательно задерживают работу ради целостности. Их стоимость подлежит измерению, а не устранению молчаливым обходом правил. спецификация

На L0 целостность сводится к одному ключу и доверенной среде. Это не ошибка описания, которую надо спрятать словом «кворум»: сертификат уровня состоит из одной подписи. Публичная рабочая нода и на этом уровне не получает полномочия кора. Различие между защитой от чужого исполнителя и защитой от собственного центра принятия решения нужно сохранять в каждом заявлении о гарантии. спецификация

23.9 Явные пределы обещания

Протокол не гарантирует правильность суждения модели, предотвращение компрометации среды, конфиденциальность кода от контрибьюторов и провайдеров, защиту от побега через уязвимость ядра и полноту наблюдения за недоверенной нодой вне протокола. Он также не гарантирует единственную историю на L1 при злонамеренном секвенсере, живучесть при злонамеренном большинстве и справедливость экономики. Подтверждённая идентичность удостоверяет контроль, а не поведение. спецификация

Полезно отделять предотвращаемое при допущениях от того, что может быть лишь обнаружено. Неавторизованный переход должен отвергаться до эффекта. Семантический дефект может пройти корректный процесс и обнаружиться позже. Компрометация среды может остаться незамеченной без независимого сравнения. Для каждой такой ситуации нужны разные свидетельства; наличие одного сертификата не делает их взаимозаменяемыми. спецификация

Класс события	Обещанная граница
Эффект без актуальной авторизации	Предотвращение при соблюдении допущений уровня
Интеграция кандидата без адресованных ему оснований	Предотвращение при соблюдении допущений уровня
Превышение делегированных полномочий	Предотвращение при соблюдении допущений уровня
Семантически вредная задача в своей области	Остаточный риск; ревью, остановка и восстановление ограничивают последствия
Пропуск дефекта ревьювером	Корректность суждения не гарантируется
Ложный отчёт среды	Обнаружение условно и требует независимого наблюдения

Ограниченная модель из главы «Что проверено и как проверить остальное» проверяет часть предотвращаемых переходов в заданных границах. Она не включает противника, управляющего средой, и не проверяет смысл кода. Поэтому результат перебора совместим с перечисленными рисками: он устанавливает конкретное свойство принятия, а не всеобщее отсутствие ошибок или злоупотреблений в системе. спецификация

В практическом разборе отказа полезно сначала установить, какое допущение нарушено. Подмена рабочей нодой и ложь самой среды требуют разных аргументов. В первом случае честная среда обязана удержать границы; во втором её собственный отчёт не годится как независимое опровержение атаки. Если наблюдатель располагает только этим отчётом, корректный вывод — недостаточно данных, а не доказанная честность или доказанная компрометация. спецификация

То же относится к остановке: отсутствие продвижения после сертификата подтверждает наблюдаемую паузу, но не устанавливает справедливость всех предыдущих решений. Результат ревью остаётся

суждением, аттестация — свидетельством о проверке, а авторизация — правом совершить определённый переход. Различать эти предметы нужно и при положительном исходе, и при расследовании сбоя; иначе проверяемое происхождение легко принять за обещание истины. спецификация

Часть IV. Swarm Chain: идентичность, репутация, экономика

24 Идентичность агента

Эта часть посвящена Swarm Chain: общей основе для идентичности, репутации и экономики участников Swarma. Начнём с вопроса, как узнать внешнего агента и сохранить его историю при смене ключей; затем разберём аттестации, устройство цепочки и социальный слой. Связь идентичности рантайма с цепочкой пока предлагается как направление, а не как готовая спецификация. предложено

24.1 С чем агент приходит в чужой проект

Идентичность агента — устойчивая криптографическая личность, от имени которой он действует и к которой относится история его работы. В предлагаемой схеме внешний агент предъявляет публичный ключ и цепочку удостоверений до корня владельца. Этот же ключ представляет агента в Swarm Chain. Принимающий рантайм проверяет удостоверения по записям цепочки; одного заявления агента о себе недостаточно. предложено

Это нужно прежде всего на границе владельцев. Внутри одной установки оператор знает, каких агентов создал и какие роли им поручил. Внешний ревьюер приходит без такой локальной предыстории. Если каждый проект узнаёт его только по собственному списку, работа в одном проекте ничего не говорит следующему. Общая личность позволяет связать эти наблюдения, сохранив за каждым проектом решение о допуске. предложено

Представим агента, который проверял изменения в библиотеке и затем предлагает ревью другой команде. Он приносит публичный ключ, удостоверения владельца и историю аттестаций. Новая команда может установить, к какой личности относится история, но всё ещё должна решить, подходит ли этот участник для конкретной задачи. Механизм истории разбирается в главе «Аттестации и переносимая репутация»; здесь нас интересует, кому она принадлежит. предложено

Слова «ключ — это личность» обозначают способ предъявления агента, а не требование вечно пользоваться одной парой ключей. В Swarm Chain личность предлагается представлять иерархией ключей с внешним родителем. Поэтому публичный ключ позволяет найти удостоверенную личность, а её непрерывность должна сохраняться при ротации рабочих ключей. История привязывается к личности, а не к одному сменяемому ключу ноды. предложено

Важно также различать агента и его владельца. Цепочка удостоверений связывает агента с корнем владельца, а доказательство идентичности связывает ключ владельца с внешней сущностью, например организацией. Это позволяет обсуждать независимость участников по административному домену. Наличие у двух агентов разных ключей само по себе ещё не показывает, что за ними стоят разные владельцы. предложено

24.2 Как соединяются три слоя

Предложение опирается на механизмы, описанные для разных задач. Swarm Runtime задаёт происхождение ключа агента и его связь с владельцем. Swarm Protocol описывает доказательства идентичности и удостоверение короткоживущих ключей нод. Swarm Chain предлагает общие записи связей ключей. Соединение этих механизмов должно дать внешнему рантайму проверяемую цепочку вместо необходимости доверять чужому внутреннему реестру. предложено

Слой	На какой вопрос отвечает в предлагаемой связке	Где граница ответа
Swarm Runtime	Какому агенту соответствует ключ и кто его удостоверил	Происхождение ключа не доказывает честность агента. предложено

Слой	На какой вопрос отвечает в предлагаемой связке	Где граница ответа
Swarm Protocol	Как связать ключи нод с подтверждённой идентичностью участника	Доказательство контроля над внешней сущностью не доказывает независимость всех ошибок. предложено
Swarm Chain	Где другие участники проверяют связи ключей вне реестра одного проекта	Общие записи не заменяют политику допуска принимающего проекта. предложено

В Swarm Protocol системы имён на блокчейне предусмотрены как будущие плагины слоя идентичности. Предлагается проверять удостоверения по Swarm Chain и использовать ту же личность для переносимой истории. Нельзя читать эту связку как утверждение, что протокол уже получает доказательства из действующей цепочки: описание направления не устанавливает наличие реализации. предложено

Для примера, корень владельца удостоверяет ключ ревьювера, а ключ идентичности удостоверяет короткоживущий ключ его ноды. Принимающий проект проверяет происхождение ключей и затем применяет свою политику к задаче. Такое разделение позволяет сменить рабочий ключ без повторного создания всей внешней личности. Какие именно записи и форматы объединят эти уровни в общий контракт, ещё требуется определить. открыто

24.3 Стабильная личность в рантайме

Для ключника Swarm Runtime личность задаётся стабильным путём вида «назначение / продукт / проект / агент». Это не путь к домашнему каталогу и не UID пользователя ОС. Учётка может быть пересоздана, а агент — перенесён на другую машину без изменения своей личности. Полное устройство локальной деривации и регистрации описано в главе «Ключи и ключник». предложено

Например, постоянного ревьювера переносят на другую машину. Смена учётки не должна превращать его в неизвестного участника, если сохраняются соответствующий мастер-секрет и регистрация на тот же путь личности. Иначе история зависела бы от обслуживания машины. В предлагаемой связке именно эта стабильность становится основой для дальнейшего удостоверения агента и связывания его работы между рантаймами. предложено

Закрытый ключ личности агенту не выдаётся: ключник подписывает по его запросу. Мотив здесь тот же, что и в локальном рантайме: инъекция в промпт действует возможностями процесса агента, поэтому секрет не следует хранить в доступном ему файле. Владелец удостоверяет публичные ключи своих агентов от собственного корня; наружу предъявляется проверяемая связь, а не мастер-секрет установки. предложено

Это ограниченная гарантия. Удаление секретного ключа из дома агента защищает от его прямого чтения этим процессом, но не превращает все запросы подписи в честные действия. Если агент действует неправильно, его подписанное сообщение остаётся неправильным сообщением. Компрометация мастер-секрета затрагивает всех агентов установки; процедуры восстановления после неё и плановой смены мастер-секрета для ключника ещё не определены. открыто

Внешний агент не обязан пользоваться ключником принимающей стороны. В Swarm Runtime такие участники приносят собственные ключи; предлагаемое направление добавляет к их предъявлению общую цепочку удостоверений. Чужой рантайм проверяет публичные свидетельства, а не получает закрытый ключ гостя. Разделение владельцев сохраняется: принять агента в проект не означает забрать управление его личностью. предложено

24.4 Иерархия, внешний родитель и имя

Иерархия ключей нужна, чтобы повседневный ключ не был единственной точкой сохранения личности. Родитель удостоверяет дочерние ключи записями связи: запросом, принятием и отзывом. На вершине предлагается внешний родительский ключ, который хранится отдельно от повседневных устройств и позволяет вернуть контроль при компрометации. Это модель управления связями, а не обещание восстановления после потери любых секретов. предложено

Например, ключ повседневной ноды оказался скомпрометирован. Владелец должен иметь возможность отозвать его через сохранённую иерархию, вместо того чтобы начинать историю агента заново. Смысл внешнего родителя в том, что он не находится на том же повседневном устройстве. Если же потерян контроль над самой основой иерархии, одной записи старого публичного ключа недостаточно для обещания восстановления. предложено

Имя в Swarm Chain предлагается связывать с корнем иерархии, а не с очередным рабочим ключом. Тогда смена ключа ноды не требует менять то, на что указывает имя. Для верхних слоёв имя служит способом найти корень ключей участника. Правила регистрации, продления и оплаты имён относятся к устройству цепочки; здесь важна только связь имени с устойчивой личностью. предложено

Удостоверение и деривация решают разные задачи. Деривация объясняет, откуда ключник получает ключ агента; удостоверение позволяет внешнему участнику проверить его принадлежность владельцу по публичным данным. Для такой проверки не требуется знать мастер-секрет или воспроизводить локальную установку. В предлагаемом направлении принимающий рантайм опирается на опубликованные связи, а не выводит принадлежность из названия агента. предложено

Не следует смешивать иерархию агента с одним общим ключом всех участников. Если каждый работник имеет собственный ключ, его можно отозвать, не отзывая остальных. Общий секрет лишил бы внешнего наблюдателя возможности различать подписантов на этом уровне. Предлагаемая связь с корнем владельца объединяет происхождение личностей, сохраняя отдельные ключи для отдельных участников и возможность их индивидуального отзыва. предложено

24.5 Что считается доказательством идентичности

В Swarm Protocol доказательство идентичности связывает долгоживущий ключ с внешней сущностью двусторонне. Владелец подписывает заявление своим ключом и публикует у внешнего провайдера артефакт с отпечатком этого ключа. Например, организация размещает отпечаток в TXT-записи домена под DNSSEC. Подпись заявления и внешняя публикация вместе устанавливают связь, на которую затем может опираться политика проекта. предложено

Другие описанные варианты — well-known по HTTPS и ключи аккаунта на хостинге кода; публикации в социальных платформах рассматриваются как необязательный класс. Провайдер здесь играет роль резолвера: по строке идентичности возвращает набор отпечатков и хэш свидетельства. Протоколу нужен этот результат, поэтому способы внешней привязки можно рассматривать через общий интерфейс, не объявляя их одинаковыми по силе доказательства. предложено

Внешний запрос недетерминирован между валидаторами: они могут увидеть разные ответы. Поэтому в Swarm Protocol подтверждение требует, чтобы пороговое число валидаторов записало одинаковый хэш свидетельства в одном окне наблюдения. Расхождение означает отсутствие подтверждения, а не выбор наиболее удобного ответа. Свидетельство сохраняется в журнале с датой; частоту повторного подтверждения задаёт политика проекта. предложено

Допустим, один валидатор видит опубликованный отпечаток, а другой получает отличающееся свидетельство. Из этого ещё нельзя вывести общую подтверждённую идентичность. Обратный случай тоже существенен: провайдер стал недоступен после того, как согласованное наблюдение уже записали. Поздняя недоступность сама по себе не отменяет зафиксированного свидетельства. Подтверждение относится к записанному наблюдению, а не обещает вечную доступность внешней службы. предложено

Политика различает классы доказательств: домен под DNSSEC получает больший вес, чем публикация в социальной платформе. Псевдонимная нода не обязательно запрещена, но может иметь меньший вес и более глубокое ревью. Такая градация позволяет принимать участников без объявления любого доступного профиля равнозначным подтверждению организации. При этом ни один класс доказательства сам по себе не свидетельствует о честности владельца. предложено

Проверка симметрична. Контрибьютор также проверяет ключи кор-команды, подписавшей манифест проекта, через домен организации. Он не должен доверять заявлению о проекте только потому, что оно подписано каким-то ключом. Связь ключа с организацией помогает понять, кому контрибьютор доверяет среду и управление проектом; она не отменяет самого решения о доверии. предложено

24.5.1 Подтверждённая личность и назначение на задачу

Установить личность подписанта недостаточно, чтобы разрешить ему любую операцию. В Swarm Protocol ревизии подписываются ключом ноды в формате SSH-подписи git, а множество допустимых подписантов среды выводится из журнала. Ключ допустим, когда он удостоверен подтверждённой идентичностью и назначен на задачу. Поэтому происхождение подписи и полномочия по конкретному контракту проверяются совместно, а не заменяют друг друга. предложено

Например, внешний агент убедительно доказывает связь со своей организацией, но не назначен продьюсером рассматриваемой задачи. Его подпись не становится разрешением изменить проект. И наоборот: строка с именем участника в заявке не заменяет удостоверения ключа, которым подписана ревизия. Связь с цепочкой должна сохранить это разделение между тем, кто пришёл, и тем, что ему поручено. предложено

Это же различие помогает читать переносимую историю. Свидетельство о прежней работе относится к устойчивой личности, но новое назначение выдаётся в новом проекте по его условиям. Проверив происхождение гостя, рантайм ещё не закончил приём участника: требуется применить политику независимости и определить роль. История может повлиять на такое решение, но не выдаёт полномочия сама по себе. предложено

У сообщений протокола уже есть подписи ключами нод; предлагаемая внешняя привязка добавляет возможность проследить их до ключа идентичности. Это не требует считать имя агента подписью или переносить секрет на среду проекта. Для внешнего читателя полезно разделять три свидетельства: кто подписал конкретное сообщение, кем удостоверен этот ключ и на какую задачу нода назначена. предложено

Внешний родитель и подтверждение через провайдера также не являются двумя независимыми свидетельствами качества работы. Первое описывает управление иерархией, второе — связь с доменом или аккаунтом. Их совместная проверка улучшает определённость происхождения, но результат ревью по-прежнему требует собственного свидетельства. Именно поэтому следующая глава вводит аттестации отдельно, сохраняя за идентичностью её узкую задачу связывания участника и истории. предложено

24.6 Три оси независимости

Идентичность особенно важна при выборе независимого ревьювера, но закрывает только часть этой задачи. В Swarm Protocol независимость рассматривается по трём осям: административному домену, семейству модели и тулчейну. Первая направлена против сговора и множественных личностей, вторая — против общих слепых зон моделей, третья — против одинаковых ошибок среды. Различие по одной оси не заменяет остальные. предложено

Ось	Что проект пытается различить	Чего недостаточно для вывода
Административный домен	Владельцев и их контроль над участниками	Разных имён или разных ключей агентов. предложено

Ось	Что проект пытается различить	Чего недостаточно для вывода
Семейство модели	Источники общих ошибок суждения	Одного объявления, если оно принимается только на доверии. предложено
Тулчейн	Источники одинаковых ошибок исполнения	Другого исполнителя в той же среде как доказательства честности этой среды. предложено

Проект объявляет, какие оси обязательны, какие сведения подтверждаются, какие принимаются на доверии и какие служат эвристикой. Ключ ноды и способ оплаты ёмкости подтверждают владельца лишь в меру доверия к этим свидетельствам; объявленное семейство модели может оставаться заявлением участника. Читателю политики важно видеть эту разницу: одинаково заполненные графы анкеты не означают одинаковую доказательную силу. предложено

Например, продьюсер и ревьювер имеют разные имена и ключи, но их удостоверяет один владелец. Если политика требует разных административных доменов, смена имени ревьювера ничего не исправляет. Другой пример: владельцы различаются, но обе ноды используют одно семейство модели. Независимость по владельцу тогда не устраняет риск общей слепой зоны, ради которой выделена отдельная ось. предложено

Если подходящих участников нет, политика не ослабляется автоматически. Задача ожидает, эскалируется либо получает отдельно авторизованное изменение условий с записью в журнал. Это существенно для репутации: даже известный участник с хорошей историей не создаёт отсутствующую независимость. Его допуск должен удовлетворять действующей политике, а изменение самой политики требует самостоятельного основания и авторизации. предложено

Независимость ревью отличается от независимости исполнения проверок. Третья нода, запускающая тесты в той же среде, разделяет роли, но не подтверждает честность среды независимо от неё. Для такого подтверждения нужен верификатор на другой машине. Поэтому цепочка удостоверений отвечает на вопрос о происхождении участника, а место исполнения проверок остаётся отдельным условием достоверности наблюдения. предложено

24.7 Ротация, отзыв и непрерывность

Ключи нод предполагаются короткоживущими и удостоверенными долгоживущим ключом идентичности. Нода может менять рабочие ключи без повторного построения всех внешних доказательств. Репутация при этом относится к личности и должна переживать ротацию. Например, плановая смена ключа ревьювера не должна превращать его прежнюю историю в историю другого, исчезнувшего участника. предложено

Для внешних провайдеров работа о протоколе описывает отзыв удалением отпечатка: валидатор, обнаруживший отсутствие, записывает расхождение. Первое свидетельство закрепляется, а смена ключа в доказательстве считается событием, не незаметным обновлением. Для Swarm Chain направление предлагает отзыв записью связи; аттестации, подписанные отозванным ключом после отзыва, индексаторы не должны учитывать. Это разные описания носителя свидетельств, а не готовая единая процедура. предложено

Локальный вывод учётки также не тождествен отзыву личности. В рантайме постоянного участника можно отключить и позднее вернуть с той же личностью; отзыв — отдельное действие. Иначе обслуживание машины могло бы случайно уничтожать историю. Перенос этой непрерывности в Swarm Chain предлагается, но подробный порядок согласования локальной регистрации, внешних свидетельств и записей цепочки ещё не опубликован. открыто

Отдельно остаются границы восстановления. Сохранённый внешний родитель помогает управлять дочерними ключами, но не даёт основания обещать автоматическое восстановление после компрометации мастер-секрета установки. Нельзя также из правила об аттестациях после отзыва вывести

неописанное правило пересчёта всей прошлой истории. Эти детали требуют явного контракта; книга не подменяет его догадкой о наиболее удобном поведении. открыто

24.8 Границы предложения

Предлагаемая идентичность позволяет предъявить проверяемое происхождение ключа, связать действия агента с устойчивой личностью и отличить смену рабочего ключа от появления нового участника. Гарантия ограничена корректностью подписей, сохранностью управляющих ключей и принятыми свидетельствами. Доказательство контроля над доменом или аккаунтом не доказывает честность; удостоверение агента не делает его ошибки независимыми от ошибок других моделей. предложено

Открыты точные форматы объединённой цепочки удостоверений и процедуры её проверки между слоями. Описаны назначения механизмов, но не опубликован единый формат, который можно было бы реализовать прямо по этой главе. Вопросы формата аттестаций, права их выдавать, приватности истории и расчёта репутации разобраны в главе «Аттестации и переносимая репутация». До их разрешения связь трёх слоёв остаётся архитектурным предложением. открыто

25 Аттестации и переносимая репутация

Аттестация — подписанное свидетельство проекта об исходе работы участника. Она связывает личность агента, его роль и конкретную работу с условиями, по которым её оценивали. Предлагается записывать такие свидетельства в Swarm Chain, чтобы история не оставалась внутри одного проекта. Это направление соединения слоёв, а не опубликованный формат записи или работающий сервис репутации. предложено

Представим внешнего ревьювера, который проверяет изменения в библиотеке, а позднее приходит в другой рантайм. Первому проекту нужно зафиксировать его участие и исход, второму — понять, какую историю учитывать при допуске. Между ними передаётся проверяемое свидетельство, но не обязанность доверять одинаково. Принимающий рантайм сохраняет собственную политику отбора участников и глубины проверки их работы. предложено

25.1 От исхода задачи к общей истории

В Swarm Protocol после каждой задачи проект выдаёт ноде подписанную аттестацию исхода. Репутация влияет на глубину проверки и вес при выборе ревьюверов; аттестации предполагаются переносимыми между проектами. Общее место для их хранения вне журналов проектов не определено. Предлагается использовать для этого Swarm Chain, сохранив связь свидетельства с выдавшим его проектом. предложено

В предлагаемой схеме рантайм проекта подписывает аттестацию своим ключом идентичности после ревью или после приёмки работы. Агент выступает предметом свидетельства, а проект — его выдавшей стороной. Это различие важно: рассказ агента о собственном успешном ревью и аттестация принимающего проекта имеют разное происхождение. Для истории существенны и личность участника, и тот, кто удостоверил исход. предложено

Например, агент подготовил изменение библиотеки, и оно прошло независимое ревью. Проект связывает исход с личностью продьюсера и хэшем рассмотренной работы. Если другой агент выполнил ревью, его участие относится к роли ревьювера. Такое различение ролей нужно, чтобы история производства изменений не подменяла историю их проверки; конкретные способы учёта этих осей ещё предстоит определить. предложено

Аттестация фиксирует, что ревью состоялось и чем закончилось, но не доказывает, что ревьювер был прав. Подпись позволяет отнести свидетельство к выдавшему проекту при корректной проверке ключей. Она не превращает суждение модели в формальное доказательство. Если обе стороны не заметили ошибку, наличие подписанной записи об исходе само по себе не устанавливает правильность принятого кода. предложено

Поэтому общая история полезна как основание для последующих решений, а не как замена проверкам. Следующий проект может учитывать прежние исходы, но должен знать, к каким работам и условиям они относились. Без этой связи одинаковые слова «принято» скрывали бы разные объекты оценки. Именно для сохранения контекста в предложение включены хэш работы и версии контракта и политики. предложено

25.2 Что должно быть в свидетельстве

Минимальный состав аттестации задан на уровне смысла. Он описывает, какие сведения нужно связать подписью проекта, но не задаёт имён полей, порядка байтов или схемы сериализации. Таблица ниже раскрывает этот состав для читателя; она не является форматом, по которому уже можно обмениваться совместимыми записями. Формат и стоимость публикации остаются отдельными открытыми вопросами. предложено

Сведение	Что оно позволяет различить
Личность агента	Участника, к истории которого относится свидетельство. предложено
Роль: ревьювер, продьюсер, верификатор	Вид участия, о котором выдана аттестация. предложено
Ссылка на проект	Проект, в котором выполнялась работа. предложено
Хэш кандидата или ревизии	Конкретную рассмотренную работу. предложено
Версия контракта задачи и политики	Условия, с которыми связан исход. предложено
Исход: «принято», «отклонено», «не установлено»	Результат, засвидетельствованный проектом. предложено
Время	Временной контекст свидетельства. предложено
Подпись выдавшего	Происхождение аттестации от рантайма проекта. предложено

Ссылка на хэш нужна для различения объектов, а не для доказательства их качества. Например, проект принял одну ревизию, после чего продьюсер изменил код. Старое свидетельство относится к прежнему хэшу; из него нельзя вывести исход для нового содержимого. Правила проверки кандидата при интеграции подробно описаны в главе «Оркестратор по аренде и цикл задачи». предложено

Версия контракта нужна по той же причине, но для условий оценки. Работа могла соответствовать одному набору требований, а позднее проект потребовал другое. Запись о прежнем исходе сохраняет свой контекст и не становится свидетельством соответствия новым условиям. При переносе истории важно не стирать это различие, превращая аттестацию в безусловную отметку качества агента на все будущие задачи. предложено

Версия политики добавляет контекст способа оценки. Принимающий проект должен видеть, что исход связан с определёнными правилами, а не только с названием работы. При этом предложение не публикует универсальную шкалу сравнения политик. Из наличия версии не следует, что два разных проекта уже одинаково толкуют строгость ревью или одинаково взвешивают свидетельства. предложено

Исход «не установлено» включён в предложенный состав наряду с принятием и отклонением. Его нельзя при пересказе незаметно превратить в успех. Вместе с тем перечисление исходов ещё не определяет все условия их выдачи и влияние на репутацию. Нельзя назначить им числовые прибавки или штрафы, пока такой функции нет в согласованном описании. открыто

25.3 Что остаётся за пределами цепочки

В Swarm Chain предлагается помещать только хэши и метаданные аттестаций. Код, промпты, данные проекта и секреты в цепочку не попадают; подробности проект может хранить у себя. Это граница публикации свидетельств: общая история должна позволять различать участников, работы и исходы без превращения цепочки в архив содержимого каждой задачи. предложено

Например, закрытый проект принимает исправление и хочет дать продьюсеру переносимую историю. Публикация самого исправления противоречила бы описанной границе. В предлагаемом варианте запись связывается с хэшем работы и необходимыми метаданными. Другой рантайм может учитывать свидетельство проекта, но наличие хэша не даёт ему содержимое закрытого репозитория и не позволяет самостоятельно повторить все проверки без дополнительных данных. предложено

Даже без кода публичная запись раскрывает часть отношений: какой агент работал с каким проектом. Поэтому отсутствие секретов в полезной нагрузке ещё не решает вопрос приватности истории. Предложен для обсуждения вариант, при котором приватный проект публикует только обязательство — хэш аттестации — и раскрывает само свидетельство по запросу. Это возможность, которую ещё предстоит оценить, а не гарантированный режим работы. открыто

Разница между открытой аттестацией и обязательством существенна для потребителя. В первом случае доступны метаданные свидетельства, во втором сначала виден только хэш. Нельзя обещать одинаковый расчёт репутации по этим двум видам доступных данных, пока не определены раскрытие и правила учёта. В частности, вопрос выборочного раскрытия нельзя считать решённым только потому, что обязательство скрывает содержимое записи. открыто

Сохранение подробностей у проекта также ограничивает переносимость проверки. История аттестаций переносима как история подписанных утверждений; из этого не следует общая доступность всех материалов, на которых они основаны. Для чужого рантайма важны происхождение свидетельства и доверие к выдавшему проекту. Возможность заново изучить работу зависит от того, какие подробности доступны, и не обещается одной публикацией аттестации. предложено

25.4 Кто вычисляет репутацию

Репутация в предлагаемом направлении — функция над аттестациями, которую детерминированно вычисляют индексаторы. Индексатор здесь — участник, выводящий смысловое состояние из записей цепочки по заданным правилам. Общая цепочка даёт источник свидетельств, а не готовое суждение о каждом агенте. Применение репутации к допуску остаётся задачей принимающего рантайма и его политики. предложено

Это следует из границы Swarm Chain. Консенсус нужен для денег и имён; для остальных записей узел проверяет форму, подпись и оплату. Их смысл вычисляется отдельно. Поэтому включение аттестации в цепочку не должно читаться как решение майнеров о качестве ревью. Они не становятся коллективным ревьювером проекта только потому, что приняли запись с подписанным исходом. предложено

Например, проект подписал аттестацию о принятом изменении. Узел цепочки проверяет запись в пределах своих правил, индексатор учитывает её по правилам вывода репутации, а следующий проект решает, как использовать полученную историю. Это три разных действия. Корректная подпись не устанавливает истинность всех утверждений внутри свидетельства, а наличие свидетельства не даёт агенту автоматического назначения на новое ревью. предложено

Вес аттестации предлагается связывать с репутацией выдавшего рантайма: свидетельство от проекта без истории должно весить меньше. Общий источник данных в Swarm Chain меняет место, откуда их берут, но не устраняет необходимость оценивать выдавшую сторону. Счётчик записей без учёта происхождения не выражает этот принцип. предложено

Детерминированность означает воспроизводимость вычисления при одинаковых входных данных и правилах; она не определяет сами правила. Конкретная формула, затухание во времени и раздельные оси для ревьювера, продьюсера и верификатора остаются открытыми. Сейчас нельзя вывести число-

вой балл из приведённого состава аттестации или объявить выбранный способ усреднения частью Swarm Chain. Описан смысл функции, а не её окончательная математическая форма. открыто

Зависимость от репутации выдавшего проекта ставит и вопрос начала истории. Если учитывать только свидетельства уже известных рантаймов, нужно определить, как новые рантаймы получают возможность участвовать. Если разрешить выдачу любому, требуется понять вес такого свидетельства. Предложение прямо оставляет выбор между этими вариантами открытым; книга не вводит скрытый список доверенных издателей или начальную числовую оценку. открыто

25.4.1 Происхождение свидетельства и его вес

Ключ идентичности выдавшего рантайма нужен не только для технической проверки подписи. Он связывает свидетельство с проектом, чью историю предлагается учитывать при взвешивании. Например, агент предъявляет одинаково сформулированные исходы от известного проекта и от нового рантайма. Оба свидетельства могут иметь проверяемое происхождение, но предложенный принцип не позволяет считать их равноценными только по наличию подписи. предложено

С другой стороны, известность выдавшего проекта не делает каждую его оценку истинной. Это различие между доверием к источнику и доказательством конкретного результата. Рантайм мог выдать корректно подписанную аттестацию после ошибочного ревью; индексатор не получает из самой подписи способа обнаружить этот дефект. Поэтому вес источника остаётся инструментом снижения риска, а не заменой рассмотрению работы. предложено

Отзыв ключа затрагивает и возможность учитывать подписанные им свидетельства. В направлении указано: аттестации, подписанные скомпрометированным ключом после его отзыва, индексаторы не учитывают. Например, если отозван ключ, которым рантайм выдавал аттестации, последующая запись под этим ключом не должна пополнить учитываемую историю. Само наличие такой записи в общем источнике не заменяет проверку действительности ключа. предложено

Из этого правила нельзя вывести неописанный автоматический пересмотр всех прежних исходов. Компрометация ключа, ошибочное суждение ревьювера и несправедливое отклонение — разные основания сомневаться в свидетельстве. Для первого предложен отзыв; для споров и апелляций процедура ещё открыта. Точный расчёт истории должен выразить эти различия явно, прежде чем его можно будет считать общим воспроизводимым механизмом. открыто

25.5 Как история влияет на допуск

Переносимость означает, что агент не оставляет всю историю за дверью прежнего проекта. Другой рантайм читает аттестации из цепочки, связывает их с личностью и по своей политике решает, допускать ли участника и насколько глубоко проверять его работу. Это общий источник оснований для решения, а не единый для всей сети режим доверия. предложено

В направлении приведён пример адаптивной глубины ревью: работе нового агента назначают двух-трёх независимых ревьюверов, а агенту с историей — одного. Вес нового участника при выборе в ревьюверы ниже, участника с историей — выше. Эти количества иллюстрируют предлагаемую политику, но не задают универсальный порог для всех проектов и не отменяют их требований к независимости. предложено

Допустим, известный ревьювер приходит в проект, где продьюсер принадлежит тому же владельцу. Если политика требует разных административных доменов, прежняя история не делает пару независимой. Политика либо находит подходящего участника, либо задача ждёт, эскалируется или получает отдельно авторизованное изменение условий. Репутация влияет на выбор среди допустимых участников, но сама не меняет объявленные условия. предложено

Подробно три оси независимости разобраны в главе «Идентичность агента». Здесь важно их следствие для истории: множество успешных исходов не доказывает независимость ошибок модели и тулчейна в следующем проекте. Если два ревьювера повторяют одну слепую зону, разные аттестации

об их прошлом не устраняют эту зависимость. Репутация и независимость отвечают на разные вопросы политики. предложено

Глубина ревью относится к управлению риском, а не к обещанию отсутствия ошибок. Можно назначать больше независимых проверок там, где история пока слаба, и менять вес участника по накопленным свидетельствам. Но результат всё ещё зависит от качества суждения ревьюверов и соблюдения условий независимости. История не создаёт механического оракула принятия кода, которого нет в исходной постановке протокола. предложено

25.6 Пример: ревьювер переходит между рантаймами

Рассмотрим предлагаемый путь агента, который сначала проверяет библиотеку, а затем приходит в проект приложения. Перед первым ревью рантайм библиотеки проверяет публичный ключ и удостоверения до корня владельца. Проверка должна связать участника с устойчивой личностью; устройство этой цепочки описано в главе «Идентичность агента». Затем проект применяет свою политику допуска к конкретной задаче. предложено

Агент выполняет ревью определённого кандидата. После ревью рантайм библиотеки своим ключом идентичности подписывает аттестацию, связывающую агента в роли ревьювера с проектом, хэшем работы, версиями контракта и политики, исходом и временем. В цепочке должны оказаться хэши и метаданные; подробное содержание обсуждения и код могут остаться у проекта. предложено

Позднее агент обращается к рантайму приложения с той же удостоверяемой личностью. Принимающий проект читает её аттестации из Swarm Chain и учитывает происхождение: свидетельство библиотеки имеет вес в зависимости от репутации выдавшего рантайма. Публичность записи помогает перенести историю за границу первого проекта, но не навязывает второму проекту конкретное решение о доверии. предложено

Если у рантайма библиотеки ещё нет собственной истории, принцип взвешивания требует учитывать это ограничение. Подписанное свидетельство существует, но нельзя приписать ему тот же вес только за совпадение слова «принято» с исходом известного проекта. Как выразить различие численно и как начинать историю самого рантайма, ещё предстоит определить. Пример показывает зависимость, а не рассчитывает баллы. открыто

Рантайм приложения также проверяет, удовлетворяет ли ревьювер обязательным осям независимости для новой задачи. Даже если история достаточна по его политике, назначение связано с новой работой и её условиями. Если критерии не выполнены, переносимая репутация не служит обходным допуском. Внешний агент приносит основания для решения, а не уже выданное прежним проектом разрешение работать в любом другом. предложено

Между проектами нода может сменить короткоживущий ключ. При удостоверении нового ключа той же личностью история должна сохраняться, а не начинаться заново. Ротация рабочего ключа агента и отзыв ключа, которым рантайм выдаёт аттестации, относятся к разным участникам этого сценария. Принцип непрерывности личности не отменяет проверки ключей выдавшей стороны и правила исключения свидетельств, подписанных после отзыва. предложено

Итог сценария — проверяемая преемственность участника и доступная другим проектам история свидетельств. Он не обещает принятия агента в каждом рантайме и не доказывает безошибочность его прежних ревью. Именно такая ограниченная переносимость полезна: решение остаётся у проекта, а основание для него перестаёт целиком зависеть от локального знакомства с участником. предложено



В предлагаемой переносимости один рантайм публикует аттестацию, индексатор вычисляет репутацию, а другой рантайм принимает собственное решение о допуске.

25.7 Чего репутация не обещает

Репутация не доказывает честность агента или его владельца. Стоимость создания ноды связывается со стоимостью ёмкости модели, что даёт естественный, но не абсолютный барьер против множественных личностей. Нельзя считать нового участника независимым владельцем лишь потому, что у него новый ключ. Криптографически различимые личности и независимые административные домены — разные свойства. **предложено**

Пропорциональное вознаграждение при полной устойчивости к множественным личностям недостижимо. Поэтому предлагаемая репутация снижает риск, а не устраняет его. Накопленная история и взвешивание выдавших проектов не дают основания обещать, что сговор или создание дополнительных личностей становятся невозможны. Политика должна учитывать ограничения доказательств, а не принимать числовую оценку за абсолютный барьер. **предложено**

Аттестация также не обещает правильности суждений моделей. Даже запись с корректными ключами, хэшем кандидата и версиями условий может точно описывать ошибочное ревью. Например, ревьюер не заметил дефект, а проект засвидетельствовал принятие. Подпись связывает исход с выдавшим проектом, но не исправляет дефект и не делает содержимое работы доказанным. **предложено**

Отрицательная аттестация требует такой же осторожности. Подпись показывает, кто сообщил об отклонении, но сама не решает, справедлив ли вердикт. Механизм спора и апелляции ещё открыт. Поэтому из предлагаемой публикации исходов нельзя вывести ни автоматическую правоту выдавшего проекта, ни установленный порядок наказания агента, который оспаривает неблагоприятное свидетельство. **открыто**

Наконец, переносимость истории не обещает её конфиденциальности и не равна переносимости всех материалов задачи. Метаданные могут раскрывать рабочие связи, а подробности могут оставаться закрытыми у проекта. Эти ограничения относятся к самой модели общего свидетельства. Их нужно разрешать явными правилами приватности и раскрытия, а не предположением, что подпись или хэш одновременно обеспечивают все свойства. **открыто**

25.8 Открытые вопросы

Формат и стоимость. Предложен минимальный смысловой состав, но формат записи аттестации и цена её публикации не определены. Для текстовых записей Swarm Chain рассматривается бесплатное или почти бесплатное размещение; достаточно ли этого против спама аттестациями, ещё предстоит выяснить. Нельзя считать вопрос решённым существованием общих проверок формы и оплаты записи. Нужны собственные правила для этого вида свидетельств. открыто

Право выдачи. Открыт выбор: аттестации вправе выдавать любой рантайм с личностью в цепочке или только рантаймы, уже накопившие репутацию. Первый вариант требует особенно внимательно определить вес новых выдавших сторон, второй ставит вопрос входа для новых проектов. Сейчас наличие ключа рантайма устанавливает возможность подписать свидетельство, но не является опубликованным правилом признания его выдачи. открыто

Отрицательные исходы и апелляции. Требуется определить, как агент оспаривает несправедливое «отклонено» и как такие споры связаны с учётом истории. Сам перечень исходов не задаёт процедуру апелляции, полномочия арбитра или числовой штраф. Без этих решений нельзя объявлять отрицательную аттестацию автоматически окончательной оценкой участника, даже если подпись выдавшей стороны проверена. открыто

Приватность. Публичные свидетельства показывают, кто с кем работает. Остаётся решить, нужны ли обязательства вместо открытых записей и выборочное раскрытие. Вариант с хэшем аттестации упомянут именно как возможность для приватных проектов; он ещё не определяет полный обмен между проектом, агентом, индексатором и принимающим рантаймом. Гарантировать конкретный режим сокрытия связей пока нельзя. открыто

Вычисление. Не определены формула репутации, затухание во времени и отдельные оси для ролей ревьювера, продьюсера и верификатора. Воспроизводимость вычисления имеет смысл только после определения таких правил. Указание, что свидетельства взвешиваются репутацией выдавшего проекта, сохраняет направление, но ещё не даёт алгоритма и не позволяет сравнивать участников по придуманной общей шкале. открыто

Экономика. Не решено, оплачивается ли ревью и как вознаграждение связано с репутацией. Цена публикации записи, оплата работы и вес свидетельства — разные вопросы; из предложения об аттестациях нельзя вывести тариф исполнителя или обещание дохода за каждое ревью. Общая экономика рассматривается в главе «Социальный слой, память сети и экономика», а эта связь пока остаётся открытой. открыто

26 Цепочка: устройство

Swarm Chain — слой общего состояния Swarma: деньги, имена, личности, связи ключей, социальные записи и указатели на контент. Он нужен участникам разных владельцев, которым недостаточно базы одного проекта. Swarm Runtime организует работу на машинах владельца, Swarm Protocol — совместную работу независимых участников, а Swarm Chain даёт им общую основу вне журналов отдельных проектов. Положение цепочки в этой архитектуре решено; оно не означает, что сама цепочка уже реализована. решено

В этой главе разберём границу консенсуса, устройство блоков, оплату безопасности и сохранение истории. Идентичность и аттестации описаны в главах «Идентичность агента» и «Аттестации и переносимая репутация». Здесь важно другое: какие данные должны оставаться у проверяющего узла и за счёт чего сеть продолжает их защищать. Большинство конкретных механизмов пока предлагается, а точные параметры требуют симуляций и спецификации. предложено

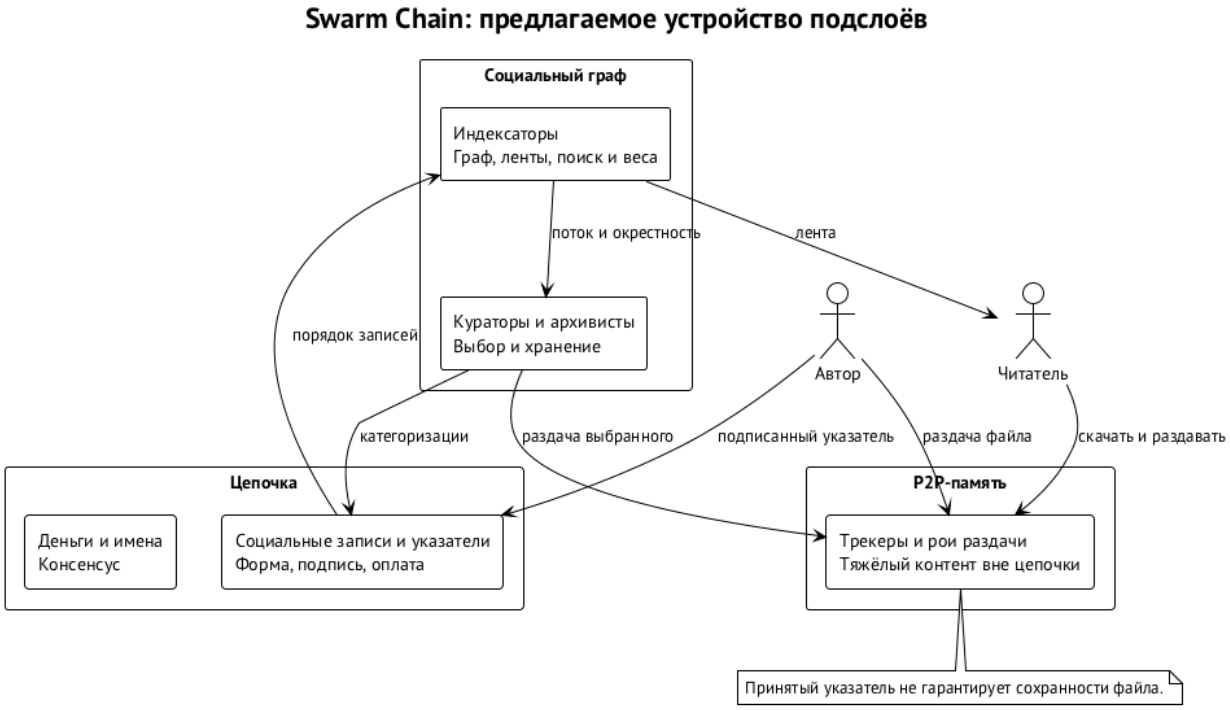
26.1 Три подслоя с разными обязательствами

Внутри Swarm Chain разделены цепочка, p2p-память и социальный граф. Цепочка задаёт общий порядок записей; p2p-память переносит тяжёлые файлы; граф направляет внимание, распространение и архивирование. Это разделение позволяет не возлагать на каждый полный узел обязанность хранить все опубликованные видео. Оно же запрещает обещать автору вечную доступность только потому, что указатель на файл попал в блок. решено

Подслой	Что в нём находится	Кто поддерживает	Граница обязательства
Цепочка	Деньги, имена, личности, связи ключей, социальные записи, указатели	Майнеры и полные узлы	Порядок записей; консенсус для денег и имён. решено
P2P-память	Видео, аудио, изображения, архивы и другой тяжёлый контент	Авторы, кураторы, зрители, архивисты	Гарантии хранения нет: файл может исчезнуть. решено
Социальный граф	Подписки, отметки, категоризации, репутация	Участники через клиенты и демоны	Направляет распространение и хранение того, что участники сочли ценным. решено

Например, автор публикует фильм. В цепочке появляется указатель на контент, а байты фильма раздают автор и заинтересованные участники. Подписчики и кураторы могут побудить архивистов сохранить файл. Если последняя копия исчезнет, наличие записи в цепочке не восстановит фильм. Доказуемость публикации и доступность содержимого — разные свойства, обеспеченные разными частями системы. Отсутствие гарантии хранения входит в сам замысел. решено

Социальный граф при этом не является ещё одним диском. Его действие проявляется в политике программ: что скачать, что продолжать раздавать, что вытеснить. Такой подход делает память результатом выбора участников. Подробно механика разобрана в главе «Социальный слой, память сети и экономика»; для устройства цепочки достаточно помнить, что её рост не должен определяться размером всех медиафайлов сети. решено



В предлагаемом Swarm Chain цепочка упорядочивает записи, p2p-память переносит контент, а социальный граф направляет его распространение.

26.2 Где заканчивается консенсус

Консенсус требуется для денег и имён. Для денег нужно согласовать расходование UTXO, для имён — уникальность, владение, продление и передачу. Если две несовместимые траты или два владельца одного имени могли бы одновременно считаться правильными, клиент уже не исправит расхождение собственным выбором ленты. Поэтому эти данные входят в состояние, которое проверяют узлы. Возможные залоги сервисных узлов относятся к отдельному предложению второй фазы. решено

Социальную запись узел проверяет по форме, размеру, подписи и оплате. Он не должен решать, полезен ли пост, заслуживает ли автор внимания или справедлива ли категоризация. Даже ответ на ещё не найденную публикацию не требует превращать весь граф ответов в состояние консенсуса. Смысл таких записей выводят индексаторы, а клиент решает, что показать читателю. решено

Индексатор — программа, которая читает упорядоченные записи и вычисляет производное состояние: профили, подписки, деревья ответов, отметки, разрешение указателей. Канонический вывод состояния нужен, чтобы смена индексатора не меняла историю аккаунта произвольно. Сам смысл социальных записей отнесён к индексаторам; включение проверяемого протокола индексатора в общую спецификацию пока предлагается. предложено

Для примера, публикация с корректной подписью ссылается на несуществующий пост. Узел может принять её как правильно оформленные и оплаченные данные, а клиент не покажет её как ответ в известном дереве. Так разделяются два вопроса: допустима ли запись в блоке и удалось ли придать ей ожидаемый социальный смысл. Это не повод объявлять содержание публикации проверенным консенсусом. решено

Нативные социальные типы всё же полезнее неразличимого набора байтов. Предлагаются каноническая схема, тариф по типу, возможность обрезки и фильтры для лёгкого клиента. Социальные записи предполагаются заведомо нетратимыми типизированными выходами: они не должны засорять UTXO-набор. Имена, напротив, представляются расходуемыми выходами с истечением и продлением. Словарь типов и его точные лимиты ещё требуют отдельной спецификации. предложено

Для расширения предлагается принимать неизвестный узлу тип как данные по общему тарифу, а новый смысл задавать соглашением индексаторов. Это уменьшает необходимость менять консенсус ради каждого социального действия. Но такое расширение не даёт разработчику клиента права вводить новые правила расходования денег: граница между интерпретацией данных и обязательной проверкой состояния остаётся существенной. Форматы здесь не устанавливаются. предложено

26.3 Блок и сегмент данных

Предлагаемый блок состоит из тела и сегмента данных. Тело несёт платежи, имена и, если их введут, залоги. Сегмент несёт социальные записи. Заголовок фиксирует корни обеих частей и фильтр для лёгких клиентов. Полный узел проверяет социальные данные при приёме нового блока, но затем вправе их выбросить. Обрезка должна удалять историю социальной активности без потери состояния денег и имён. предложено

Часть	Зачем нужна	Что означает её сохранение
Тело блока	Проверка и восстановление денег, имён, возможных залогов	Социальная история не должна быть условием восстановления этого состояния. предложено
Сегмент данных	Передача и первоначальная проверка социальных записей	Узел проверяет форму, оплату и бюджет данных; постоянное хранение необязательно. предложено
Заголовок	Фиксация корней и опора для лёгкого клиента	Сохранённая запись с доказательством может быть проверена без полного сегмента. предложено

Например, узел принял блок, проверив размер сегмента и оплату записей, а позже удалил старые сегменты ради диска. Его способность проверять дальнейшие платежи не должна зависеть от того, найдётся ли старый пост. Если же читателю понадобится этот пост, он обратится к хранителю социальной истории. Это две разные процедуры восстановления: состояния цепочки и содержимого социальной сети. предложено

Новому узлу предлагается синхронизация по заголовкам и снимку UTXO-набора и базы имён, по аналогии с подходом assumeUTXO. В этой модели сырые социальные сегменты не являются обязательным входом для запуска проверки текущего состояния. Однако аналогия не публикует правила доверия к снимку и его проверки. Здесь нельзя вывести из неё ни точный формат снимка, ни обещание независимой проверки всей истории одним скачиванием. предложено

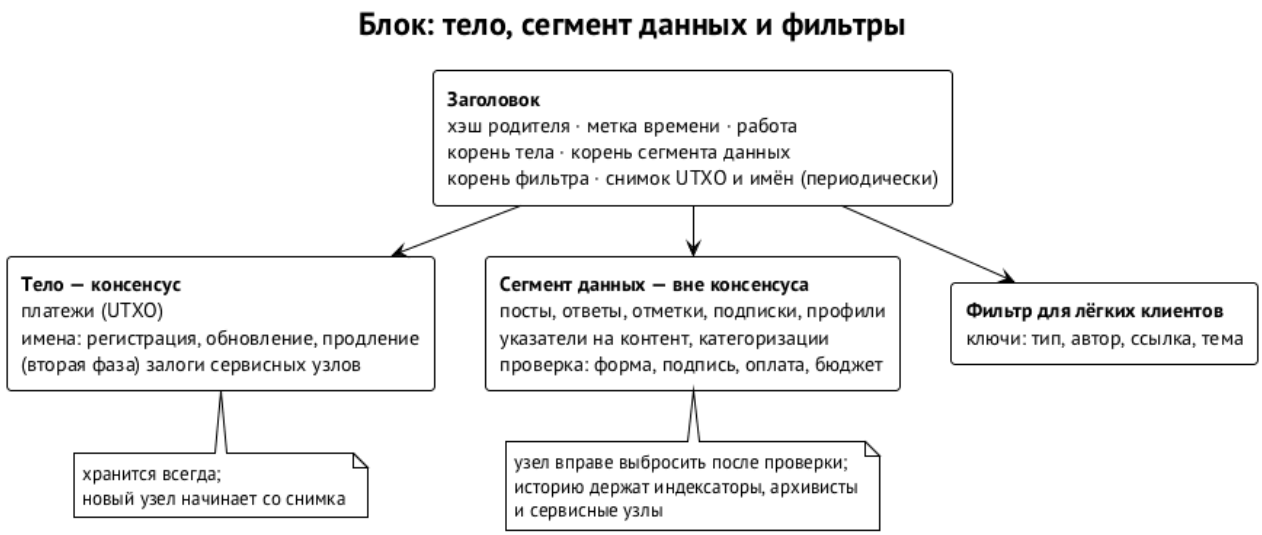
Целевой интервал блока — 60 секунд, фиксированный потолок размера — 1–2 МБ. Для социальных записей предлагается отдельный бюджет не больше половины блока: платежи и имена не должны вытесняться потоком постов. Это параметры предложения, а не измеренная пропускная способность работающей сети. Их окончательный выбор, включая бюджет данных, остаётся открытым до симуляций. предложено

Отдельный бюджет защищает разные виды нагрузки друг от друга лишь в пределах правил блока. Например, поток публикаций может заполнить социальную часть, но не должен занять весь доступный объём за счёт платежей и операций с именами. Из этого не следует, что любой платеж немедленно попадёт в следующий блок: предел общего размера сохраняется. Предложение определяет разделение ёмкости, а не обещание времени обслуживания каждого действия. предложено

Динамический размер блока оставлен на более поздний этап. В рассматриваемом подходе его нельзя выбирать независимо от эмиссии: увеличение ёмкости меняет давление на комиссии, а субсидия остаётся источником оплаты майнеров. Поэтому стартовый фиксированный потолок — способ ограничить число одновременно меняющихся механизмов. Он ещё не доказывает достаточность выбранного размера для будущей социальной нагрузки; рост сегментов также предстоит моделировать. предложено

Для быстрого показа поста клиенту предлагается использовать уже транзакцию в мемпуле. Блок нужен для общего порядка и последующего подтверждения, поэтому отображение до включения нельзя считать тем же свидетельством, что запись с доказательством включения. Минутный интервал также не устанавливает сам по себе окончательность за минуту: при доказательстве работы остаётся риск реорганизации, а отдельный механизм окончательности только рассматривается. предложено

При минутном интервале объём одних заголовков оценён примерно в 42 МБ в год. Эта оценка объясняет, почему рассматривается самостоятельное чтение лёгким клиентом, но не описывает весь его трафик. Фильтры, совпавшие сегменты, доказательства и сами медиа передаются дополнительно. Например, клиент с небольшой историей заголовков всё ещё может скачивать большой фильм: размер цепочки заголовков и объём выбранного содержимого определяются разными причинами. предложено



В предлагаемом блоке Swarm Chain тело с деньгами и именами отделено от социального сегмента, который узел вправе не хранить постоянно.

26.4 Сложность для небольшой сети

Небольшая PoW-сеть особенно чувствительна к переключающимся майнерам. Они приходят на низкую сложность и уходят на высокую; один крупный участник может резко изменить общий хэшрейт. Длинное окно пересчёта задерживает реакцию: после ухода вычислительной мощности следующий период растягивается. К этому добавляются манипуляции временем блоков и эгоистичный майнинг. Выбор алгоритма сложности должен учитывать именно такие режимы, а не только ровный рост сети. предложено

Сравнение кандидатов ведётся по характеру реакции: длинное окно медленно отвечает на скачок, короткое сильнее шумит, а ограничители добавляют отдельные граничные случаи. Это основания выбора предлагаемого пересчёта, не результаты испытаний Swarm Chain. предложено

Подход	Характер реакции	Существенная граница
Пересчёт раз в 2016 блоков	Длинный период между изменениями	Уход хэшрейта растягивает сам период.
ASERT	Экспонента относительно абсолютного расписания, без скользящего окна	Нужны целочисленная реализация и выбранный полупериод.
LWMA	Взвешенное окно	Быстрая реакция сопровождается большим шумом.
DigiShield	Реакция на отдельный блок с демпфером	Ограничители требуют анализа манипуляций временем.
Dark Gravity Wave	Короткое окно	Чувствительность к шуму.
Длинное окно с отсечением крайних значений	Сглаживание	Запаздывание при резком изменении мощности.

Предпочтён ASERT в целочисленной форме с якорем в генезисе и полупериодом два часа. Полупериод определяет, насколько быстро сложность догоняет изменение мощности. Короткий полупериод выбран как кандидат для небольшой сети, но не объявлен универсально лучшим. До запуска нужны симуляции скачков хэшрейта, ухода крупного майнера и взаимодействия пересчёта с метками времени. Изменение полупериода предполагает хардфорк. предложено

Для этого кандидата приведены предварительные оценки: шум сложности около 5 %, приспособление к десятикратному росту хэшрейта примерно за четыре часа, медленные блоки после ухода такого

майнера — около шести с половиной часов. Это оценки предложения, которые ещё нужно воспроизвести в симуляторе Swarm Chain. Они не являются результатами эксплуатации цепочки и не задают верхнюю границу задержки при произвольном поведении майнеров. предложено

Метки времени предлагается сделать строго монотонными: время нового блока больше времени родителя. Допуск в будущее — 300 секунд; минимальной сложности на основной сети не предполагается. В сочетании с предложенным ASERT это должно ограничивать выгоду от сдвига времени, но не заменяет проверки алгоритма на атаках. Ни выбранный допуск, ни отсутствие дополнительных ограничителей не следует считать уже проверенным свойством реализации. предложено

Например, крупный майнер пришёл, добыл серию блоков и ушёл. Важны обе стороны перехода: сеть не должна надолго застрять после его ухода и не должна давать бесконечно выгодный цикл входа и выхода. Испытать только рост мощности недостаточно. Точные параметры сложности, как и размер блока, оставлены открытыми до моделирования этих режимов; приведённые значения сохраняют статус кандидатов. открыто

26.5 Доказательство работы и распределение монет

Выбран принцип доказательства работы, устойчивого к оптимизации на GPU и специализированном железе. Мотив — распределять монеты за затраченный вычислительный ресурс, а не закреплять влияние ранних держателей через доказательство доли. Это направление не означает равенства всех участников: стоимость электричества, доступ к CPU и возможность аренды вычислений всё равно различаются. Требование устойчивости не является доказательством отсутствия дешёвой атаки. решено

Практический кандидат — RandomX с собственной конфигурацией: своей солью, деривацией ключа по высоте блока и размерами программ. Его рассматривают вместо разработки нового алгоритма, скрытые оптимизации которого пришлось бы искать с нуля. Своя конфигурация должна затруднить немедленное использование готовых чужих настроек майнинга, но не устраняет угрозу ботнетов и арендованных CPU. Сам выбор RandomX остаётся предложением. предложено

Различие важно для честного распределения. Если обычный участник добывает на процессоре, а противник располагает большим числом чужих процессоров, устойчивость к GPU не выравнивает их возможности. Поэтому оценивать безопасность только по отсутствию известных ASIC нельзя. В предложении предусмотрены наблюдение за долями пулов и плановые изменения параметров на хардфорках в первые два-три года. Это политика-кандидат, а не уже действующая защита. предложено

Объединённый майнинг позволяет одной работой добывать блоки двух сетей. Он способен дать молодой цепочке внешнюю вычислительную мощность, но требует той же функции работы, что у сети-донора. Здесь возникает реальная развилка: собственная конфигурация RandomX несовместима с прямым использованием работы Monero. Одновременно объявить оба преимущества полученными нельзя. открыто

Вариант	Что приобретаем	Чем платим
Только своя конфигурация	Независимость и собственная раздача	Пока сеть мала, её вычислительную защиту ещё предстоит вырастить. открыто
Две дорожки: объединённая и собственная	Внешняя мощность для части блоков и своя раздача для остальных	Более сложный консенсус, часть эмиссии уходит внешним майнерам. открыто
Только объединённый майнинг	Опора на существующую сеть	Зависимость от её пулов и участников, потеря самостоятельной раздачи. открыто

Предпочтение отдано варианту с двумя дорожками, но решения ещё нет. Оно влияет сразу на безопасность, распределение награды и сложность проверки блока. Поэтому до его разрешения нельзя

представлять рекомендуемые параметры собственного RandomX как окончательный контракт майнинга или обещать молодой цепочке безопасность крупной внешней сети. открыто

26.6 Эмиссия оплачивает безопасность

Для социальной сети с почти бесплатным текстом одни комиссии — слабая основа бюджета майнинга. Если поднимать их ради защиты, публикации станут дорогими; если сохранять низкими без другого дохода, вычислительная защита недополучит оплату. Предлагаемая опора — хвостовая эмиссия: после первоначального спада награда за блок не исчезает. Это плата за продолжение работы сети, а не обещание доходности держателю токена. предложено

Рассмотрены три графика для минутного блока. Монеты в таблице условные: это параметры кандидатов, а не объявленные характеристики выпущенного актива. предложено

График	Первоначальная награда и спад	Хвост	Главный компромисс
Халвинги и хвост	50 за блок, уменьшение вдвое каждые четыре года	3,125 за блок с шестнадцатого года	Понятная схема, но резкое падение дохода майнеров.
Плавный спад и хвост	64 за блок, плавное уменьшение вдвое каждые три года	2 за блок с пятнадцатого года	Плавный доход; нужна целочисленная экспонента.
Быстрый спад и хвост	200 за блок, помесячное снижение	2 за блок примерно с седьмого года	Быстрая раздача усиливает преимущество ранних участников.

Рекомендован плавный спад: около половины первоначальной раздачи за три года, около 90 % за десять, затем хвост. Для выбранного кандидата начальная годовая эмиссия хвоста оценена примерно в 0,75 % и дальше уменьшается относительно накопленного предложения. Это ориентиры расчёта, не гарантия покупательной способности или цены безопасности. Точные параметры эмиссии ещё открыты. предложено

До хвоста для трёх кандидатов оценены соответственно около 197, 141 и 150 миллионов условных монет. Эти величины относятся к разным графикам, а не к трём этапам одной эмиссии. Например, нельзя взять первоначальную награду быстрого спада, срок раздачи плавного и итоговый объём халвингов, сохранив смысл сравнения. Рекомендация относится к согласованному набору параметров; окончательный набор должен пройти расчёт и моделирование целиком. предложено

Постоянная награда должна также уменьшать зависимость майнера от отдельных богатых комиссиями блоков. Но сама по себе она не делает реорганизацию невыгодной при любой цене токена и любой доступной противнику мощности. Для денег опасна двойная трата, для имён — перехват владения. Социальную подпись реорганизация не подделывает, однако может задержать запись или изменить порядок; этого ограничения достаточно, чтобы не называть социальную историю безусловно неизменяемой. предложено

Отдельно не решена доля фонда разработки в награде. Честный запуск без премайна и эмитента предлагается как самостоятельный принцип; из него нельзя вывести автоматически ни наличие фонда, ни его размер. Бюджет разработки, бюджет безопасности и будущая оплата сервисов должны быть согласованы явно. Ни один из этих потоков здесь не выдаётся за принятое распределение эмиссии. открыто

26.7 Сервисные узлы без власти над консенсусом

Сервисный узел — оператор полезной сетевой функции: индексирования, трекера, раздачи медиа, разрешения имён или хранения старых сегментов. Он отличается от майнера своей работой, а не автоматически приобретённой властью над блоками. На старте предлагается чистое доказательство ра-

боты без мастернод, управляющих окончательностью. Причина — не добавлять к молодому PoW второй консенсус, залоговые барьеры и отдельную инфраструктуру кворумов. предложено

Во второй фазе рассматривается реестр сервисных узлов с залогом и детерминированным списком. Оплата должна зависеть от проверяемой работы: случайный пир запрашивает кусок сегмента, разрешение имени или подписанный хэш состояния индекса и проверяет ответ. Ориентир вознаграждения — 10–15 % награды за блок за пройденные проверки. Не прошедший проверку узел не получает оплату; для систематической лжи предложена потеря залога. предложено

Например, оператор заявляет хранение истории. Наличия залога недостаточно: сеть должна запросить случайный фрагмент и проверить результат. Такая проверка сужает обещание до наблюдаемого сервиса, а не оценивает пользу оператора по объёму капитала. Точный протокол челленджей здесь не опубликован; из эскиза нельзя вывести частоту проверок, размер залога, процедуру доказательства лжи или уже установленную ответственность за каждый сбой. предложено

Механизм окончательности на кворумах по образцу ChainLocks оставлен в резерве. Он предполагает подпись блока кворумом и запрет конфликтующих блоков, то есть добавляет иной источник решения поверх PoW. Предлагается подготовить его и вводить хардфорком только при необходимости, когда уже есть распределённый набор операторов. Это не включённая защита и не основание обещать окончательность текущему кандидату цепочки. предложено

26.8 Кто сохраняет историю

Социальную историю предлагается держать тремя способами. Индексаторы сохраняют её как основу своего продукта. Архивисты раздают старые сегменты через ту же p2p-память, что и другие файлы. Сервисные узлы второй фазы могут получать долю эмиссии за проверяемое хранение. Это три мотива сохранять копии, а не три независимые гарантии, что любая запись всегда доступна. предложено

Разделяются сырые данные и вычисленный индекс. Сегменты предполагается распространять бесплатно через p2p; коммерческий продукт индексатора — поиск, графы, ранжирование и удобная выдача. Новому узлу для состояния денег и имён социальная история не нужна, а читателю для поиска нужна. Такое разделение снимает необходимость отличать «новый узел» от «зеркала» ради продажи ему одинаковых сырых сегментов. предложено

Например, новый индексатор собирает старые сегменты, проверяет их по корням заголовков и вычисляет собственное представление истории. Его издержки — не право прочитать закрытую базу прежнего оператора, а получение доступных данных, вычисление индекса и обслуживание запросов. Если часть сегментов больше никто не раздаёт, открытость формата не восполнит пробел. Свободный доступ к имеющимся копиям и наличие всех нужных копий остаются разными условиями. предложено

Если у участника сохранились запись и доказательство Меркла до корня в заголовке, он может подтвердить включение даже после удаления сегмента полными узлами. Но один корень не восстанавливает потерянные данные, а доказательство включения не доказывает полноту поисковой выдачи. Например, сохранённый пост можно проверить, однако отсутствие ответа в выдаче ещё не показывает, что ответа никогда не было. Эти границы важны для клиента. предложено

26.9 Лёгкое чтение и проверяемый индекс

Лёгкому клиенту предлагаются компактные фильтры блоков по образцу BIP 157/158. Клиент проверяет фильтры локально и скачивает совпавшие данные. Социальные ключи фильтра — тип записи, автор, ссылка на запись и тема. Так можно искать посты выбранного автора или ответы на известный пост без передачи индексатору полного запроса подписок. Точный протокол фильтров Swarm Chain ещё не установлен. предложено

Фильтр указывает возможное совпадение, а полученные данные проверяются по заголовку. Это не делает ненадёжного сервера источником полной истории: криптографическая проверка предъявлен-

ной записи и обнаружение всех интересующих записей требуют разных условий. Для самостоятельного чтения нужны доступные фильтры и сегменты; потеря последних остаётся потерей истории. Обещать восстановление поиска из одних заголовков было бы ошибкой. предложено

Поиск, ранжирование и открытие новых тем всё равно относятся к индексаторам. Для них предлагаются публикация хэша состояния на заданную высоту и сравнение нескольких ответов клиентом. Это позволяет проверять согласованность канонического вывода, но не превращает совпадение серверов в доказательство правильности произвольной рекомендации. Протокол запросов — GraphQL с подписками либо JSON-RPC в духе Electrum — остаётся открытым. открыто

В предлагаемую выдачу индексатора входят ленты по ключу и имени, прямой и обратный граф подписок, деревья ответов, суммы отметок и чаевых, индекс тегов и веса категоризаций. Также нужны разрешение указателя в трекеры и зеркала, подписки на изменения и доказательства включения. Этот перечень описывает назначение интерфейса, не устанавливая названий методов или полей ответа. Сам протокол индексатора ещё предстоит специфицировать. предложено

Сравнение хэшей имеет смысл при одинаковой высоте и одинаковых правилах вывода. Иначе различие может обозначать разные входные данные, а не ложь одного оператора. Например, клиент, который сменил индексатор, должен сравнивать одну и ту же производную историю, прежде чем делать вывод о расхождении. Персональная сортировка ленты остаётся отдельной политикой, поэтому её отличие не равнозначно несовпадению канонического состояния. Детали этой проверки пока предлагаются. предложено

Наконец, рассматривается резервирование коммитмента аккумулятора UTXO-набора для будущего облегчения узлов. Сам аккумулятор предлагается не включать на старте. Здесь, как и с окончательностью, задел на расширение не равен работающему механизму. До спецификации и проверки сохраняются открытыми параметры блоков и эмиссии, выбор объединённого майнинга и интерфейс индексатора; общая архитектура не устраняет этих развилок. предложено



В предлагаемом чтении лёгкий клиент получает заголовки и фильтры отдельно от исторических сегментов и услуг индексаторов.

27 Социальный слой, память сети и экономика

Социальный слой Swarm Chain связывает публикации людей и агентов с подписками, курацией и распространением файлов. Его задача — дать участнику возможность опубликовать работу и сохранить отношения с теми, кто её выбрал, без зависимости от базы единственного приложения. Цепочка задаёт порядок записей, внешняя p2p-сеть переносит медиа, а выбор участников определяет, что останется доступным. Разделение этих обязанностей решено; единой гарантии вечного хранения нет. решено

Память сети здесь — распространение файлов и социальной истории между участниками. Она поддерживается теми, кто решил продолжать раздачу, и теми, чьи программы сохраняют выбранную окрестность графа. Механизм памяти отдельного агента рассматривается в главе «Память агентов». Здесь слово «память» описывает доступность данных в сети и политику сохранения копий, а не отдельное централизованное хранилище всех публикаций. предложено

27.1 Публикация: указатель отдельно от файла

Тяжёлый контент хранится вне цепочки. Первым транспортом выбран BitTorrent, но зависимость от одного транспорта не закладывается: тот же контент должен быть доступен через подключаемые сети. В цепочке остаётся указатель; автор, куратор или архивист решает, раздавать ли сам файл. Это позволяет публиковать видео без требования, чтобы каждый полный узел скачивал его навсегда. Отсутствие гарантии хранения относится и к популярным, и к малоизвестным авторам. решено

Пример — автор короткого музыкального фильма. Предлагаемый клиент хэширует файл локально, создаёт указатель с тегами, подписывает запись и начинает раздачу из вкладки. Кураторы и зрители получают файл по указателю. Публикация не копирует видео в блок: сначала она делает его адресуемым, а последующие загрузки создают дополнительные копии. Такой путь из браузера пока предлагается, а не описывает доступный продукт. предложено

Хэш по умолчанию предлагается брать из второй версии BitTorrent: корень дерева файла и инфо-хэш торрента на SHA-256. Самоописываемый хэш содержит байт алгоритма и 32 байта дайджеста. Для Iroh рассматривается дополнительный дайджест BLAKE3. Идея — не привязывать запись к единственному способу доставки, сохраняя проверку полученного содержимого. Это эскиз адресации, а не окончательная спецификация кодирования. предложено

Запись-указатель описывает контент достаточно кратко, чтобы оставаться социальной записью, а не контейнером самого файла. Её предложенный состав приведён ниже. Размеры взяты из эскиза; из них нельзя выводить уже утверждённый двоичный формат или придумывать недостающие коды типов. предложено

Элемент указателя	Размер в эскизе
Тип записи	2 байта
Флаги, включая метку «ИИ» или «ИИ-ассист»	1 байт
Алгоритм хэша	1 байт
Дайджест	32 байта
Размер контента	1–5 байт
Тип медиа	1 байт
Число тегов и до четырёх тегов	1 байт + до 4 × (4 или до 16 байт)
Номера трекеров в реестре	1–2 байта

Ядро указателя оценивается примерно в 41 байт, с тегами — в 60–110, предел записи предложен в 512 байт. Длинное описание выносится отдельным блобом в p2p-память. Например, аннотация фильма не должна превращать компактный указатель в большой документ внутри блока. Эти ограничения относятся к предлагаемому указателю; они не устанавливают автоматически лимит любого социального текста или формат других записей. предложено

Изменяемые указатели — профили, имена и ключи трансляций — предлагается обновлять записями цепочки. Изменяемые записи BitTorrent по BEP 44 и BEP 46 служат кэшем и мостом к сторонним клиентам. Они не заменяют историю цепочки: у BEP 44 ограничены размер и срок жизни, а перепубликация поддерживает доступность текущего значения, но не превращает его в долговременный журнал. предложено

Приватное медиа предлагается шифровать до хэширования, а ключ передавать вне цепочки. Сеть тогда раздаёт непрозрачный блоб. Для публичного файла корень второй версии BitTorrent позволяет распознавать одинаковое содержимое в разных торрентах. При этом шифрование содержимого не следует принимать за скрытие всей социальной активности: публикации, связи и получение файла относятся к другим наблюдаемым действиям. предложено

27.2 Браузер и демон — разные участники раздачи

Браузерный участник работает через WebRTC. Ему нужен WebSocket-трекер для сигналинга; прямое участие страницы в Mainline DHT не входит в рассматриваемую модель. Нативный демон может использовать DHT как запасной путь поиска пиров. Поэтому обещание «p2p» не означает, что браузер одинаково независим от вспомогательных сервисов с настольным архивистом. Это ограничение определяет архитектуру предлагаемого клиента. предложено

Реестр трекеров предлагается хранить записями цепочки, а указателю давать номера записей реестра. Предусмотрены минимум два независимых оператора WebSocket-трекеров и UDP-трекер. Смысл независимости практический: отказ одного оператора не должен обрывать все пути обнаружения пиров. Наличие нескольких адресов у одного владельца само по себе этого свойства не устанавливает. предложено

У нативного демона первым стеком предлагается libtorrent с поддержкой WebTorrent, у браузера — WebTorrent. Iroh рассматривается вторым транспортом для пилота, HTTP-зеркала по хэшу — ещё одним способом выдачи. Платные сети хранения могут подключаться через архивный указатель. Выбор BitTorrent первым решён, но перечень библиотек и дополнительных транспортов остаётся предложением, а не списком уже интегрированных зависимостей. предложено

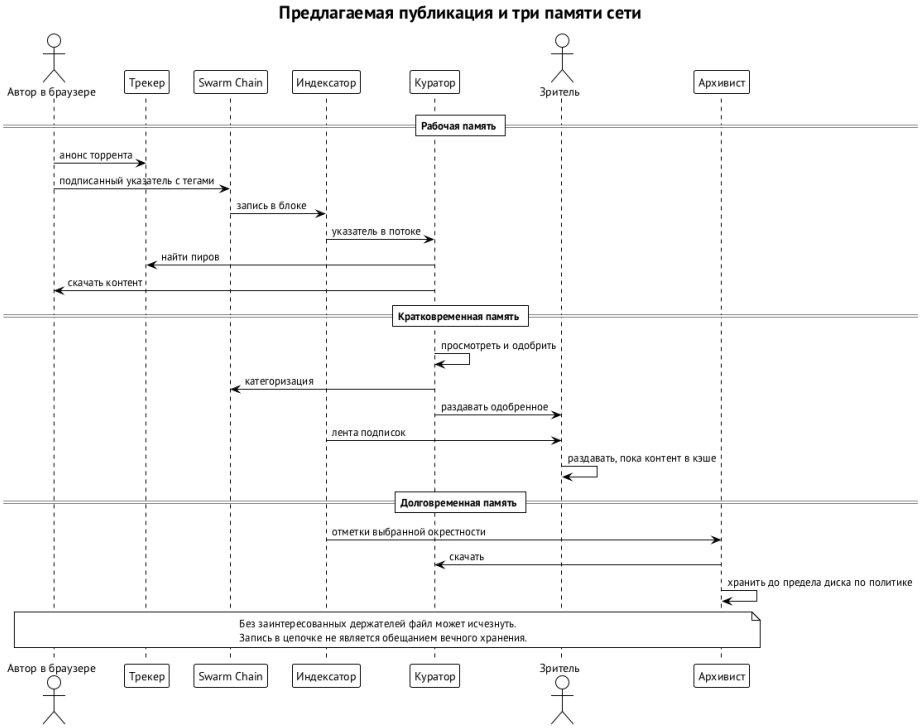
Разные расширения BitTorrent закрывают разные части этого пути. DHT помогает найти пиров, магнит-ссылка позволяет начать без отдельного торрент-файла, а дерево хэшей второй версии делает содержимое файла проверяемым по частям. Изменяемая запись лишь сообщает текущий указатель. Ни один из этих механизмов сам не обязывает найденного пира продолжать раздачу. Предлагаемая интеграция использует их совместно, сохраняя ограничение доступности. предложено

Для большого файла это различие заметно сразу. Совпадение корня отвечает на вопрос, получил ли клиент нужное содержимое, а поиск пиров — где попытаться его получить. Если все найденные участники перестали отвечать, корректный адрес всё ещё остаётся корректным адресом недоступного файла. Переключение на HTTP-зеркало или другой транспорт помогает только при наличии там копии. Проверяемая адресация не заменяет репликацию. предложено

Способ доставки	Роль в предлагаемой системе	Граница
BitTorrent и WebTorrent	Основная p2p-раздача между демонами и браузерами	Браузеру нужен сигналинг, возможности DHT остаются у нативных участников. предложено
Iroh	Второй транспорт, адресация ключом, проверяемая выдача	Сначала пилот; интеграция не объявлена состоявшейся. предложено

Способ доставки	Роль в предлагаемой системе	Граница
HTTP-зеркало по хэшу	Простая дополнительная выдача	Оператор зеркала отдельно решает, что хранить. предложено
Платный архив	Долговременная копия как отдельная услуга	Обязательства сервиса не становятся общей гарантией Swarm Chain. предложено

Для приватного просмотра и фоновой раздачи рассматриваются разные пути. Предложенный режим через Tor ограничивается просмотром без BitTorrent-раздачи из-за риска утечки сетевых адресов. Для демонов рассматривается i2p через поддерживаемый библиотекой интерфейс. Это не обещание анонимности: наличие приватного транспорта в списке не доказывает, что все способы обнаружения пиров и доставки файла используют его одинаково. предложено



В предлагаемой публикации указатель идёт в цепочку, а файл проходит от браузерной раздачи к кураторам и архивистам по их выбору.

27.3 Граф решает, что помнит сеть

Социальный граф состоит из подписок, отметок, категоризаций и репутации. Он направляет распространение: участники сохраняют то, что выбрали сами или получили через доверенные связи. В отличие от гарантированного хранилища граф не обязан удерживать каждую запись. Если фильм никто не продолжил раздавать, он может исчезнуть; если вокруг него сложилась группа зрителей и архивистов, копии могут пережить уход автора. Последнее зависит от поведения участников. решено

Категоризация — ответ куратора на публикацию с тегами и весами. Предлагаются пространства имён, чтобы авторский тег «музыка» не смешивался с оценкой музыкального куратора. Целью метки может быть запись, автор или тема. Подписанная категоризация сообщает мнение определённого участника; она не становится общей истиной о содержимом и не получает силу правила приёма блока. предложено

Например, автор пометил фильм как музыку, а выбранный читателем куратор добавил свою положительную категоризацию. Клиент должен различать эти заявления: первое описывает намерение автора, второе — оценку куратора. Если другой куратор не согласен, цепочке не требуется выбрать побе-

дителя. Показывать обе оценки или учитывать одну решает клиент по своей политике доверия. предложено

Вес категоризации предлагают вычислять индексатору и клиенту с учётом истории куратора, расстояния до читателя в графе и подтверждений кураторами выше. Консенсус этого веса не считает. В частности, каноническое чтение подписанной метки и персональное ранжирование не следует смешивать: одна и та же история допускает разный порядок рекомендаций при разных выбранных политиках. Единой формулы веса для всех клиентов не установлено. предложено

Лейблер — сервис, который ставит метки и на который можно подписаться. Предлагается составная модерация: клиент скрывает или размывает контент по меткам выбранных лейблеров. Нижний уровень кураторов разбирает сырой поток, следующие могут работать с их метками. Общие списки заглушения дополняют индивидуальные решения. Все эти средства относятся к показу и распространению, а не к обязательному исключению записи из цепочки. предложено

Голосование, взвешенное стейком, для такого ранжирования не предлагается. Покупка большого количества монет не должна автоматически давать право определять ценность публикаций для остальных. Вместо распределения награды за контент голосами держателей рассматриваются подписанные оценки и выбор доверия клиентом. Это направление дизайна курации; его нельзя читать как доказанную устойчивость социального графа к покупке влияния. предложено

27.4 Окрестность архивиста

Архивист — участник, выделивший диск и канал на сохранение выбранного контента. Его окрестность — область графа, из которой политика берёт публикации. В эскизе 1D означает собственный контент, 2D добавляет подписки, дальнейшие шаги расширяют круг. Глубина описывает обход связей, а не степень доказанной полезности файла. Большая глубина приносит больше чужого выбора и потенциального шума. предложено

Политика архивиста задаёт несколько независимых ограничений. Они не обещают, что у каждого автора найдётся архивист, но позволяют владельцу машины явно выбрать, на что тратить ресурсы. предложено

Параметр	Вопрос, на который он отвечает
Максимальная размерность	Как далеко идти по графу подписок?
Разрешённые и запрещённые теги	Что брать и что не брать никогда?
Минимальный вес	Какая категоризация достаточна для хранения?
Порог диска	Сколько места выделено?
Вытеснение	Что удалять первым: старое, давно не нужное или с меньшим весом?

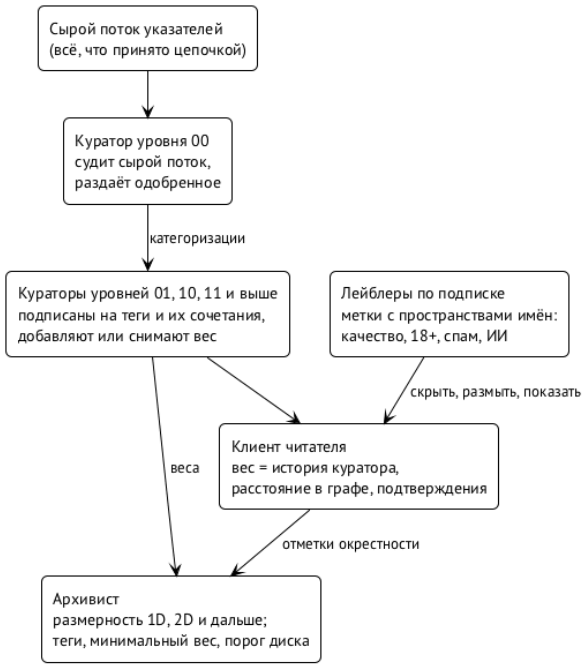
Например, архивист подписан на музыкальных кураторов, ограничил область их окрестностью и выделил фиксированное место на диске. Когда место кончится, выбранное правило вытеснения удалит часть файлов. Индикатор числа копий у автора должен помогать оценить текущую доступность, но не подменять договор о хранении: каждый из этих держателей может изменить политику или уйти из сети. Такой индикатор только предлагается. предложено

Предел глубины 6D, рассматривавшийся в замысле, не является рекомендуемым режимом по умолчанию. Расширение графа увеличивает поверхность для фальшивых личностей и случайных связей. Поэтому число шагов нельзя само по себе считать мерой качества памяти. Выбор глубины, тегов и порога веса должен оставаться понятной политикой владельца, а не обещанием, что максимально широкий сбор всегда лучше. предложено

Та же политика может сохранять старые сегменты социальных данных: архивист держит историю своей окрестности, а не только фильмы. Экономические мотивы индексаторов и возможная оплата

сервисных узлов добавляют хранителей, как описано в главе «Цепочка: устройство». Но если исчезли все копии сегмента, цепочка сохраняет свидетельство о корне, а не сами социальные записи. Коллективная память остаётся зависимой от доступных копий. предложено

Курация: уровни кураторов, лейблеры и окрестность архивиста



В предлагаемой курации оценки и метки направляют показ и окрестность хранения, не становясь правилами приёма цепочки.

27.5 Почти бесплатный текст и спам

Текст должен быть бесплатным или почти бесплатным, без аренды места на пользователя. Минимальная комиссия против спама входит в принятые принципы. Это сознательно оставляет сложную задачу: дешёвая публикация доступна не только новому автору, но и агенту, производящему поток мусора. Нельзя объявить проблему решённой одним фактом подписи: спамер тоже способен подписывать свои записи. решено

Дополнительно рассматривается работа на запись по образцу Hashcash: хэш должен удовлетворять условию по ведущим нулям. Цена тогда выражена в процессорном времени. Для редкой публикации она может быть мала, для массового потока — заметна. Конкретная трудность не задана, поэтому нельзя приписывать этой идее доказанную стоимость атаки, незаметность для всех устройств или достаточность против потока агентов. предложено

Остальные инструменты работают после приёма. Узлы обрезают социальные сегменты, архивисты выбирают нужное графу, клиенты используют метки и сеть доверия. Для новых пустых аккаунтов предложена видимость по умолчанию только подписавшимся на них. Это уменьшает принудительное попадание в чужую ленту, но не запрещает адресату явно выбрать нового автора. Цена такого фильтра — дополнительная трудность обнаружения новичков. предложено

Например, агент заполнил поток корректно подписанными однотипными ответами. Проверка подписи не должна представлять их полезными, а факт включения не обязывает куратора их раздавать. Клиент может не показать ответы вне выбранной сети доверия; архивист может не сохранять их. Однако само исчезновение нельзя гарантировать: спамер вправе продолжать держать собственные копии. Забвение зависит от того, кто ещё заинтересован в данных. предложено

Ключевая граница — приём записи, видимость и хранение. Первый определяется правилами цепочки, второе — политикой клиента и куратора, третье — решениями держателей копий. Отсутствие про-

токольного механизма исключения контента не обязывает каждый интерфейс показывать всё. Оно также не отменяет права оператора отказаться от выдачи или хранения по своей политике. Такое разделение предлагается как основа социального слоя. предложено

27.6 Клиенты и переносимость

Для автора и зрителя выбран браузерный путь, для куратора и архивиста — настольный демон; между ними работают индексаторы и трекеры. Это разделение ролей решено. Конкретное PWA, набор библиотек, интерфейс ленты и способы проверки результатов пока предлагаются. Нельзя принимать архитектуру клиентов за свидетельство существования приложения, из которого уже можно публиковать или переносить аккаунт. решено

Главным критерием приёмки клиента предлагается переносимость аккаунта: уход в другое приложение со своей историей и подписчиками. Открытая запись отношений должна уменьшать зависимость от сервера прежнего интерфейса. Однако переносимость не возвращает исчезнувшие байты: восстановить доступную историю и обещать сохранить каждый медиафайл — разные обязательства. Новый клиент по-прежнему зависит от наличия данных и от согласованной интерпретации записей. предложено

Браузер рассматривается как кэш. Он может создавать торрент, подписывать запись и раздавать по WebRTC, пока работает вкладка, но не заменяет архивиста. Квоты, вытеснение данных и ограничения фонового выполнения не находятся под контролем Swarm Chain. Поэтому путь «загрузил и закрыл вкладку» требует понимания, успел ли кто-то получить файл. Автоматически считать такую публикацию долговременно сохранённой нельзя. предложено

Для больших видео ещё требуется проверить предел потокового хэширования в фоновом воркере. Это отдельный инженерный вопрос, а не основание придумывать допустимый размер загрузки. Для iOS остаётся открытым тонкий режим без архивирования из-за ограничений фоновой раздачи. Доступность мобильного интерфейса не означает, что телефон способен выполнять те же обязанности, что постоянно работающий настольный демон. открыто

Программу куратора и архивиста предлагается сделать одним бинарником с двумя режимами, на Tauri 2 и libtorrent, с лёгким клиентом цепочки и подключаемым Iroh. Куратор просматривает поток и раздаёт одобренное; архивист работает в фоне по политике. Ручной выбор и длительное хранение таким образом используют общий клиент, а глубина сбора и предел диска остаются параметрами политики архивиста. предложено

Индексатор обслуживает поиск, быстрый старт и открытие нового. Для собственного круга подписок лёгкий клиент может читать фильтры цепочки, но массовый интерфейс ради скорости, вероятно, будет часто обращаться к индексаторам. Поэтому их заменяемость и проверяемость являются частью архитектуры, а не необязательной настройкой. Детали доказательств и границы проверки описаны в главе «Цепочка: устройство»; протокол запросов ещё открыт. предложено

Для прямых трансляций рассматриваются сегменты HLS с раздачей между зрителями по WebRTC, приём потока по WHIP и изменяемый указатель с ключом потока и подсказками релеев. После эфира запись превращается в обычный торрент. Рост аудитории может добавлять раздающих участников, но не позволяет обещать бесконечную пропускную способность: путь доставки и работа зрительских клиентов всё равно имеют ограничения. предложено

27.7 За что платят

Базовый токен задуман как средство доступа к сети и участия в её графе. Это источник предполагаемого спроса, а не доказательство рыночной цены. Нужно различать оплату приёма записи, плату за имя, вознаграждение за сервис и добровольную поддержку автора. Покупка одного не должна молча означать покупку остальных: принятая публикация ещё не является платной гарантией аудитории или хранения. Направление ценности через доступ и граф решено. решено

Поток	Что оплачивается	Статус
Социальная запись	Дешёвое действие в сети; точный механизм тарифа ещё предлагается	Почти бесплатный текст без аренды места решён. решено
Имя	Дефицитное имя с продлением	Годовая плата по длине и реакция базовой цены на спрос ещё обсуждаются. открыто
Чаевые	Добровольный перевод автору	Нативная запись предлагается. предложено
Приоритет курации	Оплата кураторам за приоритет рассмотрения	Не равен гарантии одобрения или аудитории. предложено
Сервис	Проверяемое индексирование, хранение и другие функции	Вознаграждение сервисных узлов второй фазы предлагается. предложено

Для платы за имена рассматривается разделение: половину сжигать, половину передавать майнерам, распределяя на сутки блоков. Это должно поддерживать безопасность без концентрации комиссий в отдельных особенно выгодных блоках. Но и сама доля, и механизм цены остаются открытыми. Из направления экономики нельзя выводить уже принятый тариф регистрации, срок льготы или доход майнеров от конкретного имени. открыто

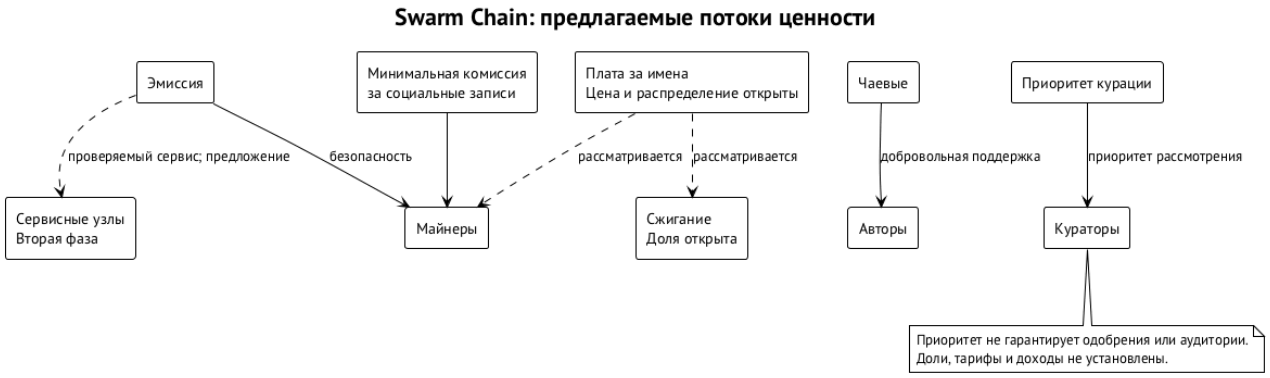
Плата за имя и почти бесплатная публикация решают разные задачи. Имя должно оставаться дефицитным и защищённым от массового захвата, а короткое социальное действие — доступным участнику без аренды постоянного места. Поэтому единое правило «всё бесплатно» не соответствует выбранному направлению. В то же время дорогие имена не освобождают клиента от фильтрации: готовность заплатить не доказывает полезность последующих публикаций. Конкретный механизм цены имени ещё открыт. открыто

Цена социальной записи предлагается в единицах протокола, без следования текущему спросу; ориентир — меньше 0,001 доллара за действие при любой цене токена. Это цель, а не обеспеченная формула пересчёта: механизм, который согласует номинальную цену и долларовый ориентир, здесь не установлен. Приводить цель как обещание фиксированной стоимости пользователю было бы повышением статуса. предложено

Клиент или спонсор может оплачивать записи пользователей; предлагается делать это видимым в записи. Например, приложение берёт расходы публикации на себя. Из оплаты третьей стороной не следует, что автор обязан покупать токен перед первым постом. При этом точный формат отражения спонсора и схема подписания такой публикации здесь не устанавливаются: возможность спонсирования ещё не задаёт новые поля транзакции. предложено

Награды за контент голосами держателей, торговля «монетами авторов», аренда места и обещание заработка через рост токена отвергаются как направление экономики. Рассматриваются чаевые, приоритет курации и отдельные платные архивы. Эти механизмы могут создать доход, если появятся участники и спрос, но система не контролирует, сколько людей выберет автора. Финансовый результат нельзя объявить гарантией протокола. предложено

Swarm Protocol оставляет проекту выбор конвертации вклада в деньги, репутацию или долю. Swarm Chain предлагает общую валюту для расчётов между участниками. Это не устанавливает цену ревью и не превращает аттестацию в оплаченный счёт: связь вознаграждения с переносимой репутацией остаётся отдельным открытым вопросом. Её ограничения разобраны в главе «Аттестации и переносимая репутация». открыто



В предлагаемой экономике Swarm Chain эмиссия, плата за записи и имена, чаевые и приоритет курации оплачивают разные действия.

27.8 ИИ-медиа: происхождение без обещания аудитории

ИИ-видео и ИИ-аудио выбраны ключевым сценарием. Автору нужны переносимые файлы, возможность найти аудиторию через выбранных кураторов и способ сообщить происхождение работы. Сам выбор сценария решён; монетизация зависит от будущего спроса. Не следует обещать автору заработок только потому, что его фильм принят как корректная запись или что он использовал разрешённый инструмент генерации. **решено**

Метка «ИИ» или «ИИ-ассист» предлагается нативной частью указателя, бесплатной и не влияющей на приём. Клиент решает, как её показать. Идея раскрытия без штрафа относится к базовому протоколу: читатель всё ещё может выбрать лейблер или фильтр, который ограничивает его собственную ленту. Отсутствие отказа на уровне цепочки не является обязанностью каждого клиента рекомендовать эту работу. **предложено**

Подпись удостоверяет публикатора, а не «человечность» произведения, правообладание или художественную ценность. Метаданные C2PA рассматриваются как добровольное дополнение внутри файла. Например, подписанный синтетический музыкальный клип имеет проверяемого публикатора, но из одной подписи нельзя заключить, что все использованные голоса разрешены правообладателями. Происхождение записи и законность использования материалов требуют разных свидетельств. **предложено**

Предлагаемый интерфейс показывает не обещание вечной загрузки, а число демонов, уже получивших файл. Автор может увидеть, что раздача вышла за пределы его вкладки, затем искать кураторов или заказать отдельный архив. Приоритет курации и чаевые дают способы взаимодействия, но не заменяют выбора аудитории. Без заинтересованных держателей контент может уйти из памяти; это ограничение принято и для ИИ-медиа. **предложено**

27.9 Операторы и правовые границы

Правовая часть архитектуры — карта рисков, а не юридическая гарантия участнику. Нужно разделять авторство протокола, работу индексатора, обслуживание трекера и хранение файла. То, что оператор не держит медиа, ещё не позволяет считать его поисковый сервис вне любых обязательств. Точная применимость требований зависит от сервиса и юрисдикции; архитектура не устанавливает универсального освобождения для p2p-инфраструктуры. **предложено**

Предложено, чтобы каждый узел, трекер и индексатор вёл собственную политику блокировки и мог подписываться на лейблеров. Например, индексатор отказывается выдавать конкретную публикацию, а архивист прекращает раздачу файла. Эти действия не равны удалению записи из всех копий цепочки. Нейтральность правил приёма и самостоятельная ответственность оператора сосуществуют именно через разделение протокола, выдачи и хранения. **предложено**

Команде ядра предлагается не вести индексаторы, трекеры и сидбоксы как собственный инфраструктурный бизнес. Отдельные операторы должны учитывать уведомления о нарушениях, контактные каналы и требования отчётности, применимые к их сервисам. Упоминаются механизмы DMCA и DSA, но их наличие в карте рисков не является готовым регламентом соответствия. Формат обязанностей в протоколе индексатора ещё требует определения. **предложено**

Для токена предложен честный запуск без премайна и эмитента, без продвижения командой листинга. Он рассматривается как менее уязвимая конфигурация, а не как автоматический юридический иммунитет. Экономическая схема и реальная деятельность операторов должны оцениваться отдельно; нельзя вывести правовой статус из одного слова «децентрализованный». Это предложение о дизайне запуска, которое ещё не устанавливает условия выпуска актива. **предложено**

Остаются открытыми точные форматы социальных записей, интерфейс индексатора, параметры защиты от спама, мобильный режим, цена имён и распределение их платы. Воспроизводимая репутация агентов также требует своих правил: метки кураторов и переносимые аттестации работы не сводятся к одной готовой шкале. Социальный слой должен соединить публикацию, выбор и оплату сервисов, сохраняя явными эти нерешённые границы. **открыто**

Часть V. Вместе

28 Сквозные сценарии

Отдельные главы разделяли полномочия, исполнение, свидетельства и принятие результата. Теперь проследим, как эти границы встречаются в одной работе. Отправной точкой будет проект, которому нужно исправление и независимое ревью; затем добавятся постоянная команда, чужая ёмкость модели и история участия между проектами. Сценарии показывают целевую последовательность и места отказа. Наличие работающих частей не означает, что вся последовательность уже реализована. предложено

28.1 От просьбы до принятого изменения

Владелец ставит задачу на доске: исправить поведение библиотеки, сохранив совместимость. Задача задаёт результат и критерии приёмки, а процесс определяет роли и проверки. Доска уже умеет вести задачи, запускать участников, хранить ресурсы и передавать изменение на ревью. Но описание желаемого результата само по себе не разрешает агенту менять произвольные файлы, выбирать платный источник модели или принимать собственную работу. Эти основания должны существовать отдельно. работает сегодня

В целевом Runtime вызывающий собирает через Swarm Curator закреплённый план разработчика. Диспетчер проверяет полномочие запуска, зависимость от вызывающего, отзыв, квоты и доступную ёмкость. Менеджер пользователей готовит отдельную учётку ОС, запускатель исполняет разрешённую программу под ней. Выбор модели в допущенном наборе может помочь уложиться в бюджет, но не выдаёт разрешения на дополнительный источник расходов. Границы плана и источник средств определяются до исполнения. спецификация

Такое разделение важно при ошибке самого разработчика. Если ему кажется, что исправление требует чужого модуля, он должен запросить расширение полномочий. Если другой агент прислал убедительную инструкцию, она проходит путь проверки сообщений, а не становится командой благодаря убедительному тексту. Принципы описаны в главах «Гранты и полномочия» и «Путь сообщения и мост»: подлинность отправителя, право действия и пригодность предложения проверяются раздельно. предложено

Разработчик работает в самостоятельном репозитории своей учётки. Связанный `git worktree` через границу разных пользователей не заменяет такого разделения. Сеть ограничена политикой исполнения; доступ к модели не означает доступ к хостингу кода. Сборки и тесты изменения должны выполняться в выделенной фазе на изолированных раннерах другой машины, а не получать секреты проекта вместе с рабочим деревом. решено

После работы наша сторона сохраняет результат, строит подписанный коммит-кандидат и предъявляет на ревью именно его. Непринятые файлы не должны стать инструкциями ревьюверу через автоматическое обнаружение конфигурации харнесом. Пока соответствующий адаптер контекста не квалифицирован, консервативный путь — инертный diff или отдельные объекты точного кандидата вне каталогов такого обнаружения. Право читать изменение не равно праву исполнять инструкции из него. решено

Ревьювер проверяет изменение и свидетельства проверок, после чего правила процесса решают, достаточно ли оснований для принятия. Реплика разработчика «исправлено» остаётся предложением исхода. Доска уже различает работу участника, опубликованную ревизию и её принятие; переход к защищённому платформенному пути не должен стирать это различие. Оно особенно важно, когда результат выглядит правильным, но обязательная проверка ещё не запускалась. работает сегодня

У этого сценария пока нет законченной платформенной цепочки до публикации. Неагентский исполнитель внешних действий репозитория, его учётные данные, право подписи и маршрут запроса ещё требуют общего контракта. Именно он должен проверять точный кандидат при публикации, передаче артефакта и подписанном продвижении ветки. Поэтому описанный путь не является инструкцией уже доступного защищённого развёртывания. Состояние отдельных частей сведено в главе «Где мы и что дальше». открыто

Есть и более простой отрицательный исход: исправление написано, но проверяющий недоступен. Тогда сохранённый результат не теряется, однако принять его только ради освобождения учётки нельзя. Очередь, повторный запуск участника и сохранение артефакта помогают продолжению, а основания принятия остаются прежними. Если проверка вернулась с неизвестным результатом, её нельзя отметить положительной. Для внешнего читателя это существенное различие между «работа участника закончилась» и «проект принял изменение». спецификация

Внутри одного владельца эти шаги опираются на местные службы и правила процесса. Между независимыми владельцами нужны дополнительные подтверждения журнала и явные допущения о среде. Поэтому слово «ревью» в обоих случаях обозначает суждение о результате, но не одинаковый набор уже развёрнутых гарантий. В последующем сценарии Protocol появятся кворум и аренда; они не должны задним числом приписываться сегодняшнему локальному запуску через доску. спецификация

28.2 Постоянная команда переживает остановку

Теперь разработчик входит в постоянную команду вместе с оркестратором и Swarm Keeper. У постоянного участника логическая роль живёт дольше отдельного исполнения. Учётка не выводится только потому, что процесс остановился; повторный запуск не требует создать новую личность. Вывод учётки, остановка исполнения и отзыв личности — разные действия. Такое разделение позволяет сохранить обязанности без предположения, что старый процесс всё ещё работает. решено

Перед плановым перезапуском хост сессий закрывает приём, доводит исполнение до покоя и сохраняет пакет передачи. Затем нужно подтвердить отсутствие прежнего исполнения, проверить закреплённый план и основания запуска преемника. Само право однажды запустить агента не даёт неограниченного права перезапускать его. Путь требует отдельно разрешённого жизненного цикла и свежего решения об отзыве, как описано в главе «Запуск агента и хост сессий». спецификация

Пакет передачи содержит относящийся к работе контекст и ссылки на свидетельства. В нём нет переносимого разрешения нажать ожидающее подтверждение, чужих токенов или права продолжить незавершённый внешний эффект. На другой машине контекст пересобирают из пакета; нативное возобновление нельзя обещать только на основании одинакового имени харнеса. Совместимость плана и среды проверяется отдельно от содержательного продолжения задачи. спецификация

После перезагрузки диспетчер сверяет записи с действительностью. Уцелевшая учётка постоянной команды не доказывает наличия работающего исполнения. До подтверждения нового раннера и его эпохи приём остаётся закрытым. При этом источник, который должен попросить поднять постоянные сессии после загрузки машины, ещё не выбран; форматы разрешений хоста тоже не опубликованы. От сохранённого описания команды нельзя перескочить к обещанию автоматического восстановления. открыто

Предлагаемый модуль кипера после запуска воспроизводит журнал обязательств. Оборванный хвост не считается целой записью, порядок определяется последовательностью журнала, а не сравнением настенных часов. Истёкшие сроки и недоставленные уведомления требуют сверки с уже записанными последствиями. Это защищает от повторного открытия закрытого эпизода и от потери обязательства между записью недоступности держателя и записью её последствий. Кода этого модуля пока нет. предложено

Если ошибка входа в провайдера остановила агентский цикл, парковка сохраняет цель, но не восстанавливает логин. Возможный путь повторного входа с участием владельца аккаунта ещё не имеет полного распределения ответственности и спецификации доставки событий. Рабочие способы входа

для конкретных харнесов рассмотрены в главе «Учётные данные и логины»; они не доказывают автоматическое восстановление всей команды после истечения учётных данных. открыто

28.3 Обязанность переходит только после действия

Постоянный участник отвечает за проверку ночного результата. Оркестратор хочет передать эту обязанность сменщику. В модели Swarm Keeper появляется отдельная передача со сроками принятия и первого действия. Ответ «беру» подтверждает принятие предложения, но прежний держатель ещё не освобождён. Только проверенное первое действие нового держателя завершает передачу. Подробные состояния разобраны в главе «Swarm Keeper». предложено

Сменщик проверяет результат и оставляет событие на доске. Доска предоставляет свидетельство, а журнал обязательств связывает его с нужной передачей. Это не второй независимый трекер задач: у доски остаются её факты, у кипера — обязательство и переход держателя. Номер передачи нужен, чтобы действие по другой договорённости или старой попытке нельзя было зачесть как исполнение текущей. предложено

Если сменщик принял передачу и затем остановился, обязанность всё время остаётся у прежнего держателя. По сроку попытка закрывается неудачей и требует решения о дальнейшем держателе. Поздняя квитанция не оживляет закрытую передачу. Это отличается от позднего исполнения ещё открытого обязательства: журнал проверяет допустимость конкретного перехода, а не просто наличие похожего сообщения с подходящим автором. предложено

Другой случай — сломался общий раннер, и ремонтирующий участник тоже остановился. Для восстановления ресурса предлагается собственное обязательство с держателем, сроком проверки и свидетельством возвращения в строй. Сообщение «разбираюсь» не закрывает поломку; успешное принятие раннером задачи может стать относящимся к ней свидетельством. Недоступность ремонтирующего поднимает вопрос о владельце восстановления, а не превращает поломку в забытый комментарий. предложено

Получатель эскалации может организовать передачу или записать ожидание до определённого срока. Эскалация не раздаёт права ремонтировать любой сервис. Если наблюдение само сломалось, ошибка чтения не считается пустым списком обязательств. Надёжное автоматическое пробуждение пока не проверено, поэтому адресный вопрос сопровождается видимым человеку вопросом к владельцу. Обещания устранить простой за определённое число минут из этой модели не следуют. предложено

28.4 Просьба из чата и действие оператора

Владелец пишет оркестратору из комнаты проекта. Короткий «звонок» сообщает о новом событии, агент вытягивает тело отдельно. Такой способ связи через Matrix уже используется. Он позволяет не вставлять весь поток разговора прямо в текущий ввод, но сам по себе не доказывает проверку полномочий на каждую просьбу и не заменяет надёжное подтверждение применения сообщения харнесом. работает сегодня

В целевом мосте событие сначала сохраняется, затем проверяются подпись, ключ отправителя и грант. Дополнительный guard может ограничить решение, но не создать полномочие. Текст без подтверждённого основания может попасть к модели как недоверенные данные; невалидная подпись не становится допустимой командой после такого переименования. Подтверждение сохранения события, его доставка и фактическая обработка остаются отдельными наблюдаемыми фактами. Спецификация моста ещё не опубликована; весь путь остаётся архитектурным предложением. предложено

Допустим, просьба требует выдать оркестратору новое право запуска. Агент не может выдать его себе на основании сообщения в собственной истории. Для оператора выбран отдельный путь: телефон показывает точное действие, человек подтверждает его, ключ телефона подписывает это содержание, защищённая команда проверяет подпись. Терминал предоставляет запасной путь с паролем; учётка наблюдателя позволяет читать состояние без полномочия его менять. решено

Подпись должна относиться к конкретному действию: адресату, объёму прав и ограничениям, а не к произвольному последующему решению модели. Если сообщение доставлено дважды, это не основание дважды выдать независимые права. Форматы операторской команды, подписываемых байтов и восстановления после потери телефона ещё не опубликованы. Выбор телефона как основного интерфейса не означает, что приложение и весь протокол уже доступны. открыто

Аналогично выглядит команда от другой организации: её цепочка делегирования должна доходить до признаваемого корня и сужаться на каждом шаге. Исполнитель перепроверяет полномочие у границы эффекта, даже если мост уже пропустил сообщение. Этот межорганизационный путь остаётся направлением архитектуры. Нельзя считать федерацию транспорта или знакомое имя отправителя готовым доказательством права перезапустить наш раннер. предложено

28.5 Донор приносит модель, проект оставляет инструменты

Проекту нужна дополнительная ёмкость модели. Внешний участник оставляет агентский цикл и учётные данные провайдера у себя; проект предоставляет ограниченное рабочее место. Это разделение полезно, когда среда содержит дорогой тулчейн, устройства или данные, которые нельзя просто скопировать каждому участнику. Оно не увеличивает ёмкость самой среды: очередь сборок и число модельных циклов учитываются раздельно. спецификация

Человек начинает присоединение и проверяет среду проекта через выбранные основания первого контакта. Владение собственными ключами ещё не означает допуска: проект сверяет приглашение или другое предусмотренное основание, закрепляет ключи и выдаёт роль. Донор устанавливает исходящее SSH-соединение к проекту. Ограниченная принудительная команда и короткий сертификат нужны именно для рабочего пути; это не обычный интерактивный вход на машину проекта. решено

Харнес у донора должен работать без локальных инструментов в квалифицированной конфигурации. Обещание не выполнять команды на своей машине слабее запрета, а отсутствие инструмента в описании не доказывает, что обходов нет. Варианты установки донора имеют разные границы: запертый режим сам по себе не создаёт разделения пользователей ОС; вариант с двумя учётками требует компонента привилегированной подготовки, которого пока нет. решено

На стороне проекта рабочее место получает отдельного пользователя и самостоятельный репозиторий. Исходящая сеть этого пользователя закрыта, включая локальные сетевые пути; входящий SSH рассматривается отдельно. Служебное реле передаёт разрешённые вызовы под нужной учёткой. Доступ к инструменту веб-поиска в агентском цикле не означает сетевого доступа рабочего места, а принятие личности в доверенный круг не переносит её модельный цикл на наш хост. решено

Перед выгрузкой результата требуется остановить изменение дерева и сохранить точную дельту. Наша сторона не должна следовать ссылке в чужой файл, исполнять конфигурацию кандидата или удалять дом при неизвестном исходе экспорта. Незавершённая выгрузка сохраняет обязательство получить результат. При доработке ревью снова относится к полному кандидату над доверенной базой; чувствительные изменения не исчезают из рассмотрения только потому, что их уже показывали раньше. спецификация

Общий жизненный цикл SSH-входа, реле, повторного соединения и всех процессов учётки ещё требует контракта. Запрет записи и запрет исходящей сети не доказывают, что новый вход не создаст исполнение во время экспорта или замены. Не выбран и полный исполнитель публикации результата. Эти ограничения главы «Удалённое рабочее место» остаются ограничениями всего сценария, даже если отдельный удалённый вызов уже удалось бы продемонстрировать. открыто

28.6 Независимые владельцы принимают точный кандидат

В Swarm Protocol тот же проект рассматривается уже как общая среда для независимых владельцев. Назначение связывает участника, роль, область записи и план; кор-ноды фиксируют допустимые записи журнала по правилам выбранного уровня доверия. Модель оркестратора предлагает работу, но

не подменяет детерминированное подтверждение её оснований. Разделение плоскостей мышления, управления и исполнения описано в главе «Модель протокола». спецификация

Продьюсер представляет ревизию относительно базы. Среда строит интеграционный кандидат над текущей головой основной ветки. Ревью и проверки относятся к этому кандидату, текущему контракту задачи, политике и окружению проверок. Выбор ревьювера должен удовлетворять объявленной политике независимости. Если подходящего участника нет, нужно ждать или эскалировать; другой ключ того же владельца не создаёт независимость автоматически. спецификация

Пока ревью шло, другая задача могла продвинуть основную ветку. Тогда прежний кандидат построен уже не над текущей головой, и атомарное сравнение перед интеграцией должно отказать. Требуется новый кандидат с повторными проверками и ревью. Совпадение текста патча не переносит прежнее одобрение: новая база меняет окружение результата, даже когда собственные строки исправления выглядят одинаково. спецификация

Ограниченная исполняемая модель проверяет именно часть этой механики: текущую голову, эпоху, версию контракта, привязку аттестации и повторную доставку. Она не моделирует семантику кода и не доказывает всю распределённую реализацию. Поэтому её результат из главы «Что проверено и как проверить остальное» нельзя превратить в утверждение, что произвольный вредный патч будет распознан или что весь сценарий уже прошёл испытание на настоящих участниках. спецификация

После принятия может понадобиться внешнее действие раннера. Если запрос ушёл, а ответ потерялся, состояние остаётся неизвестным. Получатель с поддержкой идемпотентности позволяет сверить ту же операцию; иначе требуется разбор, а не слепой повтор. Смена оркестратора не обнуляет этот долг: преемник восстанавливает его по журналу. Принятая интеграция и установленный исход внешнего эффекта — разные факты. спецификация

28.7 Ревьювер переносит историю в другой Runtime

Внешний агент проверяет изменение библиотеки в одном Swarm Runtime и позже хочет проверять приложение в другом. Переносимым результатом должна стать история подтверждённого участия. Права первого проекта, его подписка на модель, незавершённые подтверждения и доступ к репозиторию не путешествуют вместе с ней. Этот сценарий соединяет Runtime, протокольное принятие результата и предлагаемый слой аттестаций Swarm Chain. В целом он остаётся предложением. предложено

Сначала первый проект самостоятельно допускает ревьювера к роли. Внешний агент пользуется своими ключами, а не нашим ключником; связь с владельцем и непрерывность личности проверяются по выбранной политике. Отдельно проверяется независимость от продьюсера. Даже уже известная личность может не подойти для этого раунда из-за общего владельца, модели или тулчейна. История полезных ревью не превращает совпадающие источники ошибок в независимые. предложено

Ревьювер читает точный кандидат и выдаёт вердикт. Проект проводит предусмотренную приёмку; один уверенный ответ ещё не создаёт принятого результата. После принятия проект как эмитент может подписать аттестацию об участии агента в роли ревьювера. Это свидетельство выдано проектом по его правилам, а не самим агентом в подтверждение собственной полезности. Смысл такой записи ограничен указанной работой и основанием её принятия. предложено

В Swarm Chain предлагается поместить хэш и необходимые метаданные, сохранив подпись эмитента. Код, промпты и секреты туда не публикуются. Проверка включения позволяет установить связь сохранённого свидетельства с цепочкой, но не доказывает правильность ревью и не обещает вечную доступность полного документа. Скрытые обязательства и выборочное раскрытие могли бы дополнительно защищать приватность, однако их устройство ещё не определено. предложено открыто

Индексатор собирает относящиеся к личности аттестации и вычисляет репутацию по опубликованным правилам. Вес зависит от доверия к выдавшим проектам, а не только от числа подписей. Детерминированность нужна, чтобы разные участники могли повторить вычисление при одинаковых дан-

ных и правилах. Формула, затухание, оси оценки, право эмиссии и разрешение споров пока открыты; сценарий не задаёт ни общего балла, ни порога допуска. предложено открыто

Второй Runtime проверяет личность, актуальность ключей, подписи эмитентов и доступные свидетельства. Затем применяет собственную политику: признаёт одних эмитентов, запрашивает дополнительные проверки или отказывает в роли. Репутация может удешевить знакомство с участником, но не выдаёт грант и не обязывает новый проект принять его. Новая задача всё равно требует местного назначения и проверки независимости именно от её продьюсера. предложено

Ротация рабочего ключа агента должна сохранять связь с долгоживущей личностью и её историей. Отдельный случай — отзыв ключа, которым проект выдаёт свидетельства. Если отозван ключ эмитента, индексаторы не учитывают аттестации, подписанные им после отзыва. Порядок исторического пересчёта аттестаций, подписанных до отзыва, остаётся открытым; автоматическое исключение всей прежней истории из этого правила не следует. Единый формат связей ключей пока не опубликован. Ни непрерывность имени, ни прежнее положительное участие не дают оснований игнорировать отзыв или считать отсутствующее подтверждение полученным. предложено открыто

28.8 Память помогает продолжить, но не разрешает действие

Между двумя задачами агент забыл, почему проект выбрал определённую границу доверия. В предлагаемой памяти он запрашивает прежнее утверждение, связанное с решением, и получает источник, обоснование, оценку применимости и последующие пересмотры. Это способ восстановить ход решения, а не заменить действующий контракт задачи. Первый пилот выбран для документальной преемственности решений с проверкой человеком, без сырых журналов и персональных профилей. предложено

Например, сохранена причина разделить тяжёлую проверку и исполнение агента. Память может показать, какое ограничение было принято и к какому случаю оно относилось. Но актуальную политику раннера агент всё равно получает из её авторитетного источника. Запись о прежнем разрешении не подтверждает, что грант ещё действует, а пересказ обязательства не определяет его нынешнего держателя. За полномочия, процесс и обязанности отвечают соответствующие службы. предложено

Запрос может вернуть поддержку утверждения и одновременно возражение. Это не ошибка формата, которую нужно скрыть выбором одного удобного ответа. Положительные и отрицательные основания рассматриваются раздельно; устаревшая оценка тоже не равна доказанной ложности. Если чтение источника не удалось или обработана лишь часть зарегистрированного графа, отсутствие найденного возражения нельзя назвать отсутствием возражений вообще. Предел поиска должен оставаться видимым потребителю. предложено

После перезапуска новый агент получает сведения как данные с происхождением. Они не становятся стартовой инструкцией только потому, что помещены рядом с ней. Даже правильная ссылка в ответе не доказывает, что все остальные входы были учтены: происхождение должно относиться ко всем контролируемым источникам представления. Такое требование особенно важно при конфликте версий решения и при неоднозначности того, какая версия считается текущей. предложено

Автоматическое включение памяти в стартовый контекст — отдельный следующий шаг после явного инструмента. Выбранное направление сохраняет отдельный блок данных, а не оверлей инструкций. Сновидение, то есть репетиция процесса на искусственном материале, также не может незаметно пополнить историю реальных событий. Для него требуется отделение от настоящих свидетельств и проверка человеком перед возможным использованием. Работающая польза такой памяти пока не измерена. предложено

Наконец, память проекта и p2p-память сети не являются одним хранилищем. Первая должна помогать прослеживать решения с учётом доступа и пересмотров. Вторая помогает участникам удерживать выбранный контент без обещания вечного хранения. Размещение хэша в цепочке не решает ни вопрос удаления чувствительных данных, ни вопрос полноты поиска по памяти. Эти ограничения разведены в главах «Память агентов» и «Социальный слой, память сети и экономика». предложено

28.9 Автор, куратор и архивист сохраняют публикацию

Другой путь через Swarm Chain начинается с ИИ-клипа. В выбранной архитектуре тяжёлый файл живёт в p2p-слое, а цепочка содержит небольшой подписанный указатель. Браузер нужен автору и зрителю; постоянное хранение и раздача относятся прежде всего к настольным участникам. Выбор BitTorrent первым транспортом не означает, что браузер становится полноценным постоянно доступным архивным узлом. решено

В предлагаемом сценарии автор публикует указатель, куратор просматривает материал и добавляет собственную подписанную категоризацию. Его метки отделены от авторских, а клиент зрителя решает, сколько доверять каждому источнику. Архивист загружает выбранную окрестность социального графа с собственными ограничениями диска и содержимого. Так доступность может поддерживаться интересом участников, однако никто не обязан хранить все публикации. предложено

Когда автор закрывает вкладку, наличие нескольких наблюдавшихся копий ещё не становится договором хранения. Полный узел вправе удалить старый сегмент социальных данных. Сохранённое доказательство Меркла помогает проверить включение записи, но не вернуть исчезнувший файл. Согласованные хэши состояния индексаторов также показывают согласованность состояния при одинаковых правилах, а не истинность рекомендаций и не полноту выдачи без дополнительных доказательств. предложено

Имена, пожертвования, платные архивы и экономика внимания могли бы связать этот путь с жизнью агентов. Но цена имени, правила выплат и доход архивиста не заданы готовым продуктом. Как и в сценарии ревьювера, подпись удостоверяет автора утверждения, а локальная политика решает, что с ним делать. Хранение, смысловая оценка и полномочие остаются отдельными вопросами во всех трёх слоях Swarma. предложено открыто

29 Где мы и что дальше

Swarma состоит из работающих инструментов, принятых архитектурных решений, спецификаций и предложений. Эти состояния нельзя складывать в один процент готовности. Работающая доска не доказывает изоляцию всех её участников, наличие схемы не доказывает реализацию, а проверка ограниченной модели не удостоверяет распределённую систему целиком. Поэтому ниже каждый статус относится к названной возможности и её границе, а не к названию продукта вообще.

29.1 Одна карта состояния

Таблица собирает состояния из соответствующих глав. «Работает сегодня» относится к описанному там пути и закреплённым условиям проверки. «Решено» сообщает о выбранном устройстве; «спецификация» — о сформулированном контракте; «предложено» и «открыто» сохраняют нерешённую часть. Соседство строк не означает, что их интеграция уже проверена. Там, где у модуля есть работающая и будущая части, они разделены внутри строки.

Слой и возможность	Подтверждённое состояние	Следующая граница и глава
Swarma, разделение слоёв	Runtime для одного владельца, Protocol для независимых участников, Chain для глобальных связей — выбранная архитектура. решено	Это не три выпущенных совместимых продукта. «Swarma целиком».
Runtime, пользователь ОС и запускатель	Отдельная учётка на автономного агента выбрана; узкие операции менеджера и запускателя описаны. решено спецификация	Код защищённого пути не заявлен. «Пользователь ОС на агента и сеть агента».

Слой и возможность	Подтверждённое состояние	Следующая граница и глава
Runtime, сеть	Кооперативные профили и проверки адаптеров существуют; принудительная сеть через ядро и прокси описана отдельно. работает сегодня спецификация	Общая заморозка поколения и сессий прокси ещё требует контракта. «Пользователь ОС на агента и сеть агента».
Runtime, ключник	Выбрано отделить ключи от агентского цикла; самостоятельного модуля и спецификации ключника нет. решено	Восстановление мастер-секрета открыто. «Ключи и ключник».
Runtime, оператор	Телефон, терминал и отдельное чтение состояния выбраны; форматы действия и восстановление телефона не опубликованы. решено открыто	Выбранный интерфейс не равен работающему приложению. «Оператор и интерфейсы».
Runtime, гранты	Общая модель делегирования предложена; узкие гранты брокера описаны спецификацией. предложено спецификация	Общий реестр действий и прав не реализован как единый опубликованный контракт. «Гранты и полномочия».
Runtime, транспорт и мост	Matrix со звонком и вытягиванием сообщений используется; полный мост с проверкой полномочий остаётся архитектурой. работает сегодня предложено	Требуются спецификация и сквозная проверка доверенной доставки. «Путь сообщения и мост».
Runtime, диспетчер	Допуск, квоты, привязки и привилегированный старт описаны; самостоятельный код не заявлен. спецификация	Связать старт с отзывом и подтверждёнными исходами. «Запуск агента и хост сессий».
Runtime, хост сессий	В доске работает `tb-sessiond`; отдельная служба и раннер имеют целевую спецификацию. работает сегодня спецификация	Аутентификация запросов хоста и разрешения жизненного цикла открыты. «Запуск агента и хост сессий».
Runtime, Swarm Keeper	Граница журнала обязательств и доски выбрана; передача, эскалация и восстановление остаются предложением без кода модуля. решено предложено	Нужны принятая спецификация, часы, хранение и проверенное пробуждение. «Swarm Keeper».
Runtime, Swarm Curator	Профили, управляемые дома, скиллы и аудит работают. работает сегодня	Снимки домов, подготовка входа и проверка готовности выбраны, но не объявлены выпущенными. «Swarm Curator». решено
Runtime, привязки MCP	Наличие серверов проверяется по имени; декларативная идентичность сервера и доставка авторизации требуют нового пути. работает сегодня предложено	Нельзя приравнять совпадение имени к удостоверению получателя секрета. «Swarm Curator».
Runtime, логины	Описанные опыты подтверждают конкретные способы входа и изоляции домов. работает сегодня	Обновление, конкуренция и принудительный выбор хранилища требуют квалификации по харнесу. «Учётные данные и логины».
Runtime, брокер учётных данных	Привязки, аренды, отзыв и единственный владелец обновления описаны; кода службы нет. спецификация	Начать с одного защищённого пути провайдера, затем расширять. «Учётные данные и логины».

Слой и возможность	Подтверждённое состояние	Следующая граница и глава
Runtime, удалённое рабочее место	Выбраны разделение донора и инструментов проекта, собственные ключи и варианты установки. **решено**	Подготовка донора и общий жизненный цикл SSH-реле не закрыты. «Удалённое рабочее место». **открыто**
Runtime, результаты и публикация	Выбран путь через сохранение дельты, точный кандидат и независимое принятие. **решено**	Неагентский исполнитель публикации, передачи артефактов и интеграции ещё требует контракта. «Удалённое рабочее место». **открыто**
Runtime, доска задач	Задачи, роли, ресурсы, запуски, Change Request и принятие с проверкой базы работают. **работает сегодня**	Платформенный процесс и права ролей — отдельное целевое устройство. «Доска задач и процесс».
Runtime, конфигурация процесса	Разделены неизменяемое ядро, процесс, роль и машинная конфигурация; выбранные ограничения не должны ослабляться скиллом. **решено**	Полный переход к платформенному исполнению не заявлен. «Доска задач и процесс».
Runtime, маршрутизатор моделей	`cmr` выбирает внутри допущенного набора, воспроизводит решение и поддерживает теневую обёртку. **работает сегодня**	Внутренний шов доски, новые замеры и включение автоматического выбора ещё требуют работы. «Модели и деньги».
Runtime, память решений	Документальная преемственность выбрана первым пилотом; модель свидетельств и пересмотров предложена. **решено** **предложено**	Нет реализации и измеренной пользы; стартовая подача данных — следующий отдельный шаг. «Память агентов».
Protocol, общая среда и клиент	Разделение агентского цикла и инструментов проекта, назначение и инвентарь ограничений описаны. **спецификация**	Полная реализация демона среды и протокола ноды не заявлена. «Модель протокола».
Protocol, журнал и уровни доверия	Кворум, защита от отката, доступность тел записей и различия L0–L2 описаны. **спецификация**	Канонические форматы и полный набор векторов соответствия ещё нужны. «Журнал, кворум и инварианты».
Protocol, контракт задачи	Версии критериев и адресация свидетельств к точному кандидату заданы. **спецификация**	Валидаторы не решают семантическую правильность кода. «Контракт задачи и правила валидаторов».
Protocol, аренда и цикл	Эпохи, отсечение прежнего владельца, ревью и атомарная интеграция описаны. **спецификация**	Выбор общего писателя и практическая цена повторных проверок открыты. «Оркестратор по аренде и цикл задачи».
Protocol, правила интеграции	Четыре условия интеграции описаны; проверяется ограниченный фрагмент. **спецификация**	Перебор модели и контрпримеры не доказывают всю реализацию; аренда, остановка и внешние эффекты вне модели. «Что проверено и как проверить остальное».
Protocol, эффекты и остановка	Неизвестный исход отделён от отказа; остановка одной подписью и кворумное возобновление описаны. **спецификация**	Требуются реальные испытания получателей эффектов и восстановления. «Внешние эффекты, остановка и федерация».
Protocol, федерация	Манифест проекта и локальный выбор доверия описаны; глобальный реестр и разделение ёмкости не определены полностью. **спецификация** **открыто**	Допущения среды и независимости должны оставаться явными. «Модель угроз».

Слой и возможность	Подтверждённое состояние	Следующая граница и глава
Chain, личность	Ключ, связь с владельцем, доказательства и непрерывность при ротации предложены. предложено	Единые форматы, веса доказательств и политика подтверждений открыты. «Идентичность агента».
Chain, аттестации и репутация	Предложены свидетельства проектов и воспроизводимые оценки индексаторов. предложено	Формула, эмитенты, споры, приватность и исторический пересчёт не определены. «Аттестации и переносимая репутация».
Chain, цепочка и имена	Разделение консенсуса денег и имён с социальными данными, PoW и внешним хранением выбрано. решено	Устройство блоков предложено; майнинг, параметры и цены имён открыты. «Цепочка: устройство».
Chain, p2p-память и социальный граф	Выбраны отдельный контент, BitTorrent первым и отсутствие гарантии хранения. решено	Клиенты, архивы, метки и индексаторы описаны как направление прототипа. «Социальный слой, память сети и экономика». предложено
Chain, экономика и эксплуатация	Доступ через граф, а не обязательная плата за внимание, — выбранное направление. решено	Вознаграждения, архивные услуги и распределение платежей не заданы готовыми правилами. «Социальный слой, память сети и экономика». открыто

29.2 Ближайший путь Runtime

Ближайший защищённый путь начинается с узкого запуска на одной машине: отдельная учётка, менеджер и запускатель, диспетчер, брокер, проверяемый план и доступ оператора. Сначала нужен один провайдерский случай с известными условиями, затем следующий. Это позволяет проверить взаимодействие реальных служб: кто хранит отзыв, кто разрешает запуск, кто наблюдает неизвестный исход и кто запрещает повтор. Добавление доски не должно скрывать отсутствие такой проверки. решено

Опыты с логинами полезны именно как ограниченные факты. Начальный успешный вход не доказывает безопасного обновления токена, отсутствия гонки между процессами или правильных прав после атомарной замены файла. Смена версии харнеса, ОС либо способа старта меняет условия опыта. Поэтому новый путь нужно квалифицировать со своим отрицательным контролем, а прежний результат сохранять вместе с условиями, при которых его получили. решено

Для сети следующий порог — принудительное ограничение, которое действует независимо от послушания модели. Кроме фильтрации новых соединений требуется совместный порядок заморозки поколения и закрытия либо перепроверки уже существующих сессий прокси. Остановка процесса сама по себе этого не доказывает. До публикации контракта нельзя называть отзыв сетевого доступа завершённым только потому, что агент перестал отвечать или исчез из наблюдаемого списка. открыто

Отдельный порог — результаты рабочего места. До внешнего донора должны согласоваться остановка всех процессов учётки, запрет создания новых исполнений через SSH, сохранение дельты и замена исполнения. Входящий канал не закрывается одним запретом исходящей сети. Политика видимости платформенных метаданных тоже требует явного описания читателей: изоляция домов не обещает скрыть все имена учёток, записи запуска и служебные квитанции. открыто

За этим следует доставка через платформенный процесс. Доска сохраняет роль источника задач и свидетельств, а хост сессий отвечает за управляемое исполнение и подачу сообщений. Проверки выноса хоста на искусственных данных не равны проверке защищённой службы под учётками агентов. Надёжное пробуждение, аутентификация управляющих запросов и начало постоянной команды после перезагрузки требуют собственных свидетельств. Простая связность интерфейсов ещё не доказывает живость. открыто

Swarm Keeper зависит от этих границ, но не должен ждать полной федерации, чтобы получить проверяемое ядро обязательств. Первым нужен однозначный журнал: переходы, восстановление оборванного хвоста, защита от повторов, передача только после действия и конечные эпизоды эскалации. Пока предложение не принято и не реализовано, наблюдение транспорта и человеческое пробуждение нельзя выдавать за автоматическую службу контроля обязательств. Интервалы и пределы реакции ещё не назначены. предложено

Swarm Curator и маршрутизатор развиваются рядом с этой работой. Для Swarm Curator важны воспроизводимая материализация домов и управляемая доставка авторизации инструментам. Для маршрутизатора — наблюдения теневого режима, внутренний шов доски и проверка локальных весов по точному отпечатку. Ни полезность более дорогой модели, ни экономия всего процесса не следуют из наличия селектора. Сравнивать нужно принятый результат, повторную работу и участие человека. предложено

Память решений имеет ещё более узкий первый шаг: несекретные документальные сведения, явный инструмент чтения и проверка человеком. Проекция памяти не должна становиться источником прав или вторым журналом задач. Следующий шаг с подачей данных при старте требует отдельной проверки происхождения и границ доступа. Накопление большого корпуса до такой проверки увеличило бы объём неподтверждённых обещаний, а не доказательную силу пилота. предложено

29.3 Что должен показать Protocol

Для распределённого протокола остаются два разных вопроса: соблюдаются ли инварианты и полезна ли организация работы. Первый требует программно воспроизводимых нарушений и сбоев: устаревшего владельца аренды, подменённого кандидата, изменённых критериев, повторной доставки и потерянного ответа внешнего действия. Отказ должен происходить у исполняющей границы. Проверка одной функции без её вызова на рабочем пути не установит это свойство для системы. спецификация

Ограниченная модель уже даёт трассы для части будущих векторов соответствия. Её результат не распространяется автоматически на сетевой обмен, файловую систему или честность среды. Следующие проверки должны охватить аренду, остановку и внешние эффекты. Отдельный опыт — остановить оркестратора, удалить его сессию и проверить продолжение из журнала и доступных артефактов. Это проверяет внешнее состояние непосредственно, вместо оценки качества пересказа контекста. предложено

Второй вопрос требует сопоставимых внешних задач и одинаковых критериев приёмки. Одиночный агент, параллельная работа без протокола и развёртывание L0 должны сравниваться по результату проекта. Число внутренних задач непригодно: оркестратор сам меняет декомпозицию. Нужно учитывать стоимость модели, проверки, ожидание среды, повторное ревью, восстановление и человеческое вмешательство. Пока таких данных нет, ускорение и экономия остаются проверяемыми гипотезами, а не достигнутыми показателями. предложено

Базовое допущение о целостности среды сохраняется и после этих опытов. Проверяющий на другой машине может независимо повторить тест, но это не удостоверяет весь контекст, показанный ревьюверу. Независимость владельцев, моделей и тулчейнов тоже требует объявленных оснований; калибровка посевом дефектов не измеряет долю злоумышленников. Выход к независимым участникам должен сохранять эти пределы, а не заменять их общим словом «децентрализация». спецификация

29.4 Что остаётся за Chain

У Swarm Chain выбран каркас, но нет заявленной работающей сети. До прототипа нужны согласованные развилки и симуляции сложности, эмиссии, цены атаки и роста данных. Собственная конфигурация RandomX и объединённый майнинг пока не образуют единого принятого механизма. Аналогич-

но нельзя собрать несколько предложенных кривых эмиссии в одну «точную» экономику или назначить доход сервисным узлам до выбора их обязанностей и оплаты. открыто

Социальному прототипу понадобятся словарь записей, правила состояния индексатора, клиенты автора и зрителя, куратор и архивист. Разделение консенсуса и социальной оценки уже выбрано, но не задаёт готовую схему каждого объекта. Доступность файла следует проверять отдельно от включения указателя в блок. Пределы браузерной раздачи и мобильного режима нельзя закрыть одной демонстрацией загрузки: вкладка может исчезнуть, а хранитель — удалить свою копию. предложено

Для переносимой репутации первым содержательным результатом стал бы проверяемый путь от принятой работы к аттестации проекта и её чтению другим Runtime. До этого нужны правила эмитентов, ключей, отзыва и воспроизводимого вычисления. Новый проект должен сохранять собственное решение о допуске. Число аттестаций без такого устройства не определяет ни полезность участника, ни его независимость, ни право действовать в чужой среде. предложено

Открытые вопросы не сводятся к реализации: кто задаёт веса доверия, как оспаривают запись, что можно раскрывать публично и как оплачивается хранение — отдельные решения. При этом подписи, данные об участии и локальные политики можно проектировать так, чтобы они не требовали заранее назначить единственную универсальную репутацию. Сохранять эту границу важно уже в первом прототипе: позднее её трудно восстановить поверх обещания общего балла. открыто

30 Словарь

Словарь объединяет названия, роли, состояния и сокращения книги; английские варианты оставлены рядом с русскими. В скобках указано название главы с объяснением и границами понятия. Определение термина не означает наличия реализации. Статусы работающих и будущих механизмов собраны в главе «Где мы и что дальше»; здесь сохраняется различие между фактом, полномочием и свидетельством.

30.1 Платформа и участники

- **Swarma** — Платформа целиком: локальный Runtime, межвладельческий Protocol и глобальный Chain. («Swarma целиком»).
- **Swarm Runtime** — Слой исполнения роя на машинах одного владельца. («Swarma целиком»).
- **Swarm Protocol** — Протокол совместной разработки независимыми владельцами над общей средой проекта. («Модель протокола»).
- **Swarm Chain** — Предлагаемый слой глобальных связей личности, аттестаций, имён и экономики. («Цепочка: устройство»).
- **Swarm Keeper** — Роль координации и предлагаемый модуль контроля обязательств; это не ключник. («Swarm Keeper»).
- **Swarm Curator** — Система профилей, пакетов, управляемых домов и подготовки агентской среды. («Swarm Curator»).
- **Агентский цикл** — Модель, контекст и последовательность обращений к инструментам для выполнения работы. («Модель протокола»).
- **Харнес, harness** — Программа, в которой работает агентский цикл: например, Claude Code, Codex или Muse. («Учётные данные и логины»).
- **Оператор** — Человек, чьи явные действия дают основания защищённому управлению платформой. («Оператор и интерфейсы»).
- **Оркестратор** — Роль декомпозиции и координации; в Protocol удерживается кор-нодой по аренде. («Оркестратор по аренде и цикл задачи»).

- **Продьюсер, ревьювер** — Создатель изменения и отдельный участник, выносящий о нём содержательное суждение. («Модель протокола»).
- **Контрибьютор, донор** — Владелец, предоставляющий модельную ёмкость; инструменты проекта могут оставаться в общей среде. («Удалённое рабочее место»).
- **Кор-нода, рабочая нода** — Проверяющий правила узел журнала и недоверенный агентский цикл контрибьютора соответственно. («Модель протокола»).
- **Верификатор** — Отдельная роль, повторяющая критические проверки на другой машине. («Модель угроз»).
- **Плоскости мышления, управления, исполнения** — Разделение выбора следующего действия, проверки его оснований и выполнения внутри Protocol. («Модель протокола»).
- **Метки статуса** — «Работает сегодня», «решено», «спецификация», «предложено», «открыто» описывают разные виды оснований («Для кого эта книга»).
- **Prompt injection, инъекция инструкций** — Попытка превратить недоверенные данные в команду; право проверяется вне суждения модели («Сквозные принципы»).

30.2 Изоляция и полномочия

- **Учётка ОС, UID** — Пользователь операционной системы и его числовой идентификатор; граница процессов и файлов. («Пользователь ОС на агента и сеть агента»).
- **Поколение учётки** — Отдельная идентичность выделения, исключающая принятие старых полномочий при повторном использовании UID. («Пользователь ОС на агента и сеть агента»).
- **Менеджер пользователей, запускатель** — Привилегированные компоненты узкого управления учётками и старта разрешённых программ под ними. («Пользователь ОС на агента и сеть агента»).
- **Граница и кооперативное ограничение** — Механизм против недоверенного поведения и правило, работающее при сотрудничестве агента. («Пользователь ОС на агента и сеть агента»).
- **Санбокс, область задачи** — Рабочая копия вместе с политикой ограничения; отдельный каталог сам по себе недостаточен. («Модель протокола»).
- **Самостоятельный репозиторий, git worktree** — Независимая база Git и связанная рабочая копия; последняя не пересекает границу пользователей. («Пользователь ОС на агента и сеть агента»).
- **Ключник, мастер-секрет** — Защищённое подписание от имени агента и корневой секрет выведения ключей; отдельный модуль ещё отсутствует. («Ключи и ключник»).
- **Иерархия ключей, hardened-выведение** — Разделение ключей по назначениям с защищёнными шагами выведения по BIP/SLIP. («Ключи и ключник»).
- **Ключ с присутствием** — Ключ, использование которого требует участия человека, например подтверждения на телефоне. («Ключи и ключник»).
- **Грант, capability** — Проверяемое разрешение на определённые действия и цели с ограничениями. («Гранты и полномочия»).
- **Делегирование, ослабление полномочий** — Передача части разрешения; потомок не получает прав сверх разрешающих оснований. («Гранты и полномочия»).
- **Отзыв, revocation** — Прекращение действия права или ключа; недоступность проверки отзыва не означает разрешения. («Гранты и полномочия»).
- **Реестр действий, обработчик** — Предлагаемое соответствие зарегистрированных действий исполнителям, проверяющим права у границы эффекта. («Гранты и полномочия»).
- **Ключ подписи и ключ SSH** — Разные назначения: удостоверить содержание и удостоверить участника соединения. («Ключи и ключник»).
- **CA, центр сертификации; KRL** — Выдающий сертификаты центр и список отозванных ключей; центры хостов и пользователей разделены. («Ключи и ключник»).

- **sudo, sudoers** — Механизм запуска с другими правами и его политика; не общий обход границ агентом. («Пользователь ОС на агента и сеть агента»).
- **Доверенный круг, размещение, класс исполнения** — Независимые условия допуска личности, местонахождения исполнения и применимых ограничений. («Удалённое рабочее место»).
- **ALLSTOP** — Специфицированное аварийное действие оператора для остановки узла; восстановление требует отдельного основания. («Ключи и ключник»).

30.3 Сеть, сообщения и удалённое исполнение

- **Egress, ingress** — Исходящие и входящие соединения; закрытие одного направления не закрывает другое. («Удалённое рабочее место»).
- **pf, nftables** — Механизмы ядра для принудительного сетевого ограничения на соответствующих ОС. («Пользователь ОС на агента и сеть агента»).
- **Прокси, CONNECT** — Посредник сетевого выхода и способ туннелирования; разрешённый адрес не исключает утечку через него. («Пользователь ОС на агента и сеть агента»).
- **DNS, DNSSEC** — Разрешение доменных имён и проверка подлинности подписанных DNS-данных при первом контакте. («Удалённое рабочее место»).
- **HTTPS, TLS; QUIC** — Защищённые сетевые протоколы; допустимость и фильтрация зависят от конкретной политики сети. («Пользователь ОС на агента и сеть агента»).
- **Loopback** — Локальные сетевые адреса машины; запрет внешнего выхода сам по себе их не закрывает. («Пользователь ОС на агента и сеть агента»).
- **Tailnet** — Защищённая сеть, выбранная для основного телефонного пути оператора. («Оператор и интерфейсы»).
- **Носитель, carrier; провайдер** — Транспорт сообщений и его адаптер; доверие к тексту проверяется отдельно от доставки. («Путь сообщения и мост»).
- **Matrix, XMPP** — Используемый носитель и возможный следующий; наличие транспорта не доказывает готовность полного моста. («Путь сообщения и мост»).
- **Мост, bridge** — Связь транспорта с агентом, включающая сохранение, проверку и управляемую подачу событий. («Путь сообщения и мост»).
- **Почтовый ящик, inbox; курсор** — Сохранённые события для вытягивания и позиция чтения после надёжного сохранения. («Путь сообщения и мост»).
- **Звонок, doorbell** — Короткий сигнал о событии; не тело сообщения и не доказательство его обработки. («Путь сообщения и мост»).
- **Ворота доверия, decision guard** — Проверка оснований сообщения и дополнительное ограничение решения; guard не создаёт полномочий. («Путь сообщения и мост»).
- **Подпись, неподписанные данные** — Проверяемое авторство байтов и содержимое без такого основания; подлинность не доказывает истинность. («Путь сообщения и мост»).
- **Проекция для аудитории** — Выбранное содержание публикации; подпись должна связывать именно отправляемые байты. («Путь сообщения и мост»).
- **Рабочее место, workplace; SSH-реле** — Учётка для удалённых инструментов и служебная передача вызовов к ней. («Удалённое рабочее место»).
- **Lockdown** — Режим харнеса без локальных инструментов; сам по себе не изоляция пользователями ОС. («Удалённое рабочее место»).
- **Квалификация** — Проверка точной конфигурации, версии и условий; результат не распространяется автоматически на другие. («Удалённое рабочее место»).
- **Export, импорт результата** — Сохранение дельты рабочего места и построение кандидата нашей стороной без исполнения чужих инструкций. («Удалённое рабочее место»).
- **Strict-context adapter** — Адаптер, отделяющий содержание непринятого кандидата от инструкций, автоматически обнаруживаемых харнесом. («Удалённое рабочее место»).

30.4 Сессии, пакеты и учётные данные

- **Диспетчер** — Служба допуска запуска, квот и привязок; физический привилегированный старт проходит через неё. («Запуск агента и хост сессий»).
- **План запуска** — Заранее собранные закреплённые входы исполнения; узкий исполнитель проверяет их, а не выбирает заново. («Запуск агента и хост сессий»).
- **Выделение, исполнение, логическая сессия** — Учётка с условиями, конкретный процесс и продолжаемая работа, способная иметь преемников. («Запуск агента и хост сессий»).
- **Хост сессий, раннер; tb-sessiond** — Управление сессиями, исполнитель одной сессии и существующий демон внутри доски соответственно. («Запуск агента и хост сессий»).
- **Забор допуска, fence; эпоха хоста** — Закрываемая граница приёма и версия управления, отделяющая старые запросы. («Запуск агента и хост сессий»).
- **Accepted, queued, delivered, applied** — Разные стадии подачи ввода; применение требует подтверждения харнеса, а не одной записи очереди. («Запуск агента и хост сессий»).
- **Парковка, quiescence** — Сохранённая остановка продвижения и подтверждённый покой исполнения перед управляемой передачей. («Запуск агента и хост сессий»).
- **Пакет передачи, handoff; resume** — Пересборка контекста из проекции и нативное возобновление совместимой среды — разные пути. («Запуск агента и хост сессий»).
- **Постоянная команда** — Участники без обычного срока истечения учётки и исполнения; остановка остаётся явным действием. («Запуск агента и хост сессий»).
- **Скилл, SKILL.md; Skillfile** — Пакет инструкций и ресурсов, его описание и состав пакетов проекта. («Swarm Curator»).
- **Профиль, overlay; lock, pin, closure** — Настройки среды, накладываемый слой и закреплённый полный набор зависимостей по содержимому. («Swarm Curator»).
- **Управляемый дом, materialize** — Подготовленное окружение харнеса и выбранное направление воспроизводимого создания его снимка. («Swarm Curator»).
- **MCP, привязка сервера** — Подключение инструментов к модели; совпадение имени сервера не удостоверяет получателя авторизации. («Swarm Curator»).
- **Аудит, реестр аудита** — Подписанная оценка пакета и учёт таких оценок; не гарантия безошибочности содержимого. («Swarm Curator»).
- **Shared, isolated; per-home** — Режимы разделения домов и вход по отдельному дому; хранилище секрета проверяется отдельно. («Учётные данные и логины»).
- **Credential store, Keychain, keyring** — Хранилище учётных данных и системные варианты; поддержка определяется конкретным харнесом. («Учётные данные и логины»).
- **OAuth, setup-token, API key** — Разные способы авторизации провайдера; успешность одного канала не подтверждает остальные. («Учётные данные и логины»).
- **Брокер, binding, credential lease** — Служба учётных данных, привязка разрешения к агенту и ограниченная аренда доступа. («Учётные данные и логины»).
- **Refresh owner, канал env** — Единственный владелец обновления токена и передача секрета окружением; секрет остаётся читаемым харнесу. («Учётные данные и логины»).

30.5 Обязательства, процесс и модели

- **Обязательство, держатель** — Явно отслеживаемая обязанность и отвечающий за неё участник; не просто упоминание в чате. («Swarm Keeper»).
- **Передача, первое действие** — Предложение смены держателя и проверяемое действие, только после которого она завершается. («Swarm Keeper»).
- **`accept_by`, `act_by`, `due`; RFC 3339** — Сроки принятия, первого действия, исполнения и полная запись времени вместо часа без даты. («Swarm Keeper»).

- **Сиротство, эпизод эскалации** — Недоступность держателя и отслеживаемый поиск решения; ни одно не расширяет полномочий. («Swarm Keeper»).
- **Свёртка, replay; seq** — Восстановление состояния из последовательности журнала, а не из порядка настенных времён. («Swarm Keeper»).
- **`owner-needed`; наблюдатель** — Видимый человеку запрос решения и источник наблюдения с собственным состоянием здоровья. («Swarm Keeper»).
- **Saved, delivered, read, done** — Уровни свидетельства: сохранено, доставлено, прочитано, сделано; соседняя ступень не доказывает следующую. («Swarm Keeper»).
- **Доска, task-board; run, spawn** — Трекер процесса, отдельный запуск участника и операция его создания для работы. («Доска задач и процесс»).
- **Change Request, исход** — Неизменяемая ревизия для проверки и предложение результата; принятие выполняется отдельно. («Доска задач и процесс»).
- **Ядро, kernel; guard** — Детерминированная часть процесса и проверка допустимости перехода, отделённые от суждения модели. («Доска задач и процесс»).
- **Playbook, действующая версия** — Конфигурация ролей и переходов, а также подтверждённая версия, продолжающая действовать до замены. («Доска задач и процесс»).
- **Authority report, confirmation** — Отчёт о покрытии изменений полномочиями и человеческое подтверждение определённой версии. («Доска задач и процесс»).
- **Goal, ресурс, свидетельство** — Цель, сохранённый материал и основание утверждения; ни один не заменяет другие. («Доска задач и процесс»).
- **Check set, protected kernel** — Именованные проверки и защищённые ограничения ядра, не отменяемые инструкцией роли. («Доска задач и процесс»).
- **Маршрутизатор моделей, `cmr`; селектор** — Выбор конфигурации внутри допущенного набора чистой функцией по закреплённым входам. («Модели и деньги»).
- **`economy`, `balanced`, `burn`** — Режимы бюджетной политики, сохраняющие ограничения допуска и требуемого качества. («Модели и деньги»).
- **`subscription`, `local`, `metered`** — Классы источника ёмкости: подписка, локальная модель и оплата потребления. («Модели и деньги»).
- **`select`, `recommend`, `shadow`** — Выбор, рекомендация и теневое наблюдение без подмены фактического выбора. («Модели и деньги»).
- **Каталог, overlay, SHA-256** — Данные оценок, поправки оператора и точный отпечаток локальных весов. («Модели и деньги»).

30.6 Доказательная память

- **Ограниченная доказательная память** — Предлагаемая система утверждений, оснований и пересмотров с явными пределами поиска. («Память агентов»).
- **Утверждение, assertion; source_ref** — Содержание суждения и отдельная ссылка на его источник. («Память агентов»).
- **Обоснование, justification; assessment** — Связь с доводами и отдельная оценка применимости либо качества. («Память агентов»).
- **Ревизия, revision; представление** — Неизменяемая запись пересмотра и вычисляемая проекция, не отдельный первичный тип. («Память агентов»).
- **Издатель, отправитель, субъект, viewpoint** — Автор утверждения, доставивший участник, предмет и точка зрения; роли не сливаются. («Память агентов»).
- **Поддержка, опровержение; `complete`, `incomplete`** — Независимые направления свидетельств и полнота просмотра зарегистрированного графа. («Память агентов»).
- **`version_conflict`, `ambiguous_current`** — Конфликт версий и неоднозначность текущей; не общий ярлык для любых разногласий. («Память агентов»).

- **`present`, `not_found`; `stale`** — Найденное основание, ненайденное в пределах поиска и устаревшая оценка, не обязательно ложная. («Память агентов»).
- **Provenance, происхождение** — Связь результата со всеми контролируемыми входами; наличие одной цитаты её не заменяет. («Память агентов»).
- **Сон, dream** — Репетиция на искусственном материале; не свидетельство реального события и не источник полномочий. («Память агентов»).

30.7 Журнал и принятие изменения

- **Журнал, запись, снимок** — Авторитетная последовательность решений, её элемент и проверяемое представление состояния. («Журнал, кворум и инварианты»).
- **Секвенсер, консенсус** — Определение порядка записей и согласование состояния; не содержательное ревью кода. («Журнал, кворум и инварианты»).
- **QC, сертификат кворума** — Достаточный по выбранным правилам набор подписей для совершения записи. («Журнал, кворум и инварианты»).
- **L0, L1, L2** — Разные допущения доверия: один узел, большинство с доверенным порядком, византийская устойчивость. («Журнал, кворум и инварианты»).
- **BFT; n, f, q** — Византийская устойчивость; число узлов, допустимых нарушителей и порог подписей. («Журнал, кворум и инварианты»).
- **Merkle proof, лёгкий клиент** — Доказательство включения и проверяющий заголовки либо снимки клиент без полного состояния. («Журнал, кворум и инварианты»).
- **Высшая принятая версия, rollback protection** — Долговременная граница, не позволяющая принять старое состояние после нового. («Журнал, кворум и инварианты»).
- **Сертификат доступности** — Подтверждение хранения тела записи; одного его хэша недостаточно для получения данных. («Журнал, кворум и инварианты»).
- **Инвариант, валидатор** — Сохраняемое свойство и детерминированная проверка допустимости; смысловую правильность они не гарантируют. («Контракт задачи и правила валидаторов»).
- **Контракт задачи, write scope** — Версионируемые условия результата и разрешённая область записи, отличная от желаемой цели. («Контракт задачи и правила валидаторов»).
- **R, B, M, I** — Представленная ревизия, её база, текущая голова и построенный над ней интеграционный кандидат. («Оркестратор по аренде и цикл задачи»).
- **CAS, compare-and-set** — Атомарное продвижение только от ожидаемой головы; предварительной проверки базы недостаточно. («Оркестратор по аренде и цикл задачи»).
- **Patch-id, перенос одобрения** — Идентичность патча не разрешает переносить ревью на новый кандидат в базовой гарантии. («Оркестратор по аренде и цикл задачи»).
- **Аренда, lease; fencing token** — Ограниченное владение ролью или областью и отсечение действий прежнего владельца. («Оркестратор по аренде и цикл задачи»).
- **Эпоха плана, аренды, оркестратора** — Разные версии полномочий; совпадение одного счётчика не заменяет проверки остальных. («Оркестратор по аренде и цикл задачи»).
- **Heartbeat, checkpoint; takeover** — Сигнал продления, сохранённая точка продолжения и явный захват после потери владельца. («Оркестратор по аренде и цикл задачи»).
- **Receipt, цепочка квитанций** — Проверяемые сведения об инструментальных обращениях; не доказательство полноты всего контекста модели. («Модель протокола»).
- **Инвентарь, host assurance** — Объявленные ограничения среды с полным, частичным либо отсутствующим обеспечением требуемых свойств. («Модель протокола»).
- **Положительная, отрицательная, неизвестная аттестация** — Разные исходы проверки; неизвестное нельзя засчитать как положительный результат. («Контракт задачи и правила валидаторов»).

- **Идемпотентность, неизвестный исход** — Защита от повторного эффекта и отсутствие установленного результата, требующее сверки. («Внешние эффекты, остановка и федерация»).
- **Раннер, reconcile; stop certificate** — Узкий исполнитель эффектов, установление исхода и основание остановки по правилам протокола. («Внешние эффекты, остановка и федерация»).
- **Манифест, федерация** — Объявленные условия проекта и работа между проектами с локальным выбором доверия. («Внешние эффекты, остановка и федерация»).
- **Оси независимости, Sybil** — Различие владельцев, моделей, тулчейнов и атака множественными контролируемыми личностями. («Модель угроз»).
- **Вектор соответствия, контрпример** — Воспроизводимый сценарий проверки контракта и исполнение, нарушающее заявленное свойство. («Что проверено и как проверить остальное»).
- **Посев дефектов, review recall** — Калибровка обнаружения известных ошибок; не измерение честности либо независимости участников. («Что проверено и как проверить остальное»).
- **Вестинг** — Предлагаемая отложенная часть вознаграждения; экономические правила ещё не определены. («Оркестратор по аренде и цикл задачи»).
- **Удалённый shell, spill, слот** — Состояние командной сессии, сохранение большого вывода отдельно и выделяемая ёмкость исполнения («Модель протокола»).
- **Конфликт свидетельств** — Несогласующиеся основания, отличные от конфликта версий и неопределённой текущей версии («Память агентов»).

30.8 Личность и переносимые свидетельства

- **Личность, идентичность; pubkey** — Долгоживущая связь участника с публичными ключами и владельцем, отличная от учётки ОС. («Идентичность агента»).
- **Резолвер, identity proof** — Получение и проверяемое основание связи ключа с доменом либо аккаунтом. («Идентичность агента»).
- **Well-known, двусторонняя публикация** — Размещение проверяемого подтверждения под контролем заявленного субъекта и подписанная связь с ним. («Идентичность агента»).
- **Ротация, отзыв ключа** — Смена ключа с непрерывностью личности и прекращение его действия — разные операции. («Идентичность агента»).
- **Эмитент, аттестация участия** — Проект как выдавшая сторона и его подписанное свидетельство участия агента в определённой роли с исходом «принято», «отклонено» или «не установлено». («Аттестации и переносимая репутация»).
- **Репутация, вес эмитента** — Вычисляемая история участия с учётом доверия к выдавшим свидетельства проектам. («Аттестации и переносимая репутация»).
- **Коммитмент, выборочное раскрытие** — Предлагаемые способы связать скрытые данные и предъявить часть свидетельства; устройство открыто. («Аттестации и переносимая репутация»).
- **Локальная политика допуска** — Правила нового проекта; переносимая история не создаёт автоматического права участия. («Аттестации и переносимая репутация»).

30.9 Цепочка и социальный слой

- **UTXO, PoW** — Модель непотраченных выходов и доказательство работы в предлагаемом устройстве Chain. («Цепочка: устройство»).
- **RandomX, ASERT** — Обсуждаемые алгоритм работы и пересчёт сложности; окончательные параметры не выбраны. («Цепочка: устройство»).
- **Объединённый майнинг, хвостовая эмиссия** — Совместная работа для цепочек и продолжающаяся награда; конкретное сочетание остаётся открытым. («Цепочка: устройство»).
- **Сегмент данных, pruning** — Социальная часть блока и удаление старых данных без обещания вечного хранения. («Цепочка: устройство»).

- **Сервисный узел, ChainLocks** — Возможные услуги и резервная окончательность; не реализованный стартовый консенсус Chain. («Цепочка: устройство»).
- **Имя, commit-reveal** — Читаемая связь с личностью и предложенная регистрация через обязательство с последующим раскрытием. («Цепочка: устройство»).
- **Индексатор, компактный фильтр** — Вычислитель представлений истории и средство отбора интересующих блоков лёгким клиентом. («Цепочка: устройство»).
- **Указатель, p2p-память** — Подписанная ссылка на содержимое и внешнее хранение самих тяжёлых файлов. («Социальный слой, память сети и экономика»).
- **BitTorrent, WebTorrent, libtorrent, Iroh** — Первый транспорт, браузерный и настольный варианты, а также возможный следующий транспорт. («Социальный слой, память сети и экономика»).
- **PWA, WebRTC, WebSocket-трекер, DHT** — Браузерный клиент и способы связи; браузер не имеет возможностей полноценного настольного узла. («Социальный слой, память сети и экономика»).
- **Социальный граф, окрестность** — Подписки и выбранная область распространения; глубина хранения определяется участником. («Социальный слой, память сети и экономика»).
- **Куратор, архивист** — Оценивающий материалы участник и хранитель выбранного содержимого; это не Swarm Curator. («Социальный слой, память сети и экономика»).
- **Категоризация, лейблер, пространство меток** — Подписанная оценка, её источник и область имён; авторские и чужие метки разделены. («Социальный слой, память сети и экономика»).
- **Hashcash, комиссия, аренда хранения** — Возможная работа на запись, оплата включения и отдельная услуга удержания данных. («Социальный слой, память сети и экономика»).
- **C2PA, подпись публикации** — Возможное свидетельство происхождения медиа; не доказательство истинности или прав на содержание. («Социальный слой, память сети и экономика»).